📖

MSA를 왜 배워야할까?

✅ MSA를 왜 배워야할까?

최근에 MSA를 배우려는 취준생 개발자분들이나 현업 개발자분들이 시간이 갈수록 점점 많아지고 있다. 이 분들이 MSA를 배우려는 이유는 다음과 같다.

> **어느 정도 규모가 있는 서비스 기업에 취업/이직을 하기 위해서**

그럼 규모 있는 서비스 기업에 취업/이직을 하려면 왜 MSA를 배워야 하는 지 조금 더 자세히 따져보자.

✅ MSA가 정말 취업/이직하는 데 큰 도움이 될까?

MSA가 정말 취업/이직하는 데 도움이 되는 지 확인하는 방법은 채용 공고를 살펴보는 것이다. 서비스 기업의 백엔드 개발자 채용 공고를 살펴보면 생각보다 많은 회사에서, MSA에 대한 경험을 우대해주고 있다는 걸 알 수 있다.

자격요건

1. 백엔드 개발 경력 3년 이상
2. Java 또는 Kotlin, Spring Boot 기반 서버 개발 경험
3. JPA, QueryDSL을 활용한 ORM 기반 개발 경험
4. 테스트 코드 및 코드 리뷰 문화에 익숙한 분
5. 제품과 사용자 경험에 대한 관심과 공감 능력을 갖춘 분

우대사항

1. MSA 환경에서의 개발 및 운영 경험
2. 대용량 트래픽 처리 경험 및 클라우드 환경(AWS/GCP) 이해도
3. Redis, MongoDB 등 캐시 및 NoSQL DB 경험
4. 푸시/알림/스케줄러 등 시간 기반 시스템 개발 경험
5. Grafana, Prometheus 등 모니터링 툴 사용 경험

우대사항

- Node.js 및 NestJS 기반의 서비스 개발 및 운영 경험이 있으신 분
- AWS 클라우드 환경에서 인프라 설계, 구축 및 운영 경험이 있으신 분
- 대규모 서비스 운영 및 성능 최적화 경험이 있으신 분
- 데이터베이스 성능 분석 및 최적화 경험이 있으신 분
- GraphQL 기반 API 개발 경험이 있으신 분
- Docker, Kubernetes 등 컨테이너 환경 실무 경험이 있으신 분
- MSA 등 확장 용이한 아키텍처 설계 및 구축 경험이 있으신 분
- 기술 블로그, 내부 문서 등 기술 지식을 공유해보신 분
- 코드 품질과 클린 코드에 대해 깊이 이해하고 계신 분
- 서비스 전체의 성과를 최우선으로 생각하고 행동하시는 분
- 비개발자에게도 기술적 내용을 쉽게 설명하실 수 있는 분
- 좋은 함수 및 변수 이름 등 코드 가독성을 위해 고민해보신 분
- 새로운 기술에 대한 지속적인 학습과 프로그래밍에 대한 열정을 가지신 분
- AI를 활용해 생산성을 향상시킨 경험이 있으신 분

우대사항

- 오픈소스에 기여해서 문제를 해결해 본 경험이 있으신 분
- AWS를 포함한 Cloud Native기반 마이크로 서비스 개발 및 운영 경험이 있으신 분
- ElasticSearch / Redis 등의 다양한 데이터베이스 활용 경험이 있으신 분
- AWS Serverless 기능 기반으로 서비스 개발을 해보신 분
- MSA 아키텍쳐 설계 및 개발경험이 있으신 분
- 대용량 트래픽 고려해서 개발을 해보신 분
- Datadog 을 활용해서 모니터링 해보신 분

자격요건

Python 및 Django, Flask, FastAPI framework 기반의 어플리케이션 개발 경험이 있으신 분
Java/Kotlin/Spring/JPA 기반의 어플리케이션 개발 경험이 있으신 분
RESTful API의 설계 규칙을 따르며 개발한 경험이 있으신 분
Git을 이용해 소프트웨어 버전 관리를 하고 PR, 코드 리뷰 등의 과정에 익숙하신 분
백엔드 애플리케이션 및 배치 프로세스 개발 경험이 있으신 분
긍정적이며 자기주도적 성장이 가능하신 분

우대사항

- Celery, RabbitMQ, Kafka 등 분산/비동기 처리에 활용되는 기술에 대한 이해와 경험이 있으신 분
- NoSQL(Redis, MongoDB 등)를 사용한 서비스 개발 경험이 있으신 분
- ElasticSearch 서비스 적용 경험이 있으신 분
- CI/CD 도구를 이용해 빌드/배포한 경험이 있으신 분
- MSA와 컨테이너화 기술(docker, k8s 등)에 대한 이해도가 있으신 분
- 신기술에 대한 지속적인 학습 및 적용이 가능하신 분

> **규모 있는 서비스 기업의 채용 공고에는 왜 이렇게 MSA에 대한 언급이 많은걸까?**

규모가 어느 정도 있는 기업에서는 조직의 효율성을 위해 MSA를 도입하는 경우가 많다. 그러다보니 회사에 이미 구축되어 있는 MSA 구조에서 업무에 참여하려면 MSA에 대한 역량을 갖추고 있어야만 한다. 그래서 채용 공고의 자격 요건과 우대 사항에 MSA에 대한 역량을 기재해놓는 것이다.

따라서 규모 있는 서비스 기업에 취업/이직을 조금 더 수월하게 하고 싶고, 채용 시장에서 차별성을 가지고 싶다면 MSA를 학습해두는 걸 추천한다.
(취업이 잘 안 되는 시기임에도 불구하고 이 강의를 수강하시는 분들만큼은, 채용 시장에서 살아남기를 진심으로 바란다.)

kh3025@gmail.com

📖

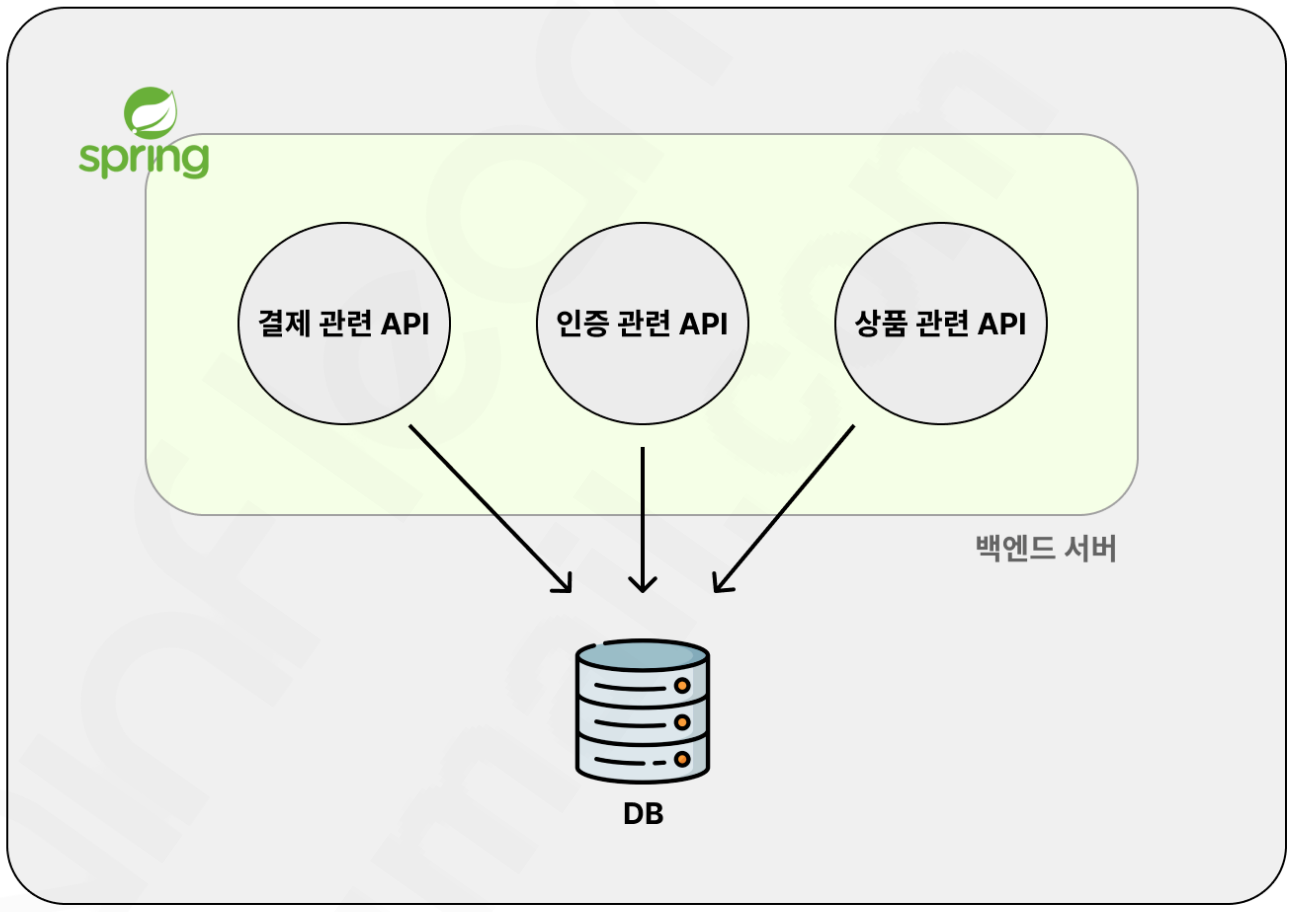
MSA(Microservice Architecture)란?

✅ MSA(Microservice Architecture)란?

MSA의 의미를 검색해보면 아래와 같이 나온다.

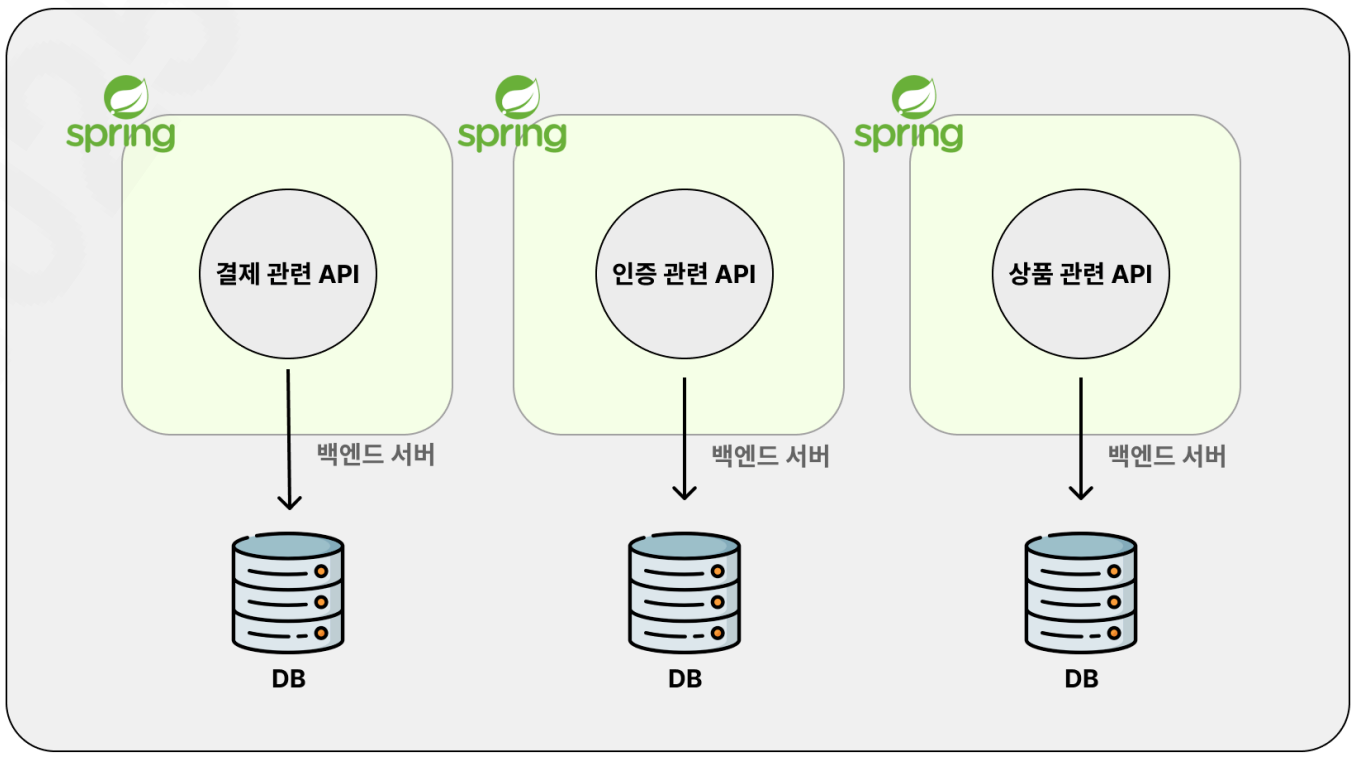
> 하나의 큰 애플리케이션을 **여러 개의 작고 독립적인 서비스로 나누어 개발하고 배포하는 소프트웨어 개발 아키텍처**를 의미한다.

아래 그림을 보면서 조금 더 직관적으로 이해해보자.



모놀리식 아키텍처

일반적으로 백엔드 서버를 구성할 때 활용하는 구조가 **모놀리식 아키텍처**다. **프로젝트 하나**에 모든 API(결제 관련 API, 인증 관련 API, 상품 관련 API)를 전부 구현하는 방식이다.



마이크로서비스 아키텍처

하지만 **마이크로서비스 아키텍처(MSA)**는 서비스에 필요한 API들을 하나의 프로젝트가 아닌 **여러 개의 프로젝트로 나눠서 구현**하는 방식이다. 위 그림을 보면 결제 관련 API들끼리 모아놓은 프로젝트와, 인증 관련 API들끼리 모아놓은 프로젝트와, 상품 관련 API들끼리 모아놓은 프로젝트를 분리해서 구성했다. 그리고 MSA에서는 **독립적으로 분리된 하나의 프로젝트**를 **서비스(service)**라고 부른다.

사실 MSA가 뭔지 궁금해서 조금 찾아본 사람들은, MSA가 대략적으로 어떤 개념인지는 이미 알고 있었을 것이다. 하지만 MSA를 언제, 어떤 상황에서 적용시켜야 하는 지 명확하게 이해하고 있는 사람들은 잘 없다.

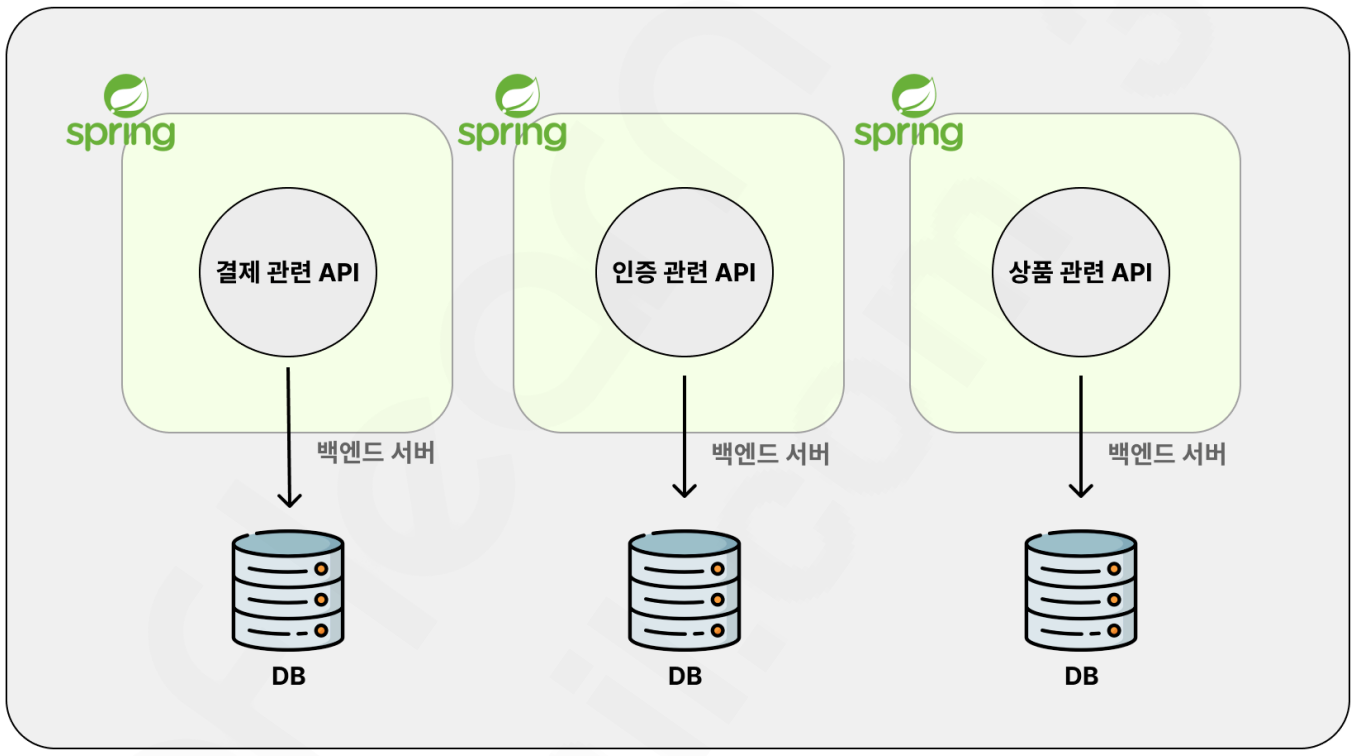
따라서 다음 강의에서는 **MSA가 도대체 어떤 특징과 장점을 가지고 있길래,** 많은 회사에서 MSA를 도입하는 지 알아보자.

@kfc3621@gmail.com
khb302500

MSA의 핵심 특징 / MSA의 독보적인 장점

✅ MSA의 핵심 특징

MSA는 다양한 특징을 가지고 있다. 그 중에서 가장 핵심 특징은 **독립성**이다. 아래 그림을 보면 결제에 관련된 기능, 인증에 관련된 기능, 상품에 관련된 기능을 **독립적으로 분리해서 프로젝트를 구성**했다. 이게 바로 MSA의 방식대로 서비스를 분리해서 구성한 것이다. 이렇게 하나의 애플리케이션을 여러 개의 독립적인 서비스로 분리시킴으로써 다양한 장점이 생기게 된다.



마이크로서비스 아키텍처

✅ MSA의 독보적인 장점

1. **서비스 별 독립적인 배포 → 배포 사이클이 빨라짐**

   개발자가 60명인 회사가 있다고 치자. 하나의 큰 프로젝트를 60명이서 작업하는 것보다, 여러 개의 프로젝트로 나눠서 각 프로젝트를 관리하는 팀을 구성해 독립적으로 작업하는 게 훨씬 효율적이다. 그 이유는 다음과 같다.

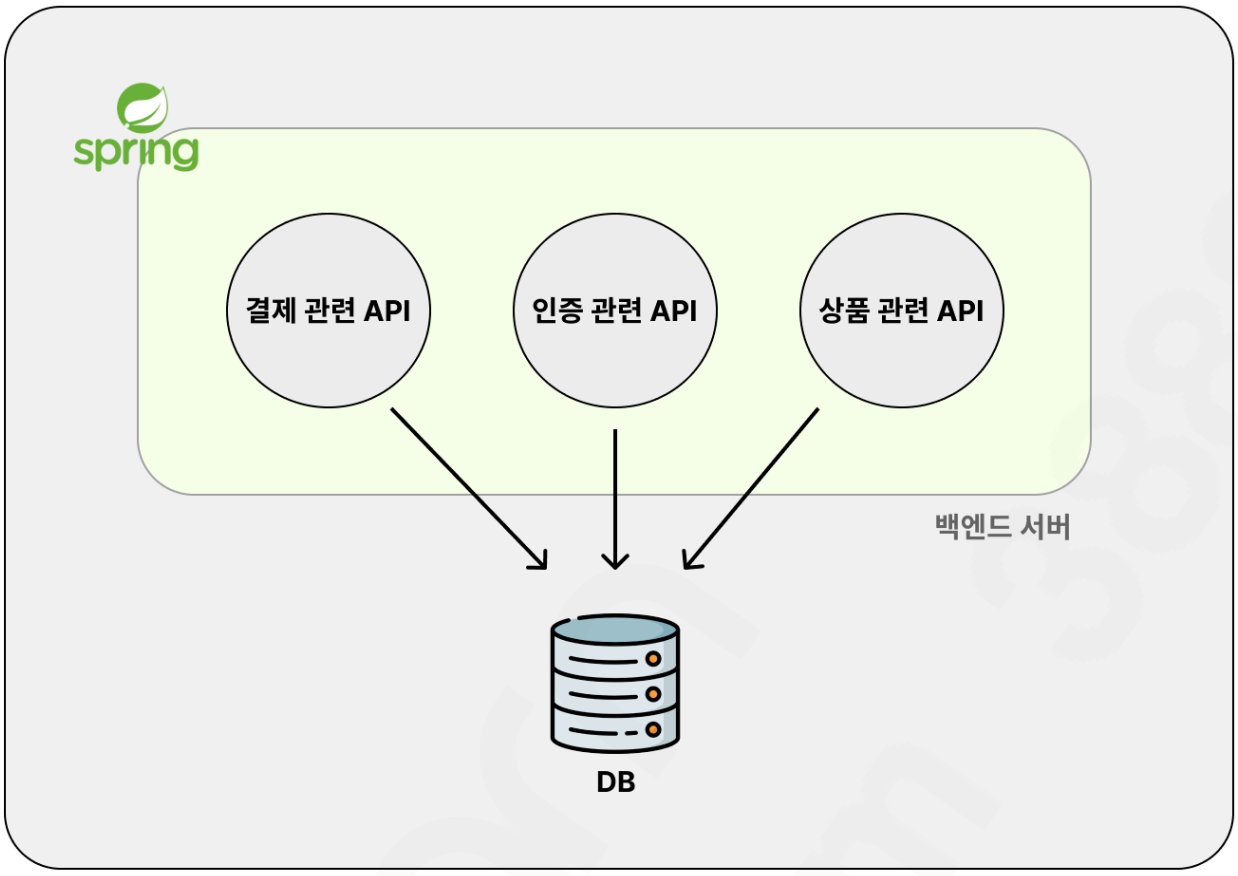
   - 서로 의사소통할 인원이 적기 때문에 프로젝트 진행이 빠르다.
   - 팀이 독립적으로 개발/배포를 진행할 수 있다.
   - 코드의 양이 비교적 작기 때문에 코드 작성, 테스트, 빌드 과정의 시간이 훨씬 적게 걸린다.

   > 실제 우아한형제들과 같이 규모가 있는 회사들의 팀 구성을 보면 배민상품시스템팀, 정산시스템팀 등 다양한 팀으로 나뉘어져있다. 배달의 민족이라는 하나의 서비스를 여러개 프로젝트로 쪼개서 각 팀에 배정해서 관리하고 있다.

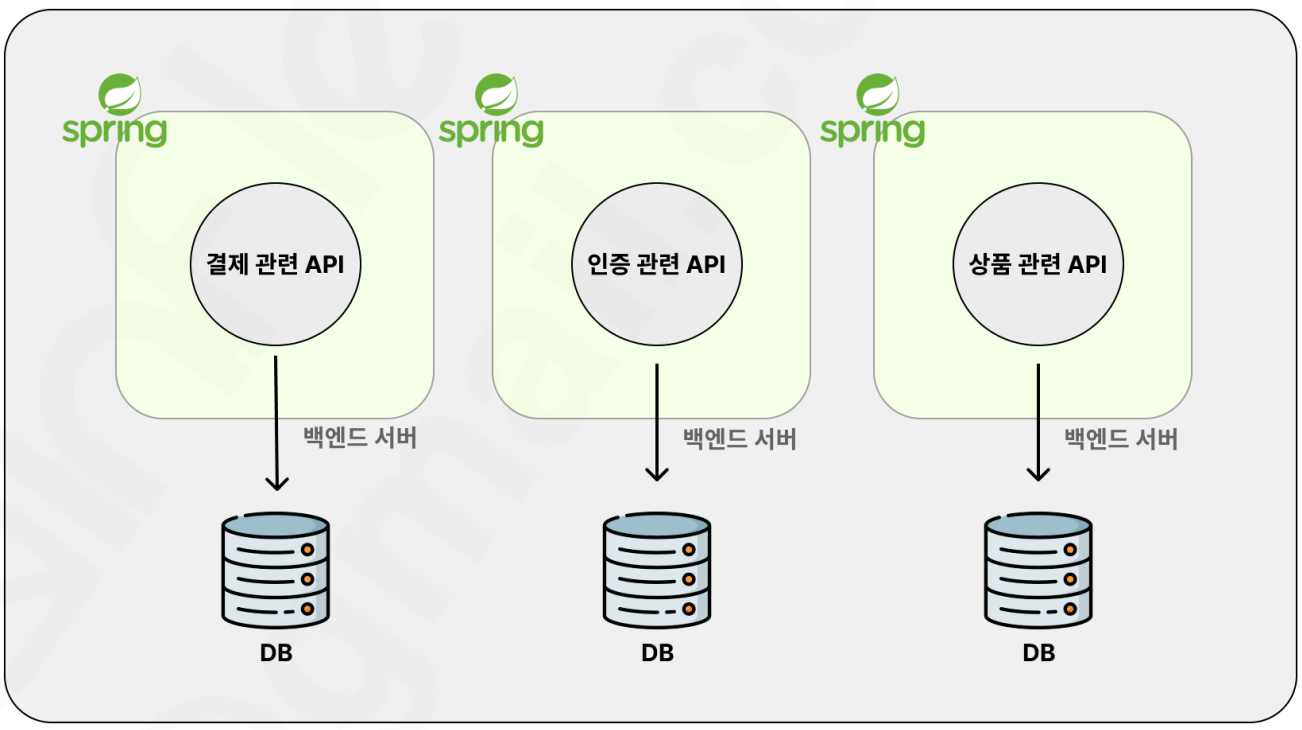
2. **데이터베이스의 독립적인 분리 → 대규모 트래픽 처리 용이**

   모놀리식 아키텍처에서는 단일 데이터베이스를 활용하는 구조를 많이 채택한다. 하지만 AWS의 단일 데이터베이스의 가장 좋은 성능의 데이터베이스로 셋팅을 한다고 하더라도 트래픽이 감당이 안 되는 순간이 온다.

   읽기 전용 데이터베이스를 추가하거나 캐시를 활용해 트래픽을 줄이려는 시도를 하지만, 이런 셋팅을 다 해도 트래픽을 감당 못할 때가 있다. 이런 경우에 MSA 구조를 활용해 각 서비스 별로 데이터베이스를 따로따로 둠으로써 대규모 트래픽을 감당하기도 한다.



모놀리식 아키텍처

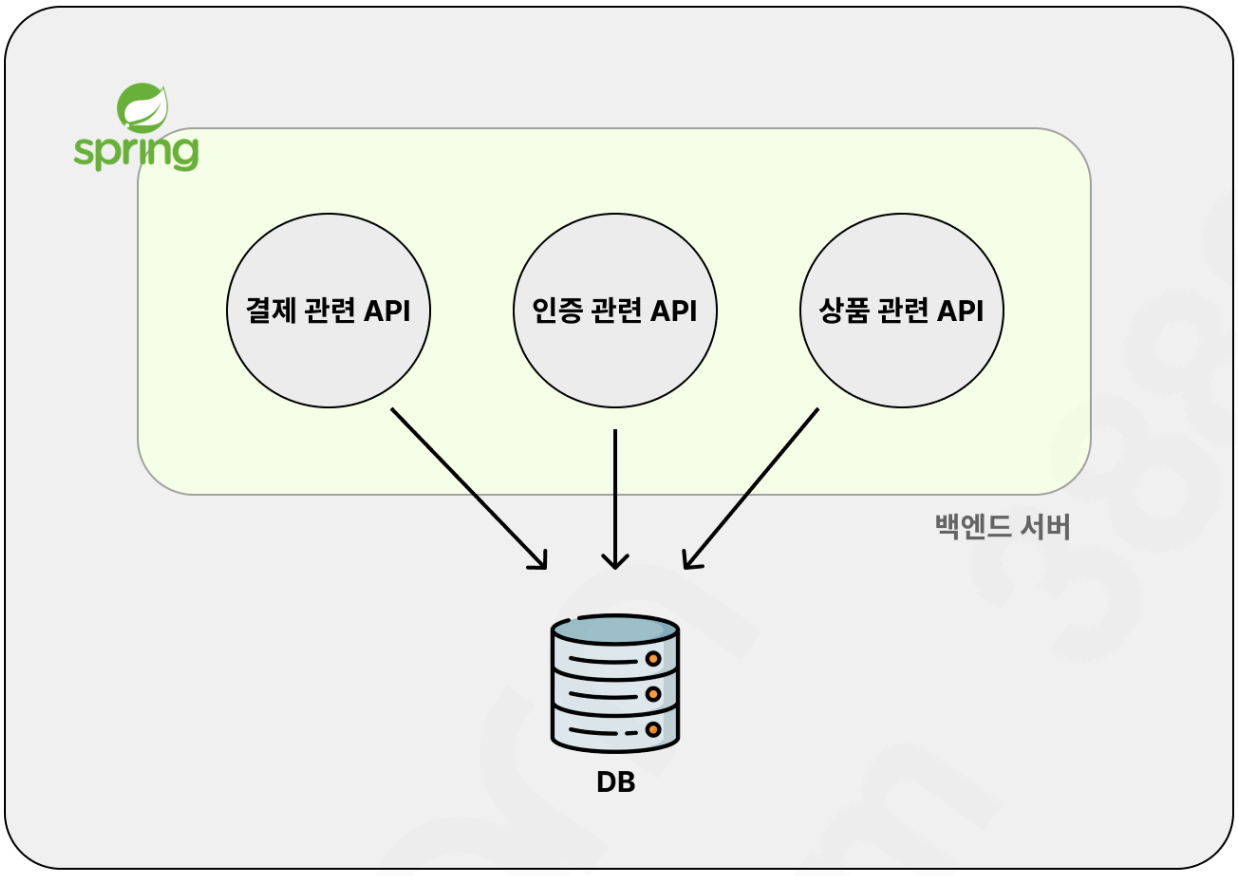


마이크로서비스 아키텍처

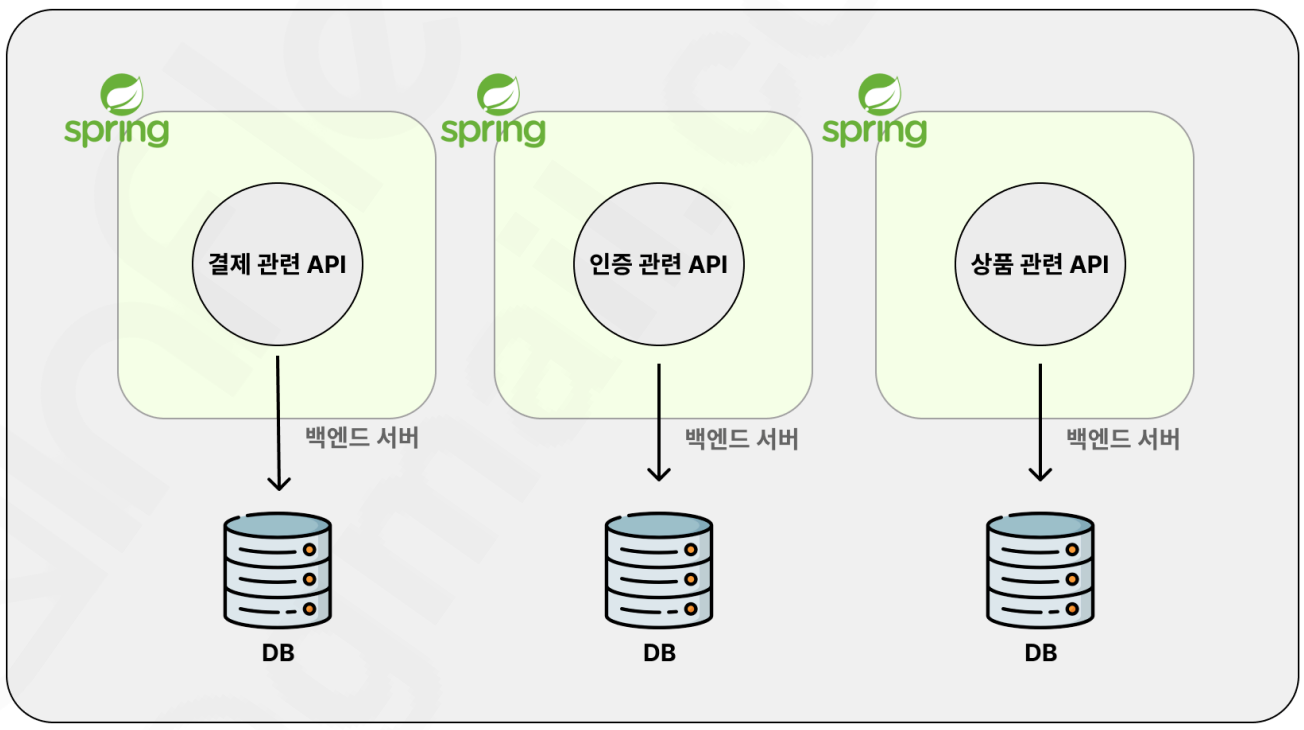
3. **서비스를 독립적으로 분리 → 장애 전파를 최소화**

   모놀리식 서비스에서는 단일 데이터베이스를 사용하고 있기 때문에, 결제 API로 인해 데이터베이스에 장애가 나면 결제와 관련되지 않은 다른 기능들도 100% 같이 장애가 난다.

   반면, MSA 구조에서는 각 서비스 별로 데이터베이스도 독립적으로 분리해서 구축하기 때문에, 결제 서비스가 장애가 나더라도 다른 서비스에는 영향을 덜 끼친다.



모놀리식 아키텍처



마이크로서비스 아키텍처

✅ 정리

MSA의 핵심 키워드를 하나 뽑자면 '**독립성**'이다. 이 '독립성'이라는 특징 때문에 아래의 3가지 장점이 생긴다고 했다.

1. **서비스 별 독립적인 배포 → 배포 사이클이 빨라짐**
2. **데이터베이스의 독립적인 분리 → 대규모 트래픽 처리 용이**
3. **서비스를 독립적으로 분리 → 장애 전파를 최소화**

(MSA는 사실 이 외에도 더 많은 짜잘짜잘한 장점들을 가지고 있다. 처음에는 위 3가지 핵심 장점만 기억해두면 충분하다. 나머지 다른 장점들은 MSA를 구축하다보면 자연스럽게 알게 된다.)

📖

언제 MSA를 도입하는 게 적절할까?

👨🏻‍🏫 MSA를 언제 도입하는 게 적절한 지 파악하기 위해, 기존에 글로벌 기업들이 어떤 상황에서 MSA를 도입했는 지 사례를 몇 가지 살펴보자.

✅ MSA 도입 사례

여러 기업들은 다양한 나름의 이유를 기반으로 MSA를 도입했다. 유명한 글로벌 기업들의 MSA 도입 사례들을 살펴보자.

넷플릭스(Netflix)

2008년 발생한 데이터베이스 손상 사고로 인해 서비스가 중단되는 경험을 한 후, **시스템의 안정성과 탄력성 확보**를 위해 MSA로 전환했다. 넷플릭스는 MSA를 통해 **매우 빠른 배포 속도**를 확보하고, **각각의 기능을 독립적으로 확장**하여 **대규모 트래픽을 안정적으로 처리**할 수 있게 되었다.

아마존 (Amazon)

아마존은 "모든 팀은 2개의 피자만으로 먹을 수 있는 규모여야 한다"는 원칙 아래 MSA를 도입했다. 이는 **소규모 팀이 각자 맡은 서비스를 독립적으로 개발하고 운영하는 것을 목표**로 합니다. 아마존은 MSA를 통해 거대한 전자상거래 플랫폼을 **유연하고 빠르게 확장**할 수 있었으며, **다양한 신규 서비스를 신속하게 배포**해서 시장에 선보일 수 있었다.

우버 (Uber)

우버는 초기 모놀리식 아키텍처가 성장 속도를 따라가지 못하는 문제에 직면했다. **새로운 기능을 추가하거나 수정할 때마다 복잡성이 증가하고 배포 시간이 길어졌습니다.** 우버는 MSA로 전환하여 승객, 운전자, 결제 등 핵심 기능을 독립적인 서비스로 분리했습니다. 이를 통해 **개발 속도를 높이고, 각 서비스의 확장성을 확보**하여 글로벌 서비스를 안정적으로 제공할 수 있게 되었습니다.

위 기업들의 사례를 보면 크게 3가지 이유 때문에 MSA로 전환을 했다.

1. 신속한 배포
2. 대규모 트래픽를 안정적으로 처리
3. 시스템 안정성 확보

위 3가지 이유는 우리가 이전에 배웠던 MSA의 핵심적인 장점 3가지랑 완전히 일치한다.

1. **서비스별 독립적인 배포 → 배포 사이클이 빨라짐**
2. **데이터베이스의 독립적인 분리 → 대규모 트래픽 처리 용이**
3. **서비스를 독립적으로 분리 → 장애 전파를 최소화**

> 이런 사례를 기반으로 언제 MSA로 전환하는 게 좋은 지 대략적으로 유추할 수 있다. MSA로 언제 전환하는 게 좋은 지 알아보자.

✅ MSA로 언제 전환하는 게 좋을까?

모놀리식 아키텍처로 프로젝트를 관리하고 운영하면서 다음과 같은 문제를 만날 때 MSA 전환을 고민하면 된다.

1. **프로젝트 규모가 크거나 또는 개발자 인원이 너무 많아서 배포 사이클이 느려진 경우**
2. **모놀리식 아키텍처에서는 도저히 감당할 수 없는 트래픽이 발생하는 경우**
3. 모놀리식 아키텍처로 인한 **장애 전파의 횟수가 잦거나**, **한 번의 장애 전파도 서비스에 치명적인 경우**

위 3가지 이유 중 하나라도 해당한다면 MSA 전환을 고려해보면 된다.

하지만 MSA로 전환해보면 알겠지만 모놀리식 구조보다 훨씬 복잡하고 전환하는 데 많은 시간과 비용이 들어간다. 따라서 **MSA로 전환했을 때 얻는 이득이 더 큰 지를 신중하게 따져보고 전환을 시도해야 한다.**

@gmail.com
khb302

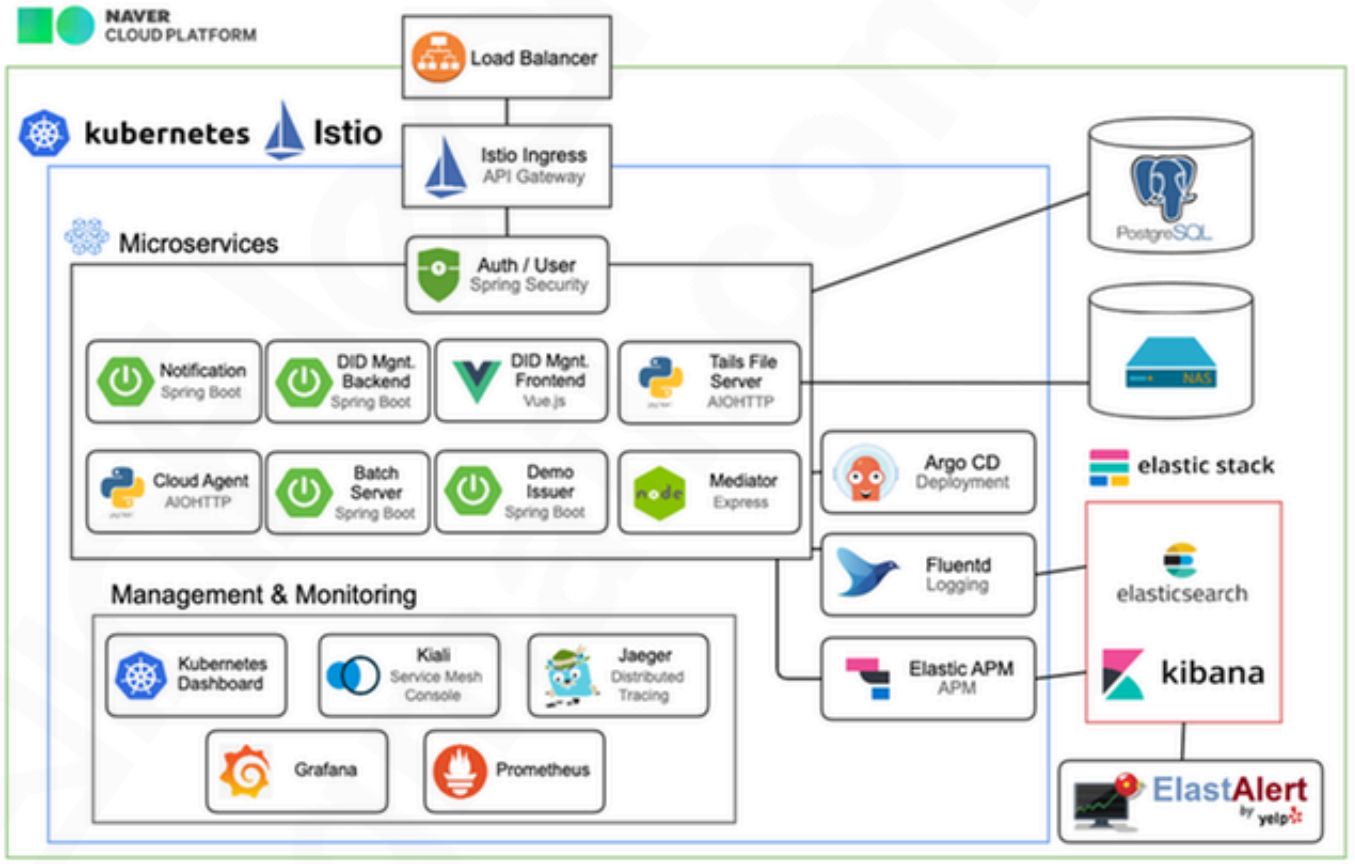
📖

많은 사람들이 MSA가 어렵다고 착각하는 이유

✅ 많은 사람들이 MSA가 어렵다고 착각하는 이유

많은 분들이 MSA를 잘못 오해하고 있는 경우가 너무 많다. 가장 대표적으로 오해하는 게 **MSA 구조로 구축하려면 거창하고 복잡한 무언가를 도입해야 한다고 잘못 알고 있다.** 구체적인 Case를 살펴보자.

1. MSA를 구축하려면 **카프카, 쿠버네티스, API 게이트웨이, 서비스 디스커버리, 프로메테우스, 그라파나, Jenkins**과 같은 다양한 기술 스택을 무조건 도입해야 한다고 착각한다.



2. MSA를 구축하려면 **서킷 브레이커 패턴, 사가 패턴, CQRS, 이벤트 소싱, DDD, EDA, BFF, 클린 아키텍처, 헥사고날 아키텍처**와 같은 다양한 패턴을 전부 알고 적용시켜야 한다고 착각한다.
3. MSA를 구축하려면 **Spring Cloud Bus, Eureka, Spring Cloud Gateway, Spring Cloud Sleuth**와 같은 다양한 라이브러리를 무조건 도입해야 한다고 착각한다.

👨🏻‍🏫 이런 착각들 때문에 MSA를 학습하는 데는 시간이 엄청 오래 걸릴 것 같이 느껴지고, '저걸 언제 다 학습하지'라는 생각이 들면서 막막하게만 느껴지는 것이다.

하지만 이 강의에서는 정말 MSA에서 핵심적인 개념들과 실무에서 자주 쓰이는 개념들 위주로만 알려줄 예정이다. 이 강의를 다 들을 때쯤에는 'MSA가 생각보다 별거 없네'라고 느낄 것이다.

📖

MSA인지 아닌지를 판단하는 기준

✅ MSA인지 아닌지를 판단하는 기준

MSA인지 아닌지를 판단하는 '명확한' 기준이라는 게 없다. 왜냐하면 MSA라는 개념이 어떤 개인이나 단체가 정의한 개념이 아니다. 그냥 서로 입소문으로 퍼진 개념 중 하나이다. 한 마디로 **'특정 기준을 만족시켜야 MSA다'라는 명확한 기준이 없다는 뜻**이다. 그래서 개개인마다, 기업마다 얘기하는 MSA의 기준이 다를 수 밖에 없다.

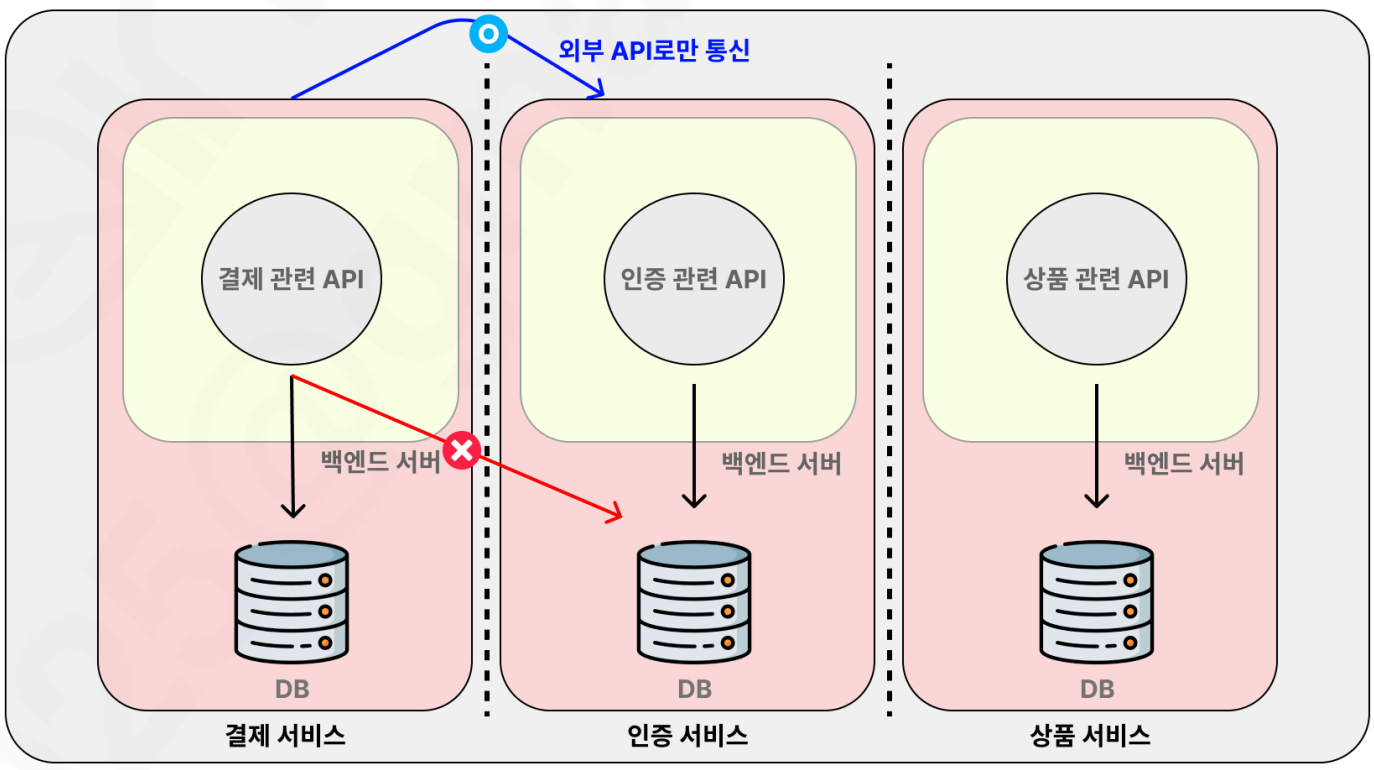
하지만 MSA라는 개념이 개발자들 사이에서 유명해지면서, MSA를 판단하는 기준이 얼추 생기긴 했다. 그 기준을 알려주겠다.

> MSA인지 아닌지를 판단할 때 핵심 개념은 **'독립성'**이다.

많은 기업들이 MSA로 전환하는 이유는 **독립성**으로 인해 생기는 장점들 때문이다. 따라서 **독립성**을 적용시켰냐 여부가 MSA를 판단하는 데 중요한 기준점이 된다.

'독립성'이라는 키워드가 추상적이니 조금 더 구체적으로 얘기해보자.

✅ MSA에서 얘기하는 '독립성'



마이크로서비스 아키텍처

1. **각 서비스를 독립적으로 개발 및 배포가 가능해야 한다.**

   각 서비스를 독립적으로 개발 및 배포가 가능해야만, MSA의 핵심적인 장점('배포 사이클이 빠르다')을 취할 수 있게 된다.

2. **서비스끼리는 API로만 통신해야 한다.**

   예를 들어, 다른 서비스의 DB에 직접적으로 접근해서 데이터를 가져오면 안 된다. 그래야 API를 제외한 다른 부분에서 의존성이 안 생겨서 독립적인 개발과 배포가 원활하게 가능해진다. 그리고 DB를 별도로 분리해서 써야 DB 장애로 인한 장애 전파를 예방할 수 있다.

✅ MSA와는 상관없는 구분 기준

1. **카프카, 쿠버네티스, API 게이트웨이, 서비스 디스커버리, 프로메테우스, 그라파나, Jenkins**와 같은 기술 스택을 썼는 지 여부는 MSA를 잘 적용시켰는 지 판단하는 데 전혀 상관없는 요소이다.
2. **서킷 브레이커 패턴, 사가 패턴, CQRS, 이벤트 소싱, DDD, EDA, BFF, 클린 아키텍처, 헥사고날 아키텍처**와 같은 패턴을 썼는 지 여부는 MSA를 잘 적용시켰는 지 판단하는 데 전혀 상관없는 요소이다.
3. **Spring Cloud Bus, Eureka, Spirng Cloud Gateway, Spirng Cloud Gateway, Spring Cloud Sleuth**와 같은 다양한 라이브러리를 썼는 지 여부는 MSA를 잘 적용시켰는 지 판단하는 데 전혀 상관없는 요소이다.
4. **서비스를 몇 개로 나눴는 지**는 MSA를 잘 적용시켰는 지 판단하는 데 전혀 상관없는 요소이다. 독립성을 지키면서 2개의 서비스로 나눈 것도 MSA라고 부른다.

MSA에 대한 잘못된 개념을 바로 잡지 않으면 MSA를 학습하는 데 방해가 될 수 있어서, MSA에 대해 오해할만한 부분을 정리해봤다.

@kh3025.com

📖

MSA를 쉽고 빠르게 배우는 방법

✅ MSA를 쉽고 빠르게 배우는 방법

> MSA를 쉽고 빠르게 배우려면 딱 3가지 법칙을 지키면 된다.

1. **원씽(One Thing) 법칙 : 낯선 개념들을 한꺼번에 다 적용시키려는 욕심을 버려야 한다.**

   MSA 프로젝트를 구성하기 위해 처음부터 쿠버네티스를 활용한 구성, 사가 패턴, API Gateway 등 다양한 개념들을 한 번에 적용시면서 프로젝트를 하려고 한다. 한 마디로 레벨에 맞지 않게 프로젝트 목표를 잡는 게 문제다.

   제일 빠르게 MSA를 학습하는 방법은 '한 번에 딱 하나의 개념만 익히겠다'라는 방식으로 접근해야 한다. 그렇게 아주 간단한 프로젝트에서 하나씩 하나씩 살을 덧붙여간다는 느낌으로 학습을 해야 한다. 사실 이 공부 방법은 모든 기술을 학습할 때 다 통용되는 방법이다.

   이 강의는 MSA에서의 수많은 개념을 하나씩 하나씩 다룸으로써, 비전공자도 충분히 따라올 수 있게 구성했다. 이 강의를 완강할 때쯤 되면 MSA에 있는 많은 개념들을 한꺼번에 적용시켜서 프로젝트를 구성할 수 있게 될 것이다.

2. **플레이그라운드(Playground) 법칙 : 연습할 수 있는 환경을 만들어야 한다.**

   어떤 개념을 빠르게 습득하려면 그 개념을 자주 쓸 수 있는 환경을 만들어야 한다. 가장 좋은 방법 중 하나는 개인 프로젝트를 할 때 MSA 방식으로 구성해서 서비스를 개발하고 운영해보는 것이다. 아주 작은 규모의 프로젝트이더라도 MSA로 구성해서 서비스를 개발하고 배포해서 운영하는 경험을 해보면 MSA를 정말 빠르게 이해할 수 있게 된다.

3. **스몰 스타트(Small Start) 법칙 : 최대한 작은 규모의 프로젝트에서 연습해야 한다.**

   많은 분들이 잘못 생각하고 있는 것 중 하나가 프로젝트는 규모가 커야된다고 생각하는 점이다. 내가 새로운 개념을 처음 학습할 때는 무조건 가장 심플한 구조의 프로젝트에서 적용을 시켜봐야 한다. 괜히 욕심부린답시고 복잡한 도메인에 구현해야 하는 API 개수도 많은 프로젝트에서 MSA를 적용시키려고 하면 막막하게만 느껴질 것이다.

   따라서 정말 간단한 도메인이면서도 정말 딱 필요한 최소한의 API 개수로만 이루어진 프로젝트에서 MSA 개념을 하나씩 적용시켜보는 걸 추천한다. 이 강의도 최소한의 기능만 구현되어 있는 프로젝트에서 MSA 개념을 적용시키는 방식으로 진행할 예정이다.

위 3가지 법칙을 활용해 본격적으로 MSA 개념을 하나씩 배워보자.

khb3025

📖

MSA 프로젝트 요구사항 및 설계

👨🏻‍🏫 **스몰 스타트 법칙(최대한 작은 규모의 프로젝트에서 연습)**에 따라, 최소한의 기능을 가진 프로젝트에서 MSA 방식으로 구축하는 연습을 해보자.

✅ 구현할 기능

1. **회원가입 API**

   고려 사항

   - 간단한 프로젝트 구조를 위해 비밀번호는 암호화하지 않고 그대로 저장한다.

   Request

   ```
   {
     "email": "jscode@naver.com",
     "name": "박재성",
     "password": "abc"
   }
   ```

   Response

   ```
   (204 No Contnet로 응답하기)
   ```

2. **게시글 작성 API**

   고려 사항

   - 간단한 프로젝트 구조를 위해 게시글 작성을 할 때, JWT 방식이 아닌 `userId`를 직접 입력받는 형태로 구성한다.

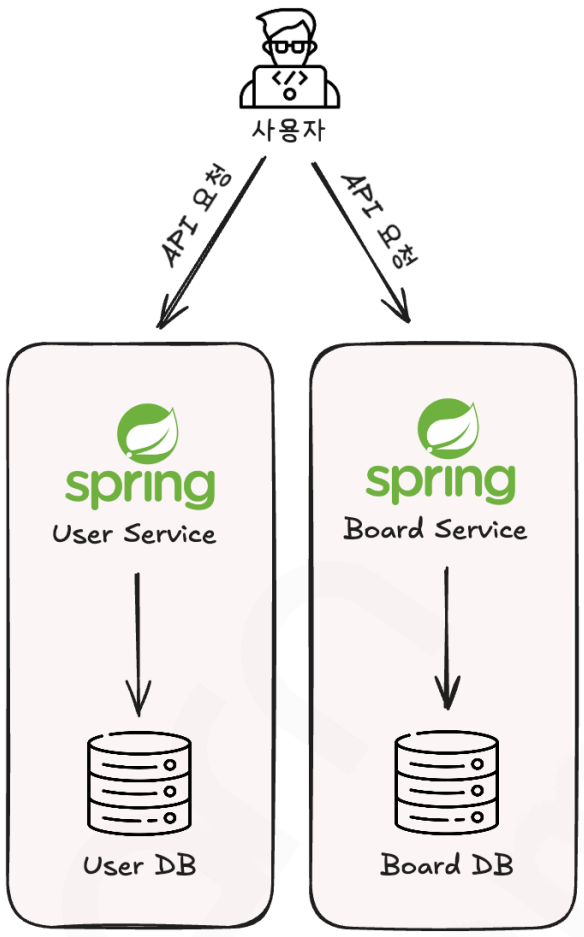
   Request

   ```
   {
     "title": "제목",
     "content": "내용"
     "userId": 1
   }
   ```

   Response

   ```
   (204 No Content로 응답하기)
   ```

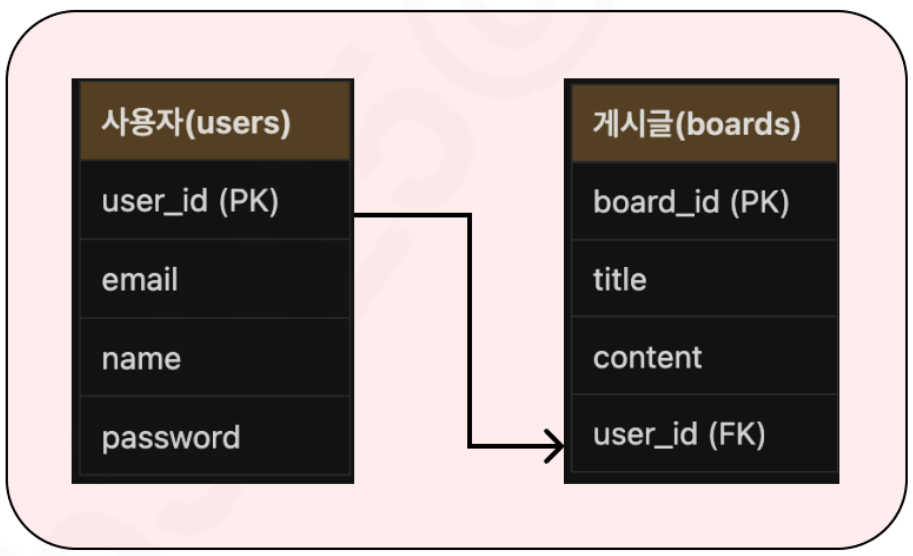
✅ 프로젝트 아키텍처



- User Service에 회원가입 API를 구현하고, Board Service에 게시글 작성 API를 구현할 것이다.
- 마이크로서비스마다 각각 다른 프레임워크(ex. Nest.js, Django)를 써도 된다. 하지만 프로젝트의 심플함을 위해 User Service와 Board Service 둘 다 Spring Boot로 구현할 것이다.
- 마이크로서비스마다 각각 다른 DB(ex. MongoDB, PostgreSQL 등)를 써도 된다. 하지만 프로젝트의 심플함을 위해 User Service와 Board Service 둘 다 MySQL로 구현할 것이다.
- DB 서버를 간편하게 띄우기 위해 Docker를 사용할 것이다.
- 실무에서 쓸 MSA를 구축하려면 각 Service와 DB를 전부 독립적인 서버에 실행시켜야 한다. 하지만 프로젝트의 심플함을 위해 로컬 컴퓨터 한 대에서 포트 번호만 다르게 해서 실행시킬 것이다.

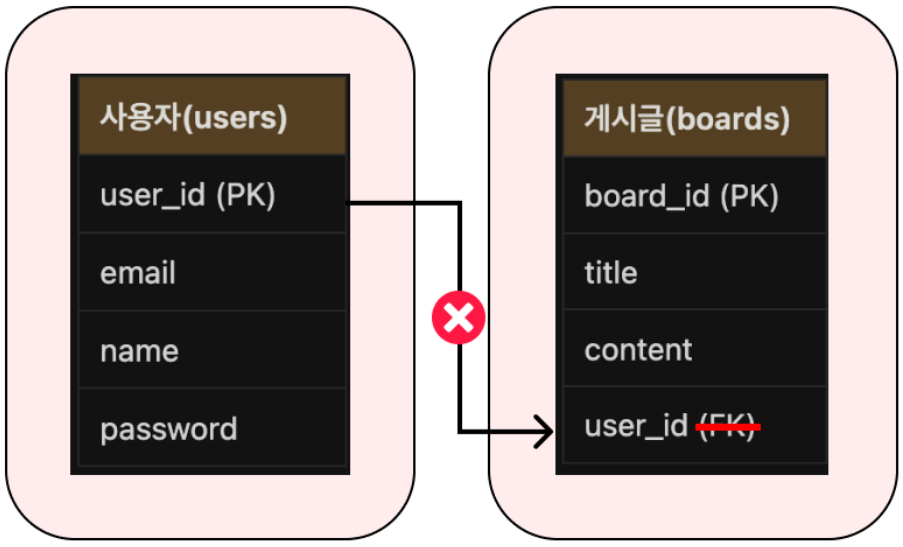
✅ DB 설계

[모놀리식 아키텍처]



모놀리식 아키텍처에서는 위 그림과 같이 `user_id`를 참조한 채로 DB를 설계하게 된다.

[마이크로서비스 아키텍처]



하지만 마이크로서비스에서는 DB가 독립적으로 분리되어 있기 때문에 `user_id`를 참조할 수 없다. 하지만 특정 게시글이 어떤 사용자의 게시글인지 파악하기 위해 `user_id`에 대한 정보는 있어야 한다. 따라서 `boards` 테이블에서 `user_id`는 FK를 설정하지 않은 채로 컬럼을 생성하면 된다.

> MSA에서는 DB를 독립적으로 분리시켜야하기 때문에 FK를 설정하지 않는 경우가 많다.

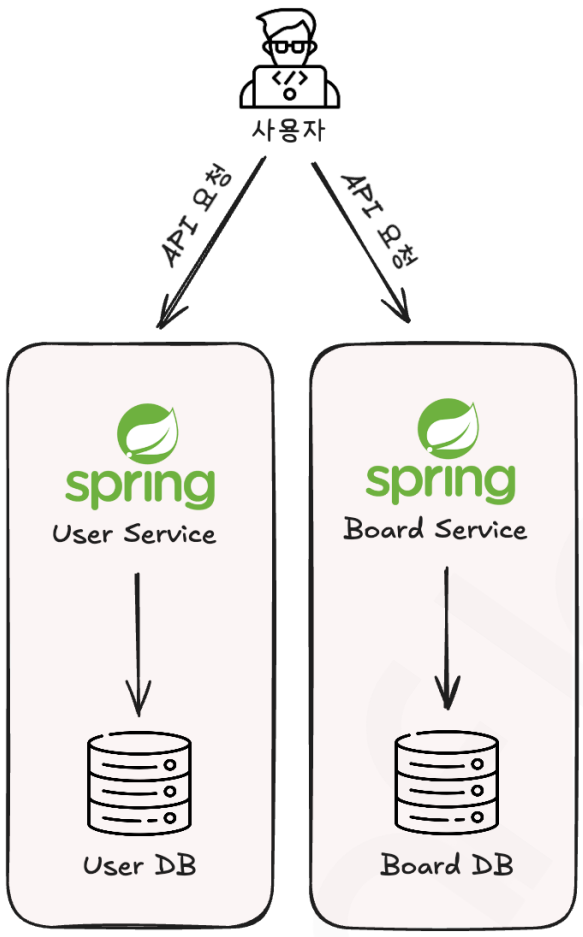
그럼 다음 강의에서 요구사항과 설계에 맞춰서 프로젝트를 구성해보자.

kb302571@gmail.com

[실습] 각 Microservice에 해당하는 DB 서버 띄우기

✅ 각 Microservice에 해당하는 DB 서버 띄우기

아래 아키텍처 구조에 맞게 Docker로 MySQL 서버 2대를 띄워주자.



1. **MySQL 서버 2대 띄우기**

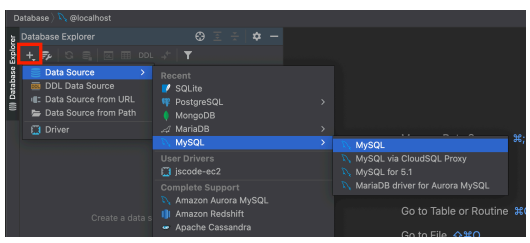
```
# MySQL 서버 2대 띄우기 (3306번 포트, 3307번 포트)
$ docker run -e MYSQL_ROOT_PASSWORD=password -p 3306:3306 -d mysql
$ docker run -e MYSQL_ROOT_PASSWORD=password -p 3307:3306 -d mysql

# 잘 띄워졌는 지 확인하기
$ docker ps
```

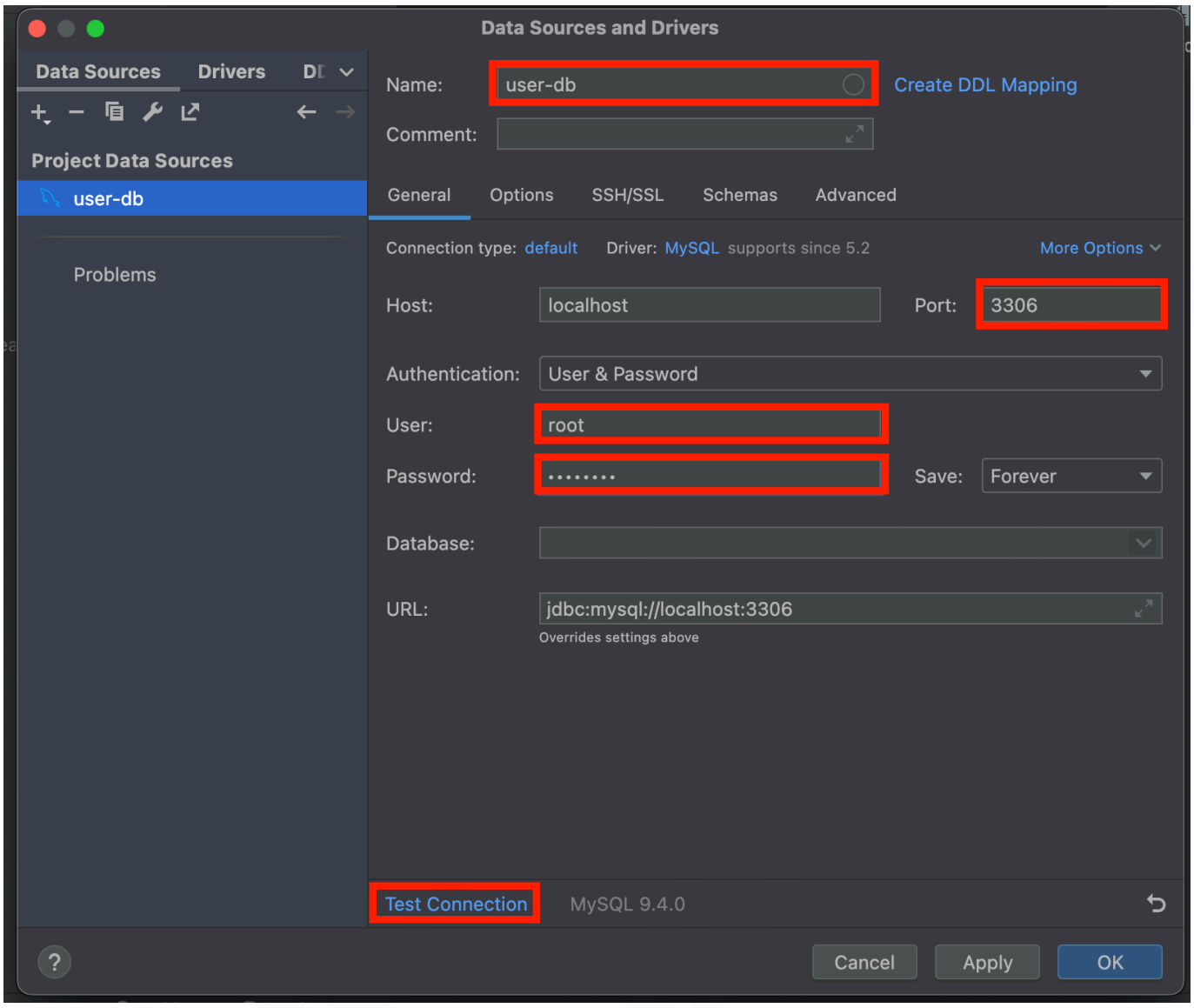
```
jaeseong ~ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                                                NAMES
eee2e8105772   mysql     "docker-entrypoint.s…"   2 minutes ago   Up 2 minutes   0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp   elegant_ganguly
bc18fff95925   mysql     "docker-entrypoint.s…"   2 minutes ago   Up 2 minutes   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp   romantic_williamson
```

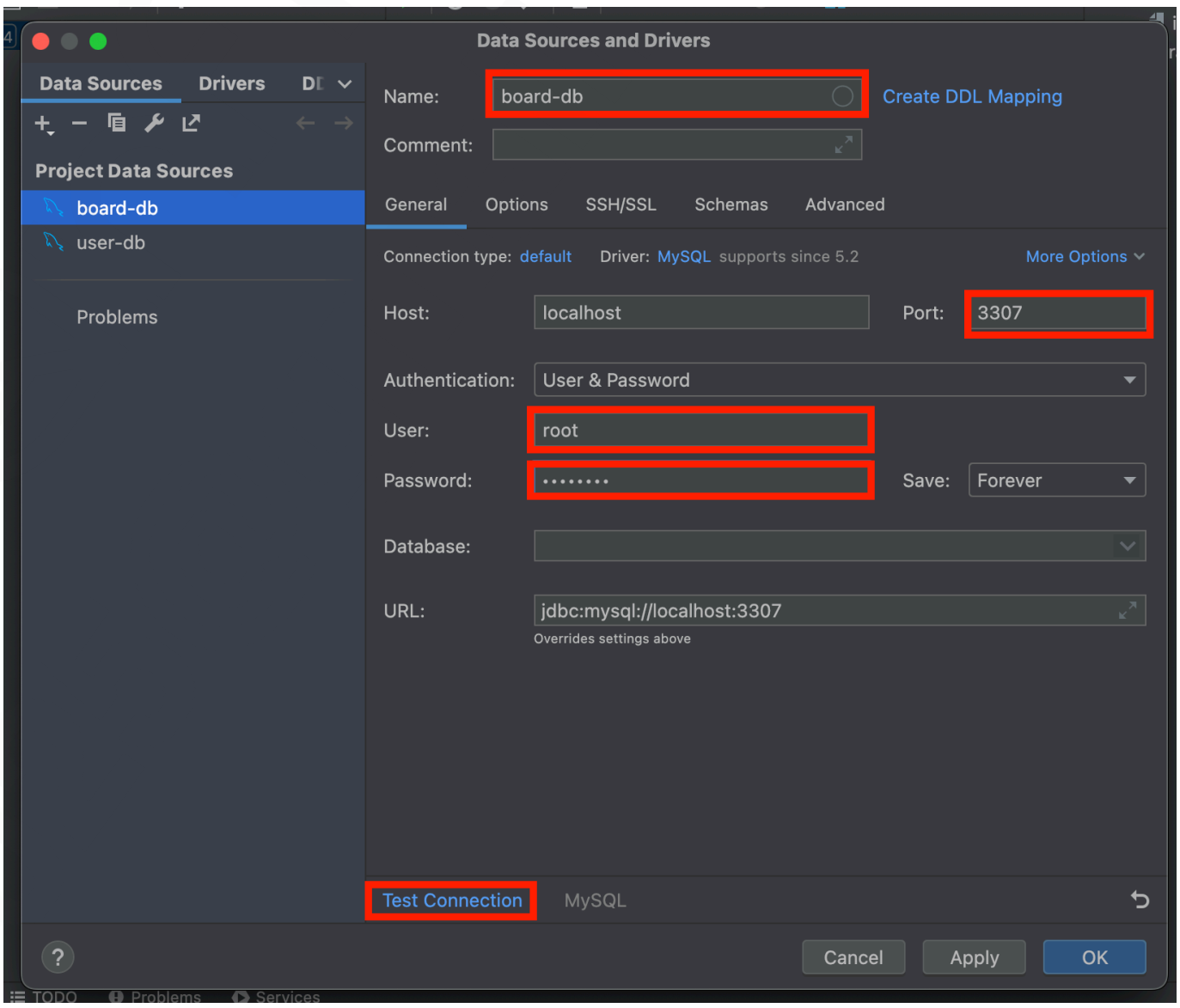
- **참고)** 개발 및 테스트 용으로 MySQL을 띄우는 것이기 때문에 일부러 볼륨(volume) 설정은 하지 않았다.

2. **GUI 툴로 MySQL 접근되는 지 확인하기**

> 이 강의에서는 GUI 툴로 Datagrip을 사용한다. 하지만 Datagrip이 아닌 다른 툴을 사용해도 상관없다.

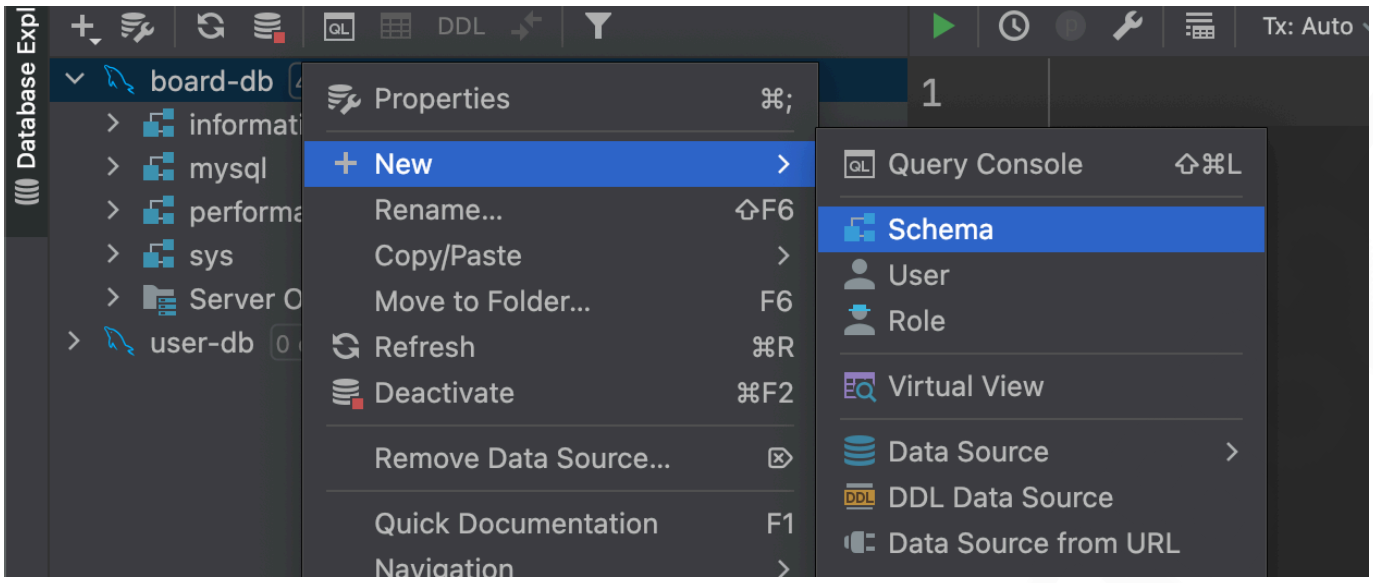


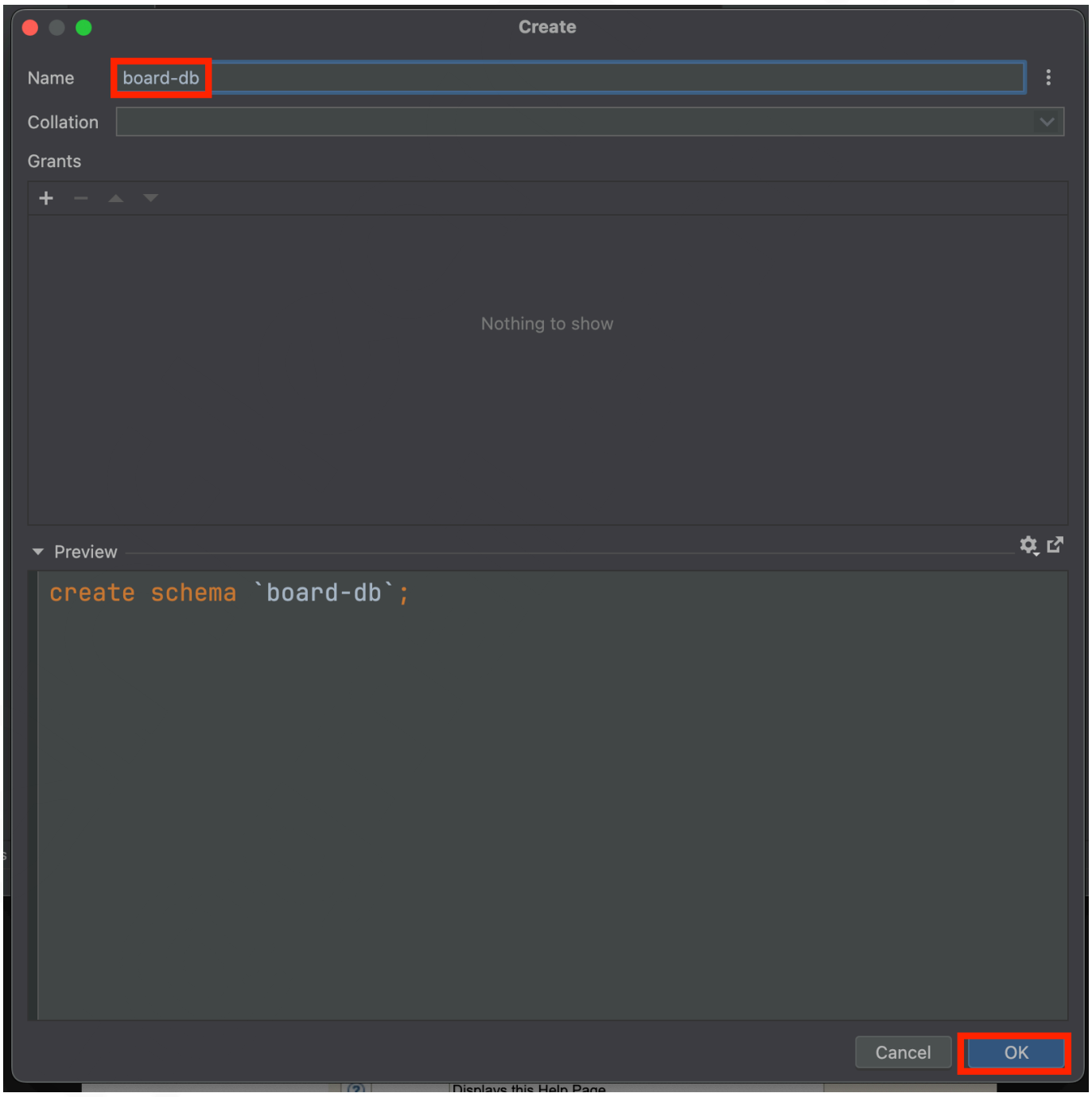




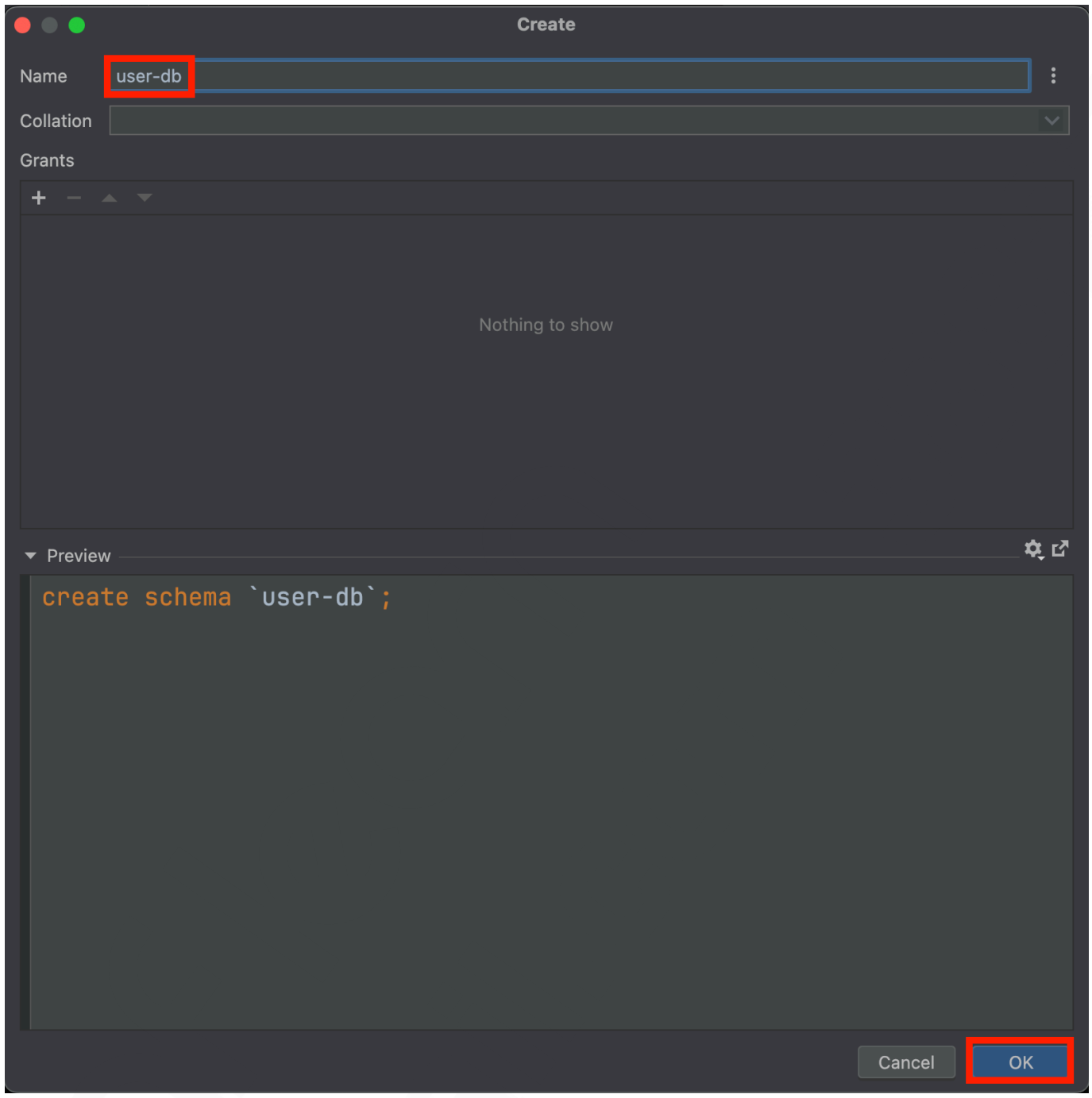
3. **데이터베이스 생성하기**

   user DB, board DB 각각에 데이터베이스를 만들어주자.


Database Expl
DDL
Tx: Auto
board-db
informat
mysql
performa
sys
Server C
user-db
1
Properties ⌘;
New
Rename... ⇧F6
Copy/Paste
Move to Folder... F6
Refresh ⌘R
Deactivate ⌘F2
Remove Data Source...
Quick Documentation F1
Navigation
Query Console ⇧⌘L
Schema
User
Role
Virtual View
Data Source
DDL Data Source
Data Source from URL


Create
Name
board-db
Collation
Grants
Nothing to show
Preview
create schema `board-db`;
Cancel
OK

khb3023

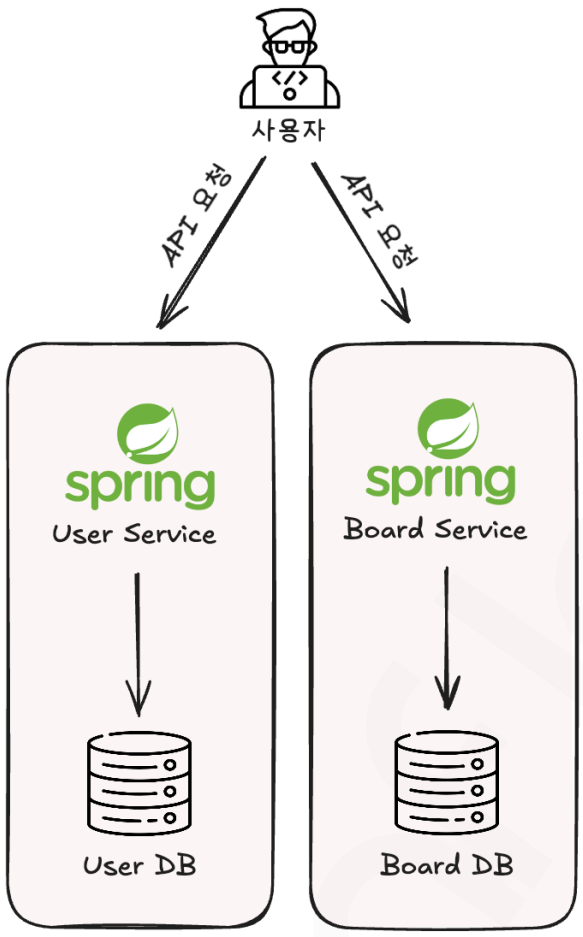


khb3025@gmail.com

[실습] User Microservice 만들기

✅ User Microservice 만들기

DB까지는 셋팅을 했으니 아래 구조에 맞게 User Service를 만들어보자.

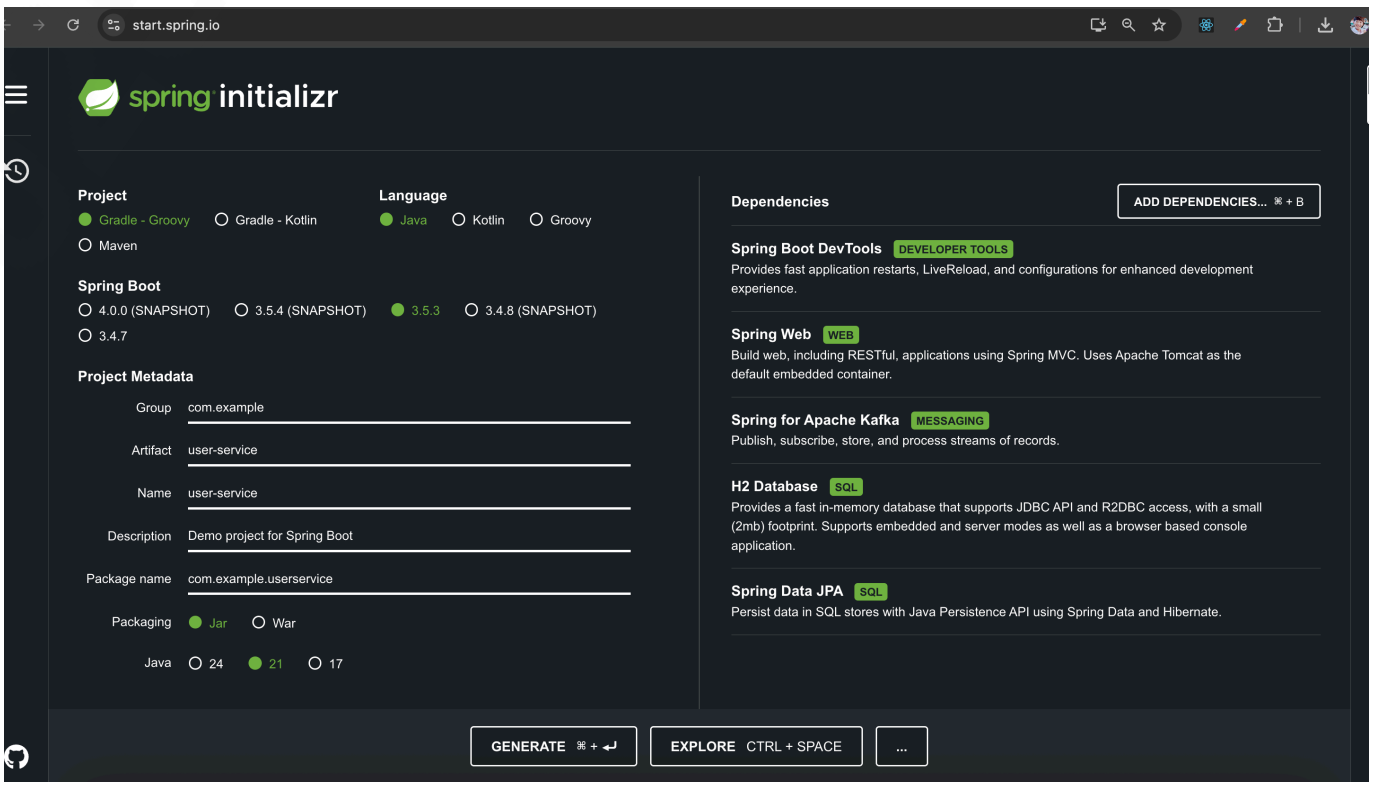


✅ Spring Boot 프로젝트 셋팅 & DB 연결

1. **Spring Boot 프로젝트 셋팅**

start.spring.io

https://start.spring.io/



- `Artifact`와 `Name`을 `user-service`라고 지어주자.
- `Package name`을 Java 컨벤션에 맞게 `userservice`라고 지어주자.
- Java 21 버전을 선택하자. 아래 과정을 Java 21 버전을 기준으로 진행할 예정이다.
- Dependencies는 `Spring Boot DevTools`, `Spring Web`, `MySQL Driver`, `Spring Data JPA`를 선택하자.

2. **application.yml 작성하기**

> 이 프로젝트에서는 `application.properties`를 지우고 `application.yml`을 생성했다.

```
server:
  port: 8080

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/user-db # DB 주소
    username: root # DB 계정
    password: password # DB 비밀번호
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

3. **Spring Boot 서버 실행시켜서 DB 연결 제대로 되는 지 확인하기**

Spring Boot 실행시켰을 때 아래와 같은 에러 안 뜨고 서버가 실행되면, 정상적으로 DB가 잘 연결된 것이다.

```
org.springframework.beans.factory.BeanCreationException Create breakpoint : Error creating bean with name
 'entityManagerFactory' defined in class path resource
 [org/springframework/boot/autoconfigure/orm/jpa/HibernateJpaConfiguration.class]: Unable to create
 requested service [org.hibernate.engine.jdbc.env.spi.JdbcEnvironment] due to: Unable to determine
 Dialect without JDBC metadata (please set 'jakarta.persistence.jdbc.url' for common cases or 'hibernate
 .dialect' when a custom Dialect implementation must be provided)
  at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.initializeBean
   (AbstractAutowireCapableBeanFactory.java:1826) ~[spring-beans-6.2.10.jar:6.2.10]
  at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.doCreateBean
   (AbstractAutowireCapableBeanFactory.java:607) ~[spring-beans-6.2.10.jar:6.2.10]
  at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.createBean
   (AbstractAutowireCapableBeanFactory.java:529) ~[spring-beans-6.2.10.jar:6.2.10]
  at org.springframework.beans.factory.support.AbstractBeanFactory.lambda$doGetBean$0(AbstractBeanFactory
   .java:339) ~[spring-beans-6.2.10.jar:6.2.10]
  at org.springframework.beans.factory.support.DefaultSingletonBeanRegistry.getSingleton
```

✅ 회원가입 API 만들기

1. **User 엔티티 생성하기**

domain/User

```
@Entity
@Table(name = "users")
public class User {

  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long userId;
```

```
private String email;

private String name;

private String password;

public User() {
}

public User(String email, String name, String password) {
  this.email = email;
  this.name = name;
  this.password = password;
}

public Long getUserId() {
  return userId;
}

  public String getEmail() {
    return email;
  }

  public String getName() {
    return name;
  }

  public String getPassword() {
    return password;
  }
}
```

2. **Repository 만들기**

   domain/UserRepository

```
public interface UserRepository extends JpaRepository<User, Long> {
}
```

3. **Controller 만들기**

   controller/UserController

```
@RestController
@RequestMapping("/users")
public class UserController {
  private final UserService userService;

  public UserController(UserService userService) {
    this.userService = userService;
  }

  @PostMapping("sign-up")
  public ResponseEntity<Void> signUp(
      @RequestBody SignUpRequestDto signUpRequestDto
  ) {
```

```
    userService.signUp(signUpRequestDto);
    return ResponseEntity.noContent().build();
  }
}
```

4. **DTO 만들기**

   dto/SignUpRequestDto

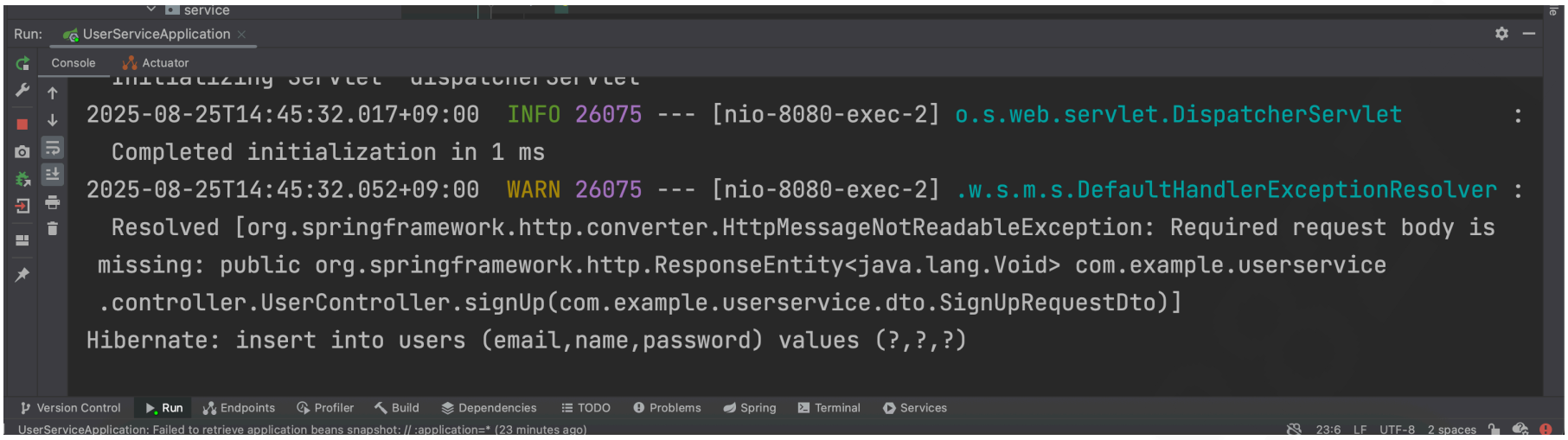
```
public class SignUpRequestDto {
  private String email;
  private String name;
  private String password;

  public String getEmail() {
    return email;
  }

  public String getName() {
    return name;
  }

  public String getPassword() {
    return password;
  }
}
```

5. **Service 만들기**

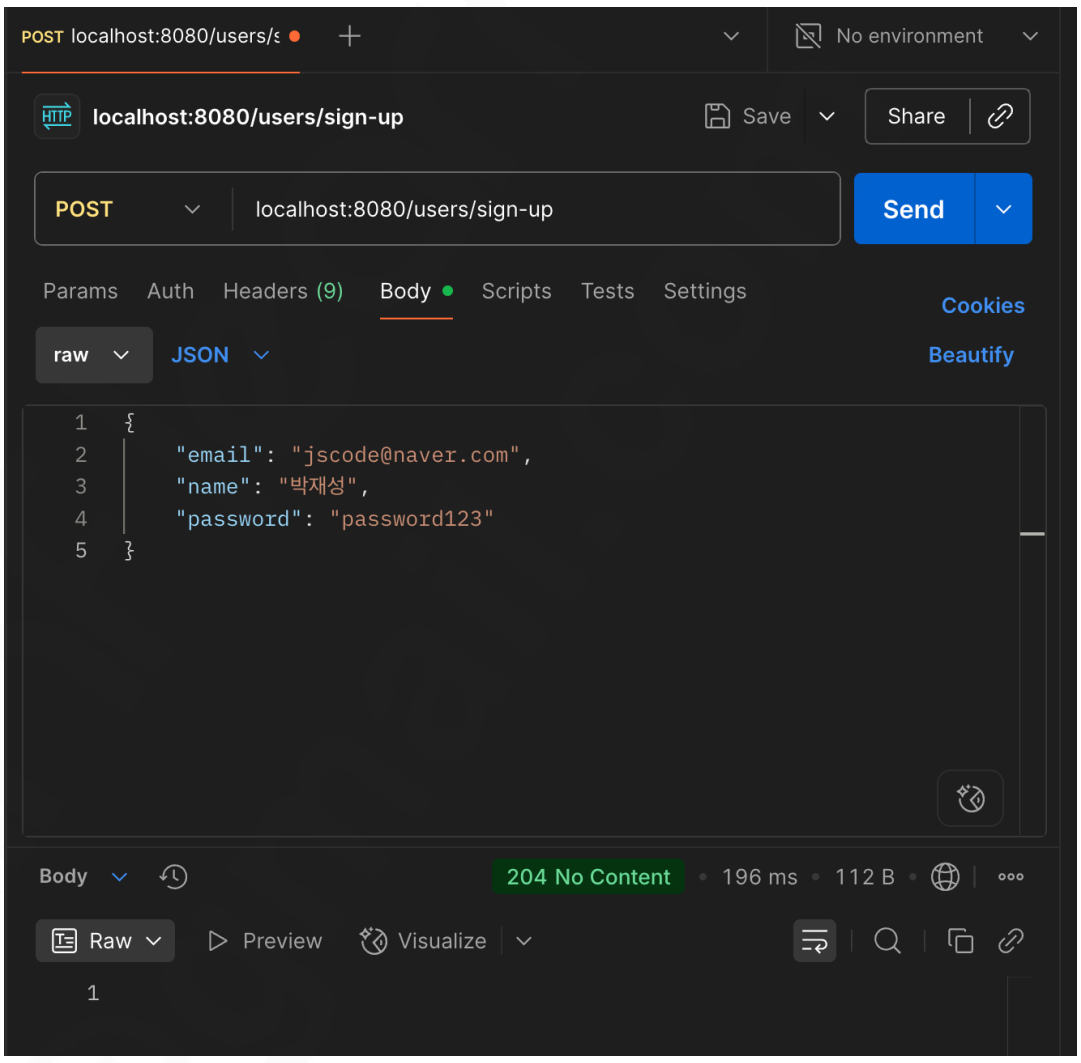
   service/UserService

```
@Service
public class UserService {
  private final UserRepository userRepository;

  public UserService(UserRepository userRepository) {
    this.userRepository = userRepository;
  }

    @Transactional
  public void signUp(SignUpRequestDto signUpRequestDto) {
    User user = new User(
      signUpRequestDto.getEmail(),
      signUpRequestDto.getName(),
      signUpRequestDto.getPassword()
    );

    this.userRepository.save(user);
  }
}
```

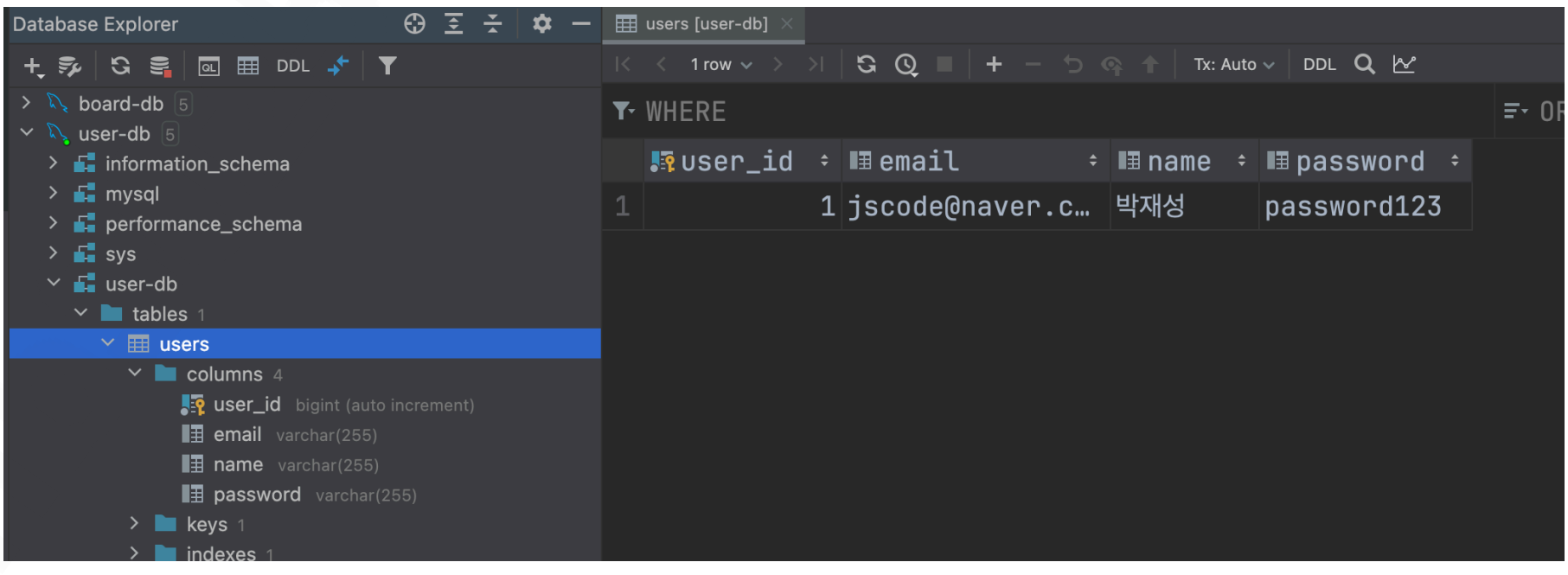
6. **서버 다시 실행시키기**



7. **API 잘 작동하는 지 테스트해보기**



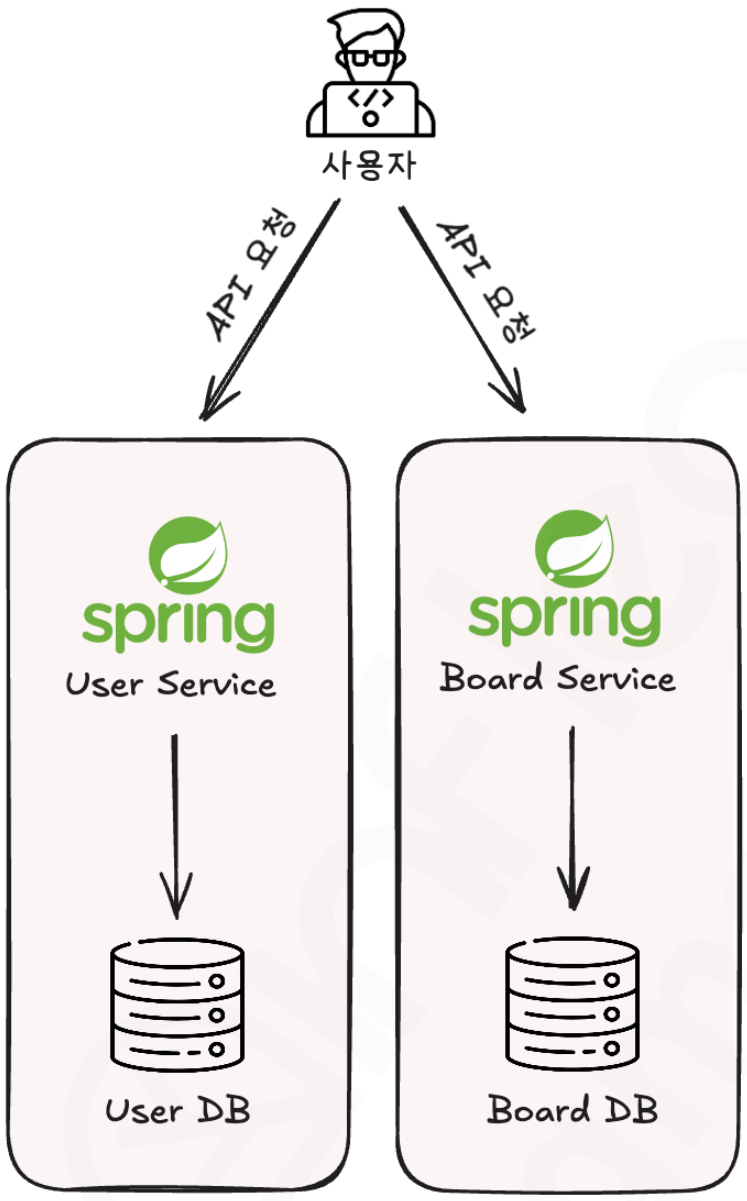
7. **DB에 데이터 잘 들어갔는 지 확인하기**



🎯 [실습] Board Microservice 만들기

✅ Board Microservice 만들기

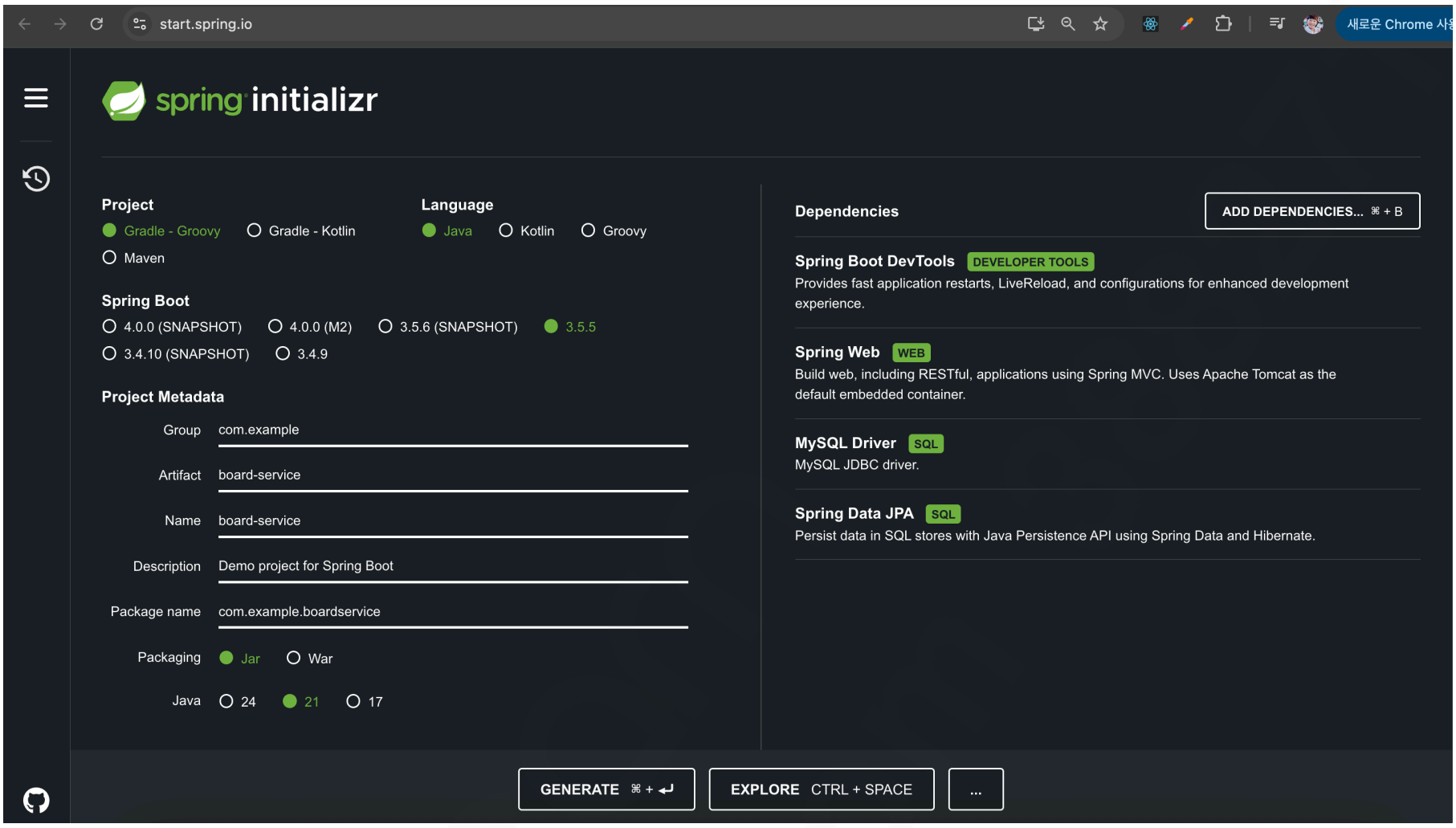
User Service는 만들었으니 마지막으로 남은 Board Service를 만들어보자.



✅ Spring Boot 프로젝트 셋팅 & DB 연결

1. **Spring Boot 프로젝트 셋팅**

> start.spring.io
>
> https://start.spring.io/



- `Artifact`와 `Name`을 `board-service`라고 지어주자.
- `Package name`을 Java 컨벤션에 맞게 `boardservice`라고 지어주자.
- Java 21 버전을 선택하자. 아래 과정을 Java 21 버전을 기준으로 진행할 예정이다.
- Dependencies는 `Spring Boot DevTools`, `Spring Web`, `MySQL Driver`, `Spring Data JPA`를 선택하자.

2. **application.yml 작성하기**

> 이 프로젝트에서는 `application.properties`를 지우고 `application.yml`을 생성했다.

```
server:
  port: 8081

spring:
  datasource:
    url: jdbc:mysql://localhost:3307/board-db # DB 주소
    username: root # DB 계정
    password: password # DB 비밀번호
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

3. **Spring Boot 서버 실행시켜서 DB 연결 제대로 되는 지 확인하기**

Spring Boot 실행시켰을 때 아래와 같은 에러 안 뜨고 서버가 실행되면, 정상적으로 DB가 잘 연결된 것이다.

```
org.springframework.beans.factory.BeanCreationException Create breakpoint : Error creating bean with name
 'entityManagerFactory' defined in class path resource
 [org/springframework/boot/autoconfigure/orm/jpa/HibernateJpaConfiguration.class]: Unable to create
 requested service [org.hibernate.engine.jdbc.env.spi.JdbcEnvironment] due to: Unable to determine
 Dialect without JDBC metadata (please set 'jakarta.persistence.jdbc.url' for common cases or 'hibernate
 .dialect' when a custom Dialect implementation must be provided)
  at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.initializeBean
   (AbstractAutowireCapableBeanFactory.java:1826) ~[spring-beans-6.2.10.jar:6.2.10]
  at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.doCreateBean
   (AbstractAutowireCapableBeanFactory.java:607) ~[spring-beans-6.2.10.jar:6.2.10]
  at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.createBean
   (AbstractAutowireCapableBeanFactory.java:529) ~[spring-beans-6.2.10.jar:6.2.10]
  at org.springframework.beans.factory.support.AbstractBeanFactory.lambda$doGetBean$0(AbstractBeanFactory
   .java:339) ~[spring-beans-6.2.10.jar:6.2.10]
  at org.springframework.beans.factory.support.DefaultSingletonBeanRegistry.getSingleton
```

✅ 게시글 생성 API 만들기

1. **Board 엔티티 생성하기**

domain/Board

```
@Entity
@Table(name = "boards")
public class Board {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long boardId;

  private String title;

  private String content;

  private Long userId; // FK 설정 안하고 그냥 컬럼으로 선언

  public Board() {
  }

  public Board(String title, String content, Long userId) {
    this.title = title;
    this.content = content;
    this.userId = userId;
  }

  public Long getBoardId() {
    return boardId;
  }

  public String getTitle() {
    return title;
  }

  public String getContent() {
    return content;
  }

  public Long getUserId() {
    return userId;
```

```
  }
}
```

2. **Repository 만들기**

   domain/BoardRepository

```
public interface BoardRepository extends JpaRepository<Board, Long> {
}
```

3. **Controller 만들기**

   controller/BoardController

```
@RestController
@RequestMapping("/boards")
public class BoardController {
  private final BoardService boardService;

  public BoardController(BoardService boardService) {
    this.boardService = boardService;
  }

  @PostMapping
  public ResponseEntity<Void> create(
      @RequestBody CreateBoardRequestDto createBoardRequestDto
  ) {
    boardService.create(createBoardRequestDto);
    return ResponseEntity.noContent().build();
  }
}
```

4. **DTO 만들기**

   dto/CreateBoardRequestDto

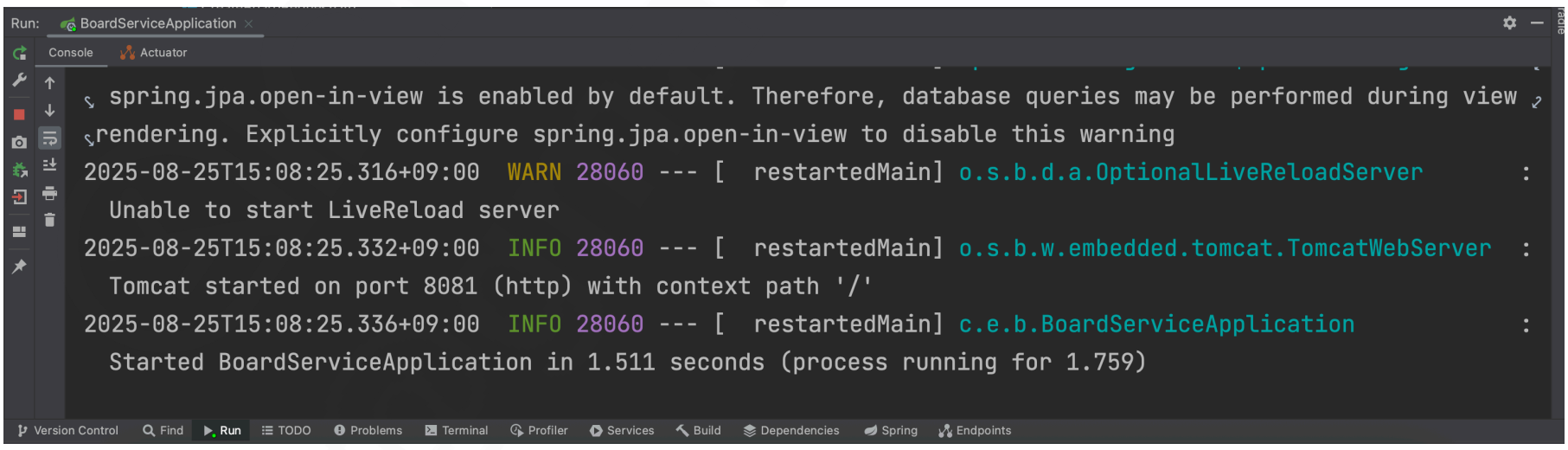
```
public class CreateBoardRequestDto {
  private String title;

  private String content;

  private Long userId;

  public String getTitle() {
    return title;
  }

  public String getContent() {
    return content;
  }

  public Long getUserId() {
    return userId;
  }
}
```

5. **Service 만들기**

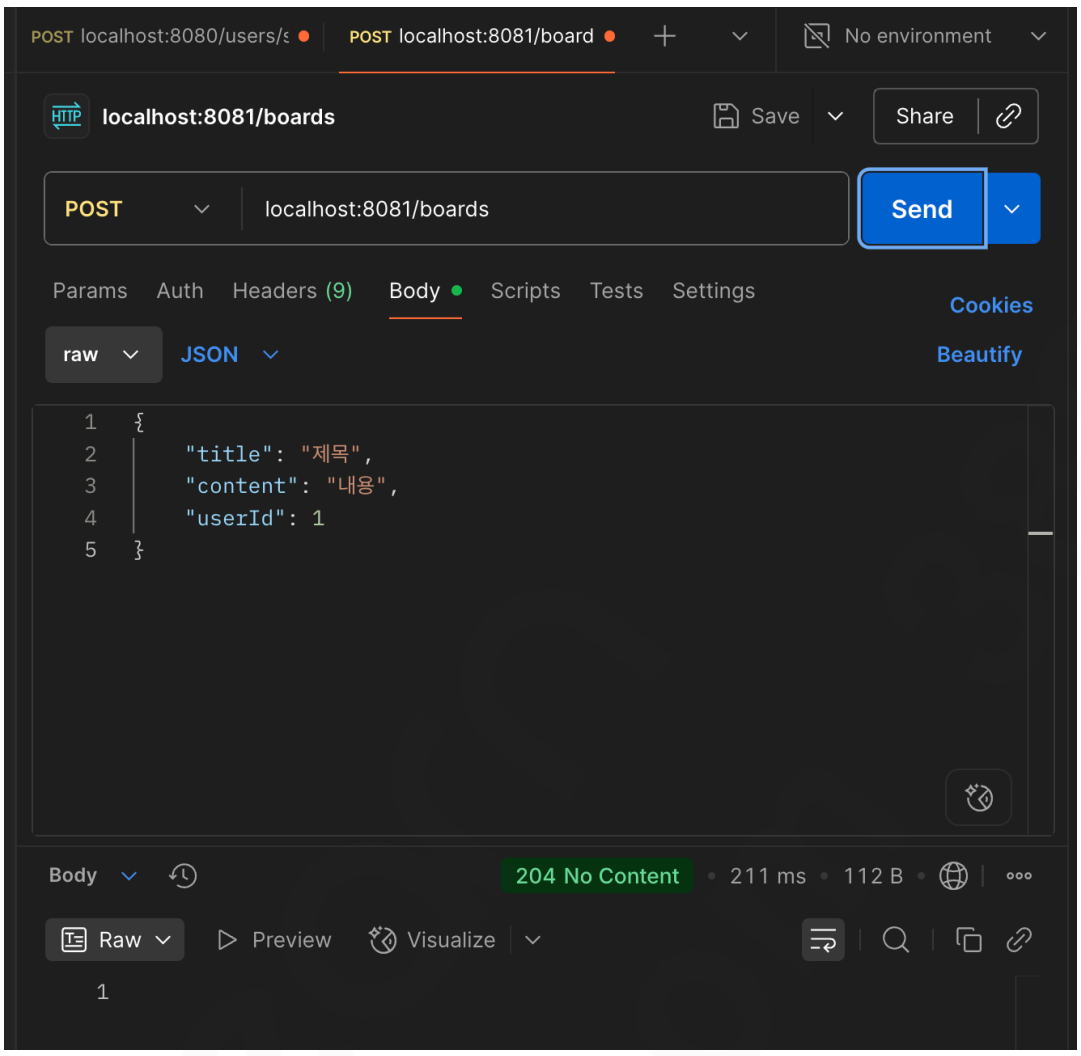
service/BoardService

```
@Service
public class BoardService {
 private final BoardRepository boardRepository;

 public BoardService(BoardRepository boardRepository) {
  this.boardRepository = boardRepository;
 }

  @Transactional
 public void create(CreateBoardRequestDto createBoardRequestDto) {
  Board board = new Board(
    createBoardRequestDto.getTitle(),
    createBoardRequestDto.getContent(),
    createBoardRequestDto.getUserId()
  );

  this.boardRepository.save(board);
 }
}
```

6. **서버 다시 실행시키기**

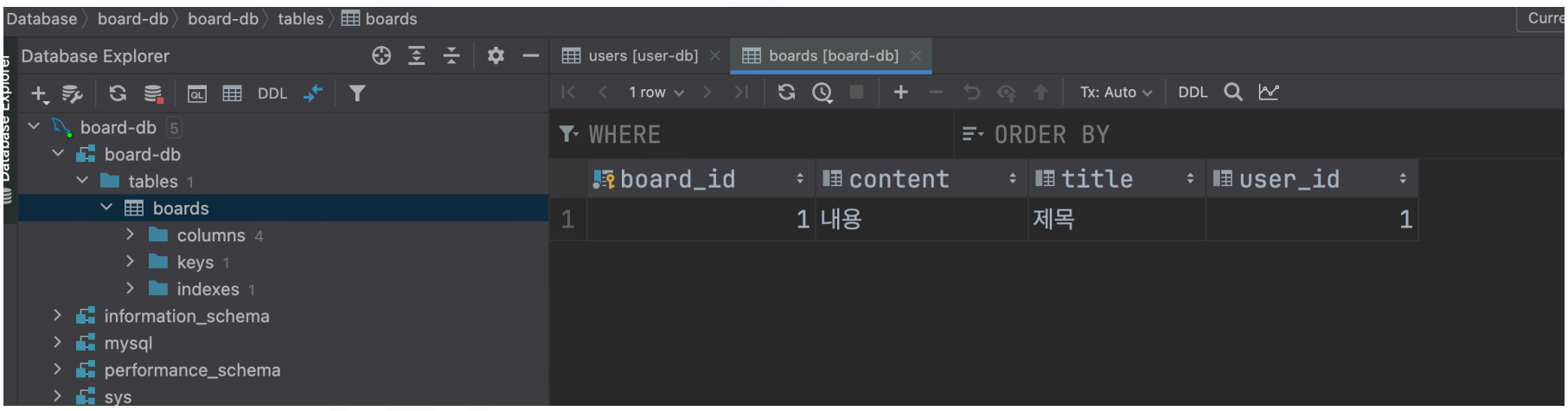


7. **API 잘 작동하는 지 테스트해보기**



주의) 8081번 포트로 요청을 보내야 한다.

7. **DB에 데이터 잘 들어갔는 지 확인하기**



khb3025@naver.com

📖

구현한 MSA 프로젝트를 잘 구현했는 지 전체적으로 검토하기

✅ 구현할 기능

1. **회원가입 API**

   고려 사항

   - 간단한 프로젝트 구조를 위해 비밀번호는 암호화하지 않고 그대로 저장한다.

   Request

   ```
   {
     "email": "jscode@naver.com",
     "name": "박재성",
     "password": "abc"
   }
   ```

   Response

   ```
   (204 No Contnet로 응답하기)
   ```

2. **게시글 작성 API**

   고려 사항

   - 간단한 프로젝트 구조를 위해 게시글 작성을 할 때, JWT 방식이 아닌 `userId`를 직접 입력받는 형태로 구성한다.

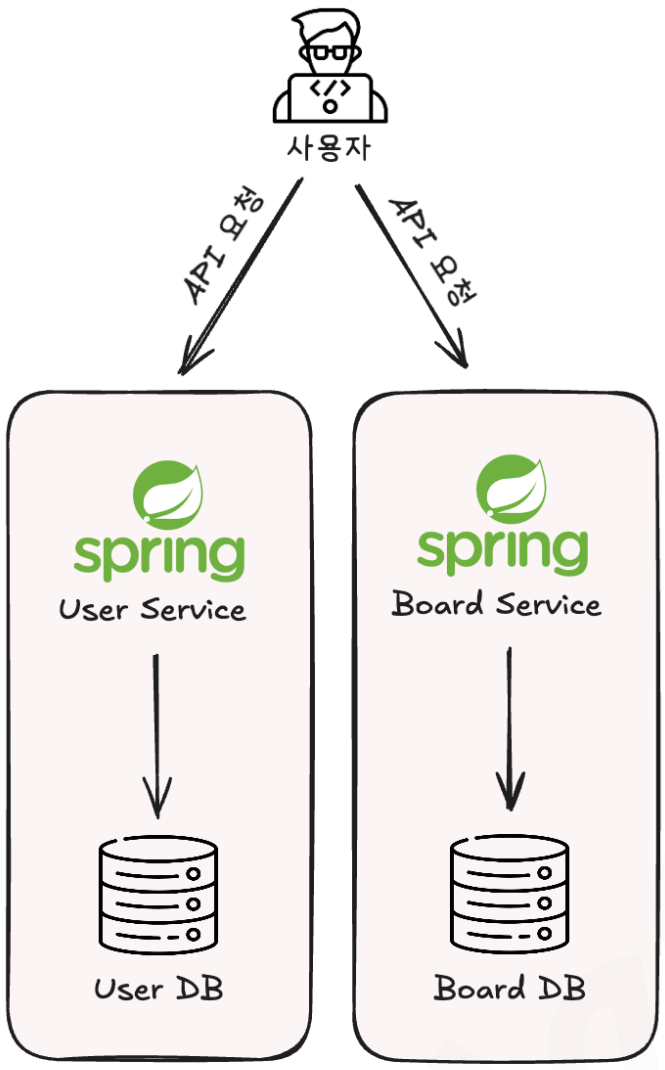
   Request

   ```
   {
     "title": "제목",
     "content": "내용"
     "userId": 1
   }
   ```

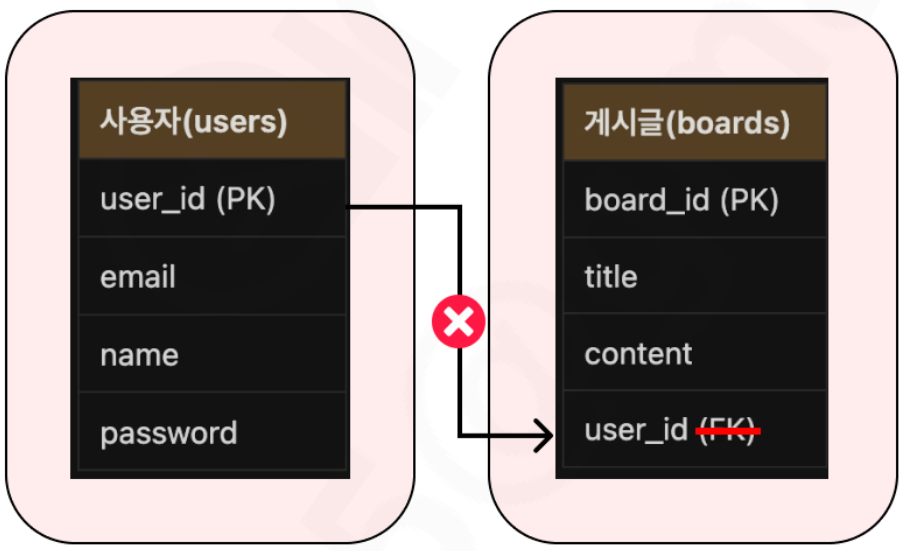
   Response

   ```
   (204 No Content로 응답하기)
   ```

✅ 프로젝트 아키텍처



✅ DB 설계



khb3023@gmail.com 38271

📖 추가 요구사항

✅ 구현할 기능

1. **특정 게시글 조회 API**

   Request

   ```
   http://{URL}/boards/{board_id}
   ```

   Response

   ```
   {
     "boardId": 1,
     "title": "제목",
     "content": "내용"
     "user": {
       "userId": 1,
       "name": "박재성"
     }
   }
   ```

2. **게시글 전체 조회 API**

   고려 사항

   - 간단한 프로젝트 구조를 위해 페이지네이션은 구현하지 않는다.

   Request

   ```
   ```

   Response

   ```
   [
     {
       "boardId": 1,
       "title": "제목",
       "content": "내용"
       "user": {
         "userId": 1,
         "name": "박재성"
       }
     },
     {
       "boardId": 2,
       "title": "제목2",
       "content": "내용2"
       "user": {
         "userId": 2,
         "name": "김재훈"
       }
   ```

```
  }
]
```

@naver.com
khb3025

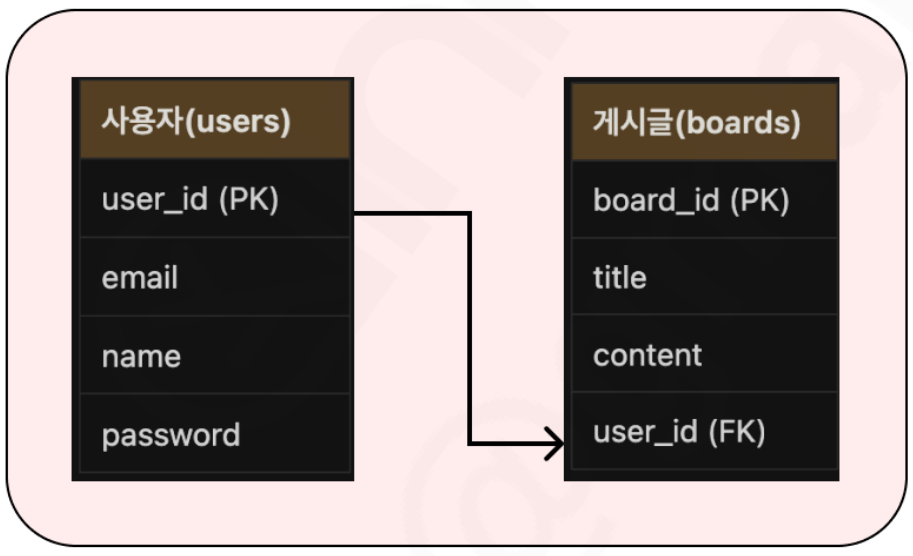
데이터 조회 기능 구현 방식 (모놀리식 vs MSA)

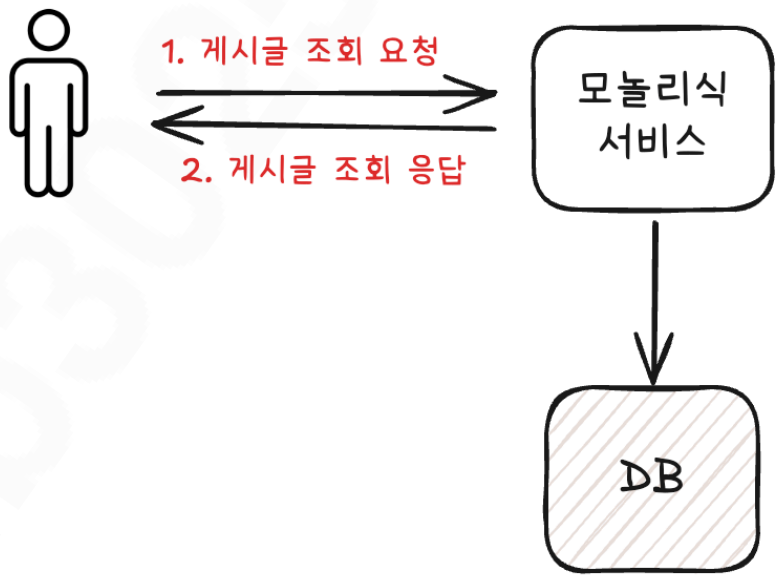
👨🏻‍🏫 요구사항에 따르면 **게시글 조회 API**를 아래와 같이 응답하게 구현해야 한다.

```
{
  "boardId": 1,
  "title": "제목",
  "content": "내용"
  "user": {
    "userId": 1,
    "name": "박재성"
  }
}
```

응답값을 자세히 보면 게시글 정보 뿐만 아니라 사용자 정보도 일부 포함되어 있다. MSA에서는 어떻게 이 기능을 구현할 수 있는지 알아보자.

✅ '모놀리식 아키텍처'에서 데이터 조회 기능 구현 방식





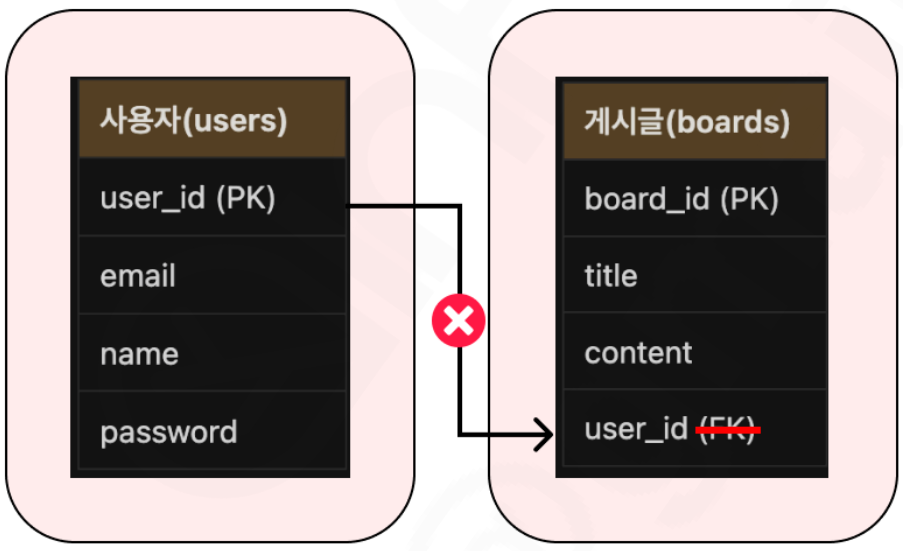
모놀리식 아키텍처에서는 하나의 DB에 **사용자 정보**와 **게시글 정보**가 한꺼번에 저장되어 있기 때문에 아래와 같이 SQL문 중 **JOIN 문**을 활용해 데이터를 쉽게 조회해올 수 있다.

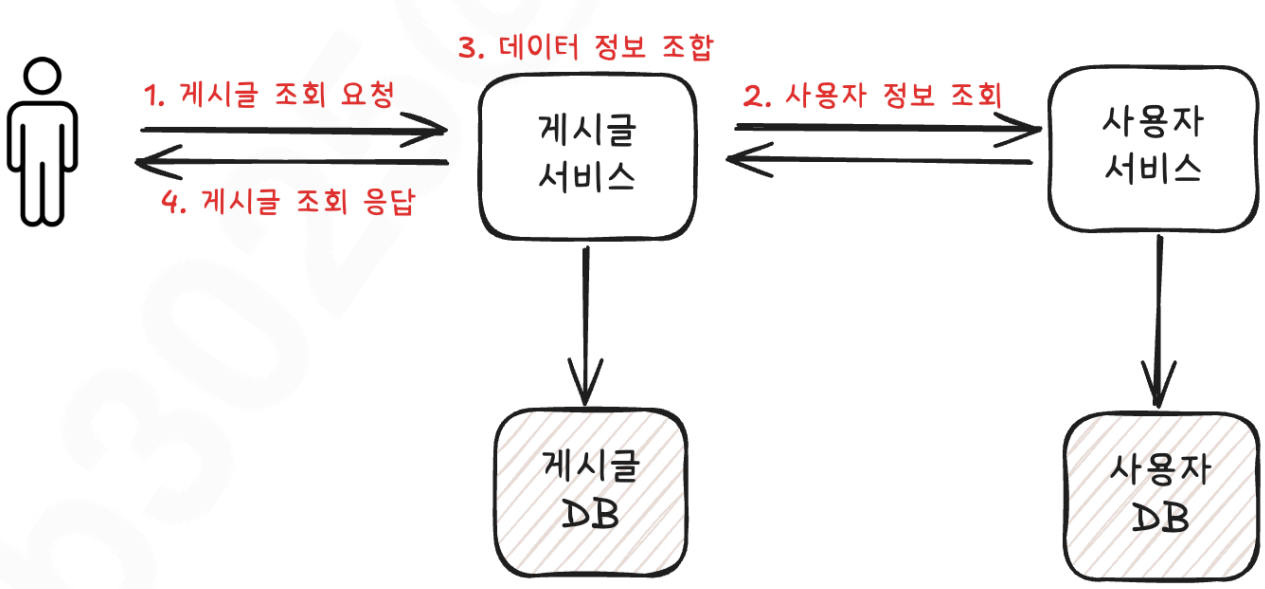
SQL문

```
SELECT
  b.board_id,
  b.title,
  b.content,
  u.user_id,
  u.name
FROM
  boards b
INNER JOIN
  users u ON b.user_id = u.user_id
WHERE
  b.board_id = 1;
```

> 하지만 MSA 구조에서는 **사용자 정보가 저장되어 있는 DB**와 **게시글 정보가 저장되어 있는 DB**가 분리되어 있기 때문에 SQL문의 **JOIN 문**을 활용할 수가 없다. 그럼 MSA 구조에서는 어떻게 데이터를 조회해와야 하는 지 알아보자.

✅ MSA에서 데이터 조회 기능 구현 방식





MSA 방식에서는 서비스 별로 DB가 분리되어 있기 때문에 **게시글 정보**와 **사용자 정보**를 각각 불러와야 한다. 그런데 MSA 구조에서는 각 서비스의 독립성을 유지하기 위해, 특정 서비스가 다른 서비스의 DB에 직접적으로 접근해서는 안 된다. 오로지 사용자 서비스의 API와 같은 인터페이스를 통해서만 접근해야 한다.

그래서 게시글 서비스가 사용자 서비스의 '사용자 정보 조회 API'를 통해 사용자 정보를 조회해와야 한다. 게시글 서비스가 외부의 다른 서비스로부터 데이터를 받아올 때는 특별한 방법이 있는 게 아니라, 외부 API와 통신하듯이 REST API 방식으로 데이터를 받아오면 된다.

참고로, 서비스 간에 데이터를 주고 받을 때는 반드시 REST API 방식을 활용해야 되는 건 아니다. 데이터를 주고 받을 수 있는 통신 방식이면 어떤 통신 방식이든 상관없다. 그 중에서 최근에 종종 언급되고 있는 통신 방식으로 gRPC, graphQL가 있다. 하지만 대부분의 기업에서 아직 gRPC, graphQL을 보편적으로 사용하고 있지는 않기 때문에 이 강의에서는 gRPC, graphQL을 따로 다루진 않을 예정이다.

@gmail.com
khb3025

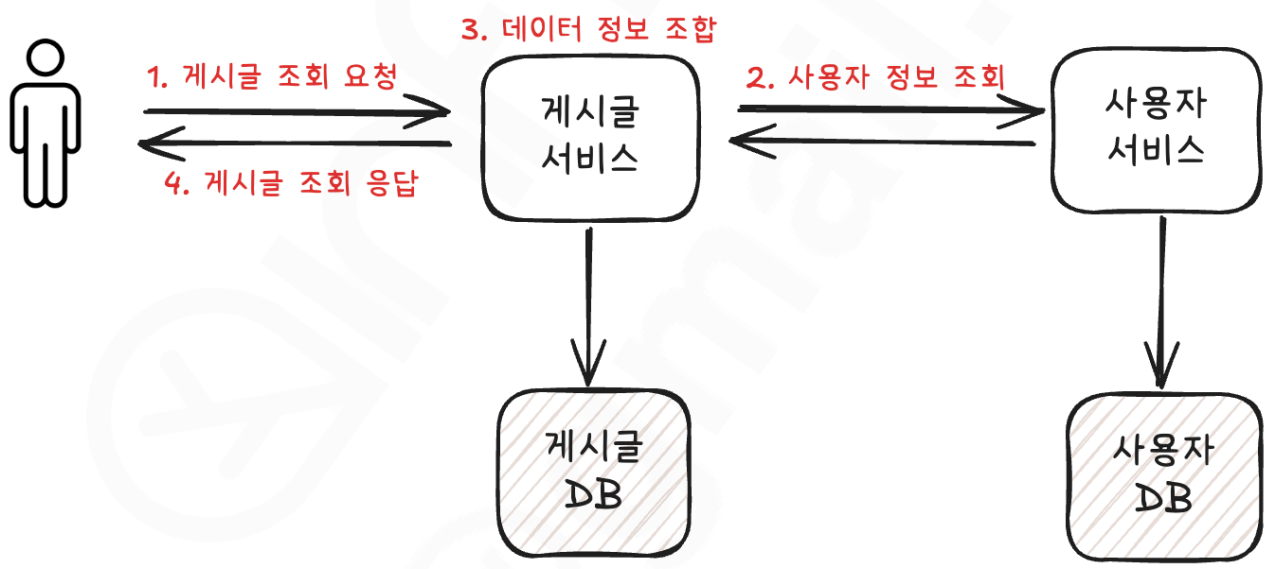
🎯 [실습] 다른 마이크로서비스가 사용할 '사용자 조회 API' 만들기

✅ 게시글 조회 API를 구현하려면, 사용자 조회 API도 만들어야 하는 이유

[게시글 조회 API의 응답값]

```
{
  "boardId": 1,
  "title": "제목",
  "content": "내용"
  "user": {
    "userId": 1,
    "name": "박재성"
  }
}
```

[구조]



게시글 조회 API 응답값에서는 사용자 정보도 필요하기 때문에, 게시글 서비스가 사용할 **사용자 정보 조회 API**가 필요하다.

✅ 사용자 조회 API 만들기

> `user-service` 프로젝트에서 아래 코드 작성하기

1. **Response DTO 만들기**

   dto/UserResponseDto

```
public class UserResponseDto {
  private Long userId;
  private String email;
  private String name;

  public UserResponseDto(Long userId, String email, String name) {
    this.userId = userId;
    this.email = email;
```

```
    this.name = name;
  }

  public Long getUserId() {
    return userId;
  }

  public String getEmail() {
    return email;
  }

  public String getName() {
    return name;
  }
}
```

- `password`는 노출되면 안 되는 정보이기 때문에 제외했다.

2. **Service 로직 작성하기**

   service/UserService

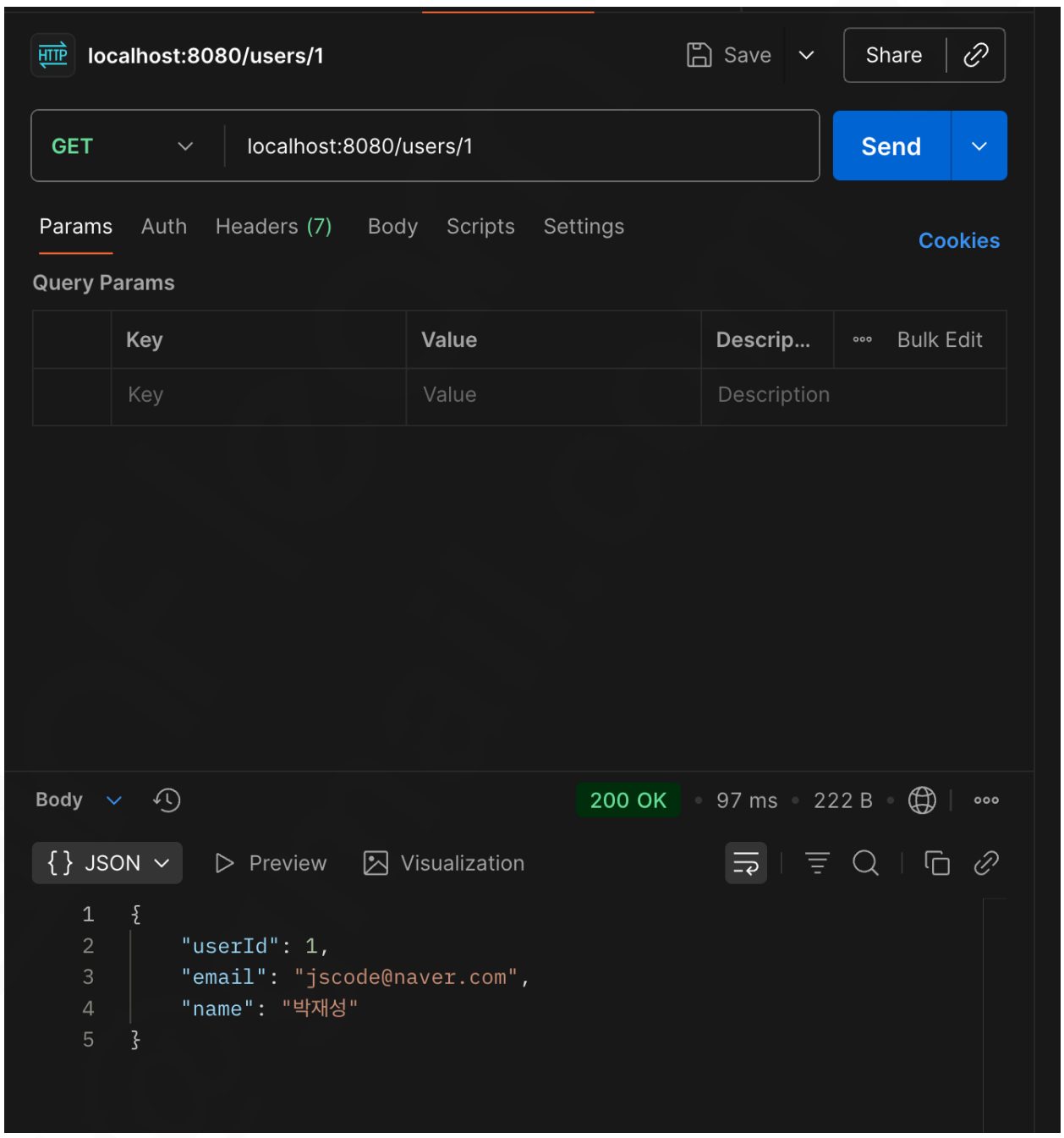
```
@Service
public class UserService {
  private final UserRepository userRepository;

  public UserService(UserRepository userRepository) {
    this.userRepository = userRepository;
  }

  ...

  public UserResponseDto getUser(Long id) {
    User user = userRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("사용자를 찾을 수 없습니다."));

    return new UserResponseDto(
        user.getUserId(),
        user.getEmail(),
        user.getName()
    );
  }
}
```

3. **Controller 로직 작성하기**

```
@RestController
@RequestMapping("/users")
public class UserController {
  private final UserService userService;

  public UserController(UserService userService) {
    this.userService = userService;
  }

  ...
```

```
  @GetMapping("{userId}")
  public ResponseEntity<UserResponseDto> getUser(@PathVariable Long userId) {
    UserResponseDto userResponseDto = userService.getUser(userId);
    return ResponseEntity.ok(userResponseDto);
  }
}
```

4. **서버 재실행시킨 후 잘 작동하는 지 확인해보기**



khb3025@naver.com

🎯 [실습] 특정 게시글 조회 API 만들기

✅ 특정 게시글 조회 API 만들기

> board-service에서 아래 코드 작성하기

1. **Response DTO 만들기**

   dto/BoardResponseDto

```
public class BoardResponseDto {
  private Long boardId;
  private String title;
  private String content;
  private UserDto user;

public BoardResponseDto(Long boardId, String title, String content, UserDto user) {
  this.boardId = boardId;
  this.title = title;
  this.content = content;
  this.user = user;
}

public Long getBoardId() {
  return boardId;
}

public String getTitle() {
  return title;
}

public String getContent() {
  return content;
}

public UserDto getUser() {
  return user;
  }
}
```

   dto/UserDto

```
public class UserDto {
  private Long userId;
  private String name;

  public UserDto(Long userId, String name) {
    this.userId = userId;
    this.name = name;
  }

  public Long getUserId() {
    return userId;
```

```
  }

  public String getName() {
    return name;
  }
}
```

- 게시글 조회 API의 응답값으로 `userId`와 `name`만 필요하기 때문에 위와 같이 코드를 작성했다.

2. **user-service의 API와 통신할 코드 작성하기**

> 이 프로젝트에서는 외부 API 통신을 할 때 `RestClient`라는 라이브러리를 사용할 것이다. 'MSA니까 반드시 `WebClient`를 써야돼'라는 건 없다. `RestTemplate`를 써도 되고 `WebClient`를 써도 되고 `FeignClient`를 써도 된다. API 통신을 할 수 있는 라이브러리라면 어떤 라이브러리를 사용해도 상관없다.

resources/application.yml

```
server:
  port: 8081

spring:
  datasource:
    url: jdbc:mysql://localhost:3307/board-db # DB 주소
    username: root # DB 계정
    password: password # DB 비밀번호
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true

client:
  user-service:
    url: http://localhost:8080
```

dto/UserResponseDto (사용자 조회 API 응답값을 담을 DTO)

```
public class UserResponseDto {
  private Long userId;
  private String email;
  private String name;

  public Long getUserId() {
    return userId;
  }

  public String getEmail() {
    return email;
  }

  public String getName() {
    return name;
  }
}
```

client/UserClient

```
@Component
public class UserClient {

  private final RestClient restClient;

  public UserClient(
      @Value("${client.user-service.url}") String userServiceUrl
  ) {
    this.restClient = RestClient.builder()
        .baseUrl(userServiceUrl)
        .build();
  }

  public UserResponseDto fetchUser(Long userId) {
    return this.restClient.get()
        .uri("/users/{userId}", userId)
        .retrieve()
        .body(UserResponseDto.class);
    }
}
```

3. **Service 로직 작성하기**

   service/BoardService

```
@Service
public class BoardService {
  private final BoardRepository boardRepository;
  private final UserClient userClient;

  public BoardService(BoardRepository boardRepository, UserClient userClient) {
    this.boardRepository = boardRepository;
    this.userClient = userClient;
  }

  ...

public BoardResponseDto getBoard(Long boardId) {
  // 게시글 불러오기
  Board board = boardRepository.findById(boardId)
      .orElseThrow(() -> new IllegalArgumentException("게시글을 찾을 수 없습니다."));

  // user-service로부터 사용자 정보 불러오기
  UserResponseDto userResponseDto = userClient.fetchUser(board.getUserId());

  // 응답값 조합하기
  UserDto userDto = new UserDto(
      userResponseDto.getUserId(),
      userResponseDto.getName()
  );
  BoardResponseDto boardResponseDto = new BoardResponseDto(
      board.getBoardId(),
      board.getTitle(),
      board.getContent(),
```

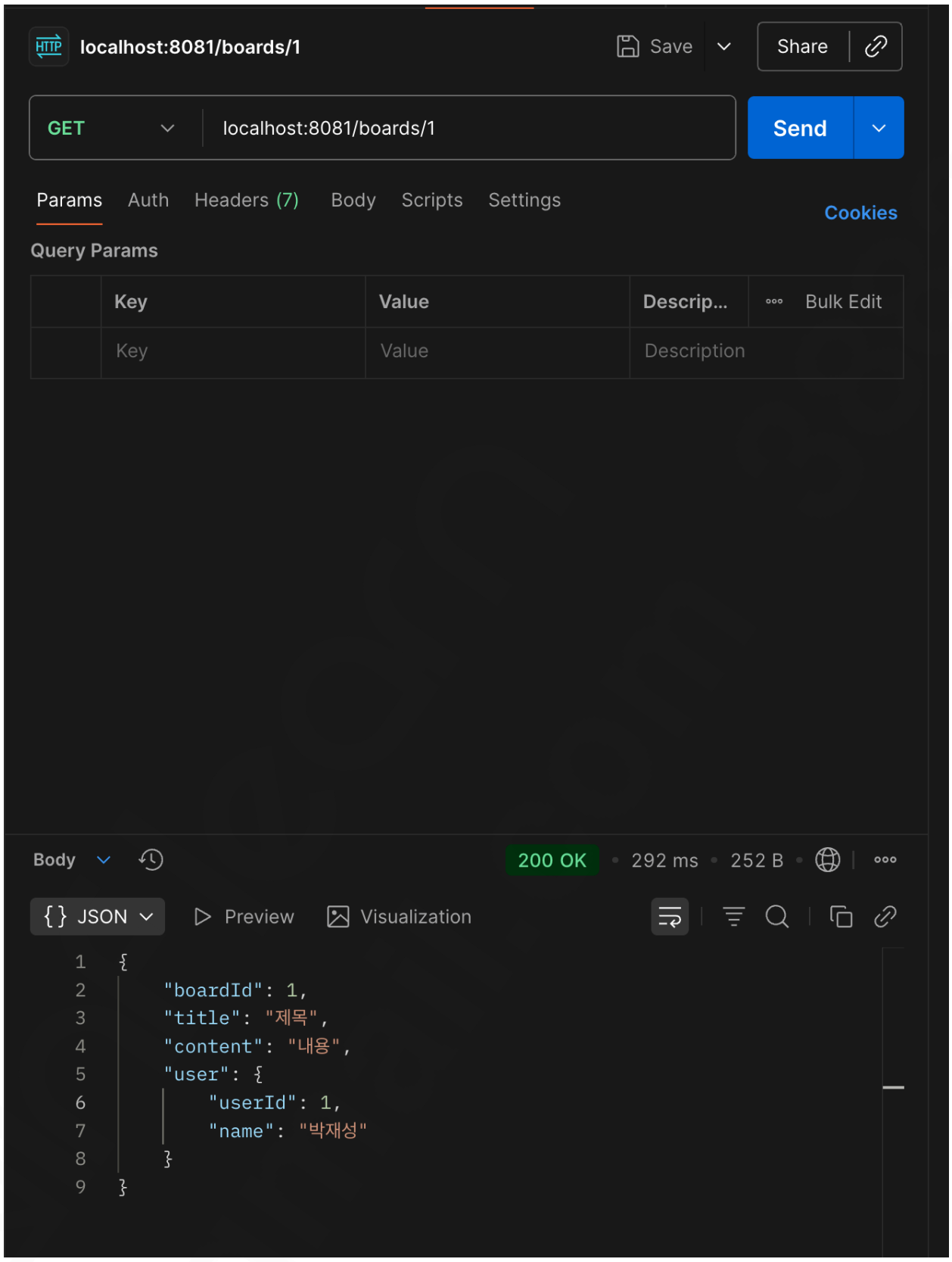
```
      userDto
    );

    return boardResponseDto;
  }
}
```

3. **Controller 로직 작성하기**

```
@RestController
@RequestMapping("/boards")
public class BoardController {
  private final BoardService boardService;

  public BoardController(BoardService boardService) {
    this.boardService = boardService;
  }

  ...

  @GetMapping("/{boardId}")
  public ResponseEntity<BoardResponseDto> getBoard(@PathVariable Long boardId) {
    BoardResponseDto boardResponseDto = boardService.getBoard(boardId);
    return ResponseEntity.ok(boardResponseDto);
  }
}
```

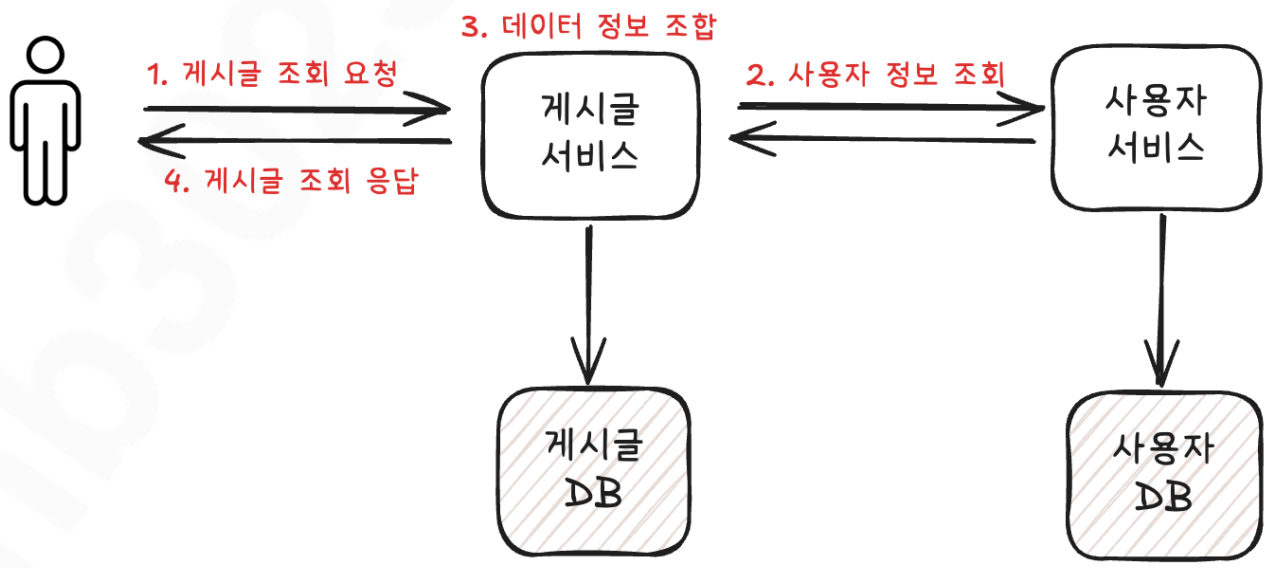
4. **서버 재실행시킨 후 잘 작동하는 지 확인해보기**

khb3025@gmail.com



DB가 분리되어 있는 MSA 환경에서 사용자 정보를 포함해 게시글 정보를 잘 조회해왔다.

✅ 정리



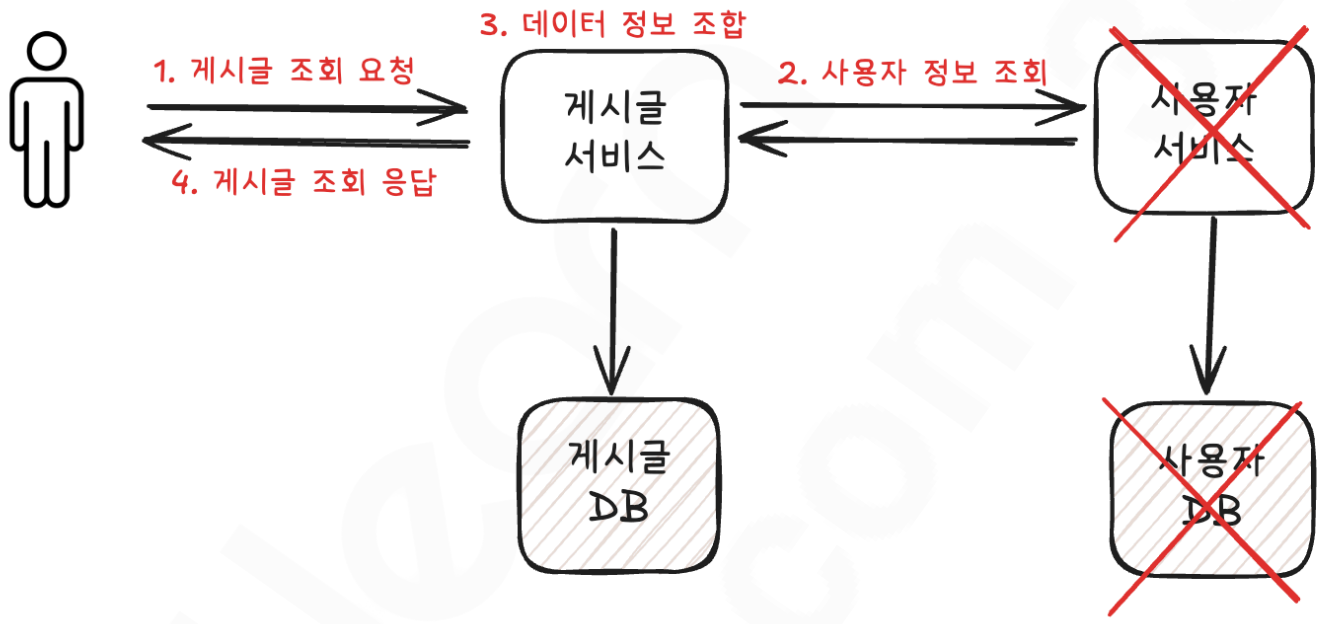
1. 특정 게시글 조회 API 주소(ex. `GET localhost:8081/boards/1`)로 요청을 보낸다.
2. 게시글 서비스에서 사용자 정보가 필요해서, 사용자 서비스(ex. `GET localhost:8080/users/1`)로부터 데이터를 조회해온다.

3. 사용자 서비스로부터 받아온 사용자 정보를 활용해 응답할 데이터를 조합한다.
4. 사용자에게 응답한다.

@kfc38827
@gmail.com
khb3025

특정 서비스가 장애났을 때, 장애가 전파되지 않게 막는 방법

✅ 특정 서비스가 장애났을 때, 장애가 전파되지 않게 막는 방법



위와 같은 구조에서 사용자 서비스의 서버 또는 DB에 장애가 났다고 가정하자. 그럼 게시글 조회 API도 정상적으로 작동하지 않을 것이다. 왜냐하면 게시글 조회 로직에는 사용자 서비스로부터 사용자 정보를 받아오는 로직이 포함되어 있기 때문이다.

이 때, 장애가 전파되지 않게 막는 가장 심플한 방법은 **장애가 난 서비스로부터 받아와야 하는 정보는 응답하지 않는 방법**이다. 즉, 아래와 같이 **정상적으로 응답할 수 있는 값만 사용자한테 응답**하면 된다. 이 방식은 사용자 정보를 필수로 보내지 않아도, 서비스 로직상 큰 문제가 안 되는 API일 경우에 자주 사용하는 처리 방법이다.

```
200 OK

{
  "boardId": 1,
  "title": "제목",
  "content": "내용"
  "user": null
}
```

만약 위 예시와는 다르게 **서비스 로직상 사용자 정보가 필수로 필요하다면 적절한 에러 메시지로 응답**하면 된다.

```
503 Service Unavailable

{
  "errorCode": "USER_SERVICE_UNAVAILABLE",
  "message": "사용자 정보를 조회할 수 없어 게시글을 표시할 수 없습니다. 잠시 후 다시 시도해주세요."
}
```

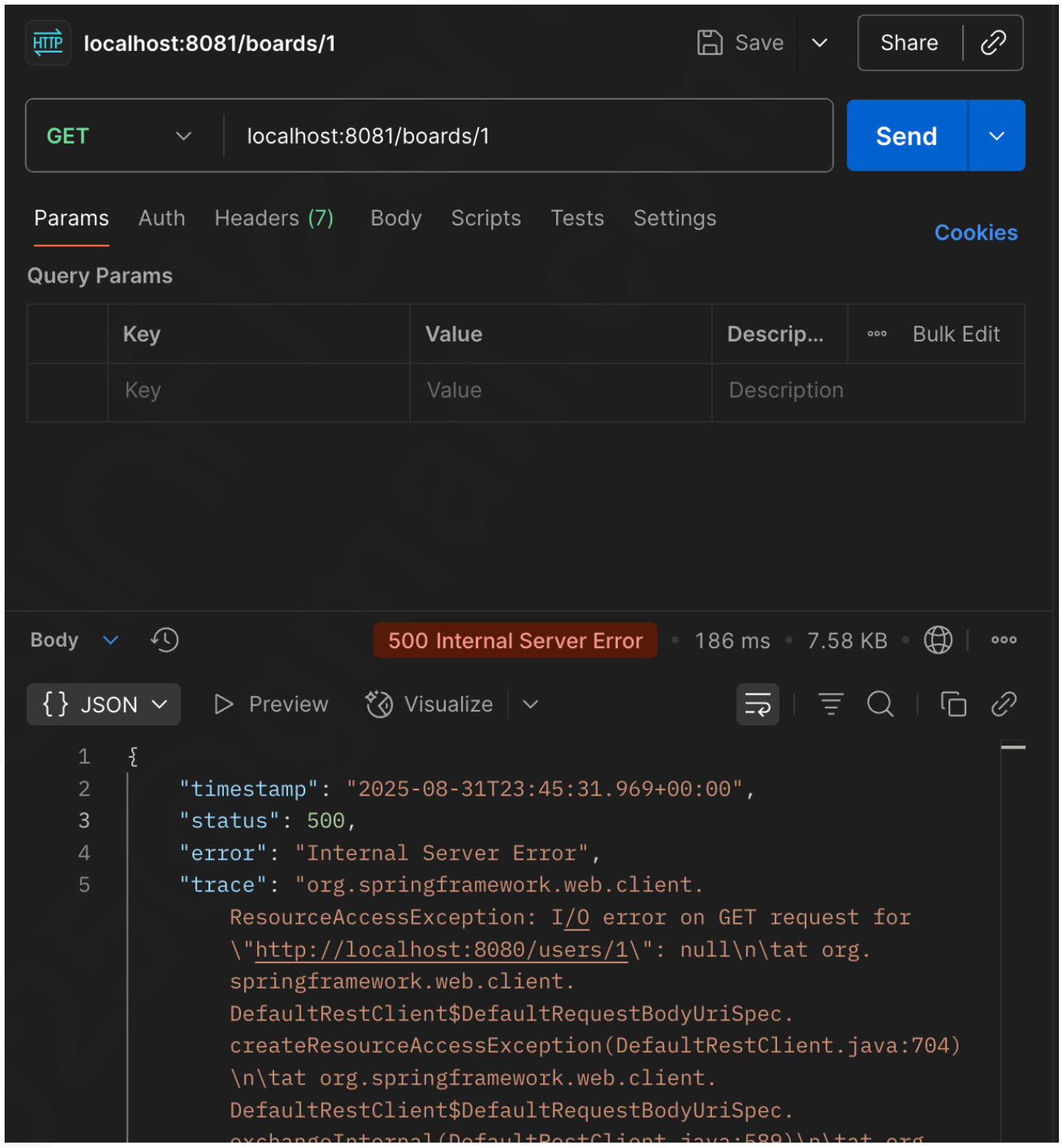
[실습] 특정 서비스가 장애났을 때, 장애가 전파되지 않게 방어 코드 작성하기

✅ 장애 전파 방어 코드 작성하지 않으면 어떻게 되는 지 테스트하기

1. **user-service 서버 종료시키기**

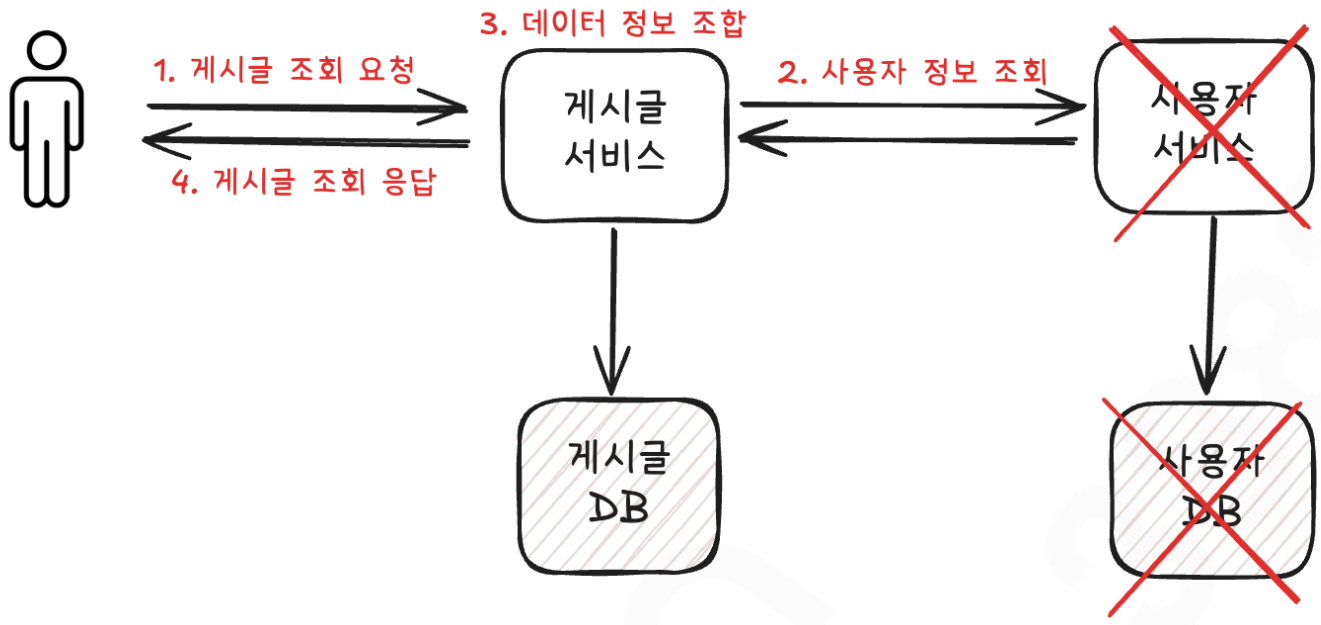
   user-service가 장애가 났다고 가정하기 위해 user-serivce를 종료시키자.

2. **게시글 조회 API로 요청 보내기**



user-service가 정상적으로 작동하지 않을 때, board-service의 게시글 조회 API도 같이 장애가 나는 걸 확인할 수 있다. 즉, 장애가 전파됐다.

✅ 장애 전파 방지 전략



User Service가 장애가 나더라도 Board Service의 게시글 조회 API의 로직은 영향을 받지 않게 만들어보자. 여러 장애 전파 방지 전략 중 '**정상적으로 응답할 수 있는 값만 사용자에게 응답하는 방식**'으로 장애 전파를 막아보자.

Response

```
200 OK

{
  "boardId": 1,
  "title": "제목",
  "content": "내용"
  "user": null
}
```

✅ 코드 작성하기

> board-service에서 아래 코드 작성하기

1. **UserClient 코드 수정하기**

   client/UserClient

```
@Component
public class UserClient {

private final RestClient restClient;

public UserClient(
    @Value("${client.user-service.url}") String userServiceUrl
) {
  this.restClient = RestClient.builder()
      .baseUrl(userServiceUrl)
      .build();
}

public Optional<UserResponseDto> fetchUser(Long userId) {
  try {
    UserResponseDto userResponseDto = this.restClient.get()
        .uri("/users/{userId}", userId)
        .retrieve()
        .body(UserResponseDto.class);
```

```
      return Optional.ofNullable(userResponseDto);
    } catch (RestClientException e) {
      // 로깅 : 예외 발생 시 로그를 남겨 문제를 파악할 수 있게 해야 함
      // log.error("사용자 정보 조회 실패: {}", e.getMessage(), e);
      return Optional.empty();
    }
  }
}
```

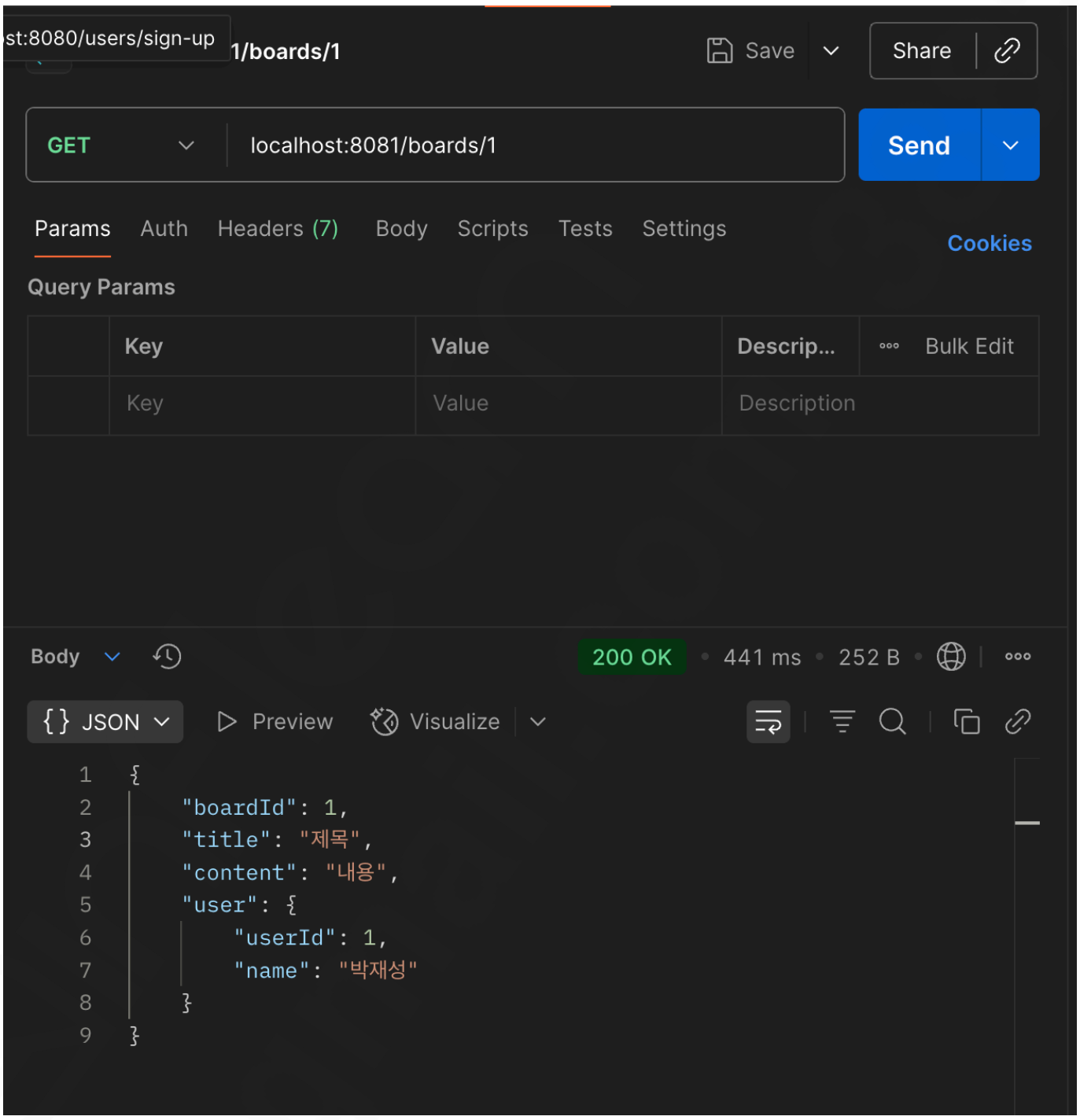
2. **BoardService 코드 수정하기**

   service/BoardService

```
@Service
public class BoardService {
  private final BoardRepository boardRepository;
  private final UserClient userClient;

public BoardService(BoardRepository boardRepository, UserClient userClient) {
  this.boardRepository = boardRepository;
  this.userClient = userClient;
}

...


public BoardResponseDto getBoard(Long boardId) {
  // 게시글 불러오기
  Board board = boardRepository.findById(boardId)
      .orElseThrow(() -> new IllegalArgumentException("게시글을 찾을 수 없습니다."));

  // user-service로부터 사용자 정보 불러오기
  Optional<UserResponseDto> optionalUserResponseDto = userClient.fetchUser(board.getUserId());

  // UserDto 생성
  UserDto userDto = null;
  if (optionalUserResponseDto.isPresent()) {
    UserResponseDto userResponseDto = optionalUserResponseDto.get();
    userDto = new UserDto(
        userResponseDto.getUserId(),
        userResponseDto.getName()
    );
  }

  // ResponseDto 생성
  BoardResponseDto boardResponseDto = new BoardResponseDto(
      board.getBoardId(),
      board.getTitle(),
      board.getContent(),
      userDto
  );

  return boardResponseDto;
 }
}
```

2. **board-service와 user-service 서버 둘 다 실행시키기**

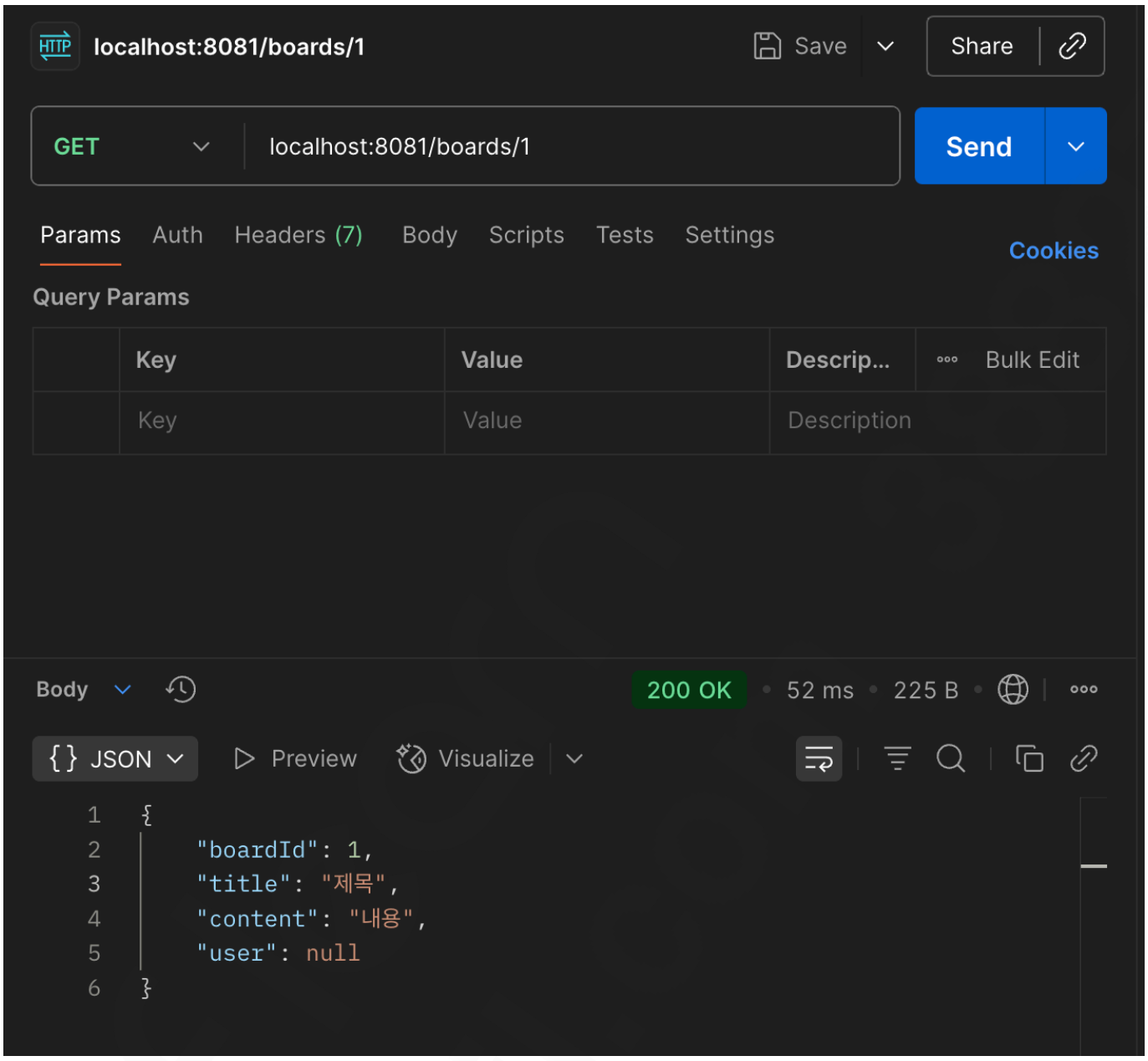
3. **게시글 조회 API 잘 작동하는 지 테스트해보기**



5. **user-service 서버만 종료하기**

   user-service에 장애가 난 상황을 가정하기 위해 서버를 종료시키자.

6. **게시글 조회 API 잘 작동하는 지 테스트해보기**



분명 user-service가 정상적으로 작동하고 있지 않는 상태인데도 불구하고, board-service에는 영향을 미치지 않았다. 이로써 장애가 전파되지 않았다.

kb302500@gmail.com

🎯 [실습] 게시글 전체 조회 API 만들기 - 1

✅ 설계

[게시글 전체 조회 API]

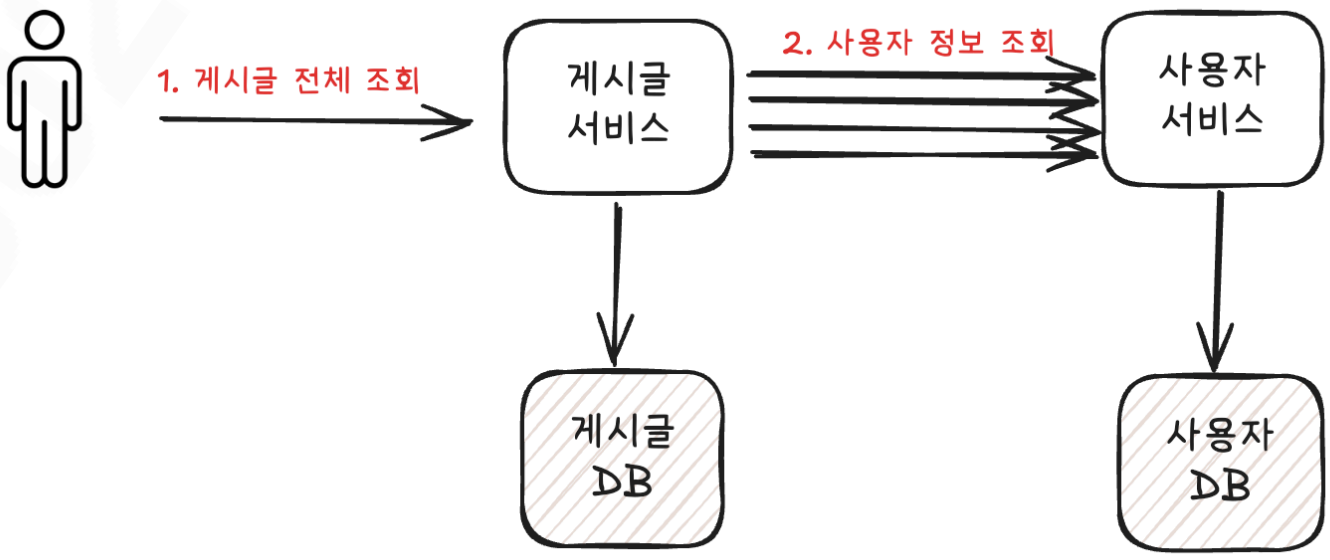
Request

```
```

Response

```
[
  {
    "boardId": 1,
    "title": "제목",
    "content": "내용"
    "user": {
      "userId": 1,
      "name": "박재성"
    }
  },
  {
    "boardId": 2,
    "title": "제목2",
    "content": "내용2"
    "user": {
      "userId": 2,
      "name": "김재훈"
    }
  }
]
```

위 요구사항에 따라 **게시글 전체 조회 API**를 만들려면 **게시글의 개수만큼 사용자 정보를 조회**해와야 한다. 즉, 10개의 게시글을 조회해서 응답하려면 각 게시글의 사용자 정보가 필요하므로 사용자 정보 조회 API를 10번 호출해야 한다.



이런 비효율성을 해결하려면 **하나의 API 요청으로 여러 개의 사용자 정보를 한 번에 조회해 올 수 있게 API를 만들면 된다.**

✅ 사용자 목록 조회 API 만들기

> user-service에서 아래 코드 작성하기

1. **Service 로직 작성하기**

   service/UserService

```
@Service
public class UserService {
  private final UserRepository userRepository;

  public UserService(UserRepository userRepository) {
    this.userRepository = userRepository;
  }

  ...

  public List<UserResponseDto> getUsersByIds(List<Long> ids) {
    List<User> users = userRepository.findAllById(ids);

    return users.stream()
        .map(user -> new UserResponseDto(
            user.getUserId(),
            user.getEmail(),
            user.getName()
        ))
        .collect(Collectors.toList());
  }
}
```

2. **Controller 로직 작성하기**

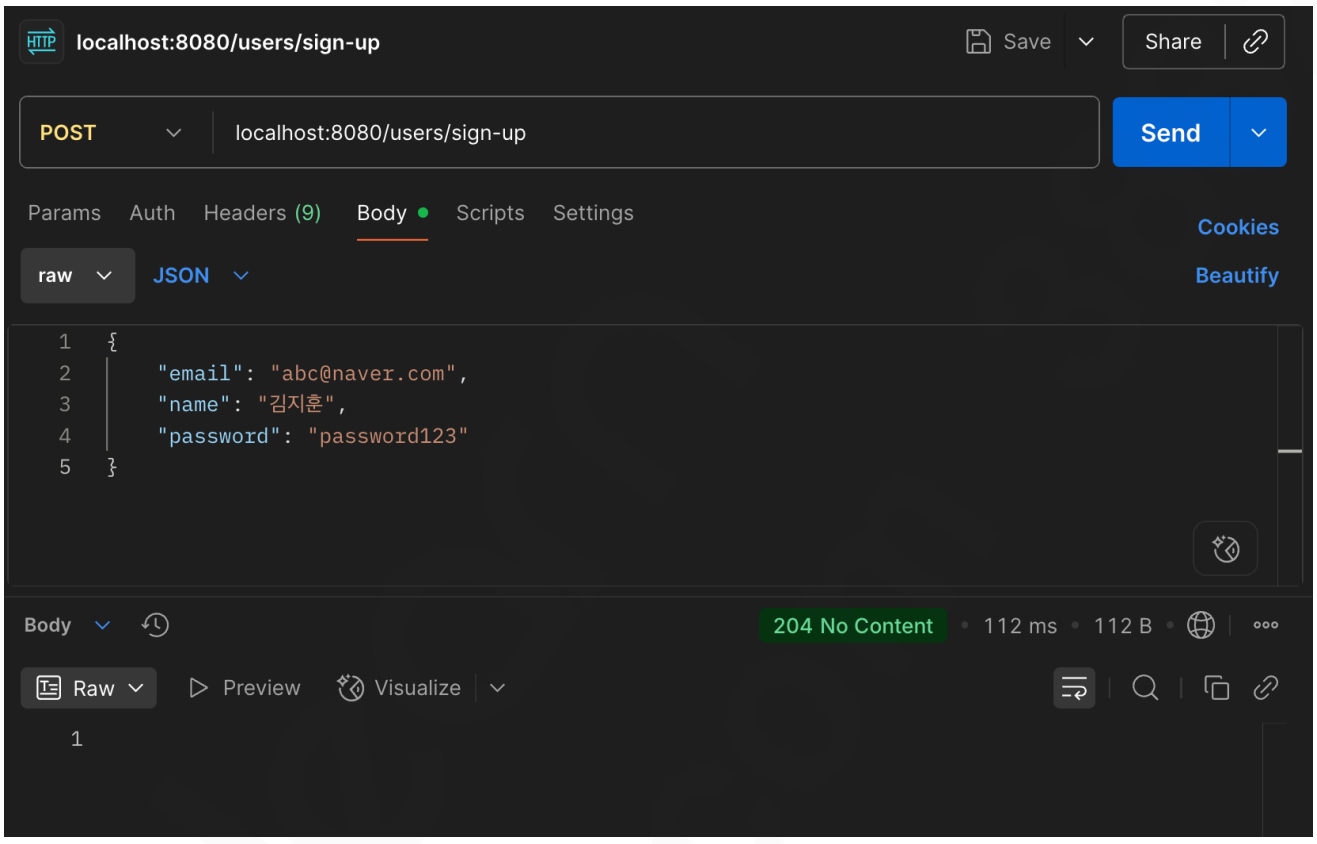
   controller/UserController

```
@RestController
@RequestMapping("/users")
public class UserController {
  private final UserService userService;

  public UserController(UserService userService) {
    this.userService = userService;
  }

    ...

  @GetMapping()
  public ResponseEntity<List<UserResponseDto>> getUsersByIds(
      @RequestParam List<Long> ids
  ) {
    List<UserResponseDto> userResponseDtos = userService.getUsersByIds(ids);
    return ResponseEntity.ok(userResponseDtos);
  }
}
```

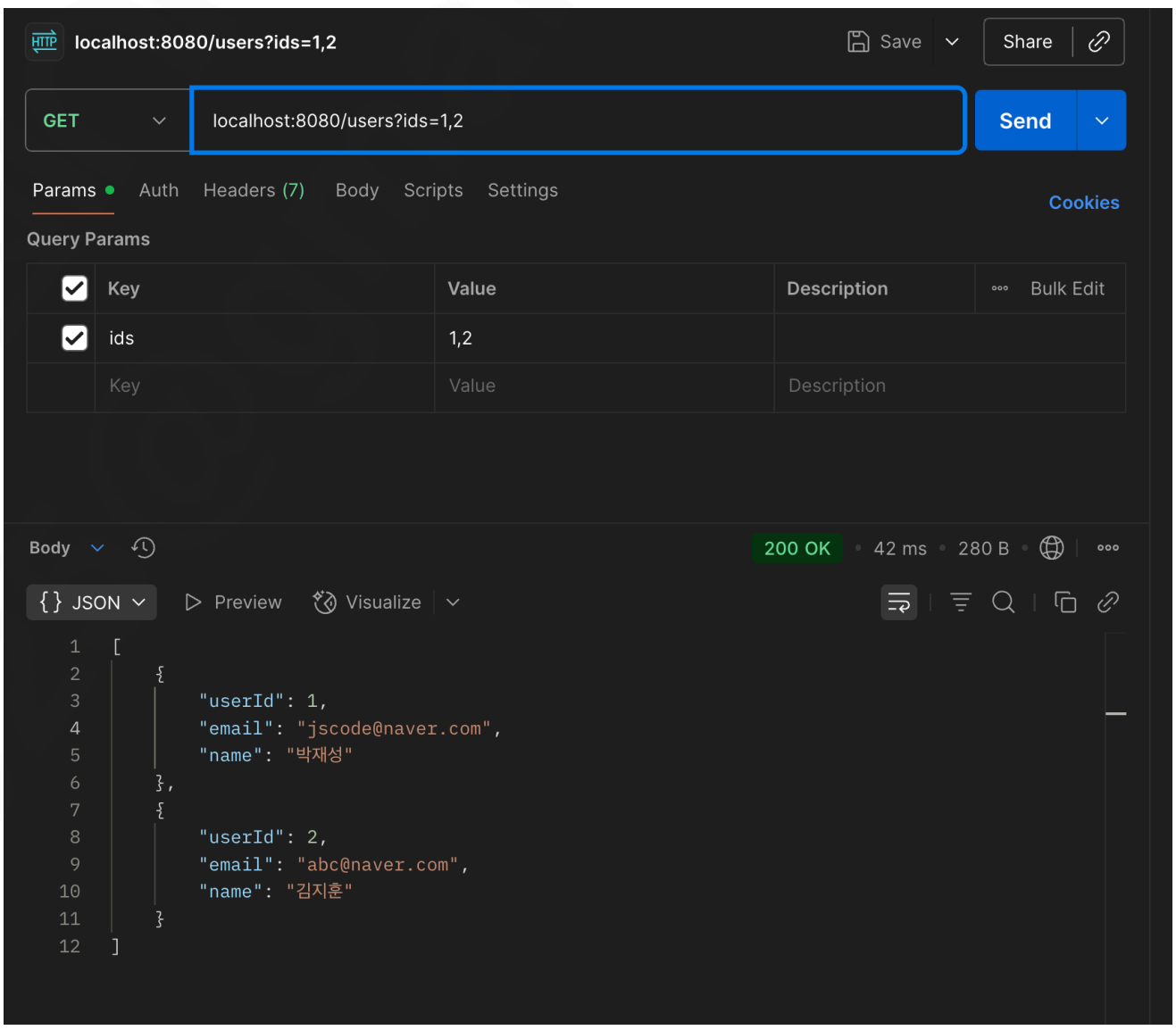
3. **서버 실행시킨 뒤 API가 잘 작동하는 지 확인하기**

   a. **회원가입 하기**

      2명의 사용자 정보를 조회해보기 위해 회원가입을 한 번 더 해주자.



   b. **사용자 목록 조회하기**



이번 강의에서 board-service가 사용할 사용자 목록 조회 API를 만들었다. 그럼 다음 강의에서 board-service의 **전체 게시글 조회 API**를 마저 만들어보자.

@kfc3882.com
khb302500

[실습] 게시글 전체 조회 API 만들기 - 2

✅ 게시글 전체 조회 API 만들기

> board-service에서 아래 코드 작성하기

1. **UserClient 로직 작성하기**

   client/UserClient

```
@Component
public class UserClient {

  private final RestClient restClient;

public UserClient(
    @Value("${client.user-service.url}") String userServiceUrl
) {
  this.restClient = RestClient.builder()
      .baseUrl(userServiceUrl)
      .build();
}

...

public List<UserResponseDto> fetchUsersByIds(List<Long> ids) {
  try {
    return this.restClient.get()
        .uri(uriBuilder -> uriBuilder
            .path("/users")
            .queryParam("ids", ids)
            .build()
        )
        .retrieve()
        .body(new ParameterizedTypeReference<>() {});
  } catch (RestClientException e) {
    // 로깅 : 예외 발생 시 로그를 남겨 문제를 파악할 수 있게 해야 함
      // log.error(...)
      return Collections.emptyList();
    }
  }
}
```

2. **Service 로직 작성하기**

   service/BoardService

```
@Service
public class BoardService {
  private final BoardRepository boardRepository;
  private final UserClient userClient;

  public BoardService(BoardRepository boardRepository, UserClient userClient) {
    this.boardRepository = boardRepository;
```

```
  this.userClient = userClient;
}

  ...

// 게시글 전체 조회
public List<BoardResponseDto> getBoards() {
  List<Board> boards = boardRepository.findAll();

  // userId 목록 추출
  List<Long> userIds = boards.stream()
      .map(Board::getUserId)
      .distinct() // 중복 제거
      .toList();

  // user-service로부터 사용자 정보 불러오기
  List<UserResponseDto> userResponseDtos = userClient.fetchUsersByIds(userIds);

    // userId를 Key로 하는 Map 생성
    Map<Long, UserDto> userMap = new HashMap<>();
    for (UserResponseDto userResponseDto : userResponseDtos) {
      Long userId = userResponseDto.getUserId();
      String name = userResponseDto.getName();
      userMap.put(userId, new UserDto(userId, name));
    }

    // 게시글 정보와 사용자 정보를 조합해서 BoardResponseDto 만들기
    return boards.stream()
        .map(board -> new BoardResponseDto(
            board.getBoardId(),
            board.getTitle(),
            board.getContent(),
            userMap.get(board.getUserId()) // 맵에서 UserDto 가져오기
        ))
        .toList();
  }
}
```

3. **Controller 로직 작성하기**

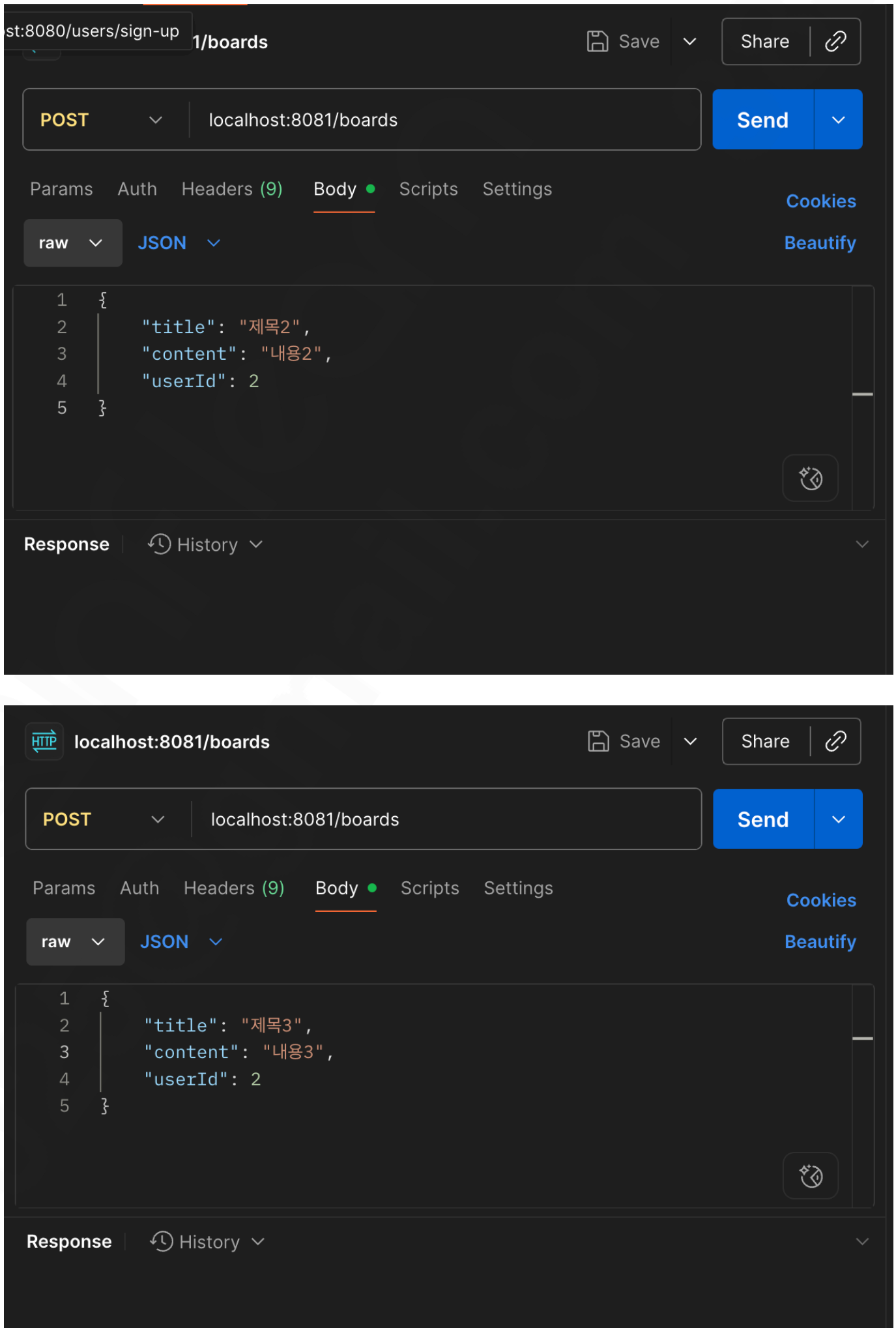
controller/BoardController

```
@RestController
@RequestMapping("/boards")
public class BoardController {
  private final BoardService boardService;

  public BoardController(BoardService boardService) {
    this.boardService = boardService;
  }

    ...

  @GetMapping()
  public ResponseEntity<List<BoardResponseDto>> getBoards() {
    List<BoardResponseDto> boardResponseDtos = boardService.getBoards();
    return ResponseEntity.ok(boardResponseDtos);
```

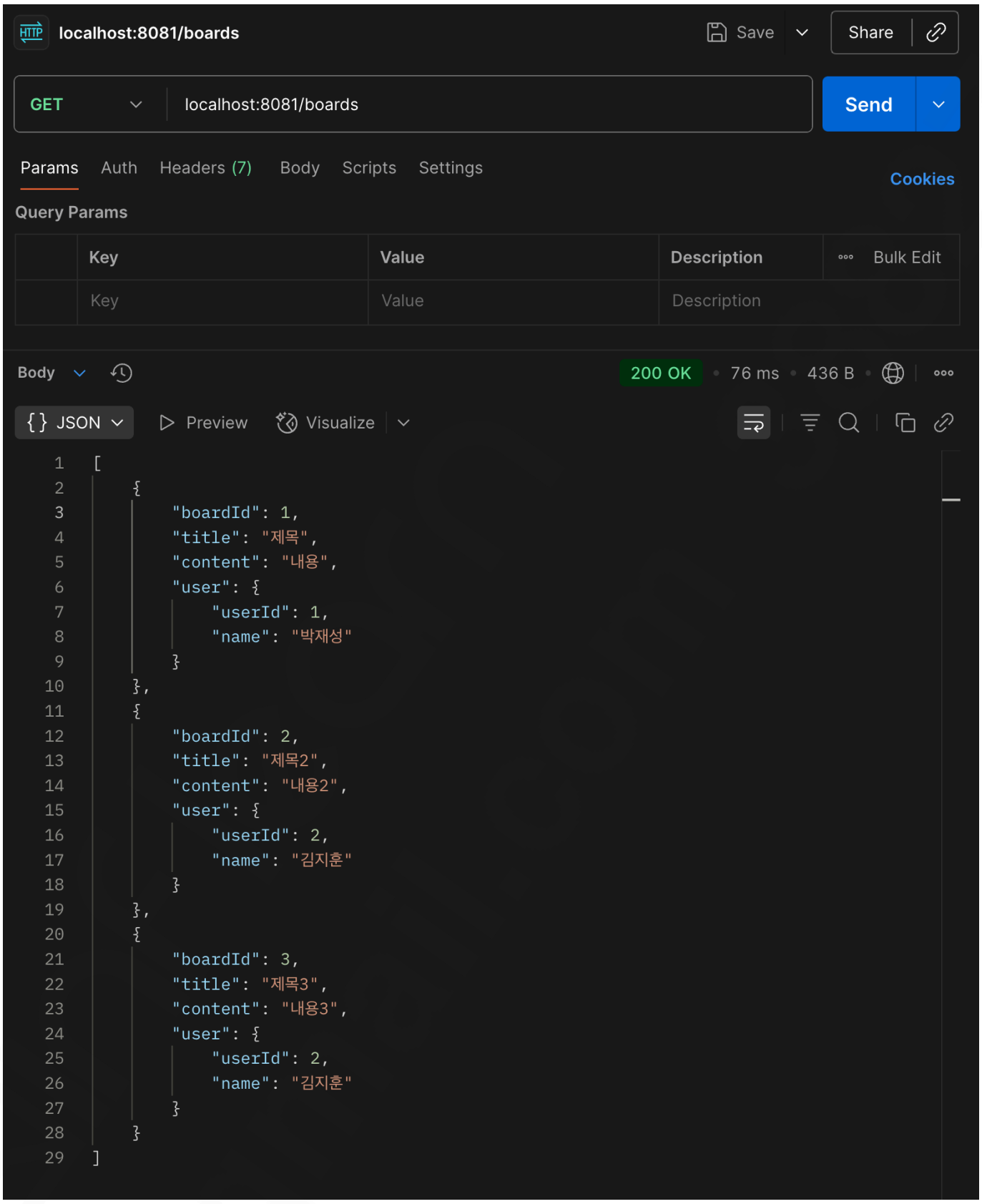
```
  }
}
```

4. **user-service, board-service 서버 실행시키기**

5. **테스트를 위한 게시글 데이터 2개 추가하기**



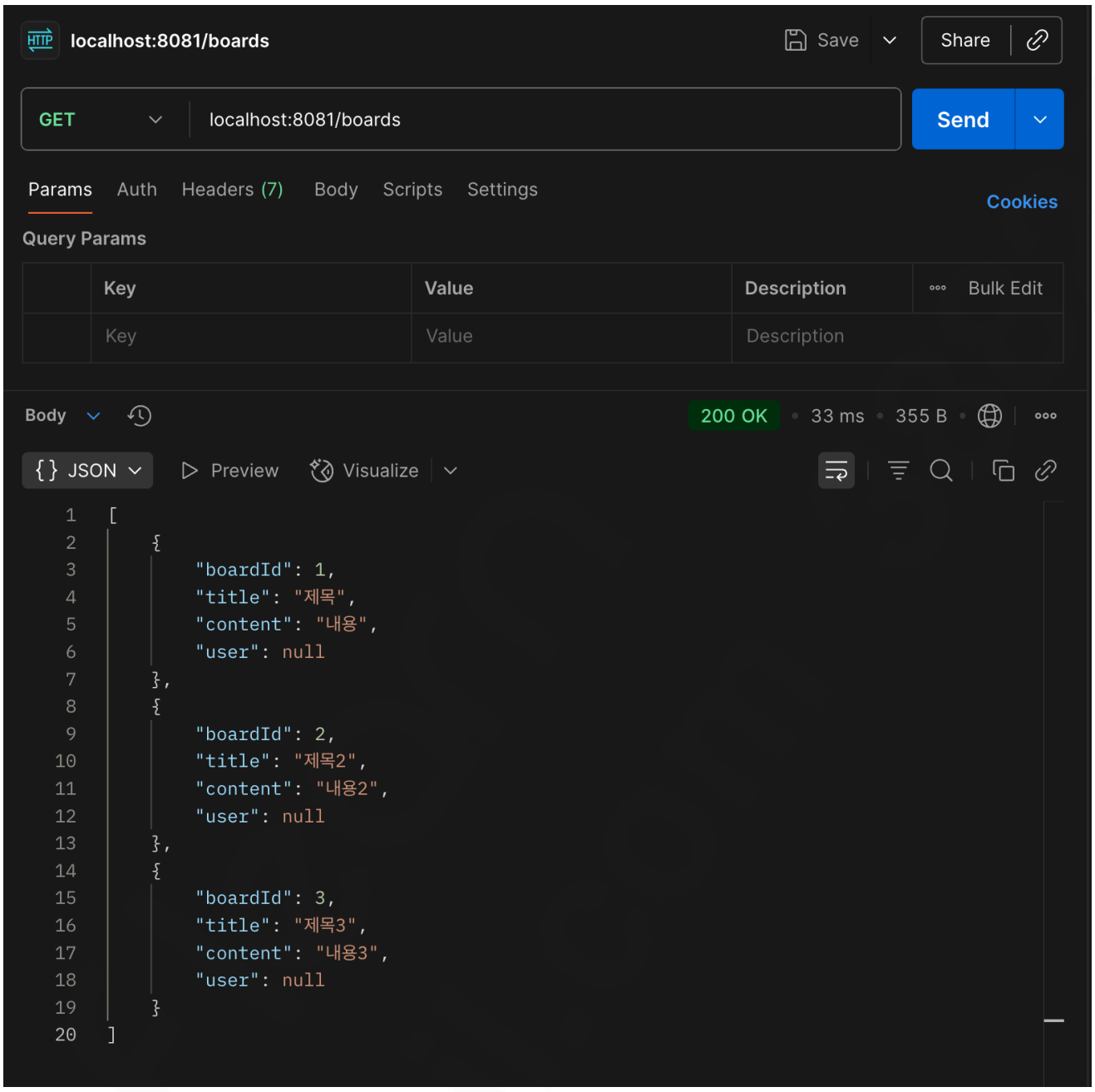
6. **잘 작동하는 지 API 요청 보내보기**
   a. **user-service, board-serivce 서버가 정상적으로 작동할 때의 API 호출**



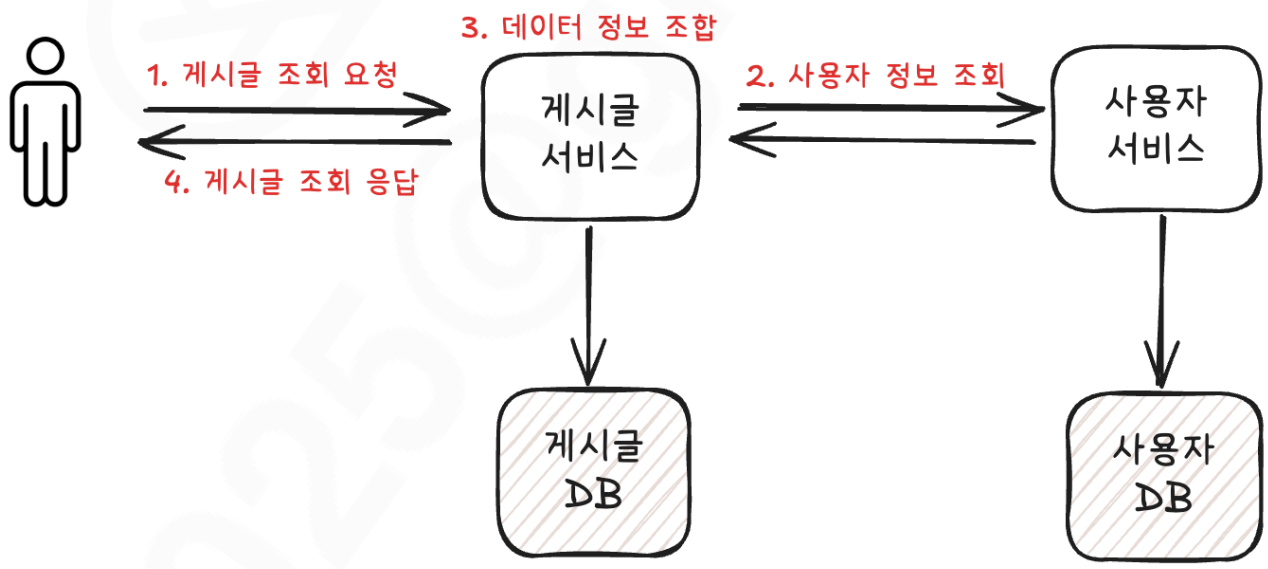
b. **user-service 서버가 장애났을 때의 API 호출**

user-service 서버가 장애난 걸 가정하기 위해, user-service 서버를 종료시키고 API 호출을 날려보자.

khb3025



✅ 정리



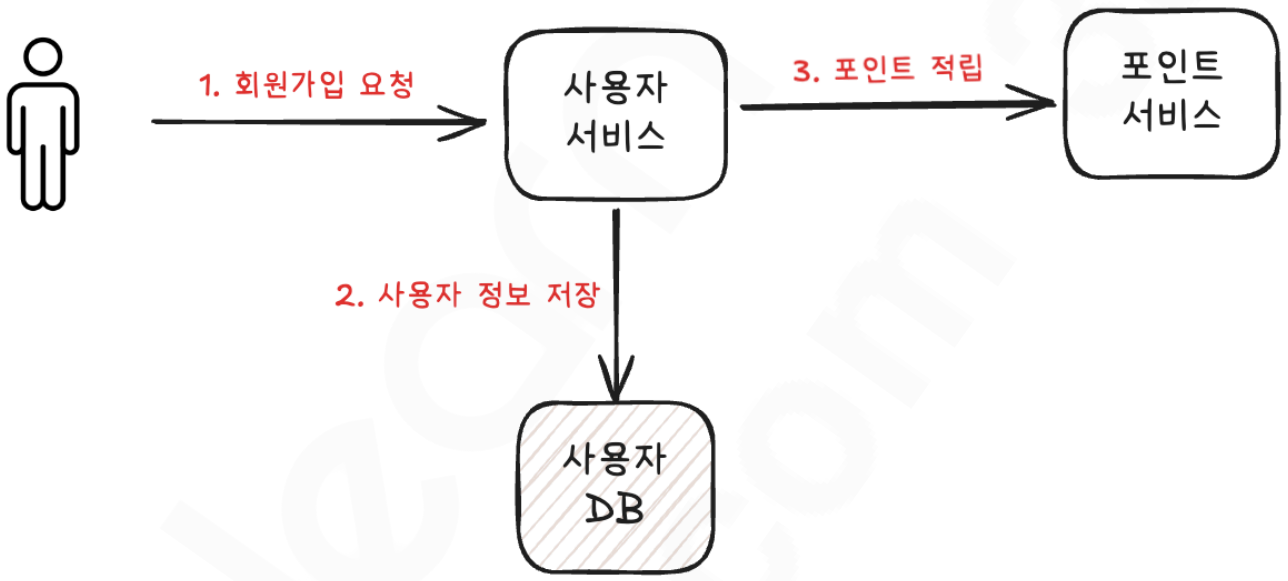
1. 게시글 전체 조회 API 주소(ex. `GET localhost:8081/boards`)로 요청을 보낸다.
2. 게시글 서비스에서 사용자 정보가 필요해서, 사용자 서비스(ex. `localhost:8080/users?ids=1,2`)로부터 데이터를 조회해온다.
3. 사용자 서비스로부터 받아온 사용자 정보를 활용해 응답할 데이터를 조합한다.
4. 사용자에게 응답한다.

추가 요구사항

✅ 추가 요구사항

1. **회원가입 API 수정하기**

   아래 그림과 같이 작동할 수 있도록 회원가입 API를 수정하자. 그리고 포인트 서비스를 새로 만들어 포인트 적립 API를 추가하자.



- **포인트 적립 API**

  Request

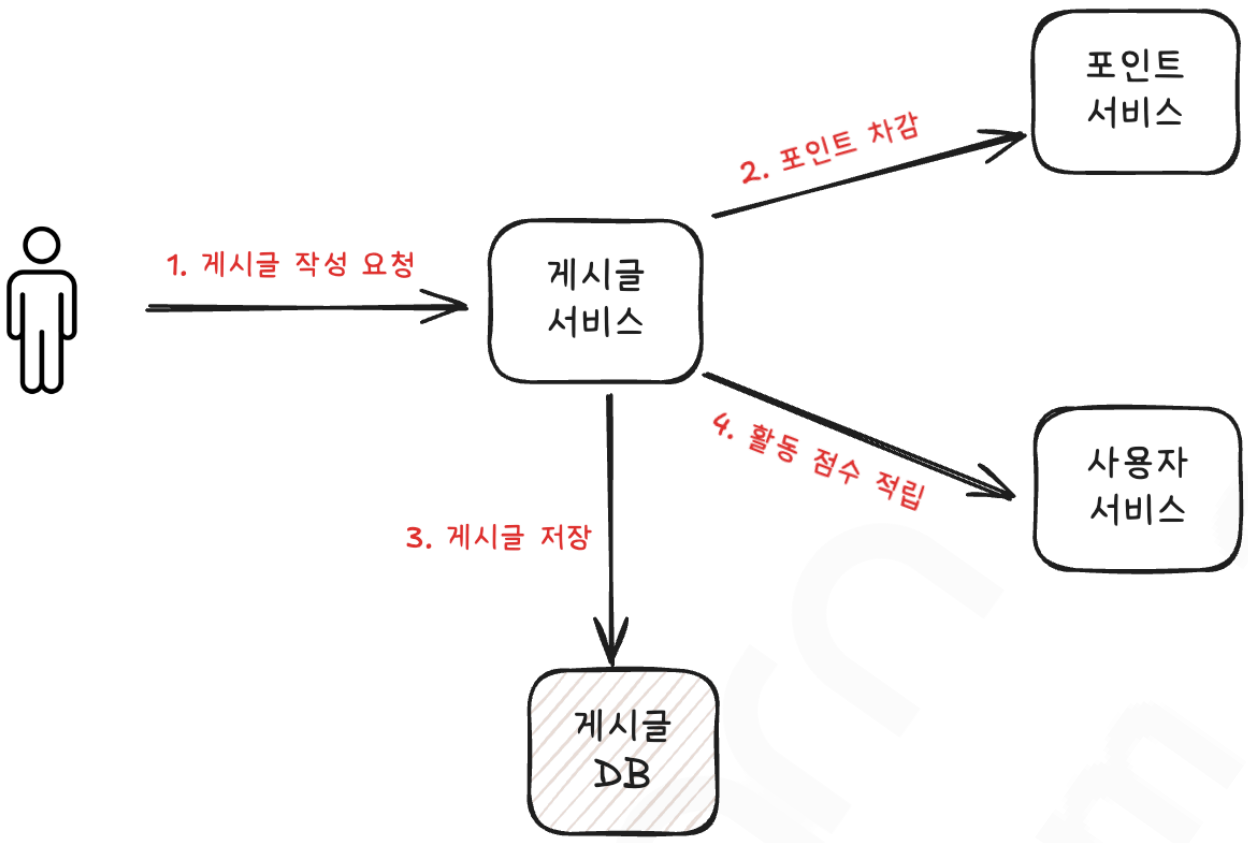
  ```
  {
    "userId": 1,
    "amount": 100 // 적립할 포인트
  }
  ```

  Response

  ```
  (204 No Contnet로 응답하기)
  ```

2. **게시글 작성 API 수정하기**

   아래 그림과 같이 작동할 수 있도록 게시글 작성 API 로직을 수정하자. 그리고 포인트 서비스에 포인트 차감 API를 추가하고, 사용자 서비스에 활동 점수 적립 API를 추가하자.



- **포인트 차감 API**

  Request

  ```
  {
    "userId": 1,
    "amount": 100 // 차감할 포인트
  }
  ```

  Response

  ```
  (204 No Contnet로 응답하기)
  ```

- **활동 점수 증가 API**

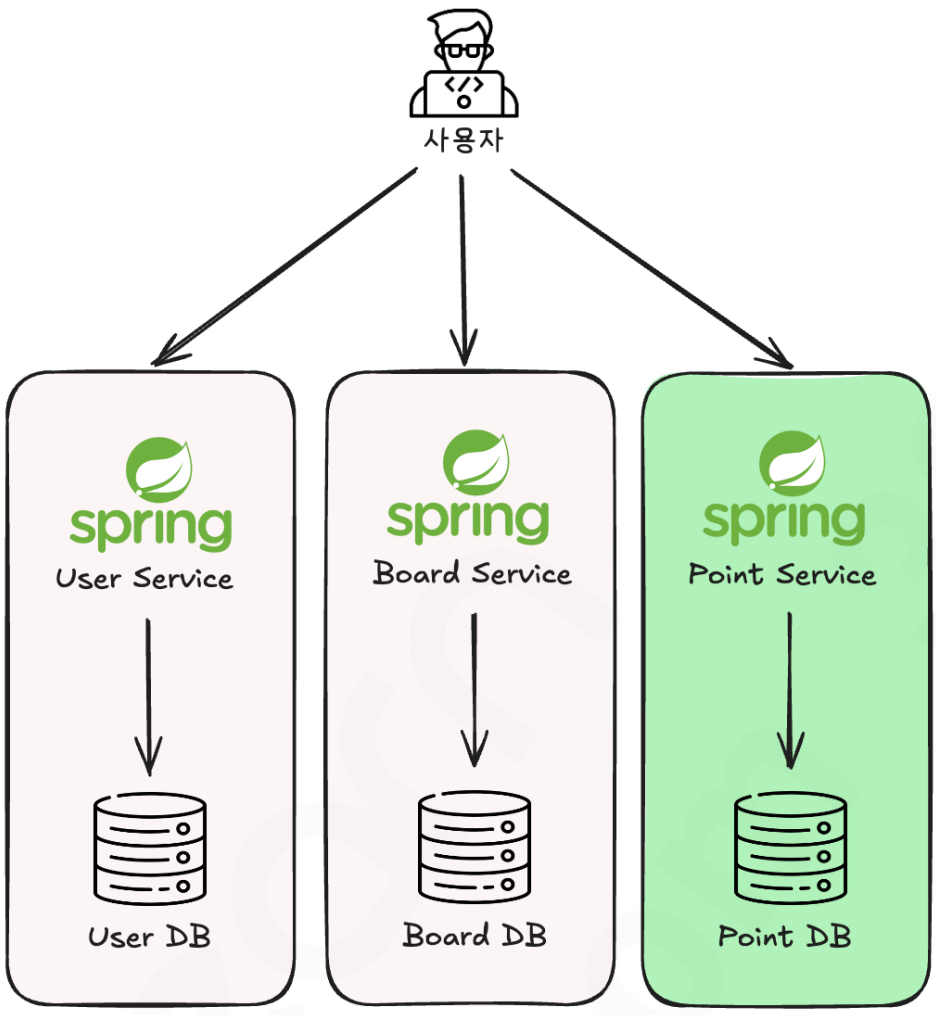
  Request

  ```
  {
    "userId": 1,
    "score": 10 // 활동 점수
  }
  ```

  Response

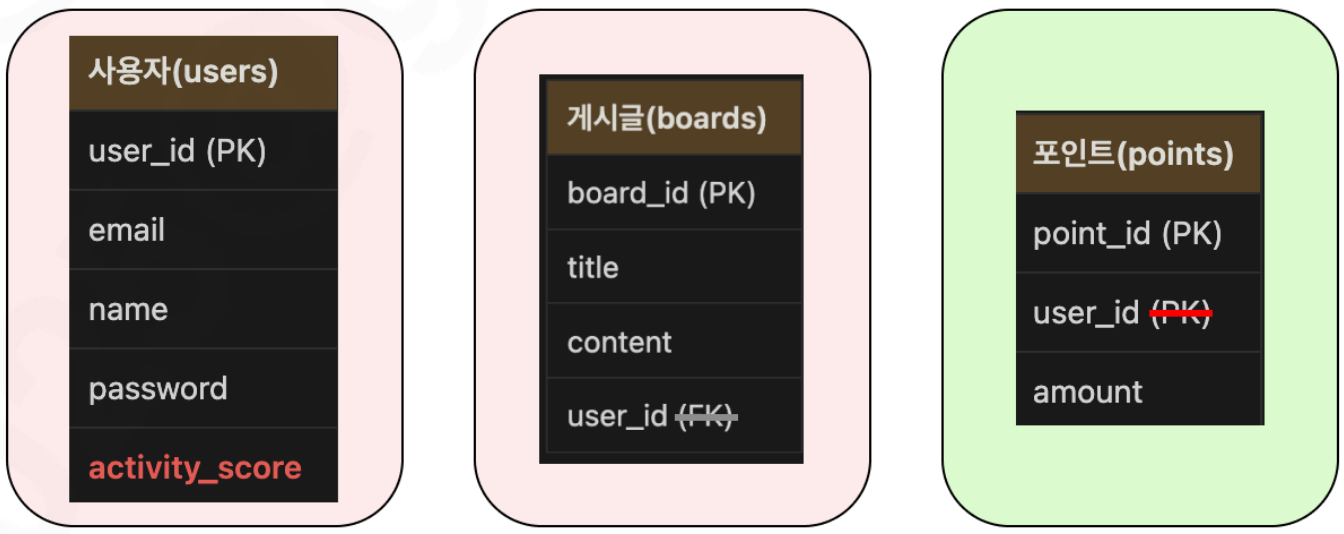
  ```
  (204 No Contnet로 응답하기)
  ```

✅ 프로젝트 아키텍처



- 게시글을 작성하면 포인트를 적립할 수 있게 만들기 위해, MSA 구조에서 Point Service를 추가로 구축하자.
- 프로젝트의 심플함을 위해 Point Service를 구현할 때 Spring Boot로 구축하고, DB는 MySQL로 구축할 것이다.
- DB 서버를 간편하게 띄우기 위해 Docker를 사용할 것이다.
- 실무에서 쓸 MSA를 구축하려면 각 Service와 DB를 전부 독립적인 서버에 실행시켜야 한다. 하지만 프로젝트의 심플함을 위해 로컬 컴퓨터 한 대에서 포트 번호만 다르게 해서 실행시킬 것이다.

✅ DB 설계



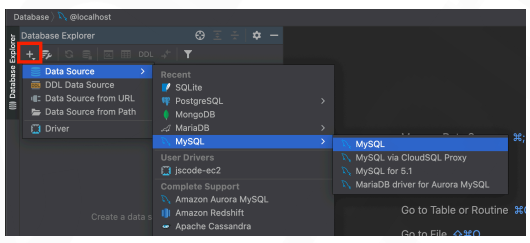
🎯 [실습] Point Microservice 만들기 - 1

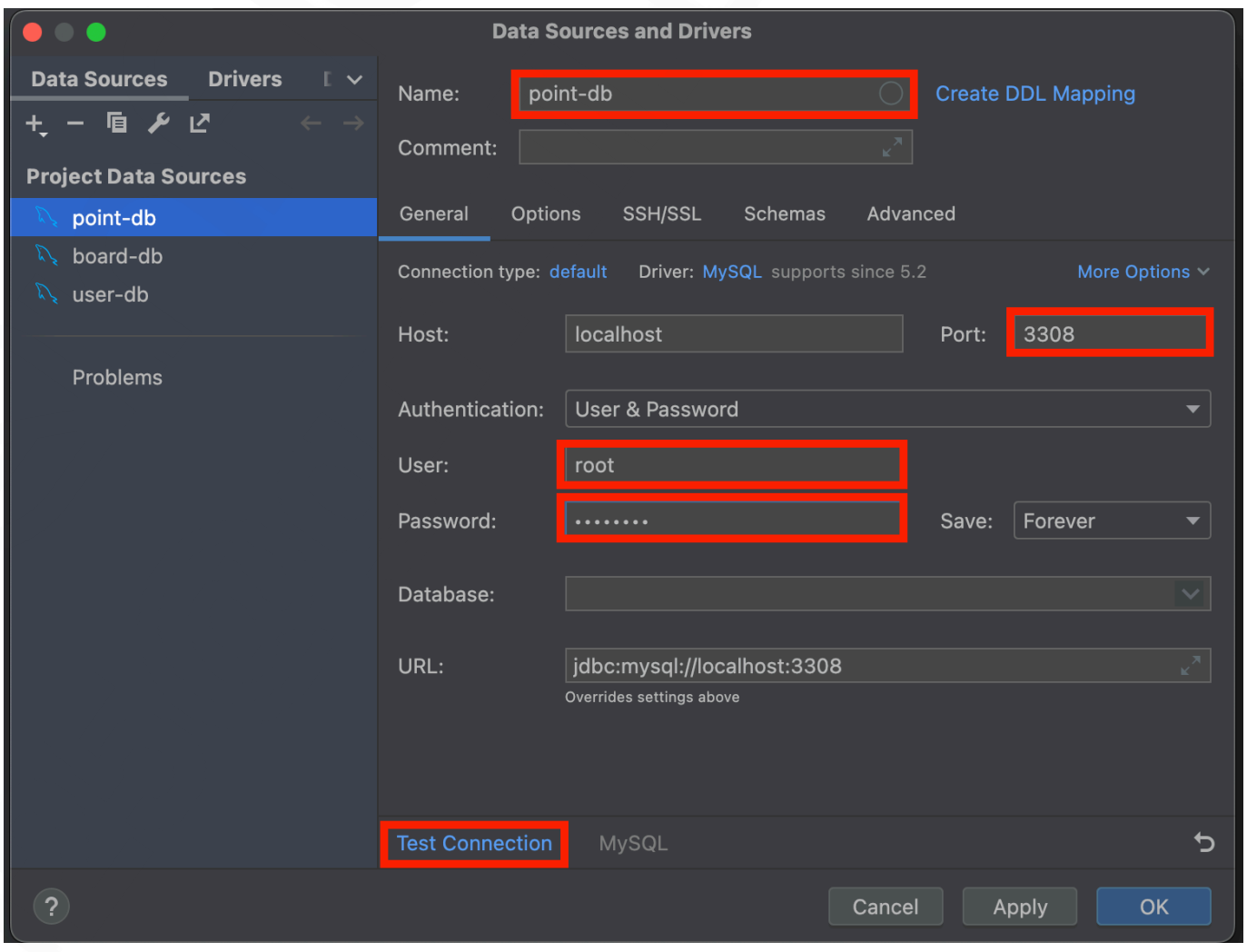
✅ Point Microservice가 사용할 DB 띄우기

1. **MySQL 서버 띄우기**

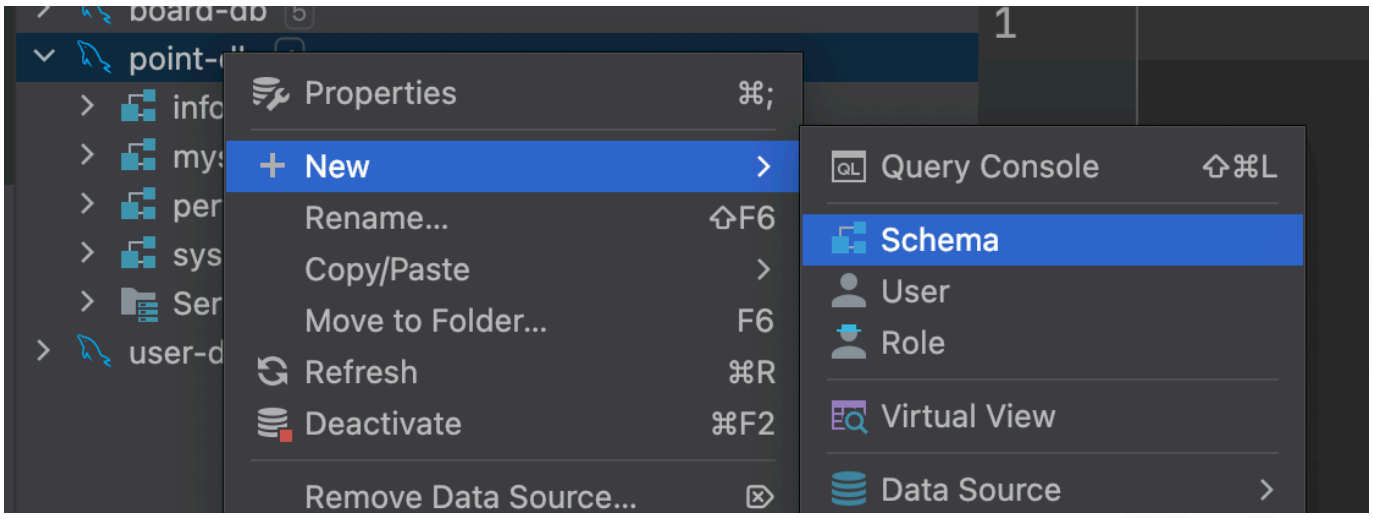
```
$ docker run -e MYSQL_ROOT_PASSWORD=password -p 3308:3306 -d mysql

# 잘 띄워졌는 지 확인하기
$ docker ps
```

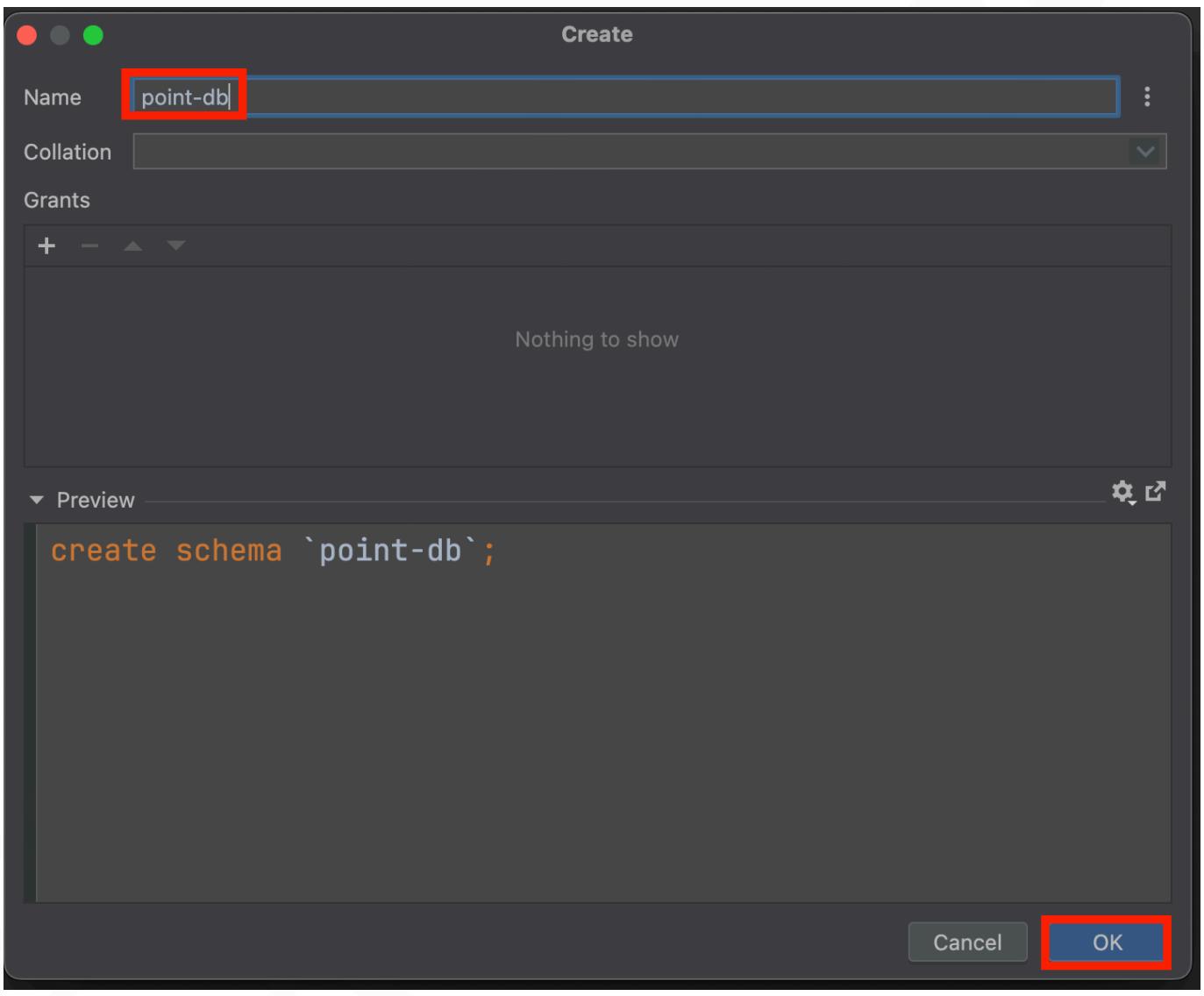
2. **GUI 툴로 MySQL 접근 되는 지 확인하기**





3. **데이터베이스 생성하기**





khb3025@gmail.com

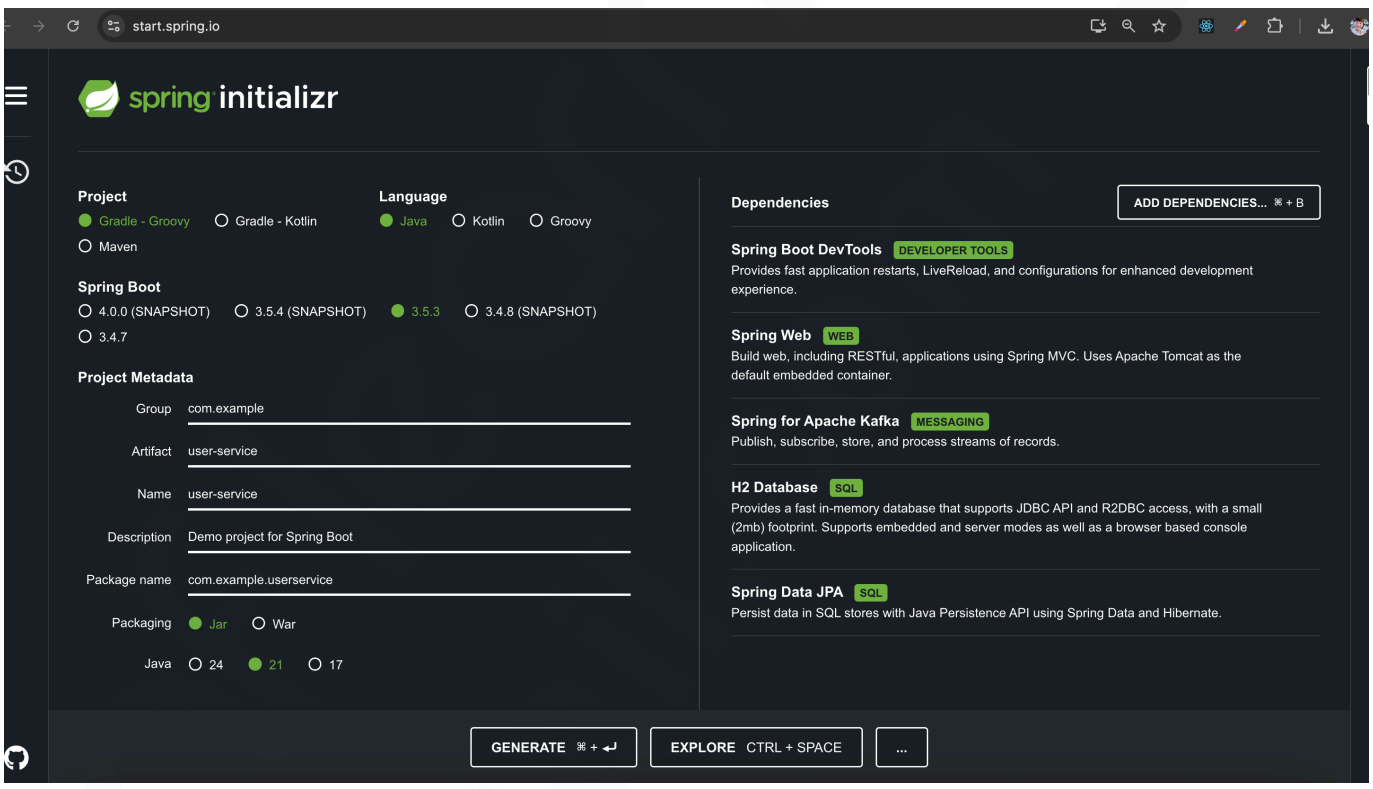
🎯 [실습] Point Microservice 만들기 - 2

✅ Spring Boot 프로젝트 셋팅 & DB 연결

1. Spring Boot 프로젝트 셋팅

start.spring.io

https://start.spring.io/



- `Artifact`와 `Name`을 `point-service`라고 지어주자.
- `Package name`을 Java 컨벤션에 맞게 `pointservice`라고 지어주자.
- Java 21 버전을 선택하자. 아래 과정을 Java 21 버전을 기준으로 진행할 예정이다.
- Dependencies는 `Spring Boot DevTools`, `Spring Web`, `MySQL Driver`, `Spring Data JPA`를 선택하자.

2. application.yml 작성하기

> 이 프로젝트에서는 `application.properties`를 지우고 `application.yml`을 생성했다.

```
server:
  port: 8082

spring:
  datasource:
    url: jdbc:mysql://localhost:3308/point-db # DB 주소
    username: root # DB 계정
    password: password # DB 비밀번호
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

3. **Spring Boot 서버 실행시켜서 DB 연결 제대로 되는 지 확인하기**

   Spring Boot 실행시켰을 때 아래와 같은 에러 안 뜨고 서버가 실행되면, 정상적으로 DB가 잘 연결된 것이다.

```
org.springframework.beans.factory.BeanCreationException Create breakpoint : Error creating bean with name
 'entityManagerFactory' defined in class path resource
 [org/springframework/boot/autoconfigure/orm/jpa/HibernateJpaConfiguration.class]: Unable to create
 requested service [org.hibernate.engine.jdbc.env.spi.JdbcEnvironment] due to: Unable to determine
 Dialect without JDBC metadata (please set 'jakarta.persistence.jdbc.url' for common cases or 'hibernate
 .dialect' when a custom Dialect implementation must be provided)
  at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.initializeBean
   (AbstractAutowireCapableBeanFactory.java:1826) ~[spring-beans-6.2.10.jar:6.2.10]
  at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.doCreateBean
   (AbstractAutowireCapableBeanFactory.java:607) ~[spring-beans-6.2.10.jar:6.2.10]
  at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.createBean
   (AbstractAutowireCapableBeanFactory.java:529) ~[spring-beans-6.2.10.jar:6.2.10]
  at org.springframework.beans.factory.support.AbstractBeanFactory.lambda$doGetBean$0(AbstractBeanFactory
   .java:339) ~[spring-beans-6.2.10.jar:6.2.10]
  at org.springframework.beans.factory.support.DefaultSingletonBeanRegistry.getSingleton
```

✅ 포인트 적립 API 만들기

1. **Point 엔티티 생성하기**

   domain/Point

```
@Entity
@Table(name = "points")
public class Point {

  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long pointId;

  private Long userId;

  private int amount;

  public Point() {
  }

  public Point(Long userId, int amount) {
   this.userId = userId;
   this.amount = amount;
  }

  public Long getPointId() {
   return pointId;
  }

  public Long getUserId() {
   return userId;
  }

  public int getAmount() {
   return amount;
  }

  // 포인트 적립 (나중에 Service 로직 짤 때 사용)
  public void addAmount(int amount) {
```

```
    this.amount += amount;
  }

  // 포인트 차감 (나중에 Service 로직 짤 때 사용)
  public void deductAmount(int amount) {
    this.amount -= amount;
  }
}
```

2. **Reposiotry 만들기**

   domain/PointRepoistory

```
public interface PointRepository extends JpaRepository<Point, Long> {
    // 나중에 Service 로직 짤 때 사용
    Optional<Point> findByUserId(Long userId);
}
```

3. **Controller 만들기**

   controller/PointController

```
@RestController
@RequestMapping("/points")
public class PointController {
  private final PointService pointService;

  public PointController(PointService pointService) {
    this.pointService = pointService;
  }

  @PostMapping("add")
  public ResponseEntity<Void> addPoints(
      @RequestBody AddPointRequestDto addPointRequestDto
  ) {
    pointService.addPoints(addPointRequestDto);
    return ResponseEntity.noContent().build();
  }

  @PostMapping("deduct") // deduct : 빼다
  public ResponseEntity<Void> deductPoints(
      @RequestBody DeductPointRequestDto deductPointRequestDto
  ) {
    pointService.deductPoints(deductPointRequestDto);
    return ResponseEntity.noContent().build();
  }
}
```

4. **DTO 만들기**

   dto/AddPointRequestDto

```
public class AddPointRequestDto {
  private Long userId;
  private int amount;
```

```
  public Long getUserId() {
    return userId;
  }

  public int getAmount() {
    return amount;
  }
}
```

dto/DeductPointRequestDto

```
public class DeductPointRequestDto {
  private Long userId;
  private int amount;

  public Long getUserId() {
    return userId;
  }

  public int getAmount() {
    return amount;
  }
}
```

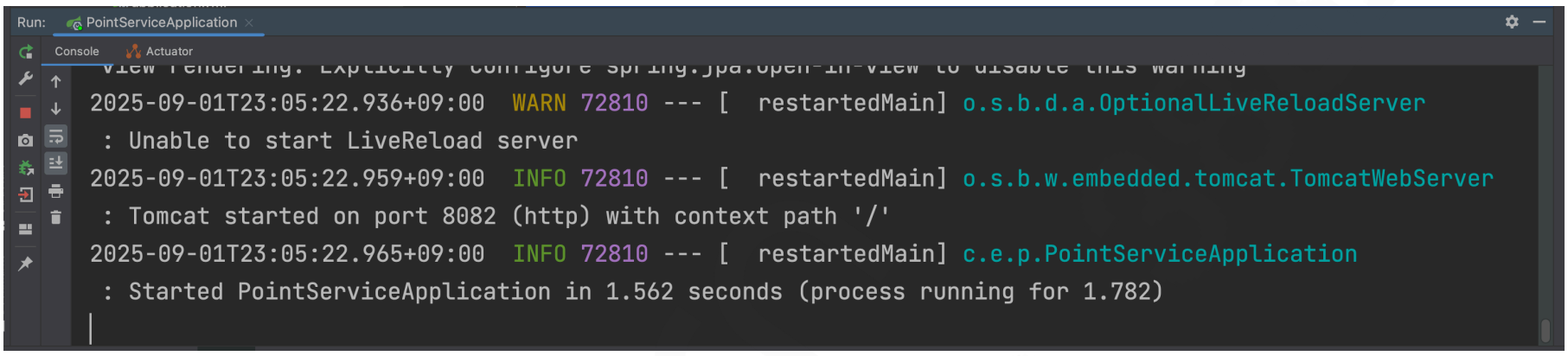
5. **Service 만들기**

   service/PointService

```
@Service
public class PointService {
  private final PointRepository pointRepository;

  public PointService(PointRepository pointRepository) {
    this.pointRepository = pointRepository;
  }

@Trasnsactional
public void addPoints(AddPointRequestDto addPointRequestDto) {
  Point point = pointRepository.findByUserId(addPointRequestDto.getUserId())
      .orElseGet(() -> new Point(addPointRequestDto.getUserId(), 0));

  point.addAmount(addPointRequestDto.getAmount());

  pointRepository.save(point);
}

@Transactional
public void deductPoints(DeductPointRequestDto deductPointRequestDto) {
  Point point = pointRepository.findByUserId(deductPointRequestDto.getUserId())
      .orElseThrow(() -> new IllegalArgumentException("사용자의 포인트 정보를 찾을 수 없습니다."));

  point.deductAmount(deductPointRequestDto.getAmount());
```

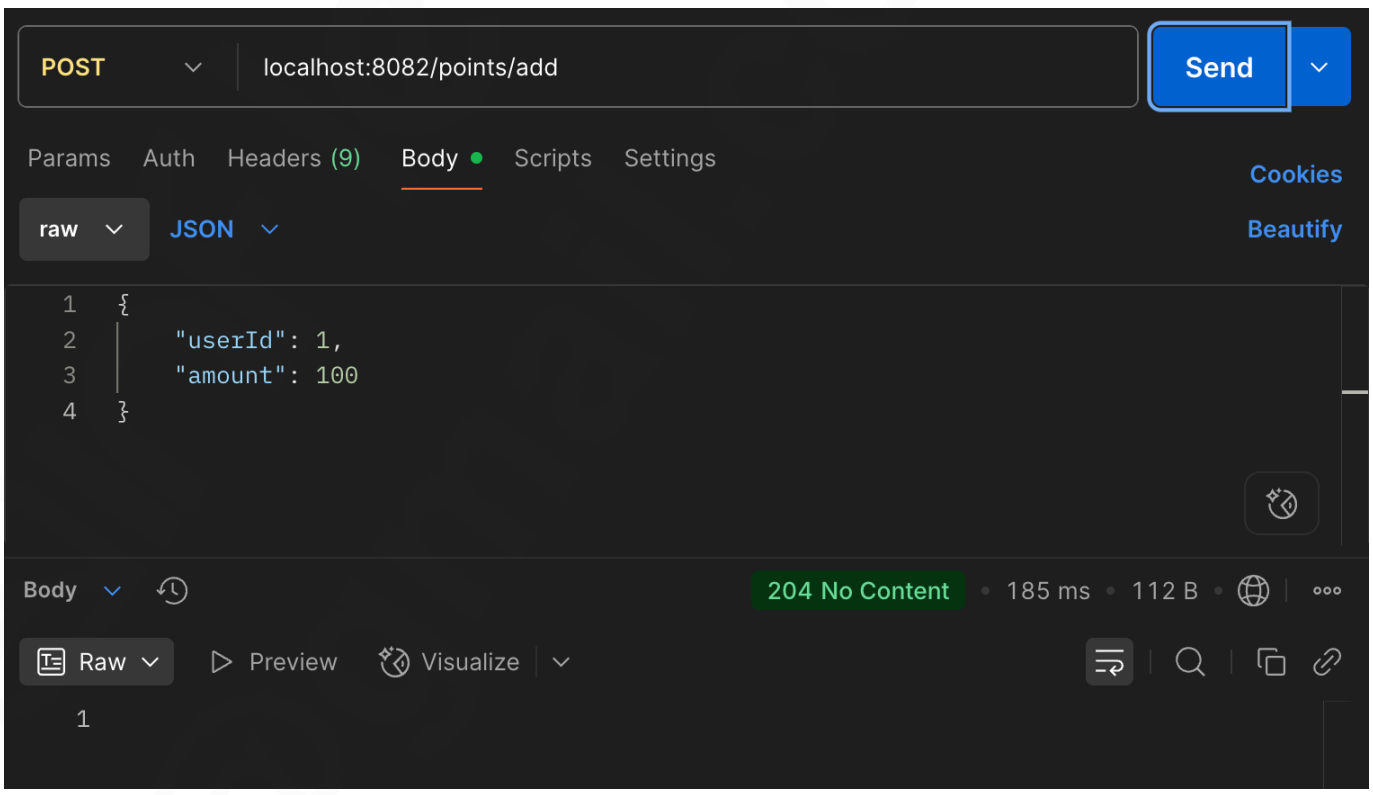
```
    pointRepository.save(point);
  }
}
```

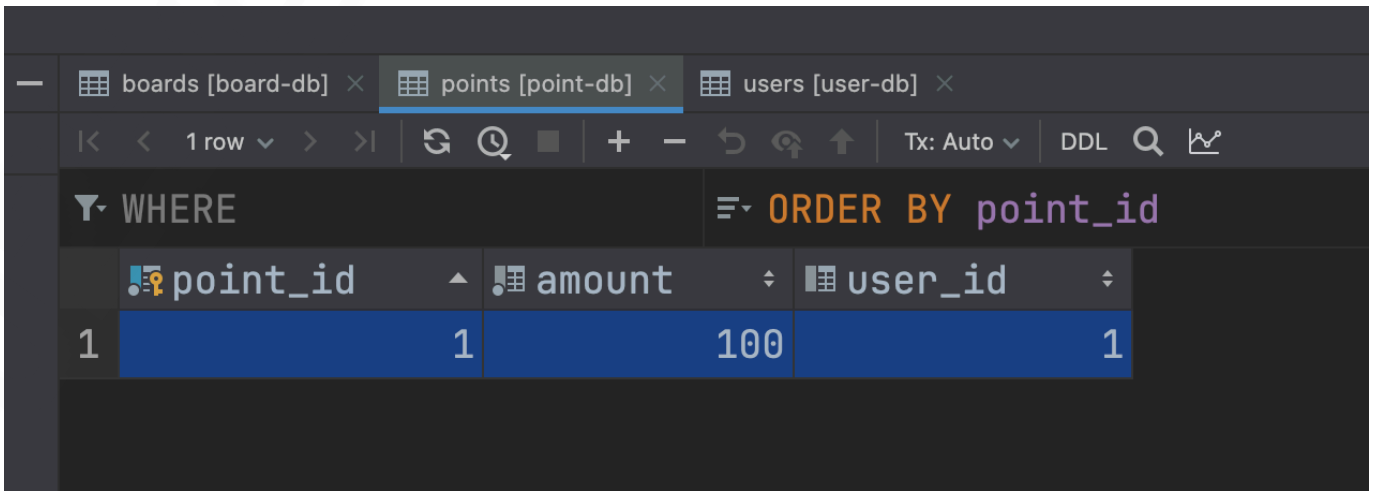
6. **서버 다시 실행시키기**



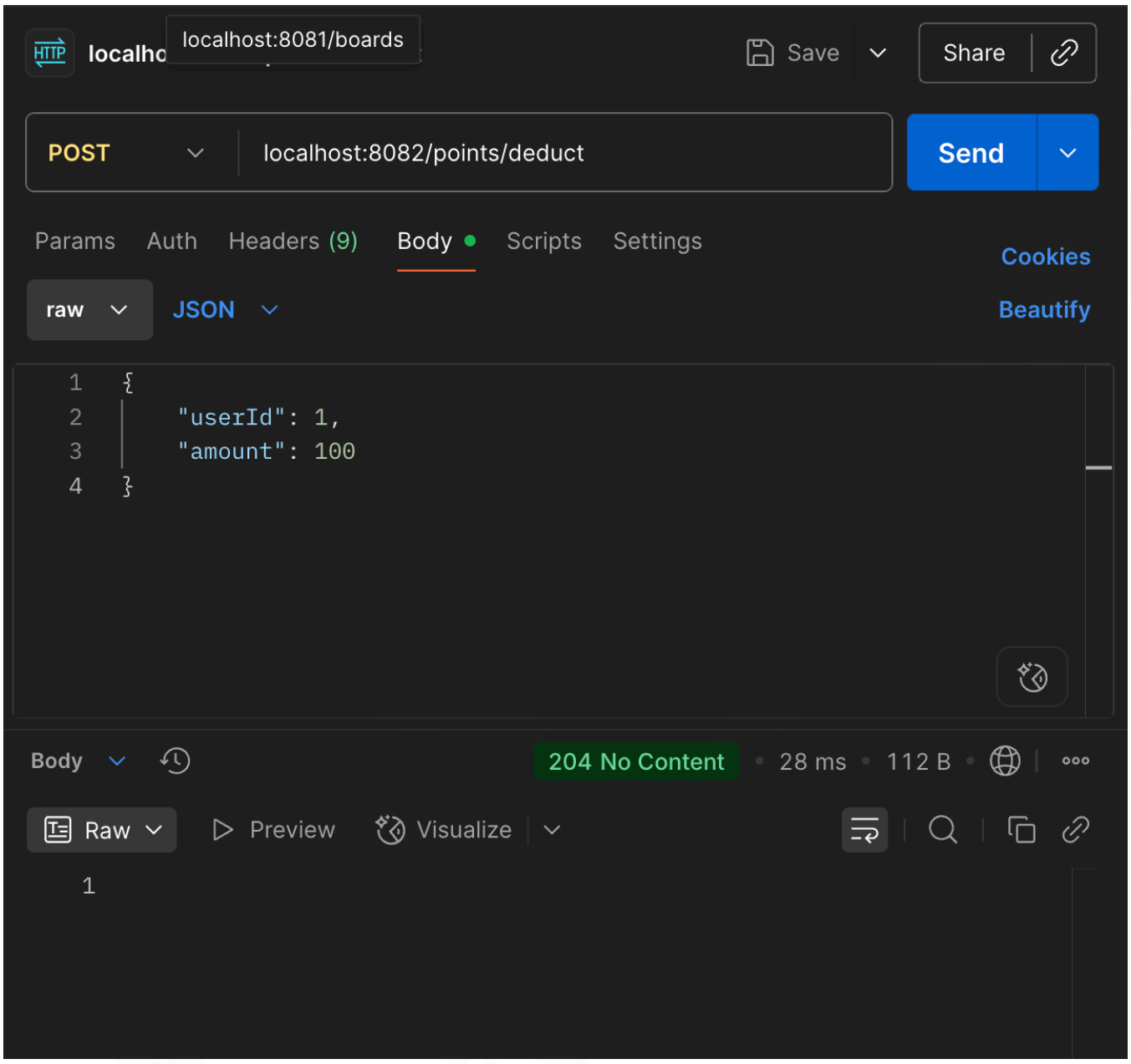
7. **API 잘 작동하는 지 테스트해보기**

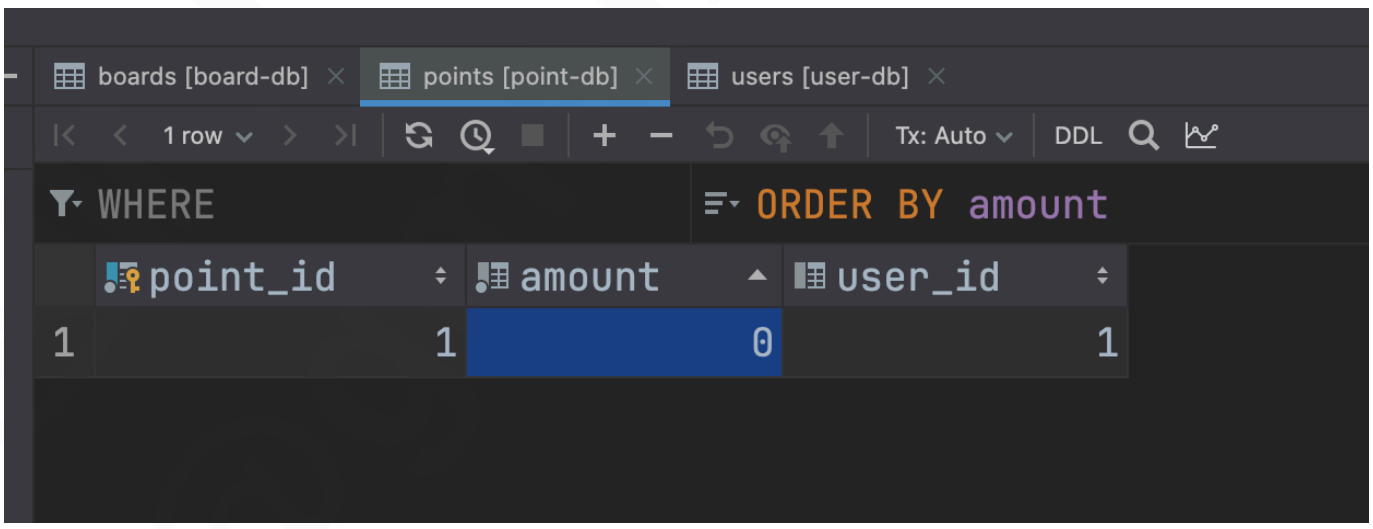
[포인트 적립 API 테스트]





[포인트 차감 API 테스트]


localhost:8081/boards
Save
Share
POST
localhost:8082/points/deduct
Send
Params
Auth
Headers (9)
Body
Scripts
Settings
Cookies
raw
JSON
Beautify
{
"userId": 1,
"amount": 100
}
Body
204 No Content
28 ms
112 B
Raw
Preview
Visualize
1

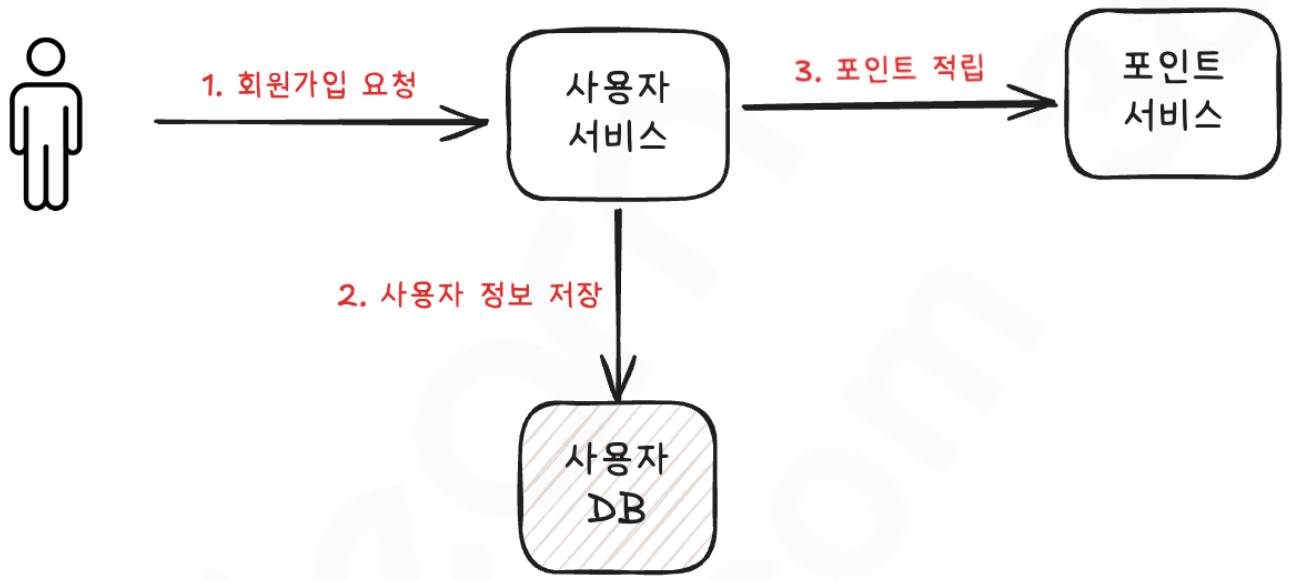

boards [board-db]
points [point-db]
users [user-db]
1 row
Tx: Auto
DDL
WHERE
ORDER BY amount
point_id
amount
user_id
1
1
0
1

khb3025

🎯 [실습] 회원 가입하면 포인트가 적립되도록 만들기

✅ 구조

아래 그림과 같이 **회원 가입**을 하면 **포인트가 적립**되도록 코드를 작성해보자.



✅ 코드 작성하기

> user-service에서 아래 코드 작성하기

1. **PointClient 코드 작성하기**

   포인트 서비스에 API 요청을 보내는 코드를 작성하자.

   application.yml

```
server:
  port: 8080

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/user-db
    username: root
    password: password
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true

client:
  point-service:
    url: http://localhost:8082
```

   dto/AddPointsRequestDto

```
public class AddPointsRequestDto {
  private Long userId;
  private int amount;

  public AddPointsRequestDto(Long userId, int amount) {
    this.userId = userId;
    this.amount = amount;
  }

  public Long getUserId() {
    return userId;
  }

  public int getAmount() {
    return amount;
  }
}
```

client/PointClient

```
@Component
public class PointClient {

  private final RestClient restClient;

  public PointClient(
      @Value("${client.point-service.url}") String pointServiceUrl
  ) {
    this.restClient = RestClient.builder()
        .baseUrl(pointServiceUrl)
        .build();
  }

  public void addPoints(Long userId, int amount) {
    AddPointsRequestDto addPointsRequestDto
        = new AddPointsRequestDto(userId, amount);
    this.restClient.post()
        .uri("/points/add")
        .contentType(MediaType.APPLICATION_JSON)
        .body(addPointsRequestDto)
        .retrieve()
        .toBodilessEntity();
  }
}
```

2. **Service 코드 작성하기**

   회원 가입 로직에 포인트 적립 로직을 추가해야 한다.

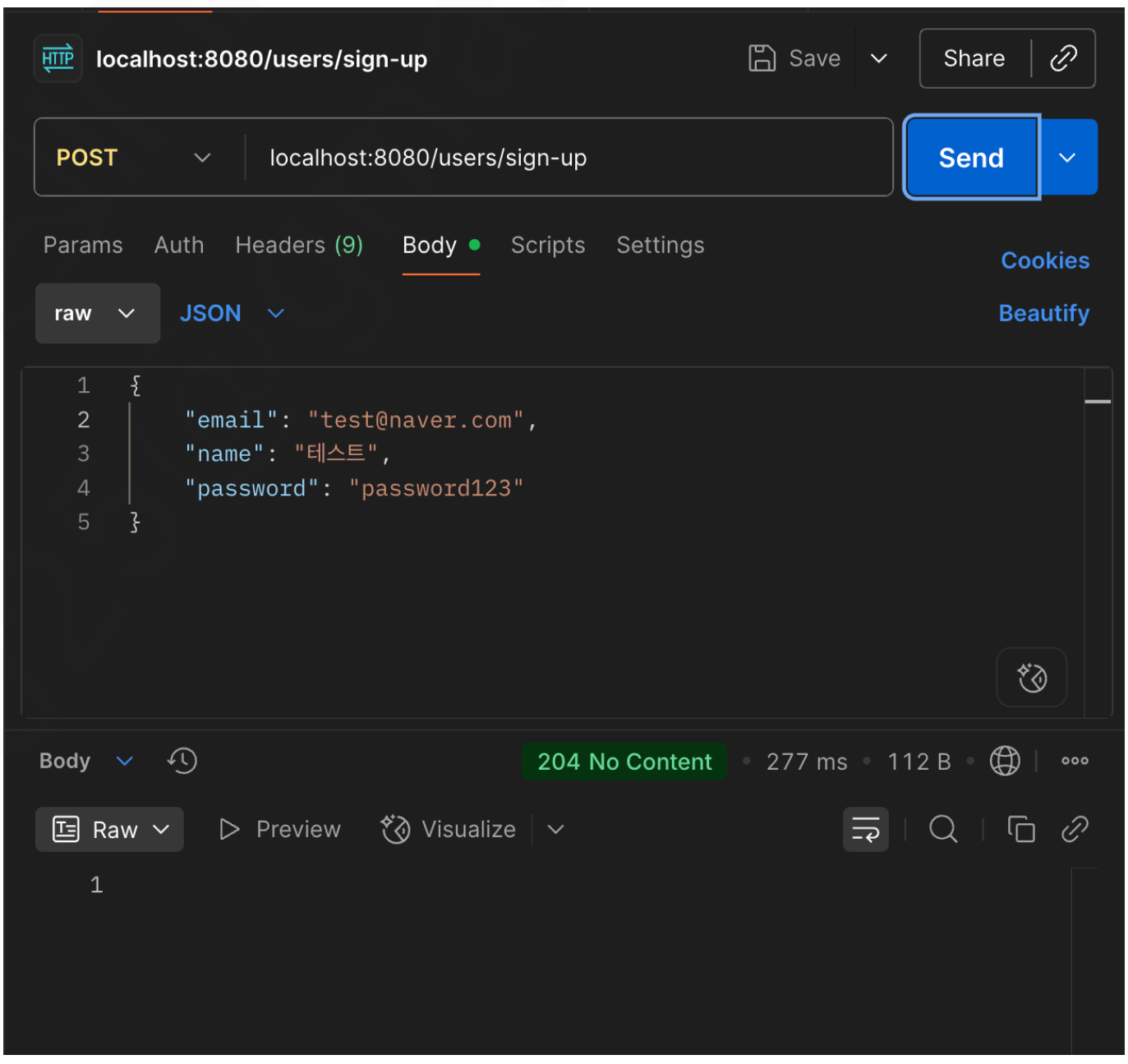
   service/UserService

```
@Service
public class UserService {
  private final UserRepository boardRepository;
  private final PointClient pointClient;
```

```
public UserService(UserRepository userRepository, PointClient pointClient) {
  this.userRepository = userRepository;
  this.pointClient = pointClient;
}

public void signUp(SignUpRequestDto signUpRequestDto) {
  User user = new User(
      signUpRequestDto.getEmail(),
      signUpRequestDto.getName(),
      signUpRequestDto.getPassword()
  );

  User savedUser = this.userRepository.save(user);

  // 회원가입하면 포인트 1000점 적립
  pointClient.addPoints(savedUser.getUserId(), 1000);
}

  ...

}
```

3. **서버 다시 실행시키기**

4. **서버 실행시켜서 API 테스트해보기**



3 rows | Tx: Auto | DDL

WHERE | ORDER BY

	user_id	email	name	password
1	1	jscode@naver.com	박재성	password123
2	2	abc@naver.com	김지훈	password123
3	4	test@naver.com	테스트	password123

users [user-db] | console [point-db] | points [point-db] | boards [board-db]

2 rows | Tx: Auto | DDL

WHERE | ORDER BY

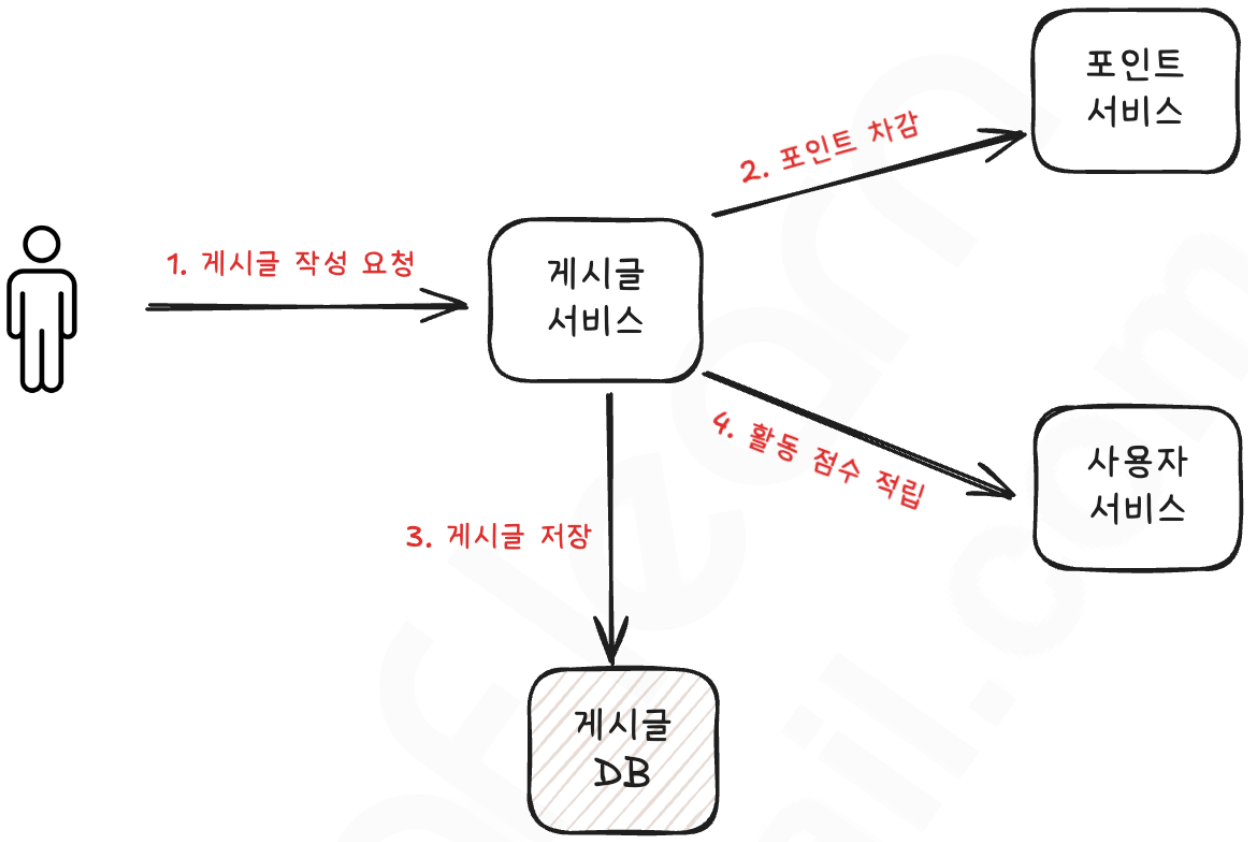
	point_id	amount	user_id
1	1	0	1
2	2	1000	4

@naver.com
khb302

🎯 [실습] 사용자 활동 점수 적립 API 만들기

✅ 구조

게시글 작성 API에서 사용할 **활동 점수 적립 API**를 만들어보자.



✅ 사용자 활동 점수 적립 API 만들기

> user-service에서 코드 작성하기

1. **User 엔티티 수정하기**

   domain/User

```
@Entity
@Table(name = "users")
public class User {

    ...

  private int activityScore = 0;

  ...

  // getter 메서드 추가
  public int getActivityScore() {
    return activityScore;
  }

  // 활동 점수 적립 (나중에 Service 로직 짤 때 사용)
  public void addActivityScore(int score) {
```

```
    this.activityScore += score;
  }
}
```

3. **Controller 만들기**

   controller/UserController

```
@RestController
@RequestMapping("/users")
public class UserController {

    ...

   @PostMapping("activity-score/add")
  public ResponseEntity<Void> addActivityScore(
     @RequestBody AddActivityScoreRequestDto addActivityScoreRequestDto
   ) {
     userService.addActivityScore(addActivityScoreRequestDto);
     return ResponseEntity.noContent().build();
   }
}
```

dto/AddActivityScoreRequestDto

```
public class AddActivityScoreRequestDto {
  private Long userId;
  private int score;

  public Long getUserId() {
    return userId;
  }

  public int getScore() {
    return score;
  }
}
```

3. **Service 만들기**

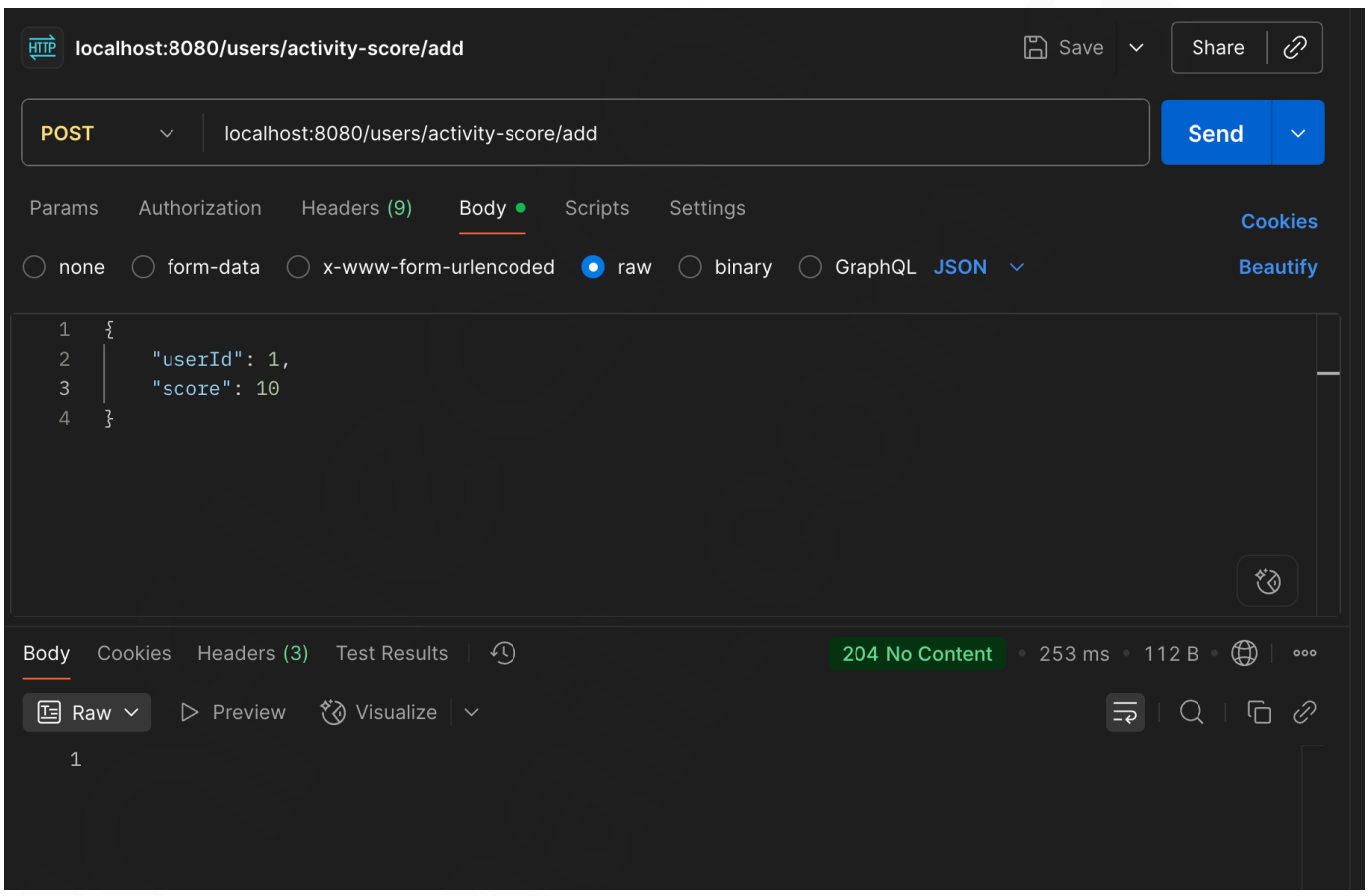
   service/UserService

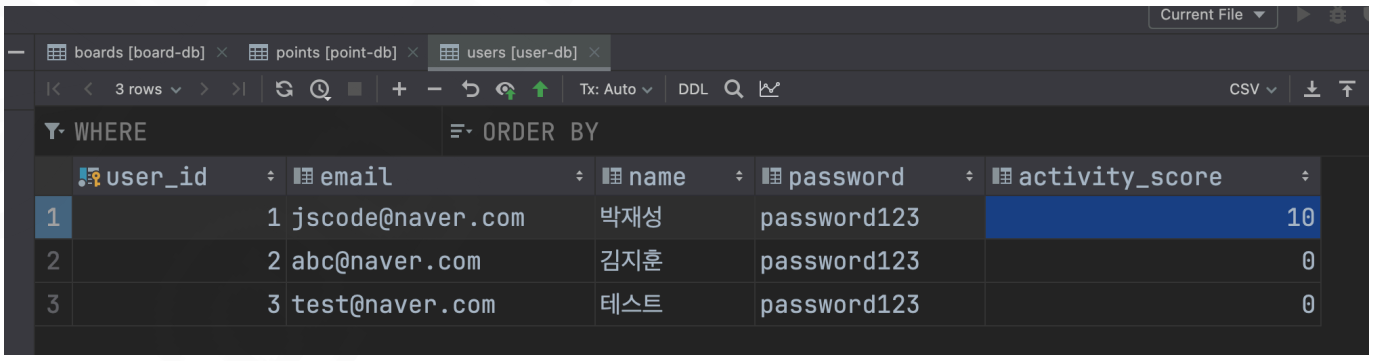
```
@Service
public class UserService {

  ...

  @Transactional
  public void addActivityScore(
      AddActivityScoreRequestDto addActivityScoreRequestDto
  ) {
    User user = userRepository.findById(addActivityScoreRequestDto.getUserId())
        .orElseThrow(() -> new IllegalArgumentException("사용자를 찾을 수 없습니다."));

    user.addActivityScore(addActivityScoreRequestDto.getScore());
```

```
    userRepository.save(user);
  }

}
```

4. **서버 다시 실행시키기**

5. **API 잘 작동하는 지 테스트해보기**



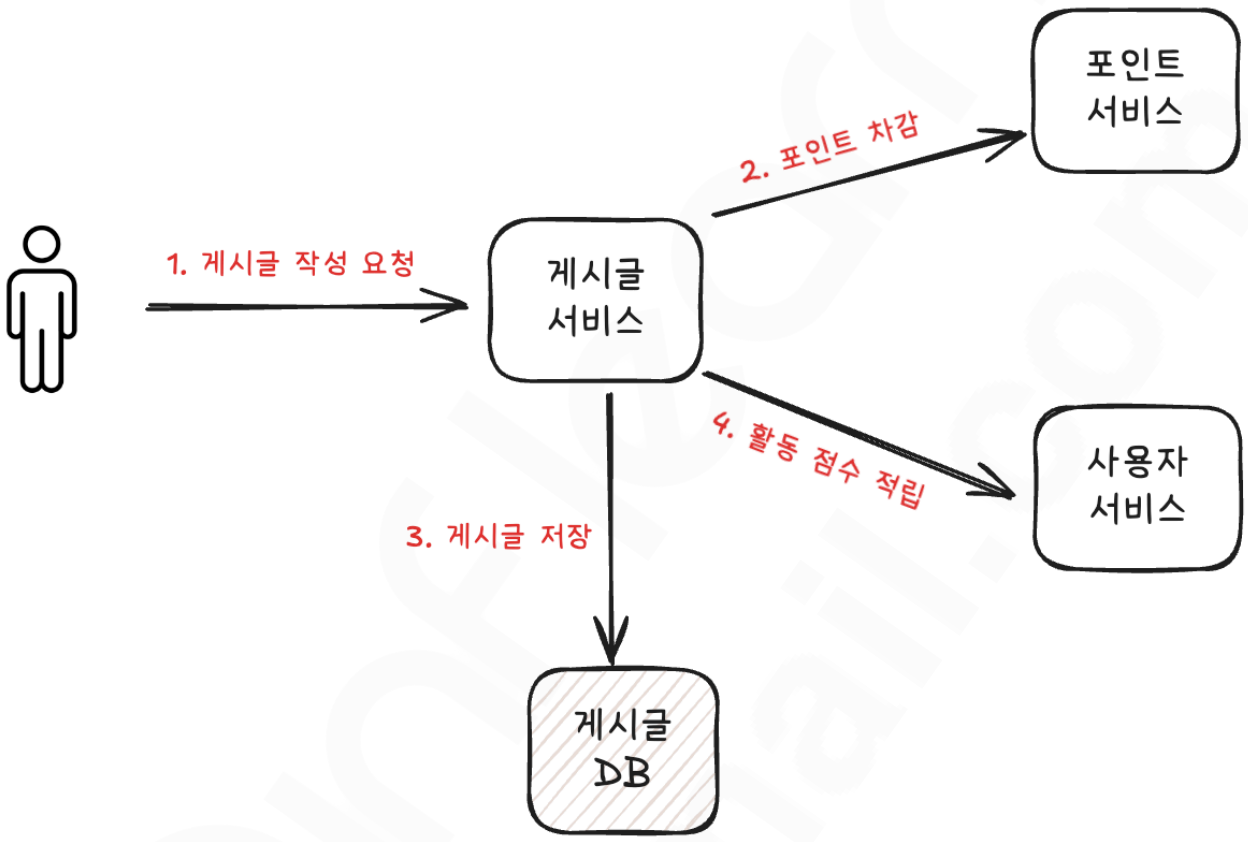


khb3025

🎯 [실습] '포인트 차감 → 게시글 작성→ 활동 점수 적립' 순으로 로직이 처리되도록 만들기

✅ 구조

이 서비스는 게시글을 작성하려면 포인트가 필요한 서비스이다. 그래서 **포인트 차감**을 한 뒤에 **게시글이 작성**돼야 하고, 게시글 작성이 완료되면 **사용자의 활동 점수가 적립**되는 서비스이다. 아래 그림과 같이 작동하도록 코드를 작성해보자.



✅ 코드 작성하기

> board-service에서 아래 코드 작성하기

1. **포인트 차감 API 호출 코드 작성하기**

   application.yml

```
server:
  port: 8081

spring:
  datasource:
    url: jdbc:mysql://localhost:3307/board-db
    username: root
    password: password
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

```
client:
  user-service:
    url: http://localhost:8080
  point-service:
    url: http://localhost:8082
```

dto/DeductPointsRequestDto

```
public class DeductPointsRequestDto {
  private Long userId;
  private int amount;

  public DeductPointsRequestDto(Long userId, int amount) {
    this.userId = userId;
    this.amount = amount;
  }

  public Long getUserId() {
    return userId;
  }

  public int getAmount() {
    return amount;
  }
}
```

client/PointClient

```
@Component
public class PointClient {

  private final RestClient restClient;

  public PointClient(
      @Value("${client.point-service.url}") String pointServiceUrl
  ) {
    this.restClient = RestClient.builder()
        .baseUrl(pointServiceUrl)
        .build();
  }

  public void deductPoints(Long userId, int amount) {
    DeductPointsRequestDto deductPointsRequestDto
        = new DeductPointsRequestDto(userId, amount);
    this.restClient.post()
        .uri("/points/deduct")
        .contentType(MediaType.APPLICATION_JSON)
        .body(deductPointsRequestDto)
        .retrieve()
        .toBodilessEntity();
  }
}
```

2. **활동 점수 적립 API 호출 코드 작성하기**

dto/AddActivityScoreRequestDto

```
public class AddActivityScoreRequestDto {
  private Long userId;
  private int score;

  public AddActivityScoreRequestDto(Long userId, int score) {
    this.userId = userId;
    this.score = score;
  }

  public Long getUserId() {
    return userId;
  }

  public int getScore() {
    return score;
  }
}
```

client/UserCleint

```
@Component
public class UserClient {

  ...

  public void addActivityScore(Long userId, int score) {
    AddActivityScoreRequestDto addActivityScoreRequestDto
        = new AddActivityScoreRequestDto(userId, score);
    this.restClient.post()
        .uri("/users/activity-score/add")
        .contentType(MediaType.APPLICATION_JSON)
        .body(addActivityScoreRequestDto)
        .retrieve()
        .toBodilessEntity();
  }
}
```

3. **Service 코드 작성하기**

   게시글 작성 로직에 포인트 적립 로직을 추가해야 한다.

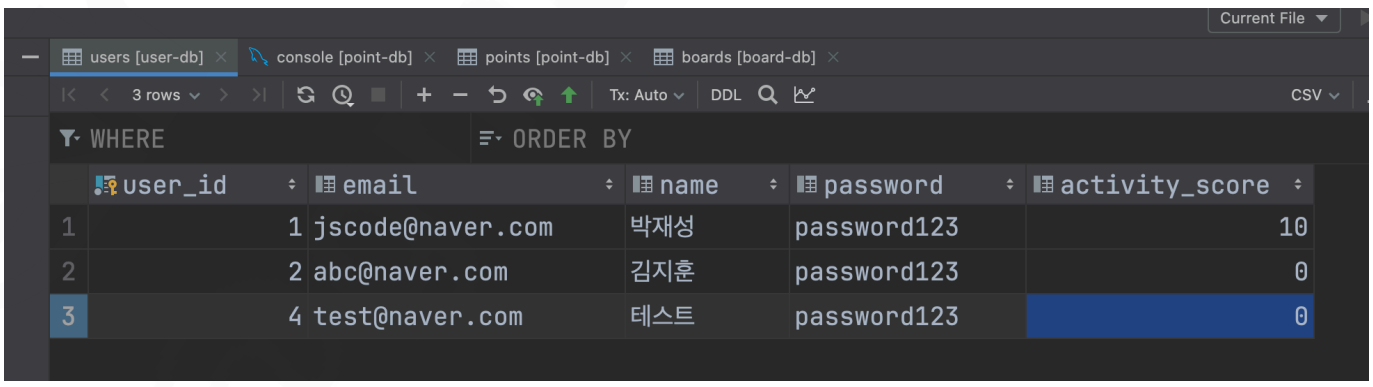
   service/BoardService

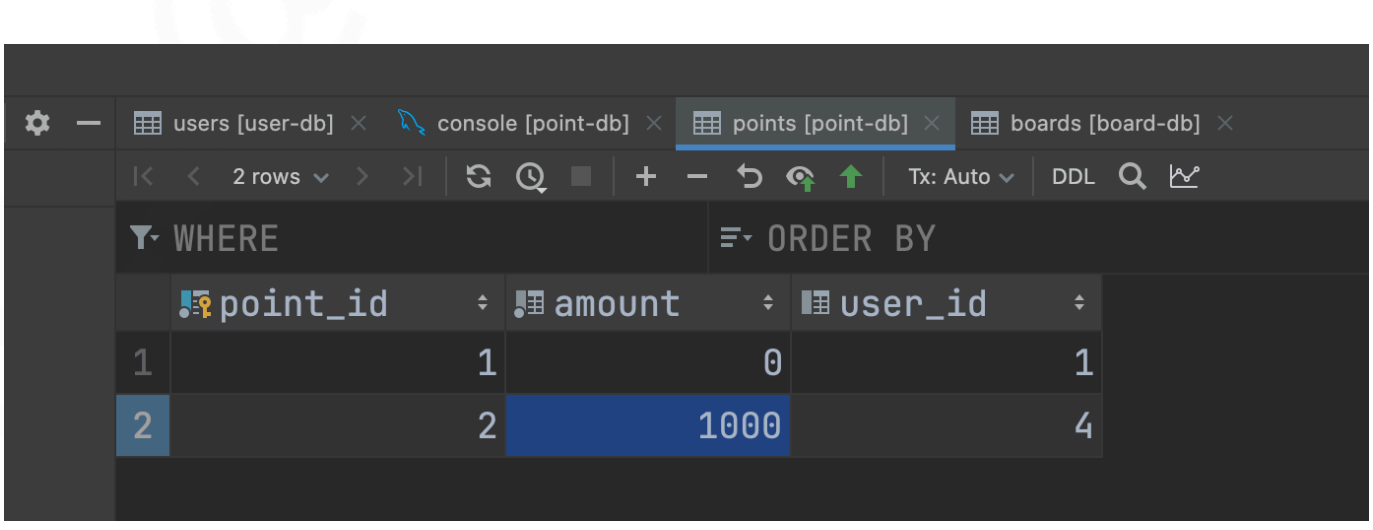
```
@Service
public class BoardService {
  private final BoardRepository boardRepository;
  private final UserClient userClient;
  private final PointClient pointClient;

  public BoardService(BoardRepository boardRepository, UserClient userClient,
      PointClient pointClient) {
    this.boardRepository = boardRepository;
    this.userClient = userClient;
    this.pointClient = pointClient;
```

```
}

  @Transactional
public void create(CreateBoardRequestDto createBoardRequestDto) {
  // 게시글 작성 전 100 포인트 차감
  pointClient.deductPoints(createBoardRequestDto.getUserId(), 100);

  // 게시글 작성
  Board board = new Board(
    createBoardRequestDto.getTitle(),
    createBoardRequestDto.getContent(),
    createBoardRequestDto.getUserId()
  );

  this.boardRepository.save(board);

  // 게시글 작성 시 작성자에게 활동 점수 10점 부여
  userClient.addActivityScore(createBoardRequestDto.getUserId(), 10);
  }

  ...

}
```

4. **서버 다시 실행시키기**

5. **기존 DB 데이터 체크**





users [user-db] | console [point-db] | points [point-db] | boards [board-db]

4 rows | Tx: Auto | DDL

WHERE | ORDER BY

	board_id	content	title	user_id
1	1	내용	제목	1
2	2	내용2	제목2	2
3	3	내용3	제목3	2

6. **서버 실행시켜서 API 테스트해보기**

HTTP localhost:8081/boards | Save | Share

POST | localhost:8081/boards | Send

Params | Auth | Headers (9) | Body | Scripts | Settings | Cookies

raw | JSON | Beautify

```
{
    "title": "테스트",
    "content": "테스트",
    "userId": 4
}
```

Body | 204 No Content | 375 ms | 112 B

Raw | Preview | Visualize

```
1
```

users [user-db] | console [point-db] | points [point-db] | boards [board-db]

4 rows | Tx: Auto | DDL

WHERE | ORDER BY

	board_id	content	title	user_id
1	1	내용	제목	1
2	2	내용2	제목2	2
3	3	내용3	제목3	2
4	4	테스트	테스트	4

users [user-db] | console [point-db] | points [point-db] | boards [board-db]

2 rows | Tx: Auto | DDL

WHERE | ORDER BY

	point_id	amount	user_id
1	1	0	1
2	2	900	4

users [user-db] | console [point-db] | points [point-db] | boards [board-db]

4 rows | Tx: Auto | DDL

WHERE | ORDER BY

	board_id	content	title	user_id
1	1	내용	제목	1
2	2	내용2	제목2	2
3	3	내용3	제목3	2
4	4	테스트	테스트	4

여기까지만 봤을 때는 잘 작동하는 코드처럼 보인다. 하지만 아직까지 문제점이 많은 코드이다. 왜 그런지 다음 강의에서 알아보자.

khb3025@gmail.com

🎯 [실습] 게시글 작성 API 로직 수행 중, 중간에 에러가 발생하면 어떻게 될까?

✅ 게시글 작성 API 로직 수행 중, 중간에 에러가 발생하면 어떻게 될까?

👨🏻‍🏫 게시글 작성 API 로직이 '**포인트 차감 → 게시글 저장 → 활동 점수 적립**' 구성으로 이루어져 있다. '**포인트 차감, 게시글 저장, 활동 점수 적립**'은 기획상 하나로 묶인 작업이기 때문에 이 중에 하나라도 실패하면 전체가 실패해야 한다. 즉, **활동 점수 적립**에 실패하면 **게시글 저장**도 실패해야 하고 **포인트 차감**도 실패해야 하는 게 정상일 것이다. 정말 이렇게 작동하는 지 확인해보자.

1. **활동 점수 증가 API에 요청을 보내면 에러가 발생하도록 만들기**

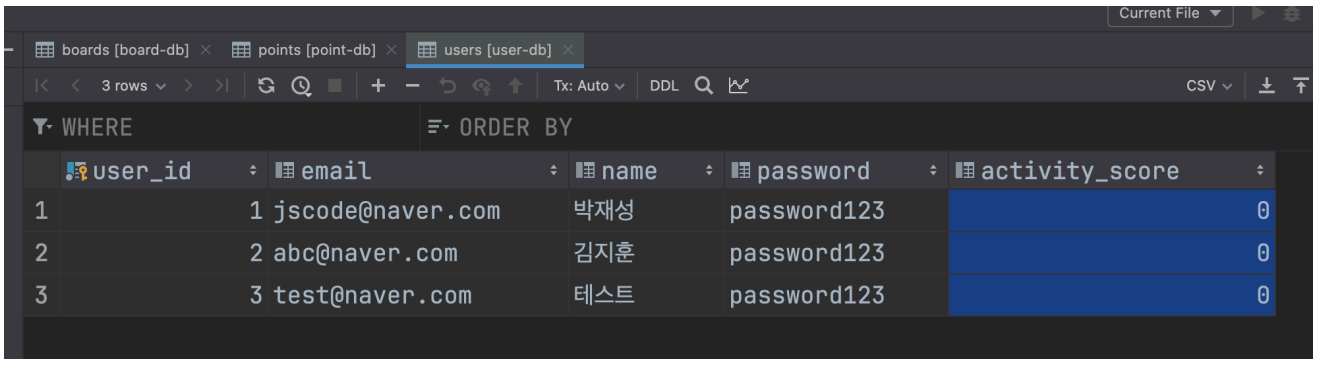
활동 점수를 증가시키다 실패하는 경우를 가정하기 위해, 에러 발생시키는 코드를 추가하자.

service/UserService

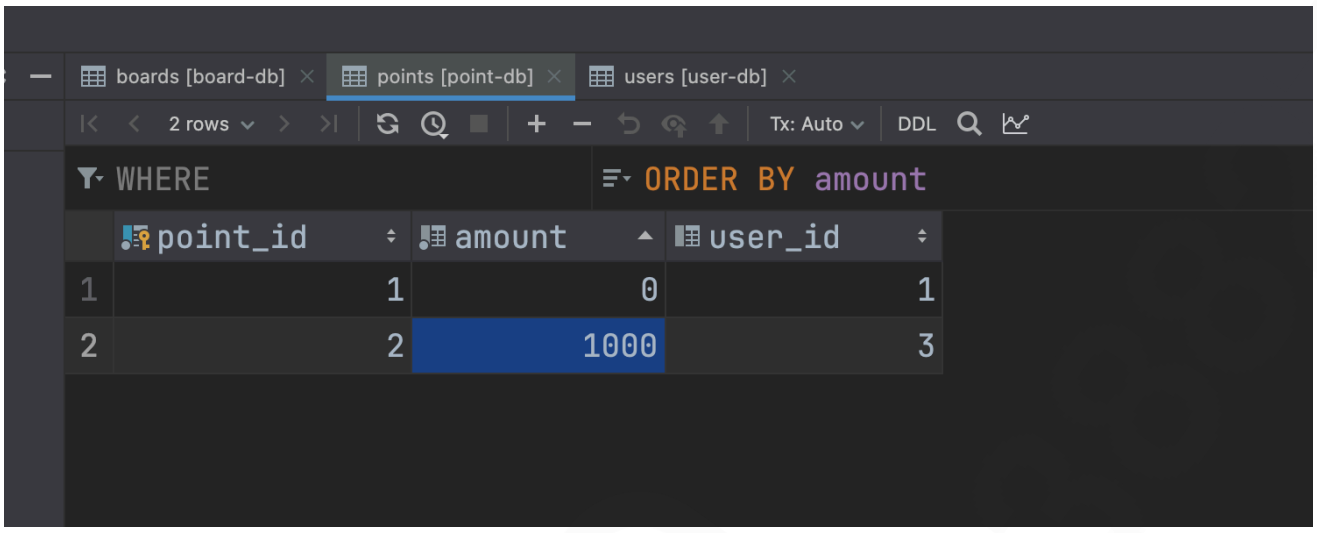
```
@Service
public class UserService {
  ...

  @Transactional
  public void addActivityScore(
      AddActivityScoreRequestDto addActivityScoreRequestDto
  ) {
    User user = userRepository.findById(addActivityScoreRequestDto.getUserId())
        .orElseThrow(() -> new IllegalArgumentException("사용자를 찾을 수 없습니다."));

    user.addActivityScore(addActivityScoreRequestDto.getScore());

    userRepository.save(user);
        throw new RuntimeException("에러 발생");
  }
}
```

2. **user-service 서버 재실행시키기**

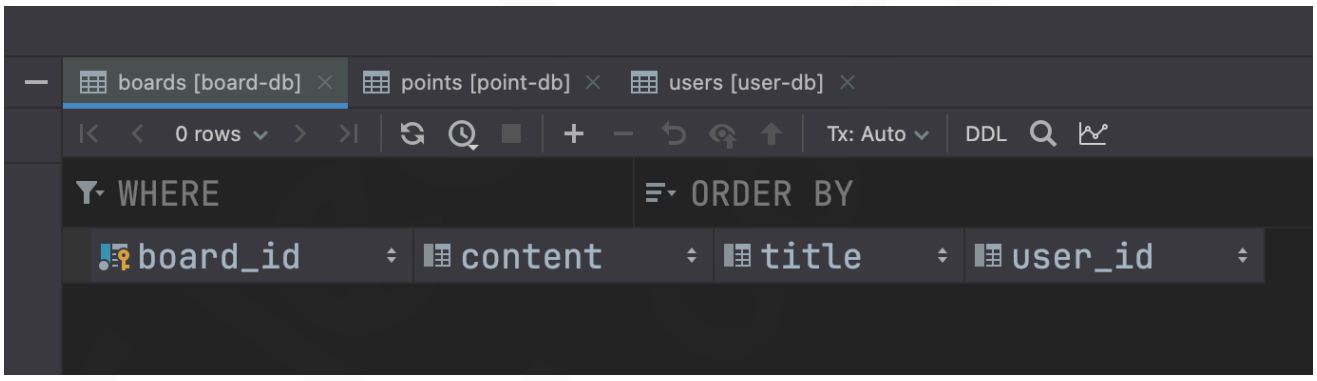
3. **테스트를 위해 DB 데이터 정리하기**
   a. **활동 점수 0으로 바꾸기**



b. **포인트 데이터 1000점으로 맞춰놓기**

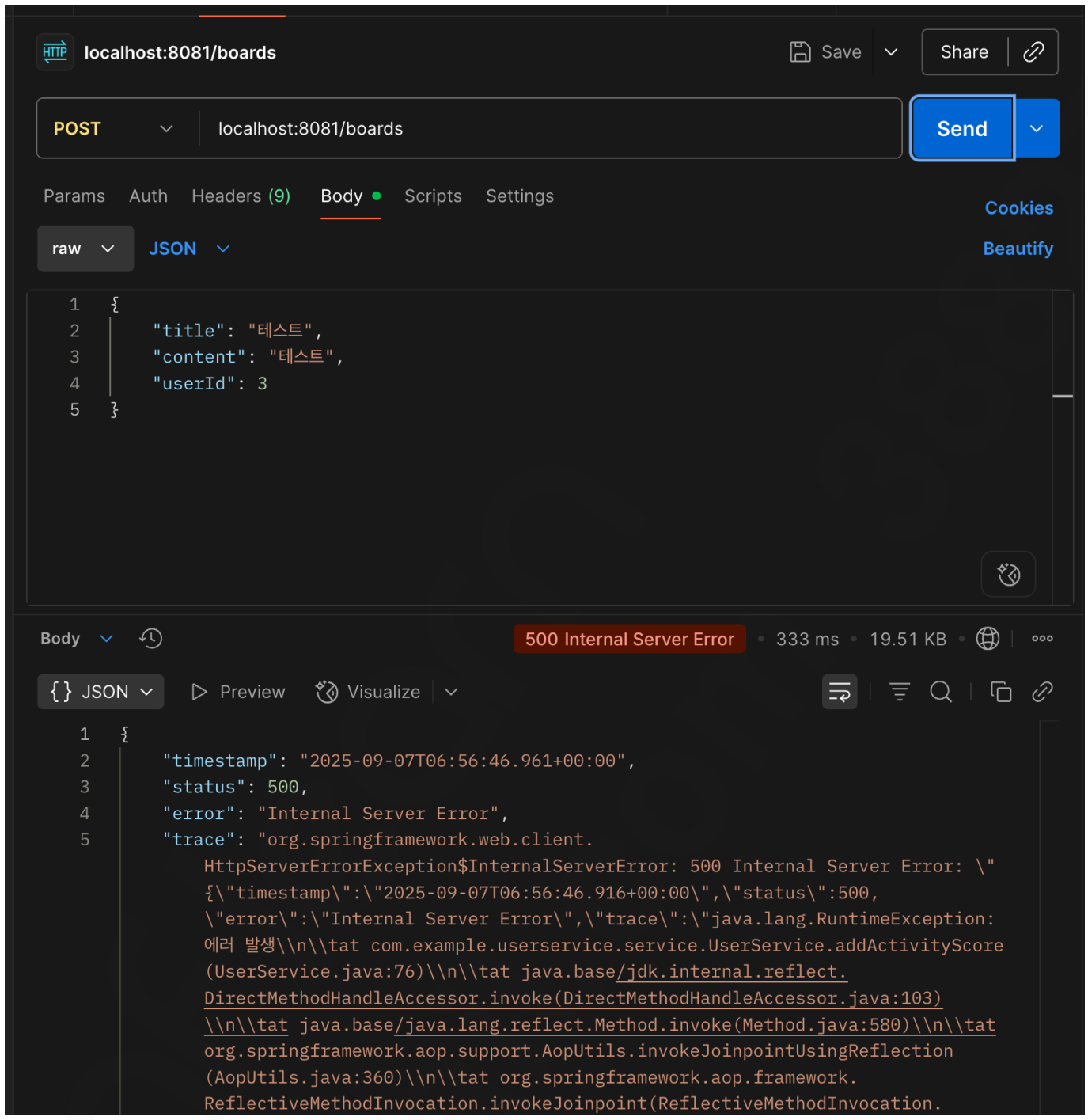


c. **게시글 데이터 전부 삭제하기**



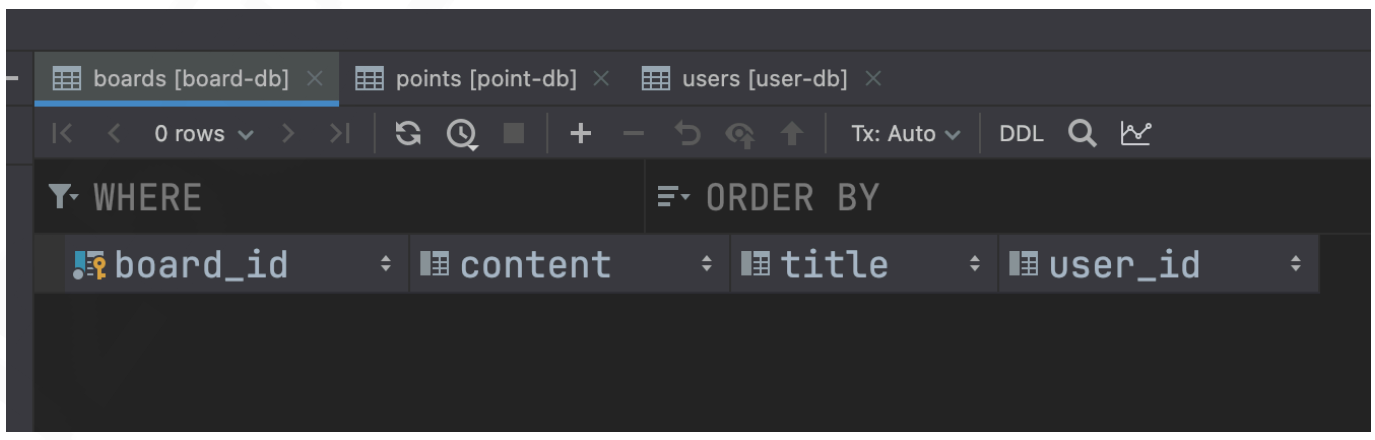
2. **게시글 생성 API 요청 보내기**

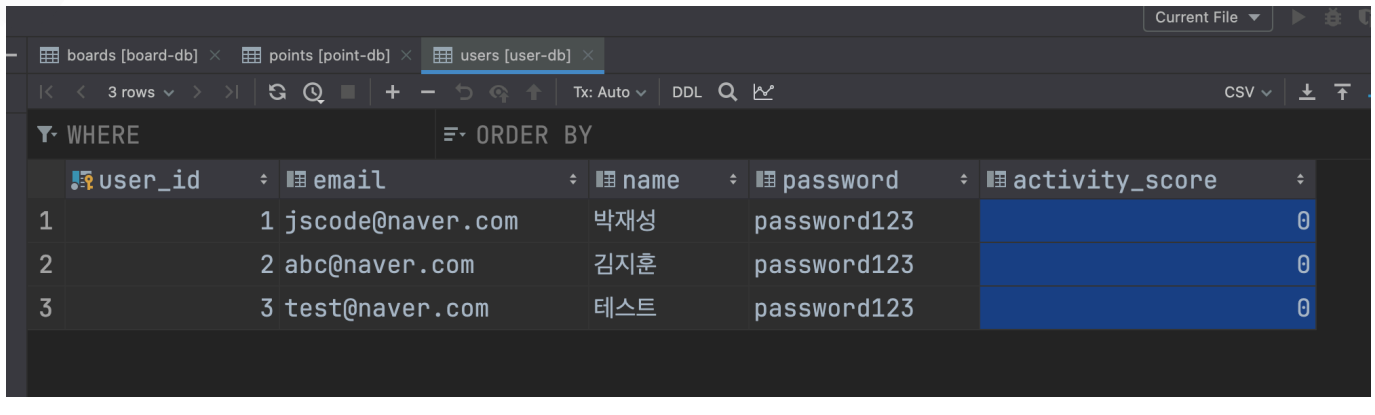
khb3025@gmail.com

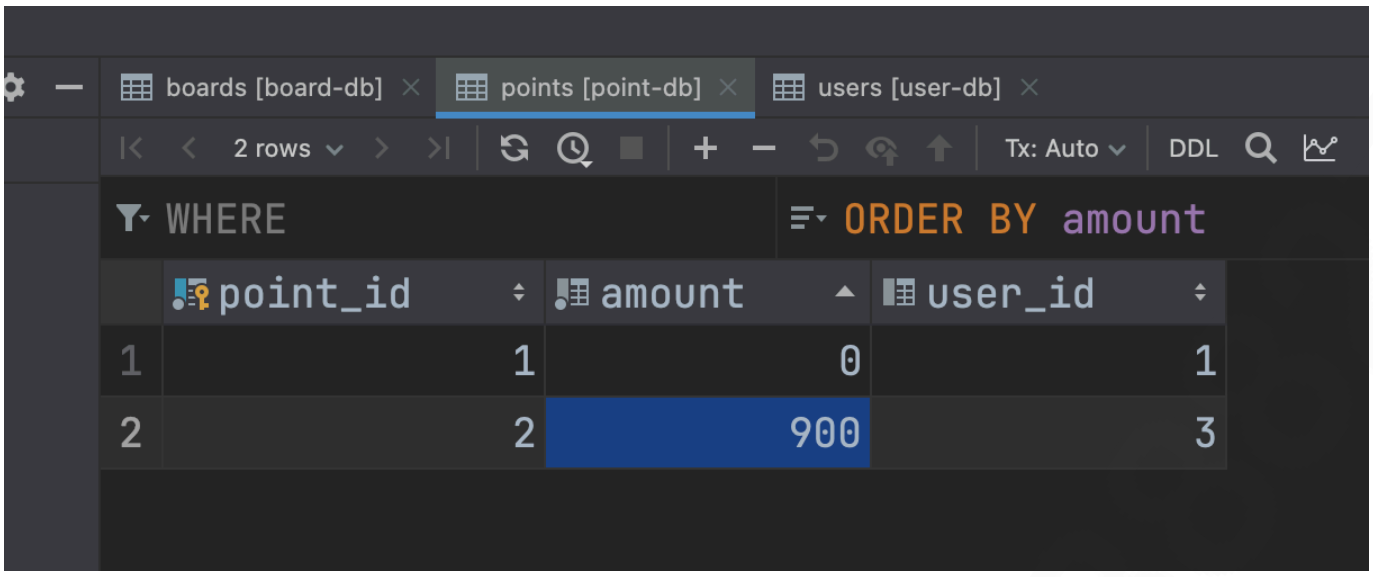


게시글 작성 API 로직의 '**포인트 차감 → 게시글 저장 → 활동 점수 적립**' 중 '**활동 점수 적립**' 단계에서 에러가 발생했기 때문에 게시글 작성 API 자체가 실패했다. 이렇게만 봤을 때는 의도한대로 '**활동 점수 적립**'이 실패함으로써 **'게시글 작성'**도 실패하고, **'포인트 차감'**도 실패한 것처럼 보인다. 정말 그렇게 작동했는 지 DB를 확인해보자.

3. DB 확인하기







DB를 보면 **게시글 작성**도 되지 않았고 **사용자 활동 점수**도 쌓이지 않았는데, **포인트 차감**은 된 걸 확인할 수 있다. 왜 이런 현상이 발생한 걸까?

✅ 원인 분석

모놀리식 아키텍처와 달리 MSA 환경에서는 서비스마다 DB가 분리되어 있다. 그래서 서로 다른 DB에서 처리하는 여러 작업(포인트 차감, 게시글 작성, 활동 점수 적립)을 하나의 트랜잭션으로 묶을 수 없다. 그러다보니 에러가 발생했을 때 어떤 작업은 실패했는데, 어떤 작업은 성공하는 모순된 상황이 발생한 것이다. 코드를 보면서 조금 더 자세히 살펴보자.

BoardService의 '게시글 작성' 비즈니스 로직

```
@Transactional
public void create(CreateBoardRequestDto createBoardRequestDto) {
  // 게시글 작성 전 100 포인트 차감
  pointClient.deductPoints(createBoardRequestDto.getUserId(), 100);

  // 게시글 작성
  Board board = new Board(
    createBoardRequestDto.getTitle(),
    createBoardRequestDto.getContent(),
    createBoardRequestDto.getUserId()
  );

  this.boardRepository.save(board); // 롤백

  // 게시글 작성 시 작성자에게 활동 점수 10점 부여
  userClient.addActivityScore(createBoardRequestDto.getUserId(), 10); // 에러 발생
}
```

- `userClient.addActivityScore(...)`에서 에러가 발생하는 순간 `@Transactional` 어노테이션 때문에 게시글 저장(`this.boardRepostiory.save()`)이 **롤백 (rollback)**된다.

PointService의 '포인트 차감' 비즈니스 로직

```
@Transactional
public void deductPoints(DeductPointRequestDto deductPointRequestDto) {
  Point point = pointRepository.findByUserId(deductPointRequestDto.getUserId())
      .orElseThrow(() -> new IllegalArgumentException("사용자의 포인트 정보를 찾을 수 없습니다."));

  point.deductAmount(deductPointRequestDto.getAmount());
```

```
  pointRepository.save(point);
}
```

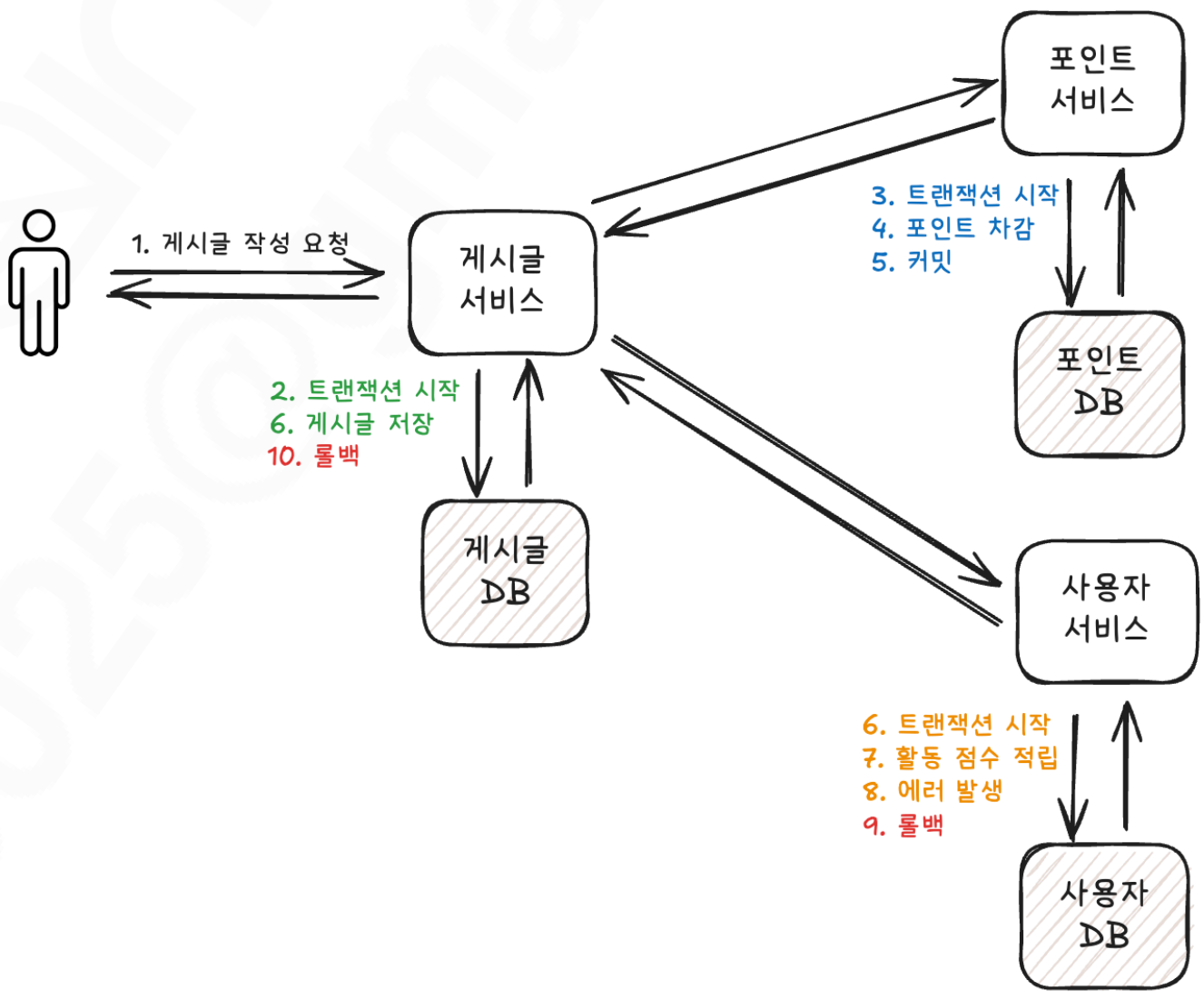
- '포인트 적립' 비즈니스 로직에서는 에러가 발생하지 않았기 때문에
  포인트 적립(`pointRepository.save()`)이 **롤백(rollback)되지 않고 커밋(commit)된다.**

UserService의 '활동 점수 증가' 비즈니스 로직

```
@Transactional
public void addActivityScore(
    AddActivityScoreRequestDto addActivityScoreRequestDto
) {
  User user = userRepository.findById(addActivityScoreRequestDto.getUserId())
      .orElseThrow(() -> new IllegalArgumentException("사용자를 찾을 수 없습니다."));

  user.addActivityScore(addActivityScoreRequestDto.getScore());

  userRepository.save(user); // 롤백
  throw new RuntimeException("에러 발생");
}
```

- '활동 점수 증가' 비즈니스 로직에서는 에러가 발생했기 때문에 `@Transactional`에 의해
  '활동 점수 증가(`userRepository.save()`)'가 **롤백(rollback)**된다.

> 그림으로 표현하면 다음과 같다.

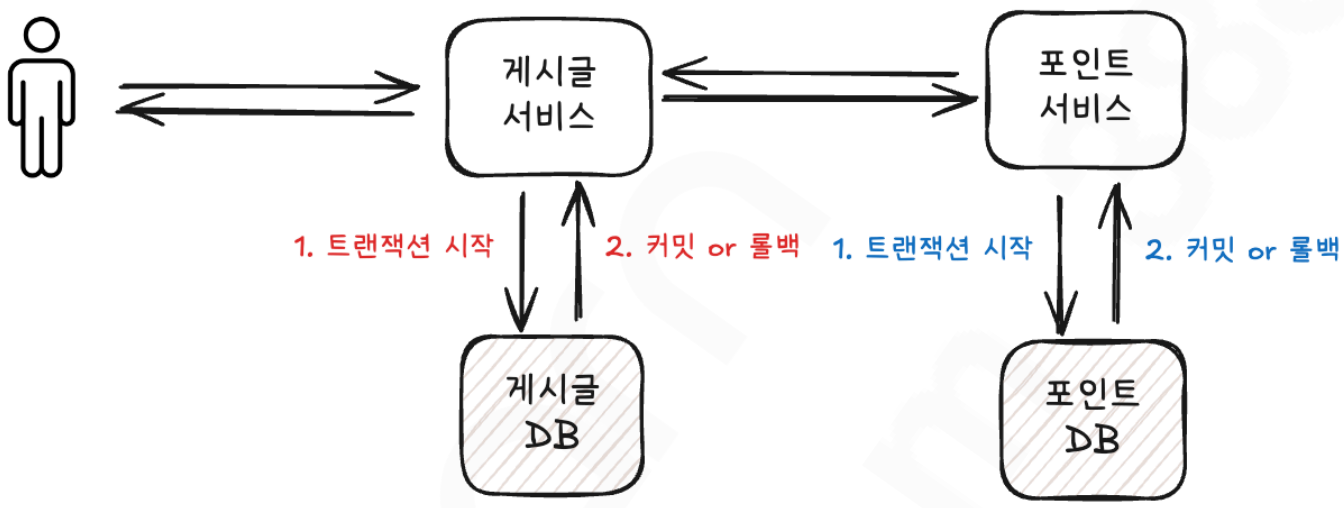


이 문제를 해결하려면 어떻게 코드를 구현해야 하는 지 다음 강의에서 알아보자.

@kfc3882@gmail.com
kfc3025

MSA 환경에서의 트랜잭션 처리 방법

✅ MSA 환경에서의 트랜잭션 처리 방법



모놀리식 아키텍처와 달리 MSA 환경에서는 서비스마다 DB가 분리되어 있다. 그래서 서로 다른 DB에서 처리하는 여러 작업을 하나의 트랜잭션으로 묶을 수 없다. 그래서 MSA 환경에서는 이런 문제점을 해결하기 위해 2PC, TCC, Saga 패턴 같은 기법을 활용한다.

✅ 2PC 패턴? TCC 패턴? SAGA 패턴?

MSA를 공부하다보면 MSA 환경에서 트랜잭션을 처리하기 위한 패턴이 크게 3가지가 나온다.

- 2PC 패턴
- TCC 패턴
- Saga 패턴

하지만 실무에서는 2PC나 TCC 패턴은 잘 사용하지 않는다. 따라서 이번 강의에서는 Saga 패턴만 배워볼 것이다.

> **"선생님, 2PC 패턴이랑 TCC 패턴도 배워야 되는 거 아닌가요? 저걸 빠트리니까 너무 불안한데요...?"**

OT에서 얘기했던 파레토 법칙을 잘 기억하자. MSA에서 언급하는 모든 기술을 익히려고 덤볐다가는, MSA를 학습하는 데 1년이 넘게 걸릴 것이다. MSA에서 자주 쓰는 내용만 익혀도 정말 배울 내용이 많다.

그래서 실전에서 MSA를 빠르게 적용시킬 수 있게 학습하려면, 실무에서 자주 사용하는 내용 위주로 배워야 한다. 그리고 MSA를 처음 공부할 때는 개념을 일일이 깊이 있게 파헤치는 것보다, 핵심 개념들 위주로 빠르게 학습해 MSA 구조로 프로젝트를 직접 구현해보면서 전체 흐름을 느껴보는 게 중요하다.

이렇게 배워야 MSA에 금방 익숙해질 수 있다. 지엽적인 내용은 나중에 MSA 핵심 개념들이 어느 정도 익숙해진 다음에 천천히 익혀도 늦지 않다.

👨‍🏫 다음 강의에서는 Saga 패턴에 대해 배워보자.

📖

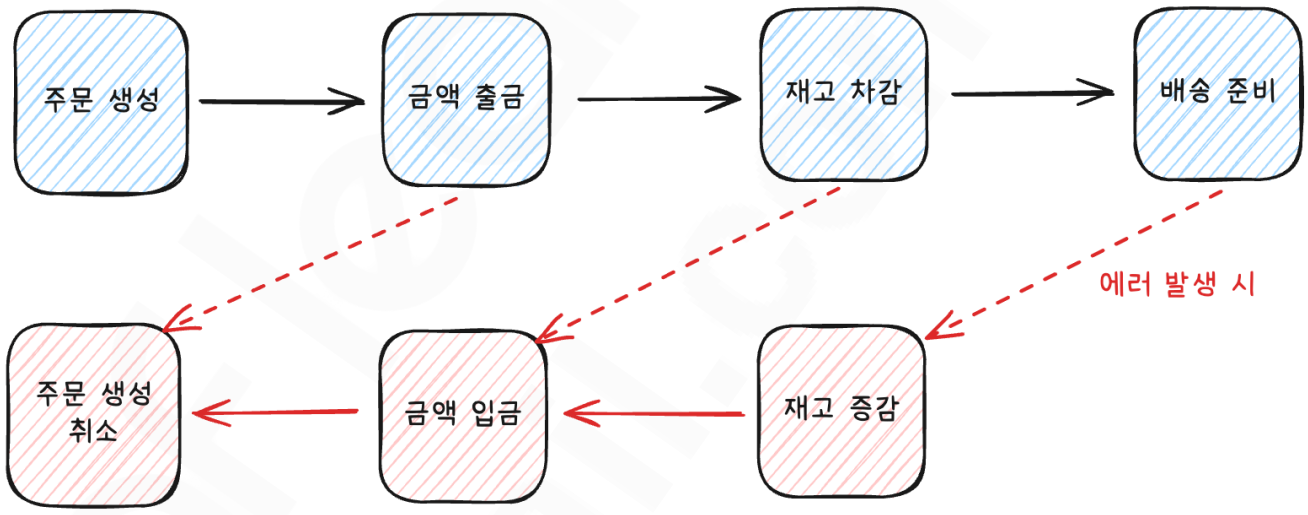
Saga 패턴이란? / 보상 트랜잭션이란?

✅ Saga 패턴이란?

MSA 환경에서의 트랜잭션을 처리할 때 사용하는 패턴 중 하나가 Saga 패턴이다. **Saga 패턴**은 **보상 트랜잭션**을 활용해 **'여러 작업을 하나의 트랜잭션으로 묶은 것'처럼 작동시키는 방식**이다.

✅ 보상 트랜잭션이란?

보상 트랜잭션이란 **수동으로 트랜잭션의 롤백(Rollback) 효과를 내는 방법**이다. 아래 예시를 바로 보자.



'주문 생성 → 금액 출금 → 재고 차감 → 배송 준비'의 작업을 하던 중 **'배송 준비'** 과정에서 에러가 났다면 지금까지 해왔던 작업을 다 취소시켜야 한다. 기존에 수행했던 작업을 취소하기 위해 기존 작업을 원래대로 돌릴 수 있는 반대되는 작업을 수행하면 된다. 즉, **재고 차감**을 했으니 다시 **재고 증감**을 하면 되고, **금액 출금**을 했으니 다시 **금액 입금**을 하면 되고, **주문을 생성**했으니 **주문을 삭제**하면 된다.

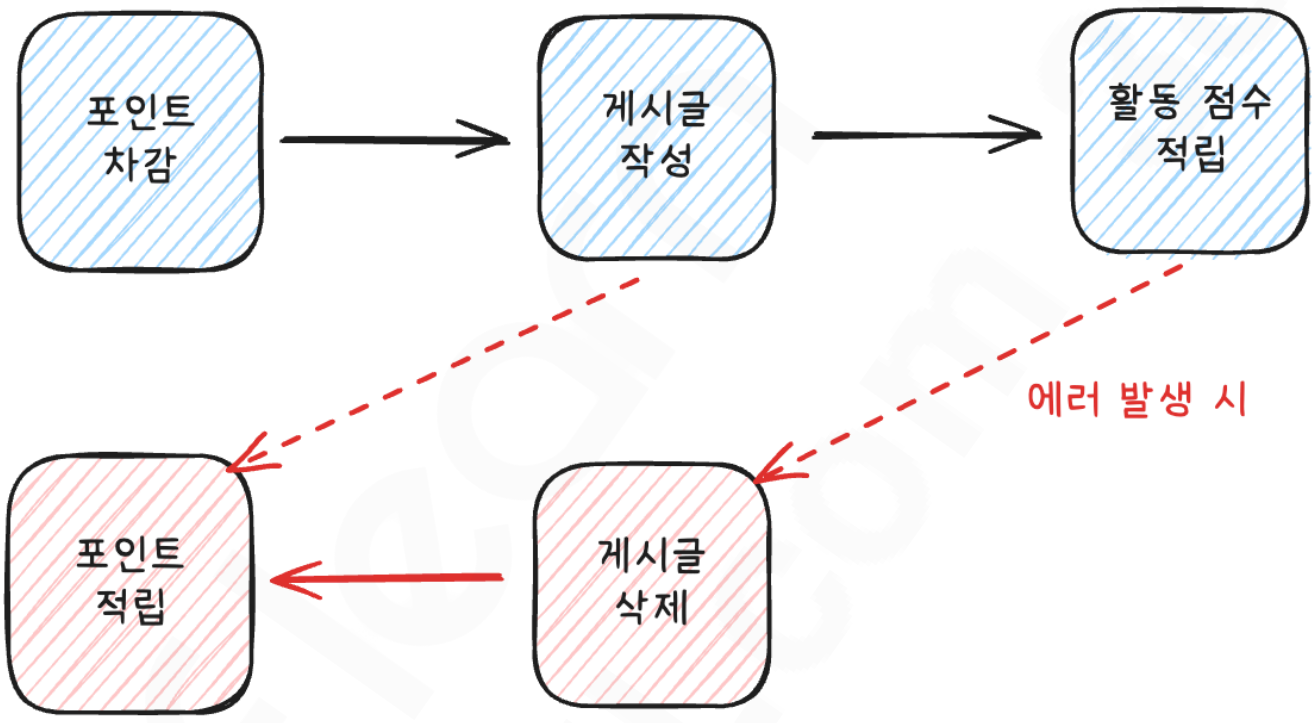
여기서 **기존에 수행했던 작업을 취소하기 위해 행하는 '재고 증감, 금액 입금, 주문 생성 취소'와 같은 취소 작업**을 보고 **보상 트랜잭션**이라고 한다. 그리고 위 예시처럼 **보상 트랜잭션을 활용해 여러 작업을 하나의 트랜잭션으로 묶은 것처럼 연결해주는 처리 방식**이 **Saga 패턴**이다.

Saga 패턴에 대한 이론은 이 정도면 충분하다. 이론을 더 자세히 설명하는 것보다 실제 프로젝트에서 코드를 짜보는 게 훨씬 이해가 빠르다. 다음 강의에서 Saga 패턴을 활용해 코드를 작성해보자.

[실습] 게시글 작성 API에 Saga 패턴 적용시키기 - 1

✅ 게시글 작성 API에 Saga 패턴 적용시키기

아래 구조처럼 기존 게시글 작성 API에 Saga 패턴을 적용시켜보자.



1. **'게시글 작성 API' 기존 로직의 문제점 확인하기**

   service/BaordService

```
@Transactional
public void create(CreateBoardRequestDto createBoardRequestDto) {
  // 게시글 작성 전 100 포인트 차감
  pointClient.deductPoints(createBoardRequestDto.getUserId(), 100);

  // 게시글 작성
  Board board = new Board(
    createBoardRequestDto.getTitle(),
    createBoardRequestDto.getContent(),
    createBoardRequestDto.getUserId()
  );

  this.boardRepository.save(board);

  // 게시글 작성 시 작성자에게 활동 점수 10점 부여
  userClient.addActivityScore(createBoardRequestDto.getUserId(), 10);
}
```

위 코드를 보면 '포인트 차감 → 게시글 작성 → 활동 점수 적립' 순으로 코드를 작성한 것처럼 보인다. 하지만 실제 코드 실행은 '포인트 차감 → 활동 점수 적립 → 게시글 작성' 순으로 실행된다. 이렇게 실행되는 이유는 `@Transactional` 어노테이션 때문이다. `@Transactional`이 어떻게 작동하는 지 아래 코드를 보자.

service/BoardService

```
public void create(CreateBoardRequestDto createBoardRequestDto) {
    transactionManager.begin();

    try {
      // 게시글 작성 전 100 포인트 차감
      pointClient.deductPoints(createBoardRequestDto.getUserId(), 100);

      // 게시글 작성
      Board board = new Board(
        createBoardRequestDto.getTitle(),
        createBoardRequestDto.getContent(),
        createBoardRequestDto.getUserId()
      );

      this.boardRepository.save(board);

      // 게시글 작성 시 작성자에게 활동 점수 10점 부여
      userClient.addActivityScore(createBoardRequestDto.getUserId(), 10);

      transactionManager.commit();
    } catch (Exception e) {
      transactionManager.rollback();
      throw e;
    }
}
```

게시글 작성은 `commit()`이 실행된 이후에 완료되기 때문에 실제로는 '포인트 차감 → 활동 점수 적립 → 게시글 작성' 순으로 작동한다. 그래서 기획에서 의도한 순서대로 작동하게 만들기 위해 `@Transactional` 어노테이션을 제거해주자. 그리고 에러로 인한 '게시글 작성'에 대한 롤백은 '보상 트랜잭션'을 활용해 처리하는 방식으로 수정하자.

2. **`@Transactional` 코드 제거하고, '게시글 저장'에 대한 보상 트랜잭션 추가하기**

   service/BoardService

```
public void create(CreateBoardRequestDto createBoardRequestDto) {
  // 게시글 저장을 성공했는 지 판단하는 플래그
  boolean isBoardCreated = false;
  Long savedBoardId = null;

// 포인트 차감을 성공했는 지 판단하는 플래그
boolean isPointDeducted = false;

try {
  // 게시글 작성 전 100 포인트 차감
  pointClient.deductPoints(createBoardRequestDto.getUserId(), 100);

  // 게시글 작성
  Board board = new Board(
      createBoardRequestDto.getTitle(),
      createBoardRequestDto.getContent(),
      createBoardRequestDto.getUserId()
  );

  Board savedBoard = this.boardRepository.save(board);
  savedBoardId = savedBoard.getBoardId();
  isBoardCreated = true; // 게시글 저장 성공 플래그
```

```
    // 게시글 작성 시 작성자에게 활동 점수 10점 부여
    userClient.addActivityScore(createBoardRequestDto.getUserId(), 10);
  } catch (Exception e) {
    if (isBoardCreated) {
      // 게시글 작성 보상 트랜잭션 => 게시글 삭제
      this.boardRepository.deleteById(savedBoardId);
    }

    // 실패 응답으로 처리하기 위해 예외 던지기
    throw e;
  }
}
```

3. **포인트 적립 API 요청할 수 있는 로직 추가하기**

dto/AddPointsRequestDto

```
public class AddPointsRequestDto {
  private Long userId;
  private int amount;

  public AddPointsRequestDto(Long userId, int amount) {
    this.userId = userId;
    this.amount = amount;
  }

  public Long getUserId() {
    return userId;
  }

  public int getAmount() {
    return amount;
  }
}
```

client/PointClient

```
@Component
public class PointClient {
    ...

    public void addPoints(Long userId, int amount) {
    AddPointsRequestDto addPointsRequestDto
        = new AddPointsRequestDto(userId, amount);
    this.restClient.post()
        .uri("/points/add")
        .contentType(MediaType.APPLICATION_JSON)
        .body(addPointsRequestDto)
        .retrieve()
        .toBodilessEntity();
  }
}
```

4. **'포인트 차감'에 대한 보상 트랜잭션 추가하기**

service/BoardService

```
public void create(CreateBoardRequestDto createBoardRequestDto) {
  // 게시글 저장을 성공했는 지 판단하는 플래그
  boolean isBoardCreated = false;
  Long savedBoardId = null;

  // 포인트 차감을 성공했는 지 판단하는 플래그
  boolean isPointDeducted = false;

  try {
    // 게시글 작성 전 100 포인트 차감
    pointClient.deductPoints(createBoardRequestDto.getUserId(), 100);
    isPointDeducted = true; // 포인트 차감 성공 플래그

    // 게시글 작성
    Board board = new Board(
        createBoardRequestDto.getTitle(),
        createBoardRequestDto.getContent(),
        createBoardRequestDto.getUserId()
    );

    Board savedBoard = this.boardRepository.save(board);
    savedBoardId = savedBoard.getBoardId();
    isBoardCreated = true; // 게시글 저장 성공 플래그

    // 게시글 작성 시 작성자에게 활동 점수 10점 부여
    userClient.addActivityScore(createBoardRequestDto.getUserId(), 10);
  } catch (Exception e) {
    if (isBoardCreated) {
      // 게시글 작성 보상 트랜잭션 => 게시글 삭제
      this.boardRepository.deleteById(savedBoardId);
    }
    if (isPointDeducted) {
      // 포인트 차감 보상 트랜잭션 => 포인트 적립
      pointClient.addPoints(createBoardRequestDto.getUserId(), 100);
    }

    // 실패 응답으로 처리하기 위해 예외 던지기
    throw e;
  }
}
```

5. **'활동 점수 적립'에 대한 보상 트랜잭션 추가하기**

   아래 로직을 보면 '활동 점수 적립' 코드는 로직의 맨 마지막에 실행된다. 따라서 이 코드가 실패하면 '활동 점수 적립' 자체가 실패하는 것이기 때문에 보상 트랜잭션을 따로 해줄 필요가 없다. 만약 이 코드가 성공하면 로직 전체가 성공으로 끝이 난다. 이런 이유에서 '활동 점수 적립'에 대한 보상 트랜잭션은 추가해줄 필요가 없다.

   service/BoardService

```
public void create(CreateBoardRequestDto createBoardRequestDto) {
  // 게시글 저장을 성공했는 지 판단하는 플래그
  boolean isBoardCreated = false;
  Long savedBoardId = null;

  // 포인트 차감을 성공했는 지 판단하는 플래그
  boolean isPointDeducted = false;
```

```
try {
  // 게시글 작성 전 100 포인트 차감
  pointClient.deductPoints(createBoardRequestDto.getUserId(), 100);
  isPointDeducted = true; // 포인트 차감 성공 플래그

  // 게시글 작성
  Board board = new Board(
      createBoardRequestDto.getTitle(),
      createBoardRequestDto.getContent(),
      createBoardRequestDto.getUserId()
  );

  Board savedBoard = this.boardRepository.save(board);
  savedBoardId = savedBoard.getBoardId();
  isBoardCreated = true; // 게시글 저장 성공 플래그

  // 게시글 작성 시 작성자에게 활동 점수 10점 부여
  userClient.addActivityScore(createBoardRequestDto.getUserId(), 10);
 } catch (Exception e) {
  if (isBoardCreated) {
   // 게시글 작성 보상 트랜잭션 => 게시글 삭제
   this.boardRepository.deleteById(savedBoardId);
  }
  if (isPointDeducted) {
   // 포인트 차감 보상 트랜잭션 => 포인트 적립
   pointClient.addPoints(createBoardRequestDto.getUserId(), 100);
  }

  // 실패 응답으로 처리하기 위해 예외 던지기
  throw e;
 }
}
```

Saga 패턴에 대한 코드는 전부 작성을 완료했다. 다음 강의에서 작성한 코드가 의도한 대로 잘 작동하는 지 테스트해보자.

khb3025@gmail.com

🎯 [실습] 게시글 작성 API에 Saga 패턴 적용시키기 - 2

✅ 게시글 작성 API의 Saga 패턴이 잘 작동하는 지 테스트해보기

1. **어떤 식으로 작동했는 지 추적하기 좋게 코드 추가하기**

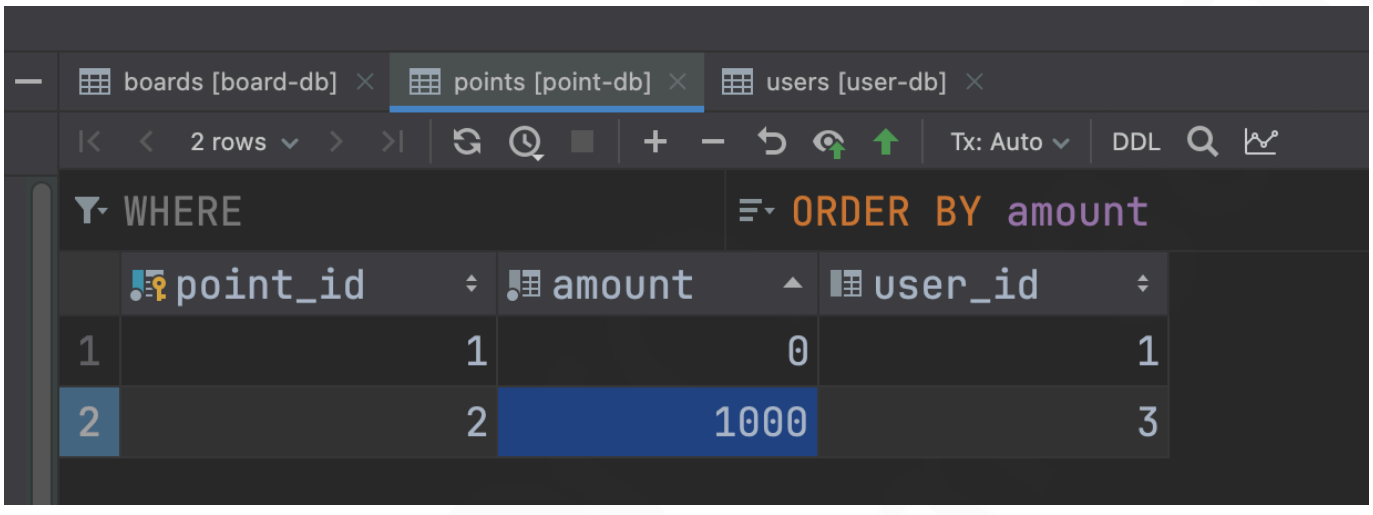
   service/BoardService

```
public void create(CreateBoardRequestDto createBoardRequestDto) {
  // 게시글 저장을 성공했는 지 판단하는 플래그
  boolean isBoardCreated = false;
  Long savedBoardId = null;

  // 포인트 차감을 성공했는 지 판단하는 플래그
  boolean isPointDeducted = false;

try {
  // 게시글 작성 전 100 포인트 차감
  pointClient.deductPoints(createBoardRequestDto.getUserId(), 100);
  isPointDeducted = true; // 포인트 차감 성공 플래그
  System.out.println("포인트 차감 성공");

  // 게시글 작성
  Board board = new Board(
      createBoardRequestDto.getTitle(),
      createBoardRequestDto.getContent(),
      createBoardRequestDto.getUserId()
  );

  Board savedBoard = this.boardRepository.save(board);
  savedBoardId = savedBoard.getBoardId();
  isBoardCreated = true; // 게시글 저장 성공 플래그
  System.out.println("게시글 저장 성공");

  // 게시글 작성 시 작성자에게 활동 점수 10점 부여
  userClient.addActivityScore(createBoardRequestDto.getUserId(), 10);
  System.out.println("포인트 적립 성공");

 } catch (Exception e) {
   if (isBoardCreated) {
     // 게시글 작성 보상 트랜잭션 => 게시글 삭제
     this.boardRepository.deleteById(savedBoardId);
     System.out.println("[보상 트랜잭션] 게시글 삭제");
   }
   if (isPointDeducted) {
     // 포인트 차감 보상 트랜잭션 => 포인트 적립
     pointClient.addPoints(createBoardRequestDto.getUserId(), 100);
     System.out.println("[보상 트랜잭션] 포인트 적립");
   }

   // 실패 응답으로 처리하기 위해 예외 던지기
   throw e;
 }
}
```

2. **board-service 서버 다시 실행시키기**

3. **DB 데이터 정리하기**

   체크하기 쉽게 포인트는 1000포인트로 맞춰놓기



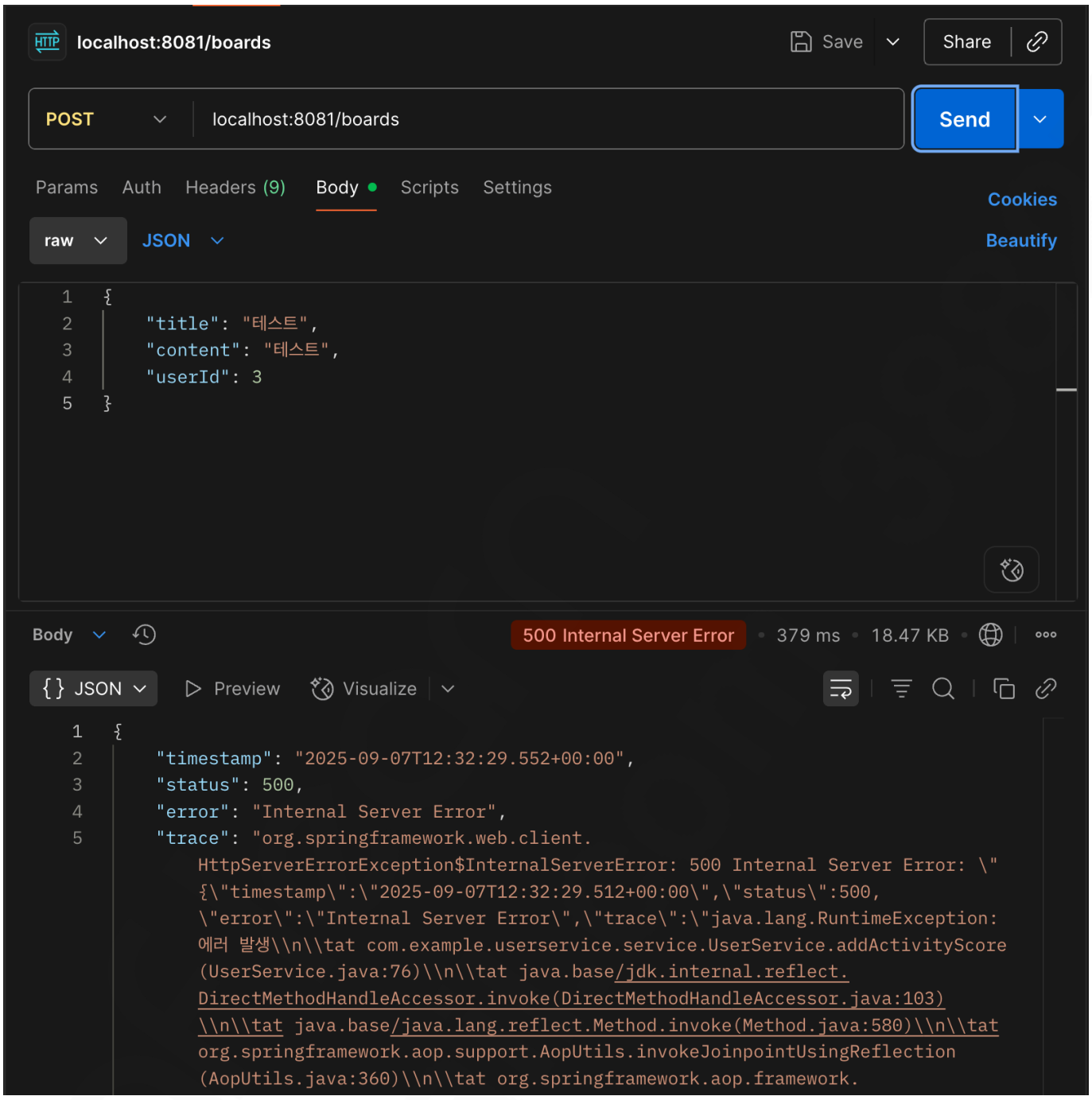
4. **활동 점수 적립 API에 예외 코드 잘 있는 지 확인**

   이전에 트랜잭션을 테스트하기 위해, '포인트 적립 → 게시글 작성 → 활동 점수 적립'의 작업 중에 '활동 점수 적립'에서 예외가 발생하도록 만들었었다.

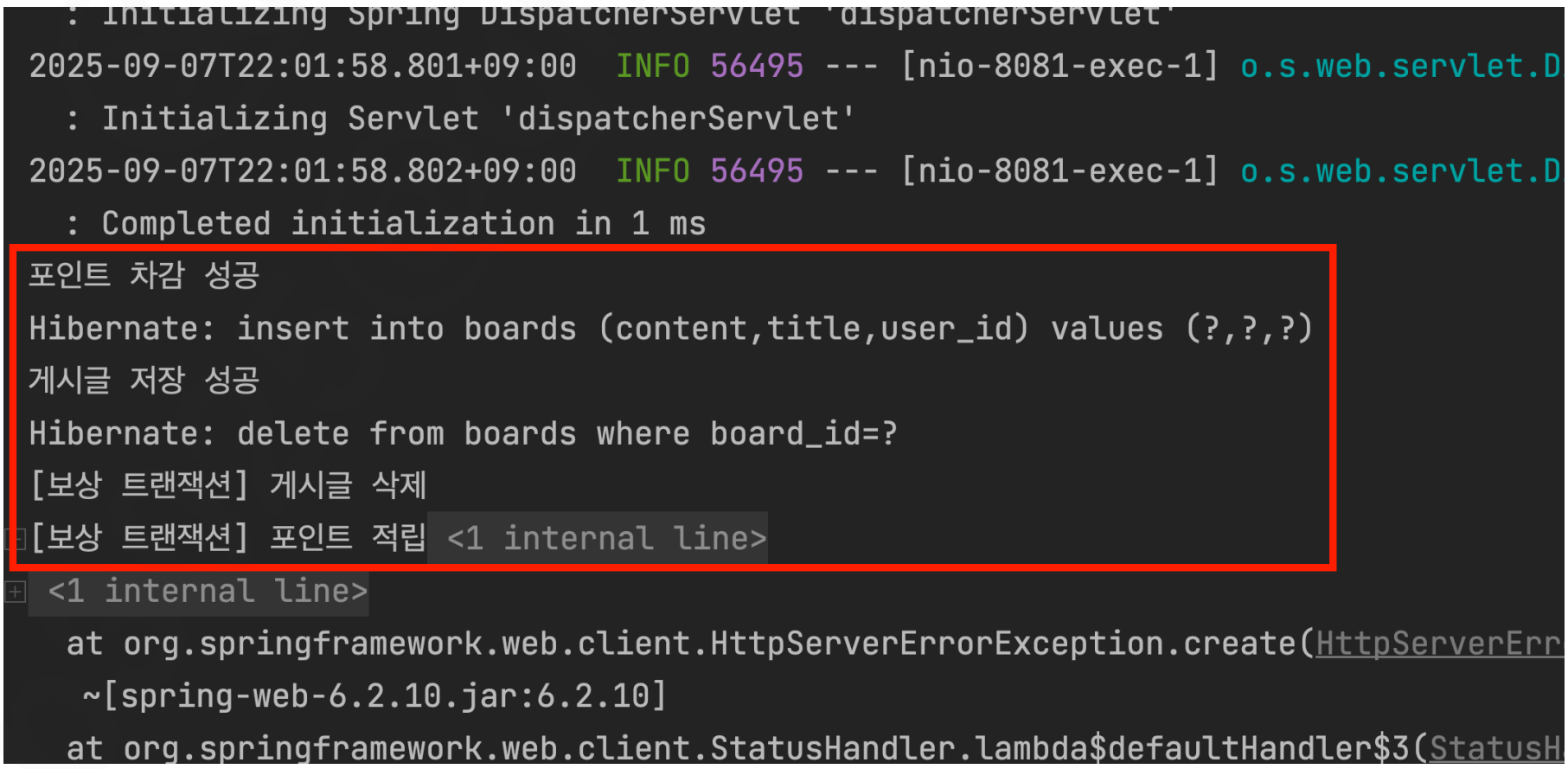
   service/UserService

```
@Service
public class UserService {
    ...

    @Transactional
  public void addActivityScore(
      AddActivityScoreRequestDto addActivityScoreRequestDto
  ) {
    User user = userRepository.findById(addActivityScoreRequestDto.getUserId())
        .orElseThrow(() -> new IllegalArgumentException("사용자를 찾을 수 없습니다."));

    user.addActivityScore(addActivityScoreRequestDto.getScore());

    userRepository.save(user);
    throw new RuntimeException("에러 발생");
  }
}
```

5. **게시글 작성 API 테스트해보기**

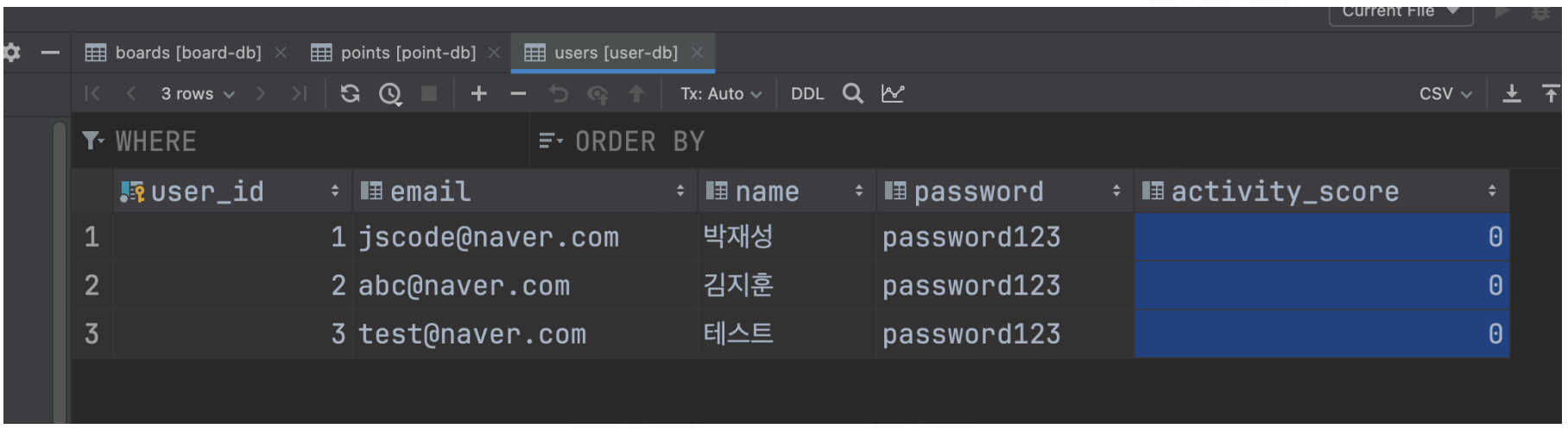


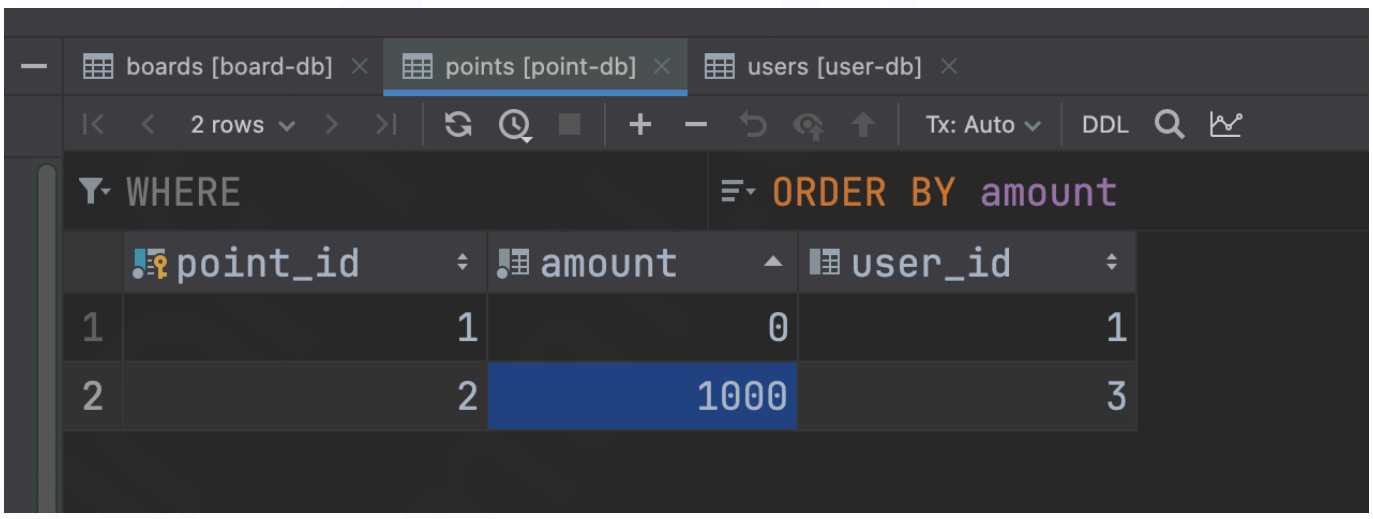
7. **board-service의 로그 확인하기**



로그만 봤을 때는 정상적으로 모든 작업이 잘 취소된 것처럼 보인다. DB도 확인해보자.

8. **DB 확인하기**





중간에 에러가 발생했을 때 일부 작업만 취소되지 않고, '포인트 차감, 게시글 작성, 활동 점수 적립'의 모든 작업이 취소됐다는 걸 알 수 있다.

9. **에러 발생하는 코드 주석 처리하기**

   그럼 이번에는 에러 코드를 주석 처리하고, 정상적으로 '게시글 작성 API'가 잘 처리되는 지도 확인해보자.

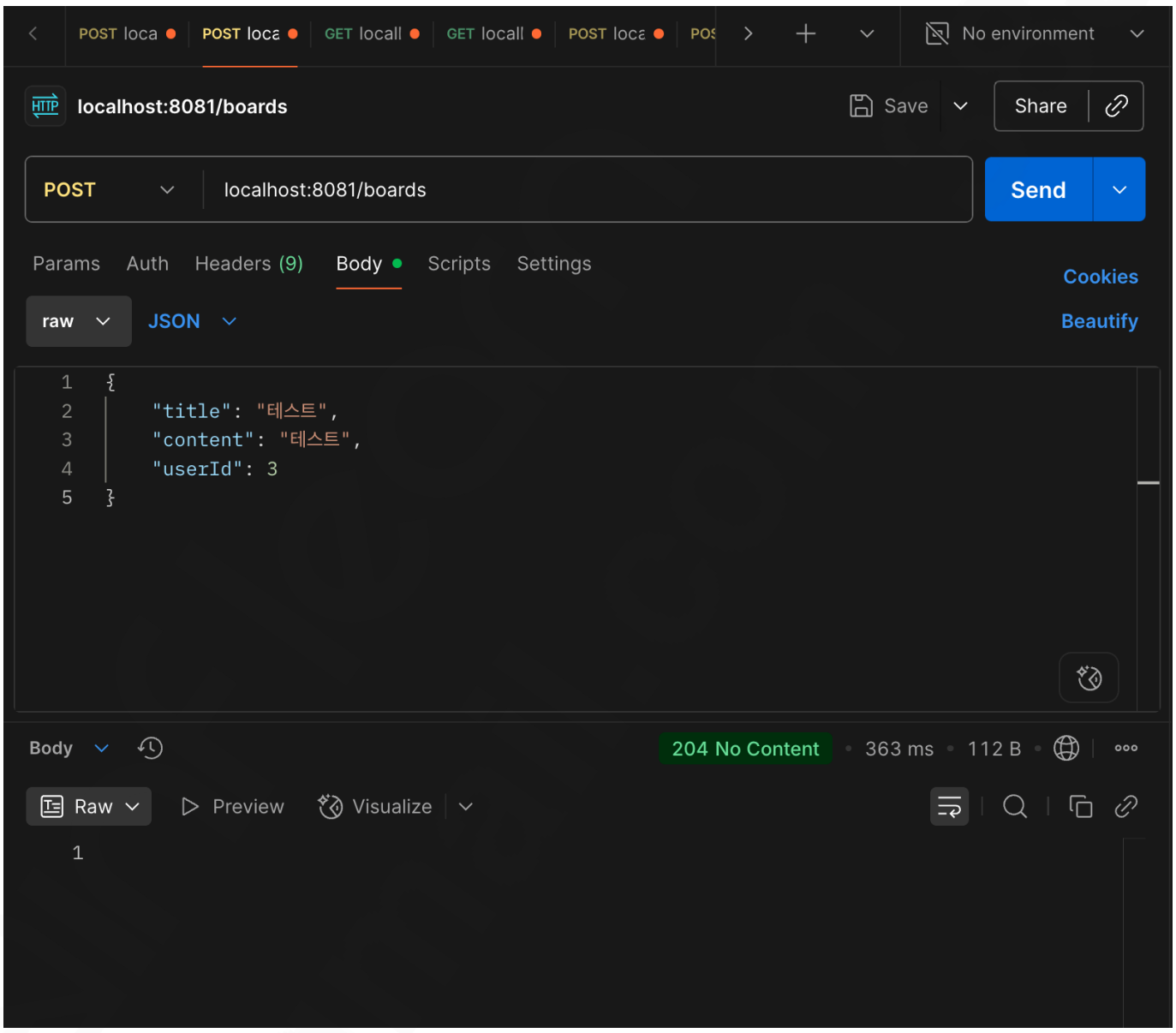
   service/UserService

```
@Service
public class UserService {
    ...

    @Transactional
  public void addActivityScore(
      AddActivityScoreRequestDto addActivityScoreRequestDto
  ) {
    User user = userRepository.findById(addActivityScoreRequestDto.getUserId())
        .orElseThrow(() -> new IllegalArgumentException("사용자를 찾을 수 없습니다."));

    user.addActivityScore(addActivityScoreRequestDto.getScore());

    userRepository.save(user);
    // throw new RuntimeException("에러 발생");
```

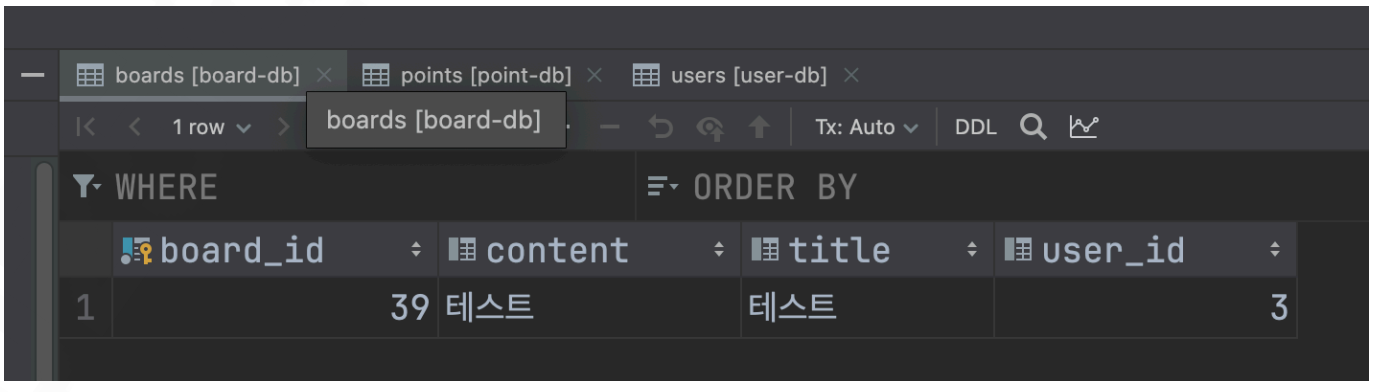
```
  }
}
```

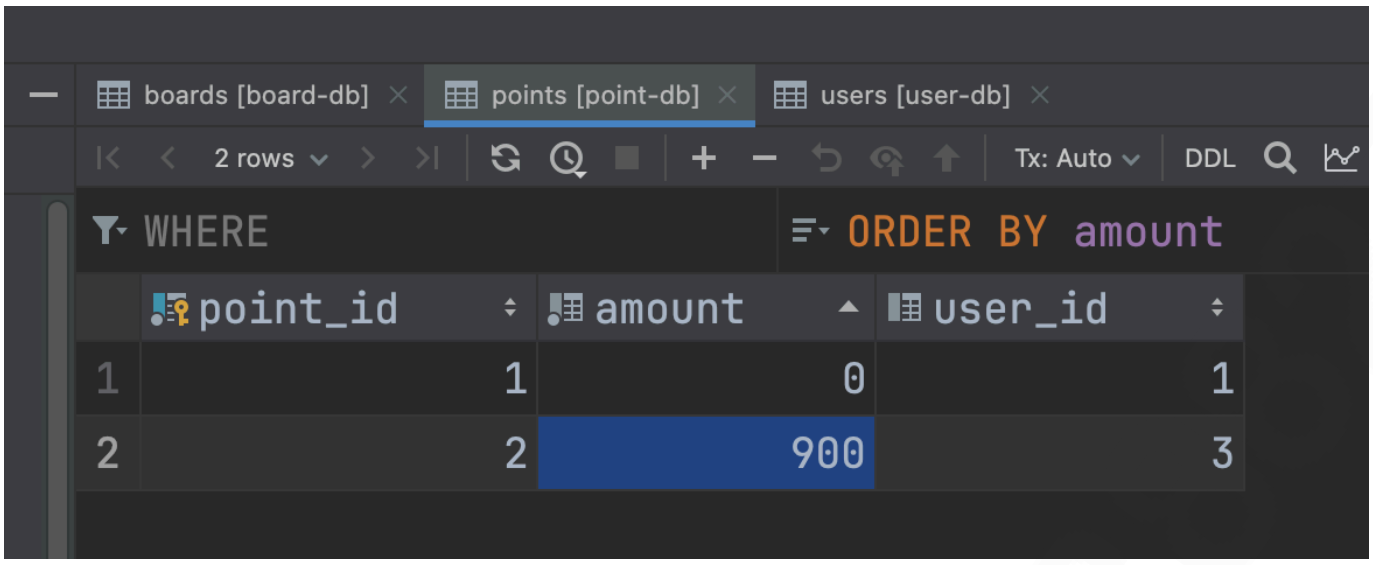
10. **user-service 서버 다시 실행시키기**

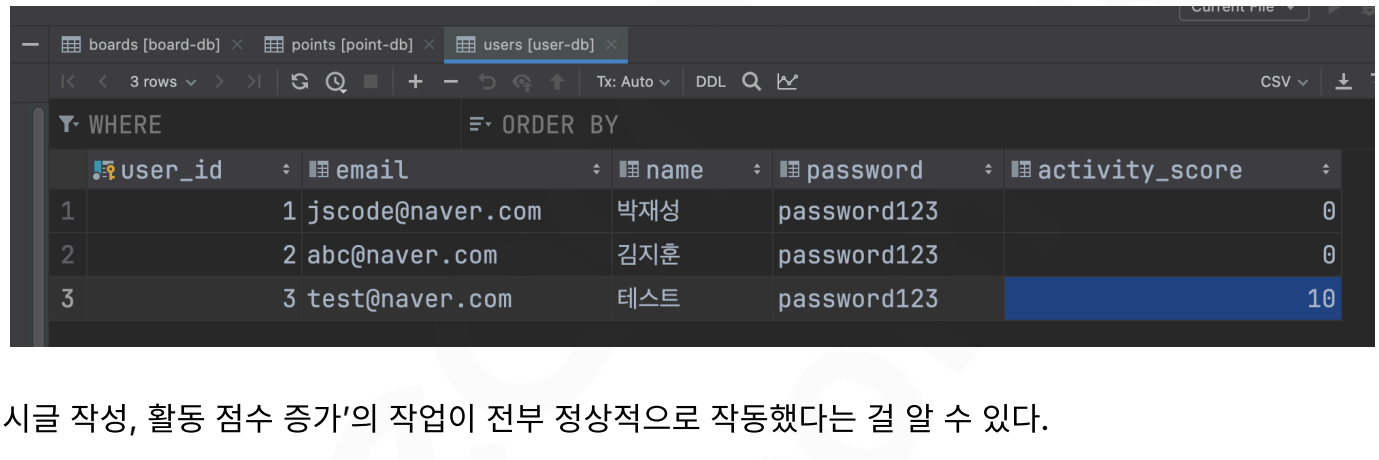
5. **게시글 작성 API 테스트해보기**



12. **DB 확인하기**



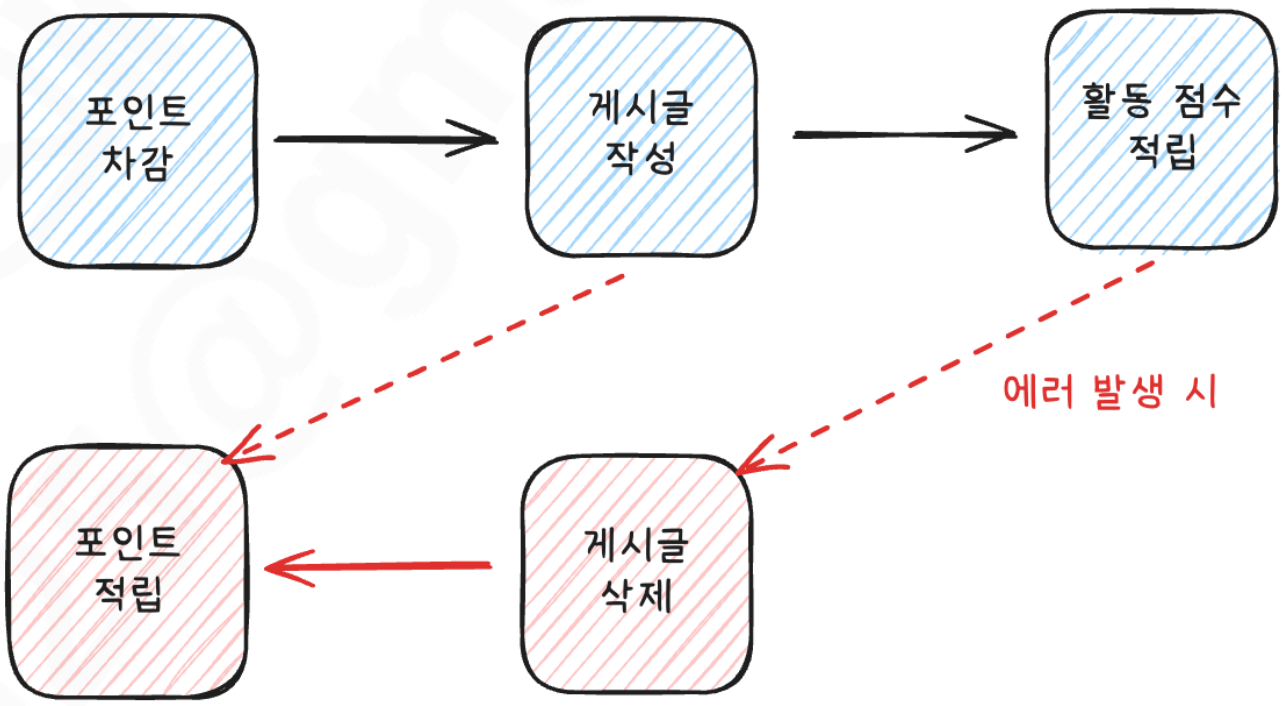




'포인트 차감, 게시글 작성, 활동 점수 증가'의 작업이 전부 정상적으로 작동했다는 걸 알 수 있다.

✅ 정리

Saga 패턴을 활용해 MSA 환경에서도 **모든 작업이 하나의 트랜잭션으로 처리되는 것처럼 만들었다.** 즉, **모든 작업이 아예 성공하거나 아예 실패하도록 만들었다.** 이로써 MSA 환경에서도 데이터 일관성을 지킬 수 있게 됐다.



📖 Eventual Consistency(최종적 일관성)이란?

✅ Eventual Consistency(최종적 일관성)이란?

> MSA 환경에서 Saga 패턴을 기반으로 로직을 처리하다보면 어쩔 수 없이 데이터 일관성이 잠깐동안 깨지는 순간이 발생한다. 이 현상을 눈으로 확인하기 위해 코드를 일부 수정해보자.

1. **활동 점수 증가 API 로직 수정하기**

   게시글 작성 API는 '포인트 차감 → 게시글 작성 → 활동 점수 적립'의 순서로 로직을 처리한다. 이 때, '활동 점수 적립' 작업을 처리하려고 하는 데, 트래픽이 몰려서 처리하는 데 시간이 오래걸리는 상황이라고 가정하자.

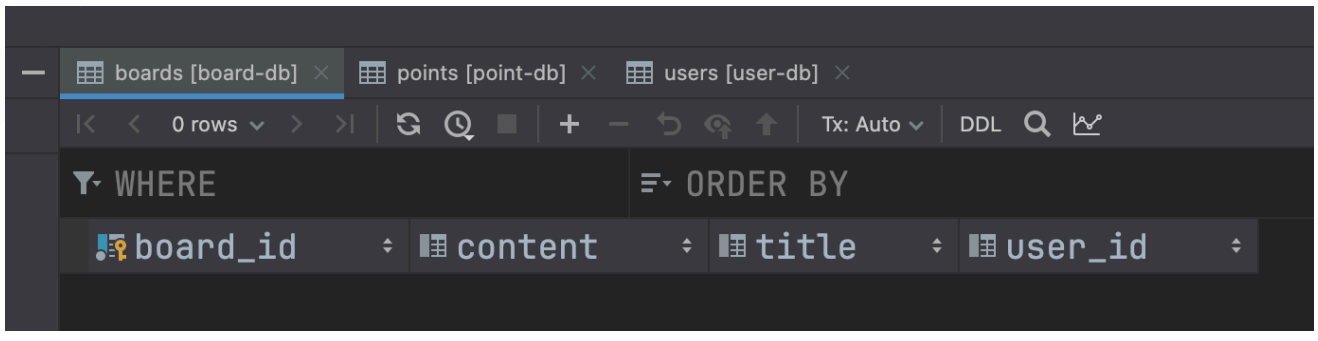
   service/UserService

```
@Transactional
public void addActivityScore(
    AddActivityScoreRequestDto addActivityScoreRequestDto
) {
  User user = userRepository.findById(addActivityScoreRequestDto.getUserId())
      .orElseThrow(() -> new IllegalArgumentException("사용자를 찾을 수 없습니다."));

  user.addActivityScore(addActivityScoreRequestDto.getScore());

  userRepository.save(user);

  // 10초 대기
  try {
    Thread.sleep(10000);
  } catch (Exception e) {}

    // throw new RuntimeException("에러 발생");
}
```

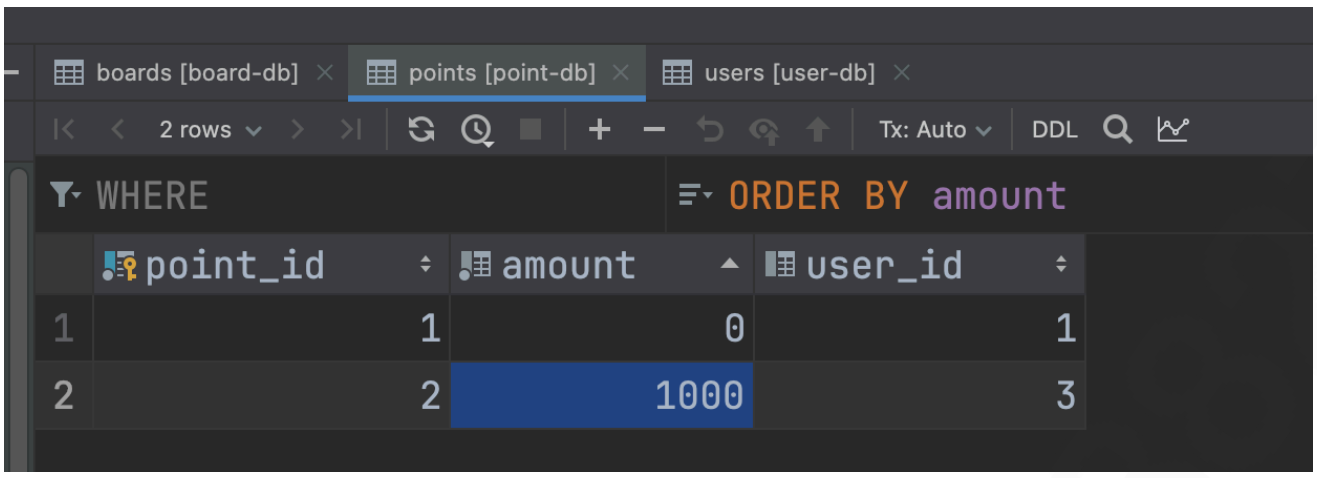
2. **user-service 다시 실행시키기**

3. **테스트를 위해 DB 데이터 정리하기**

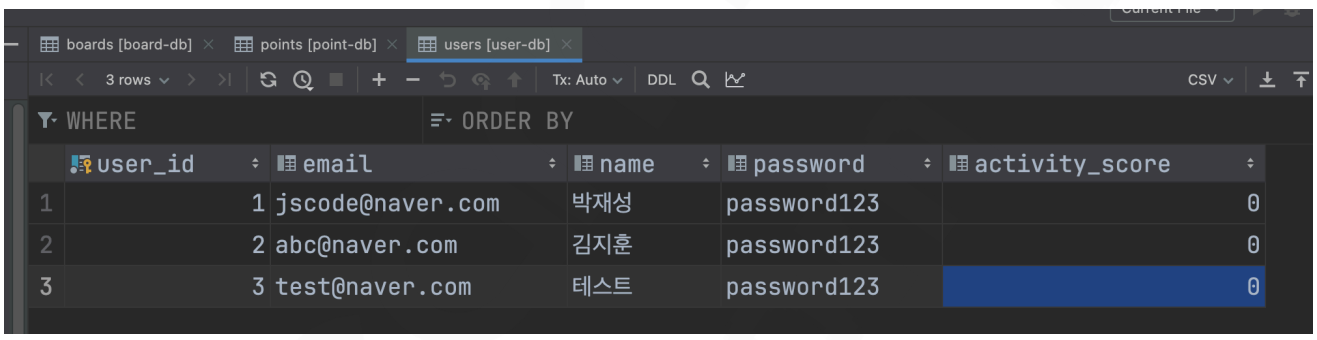
   a. **게시글 데이터 전부 삭제하기**



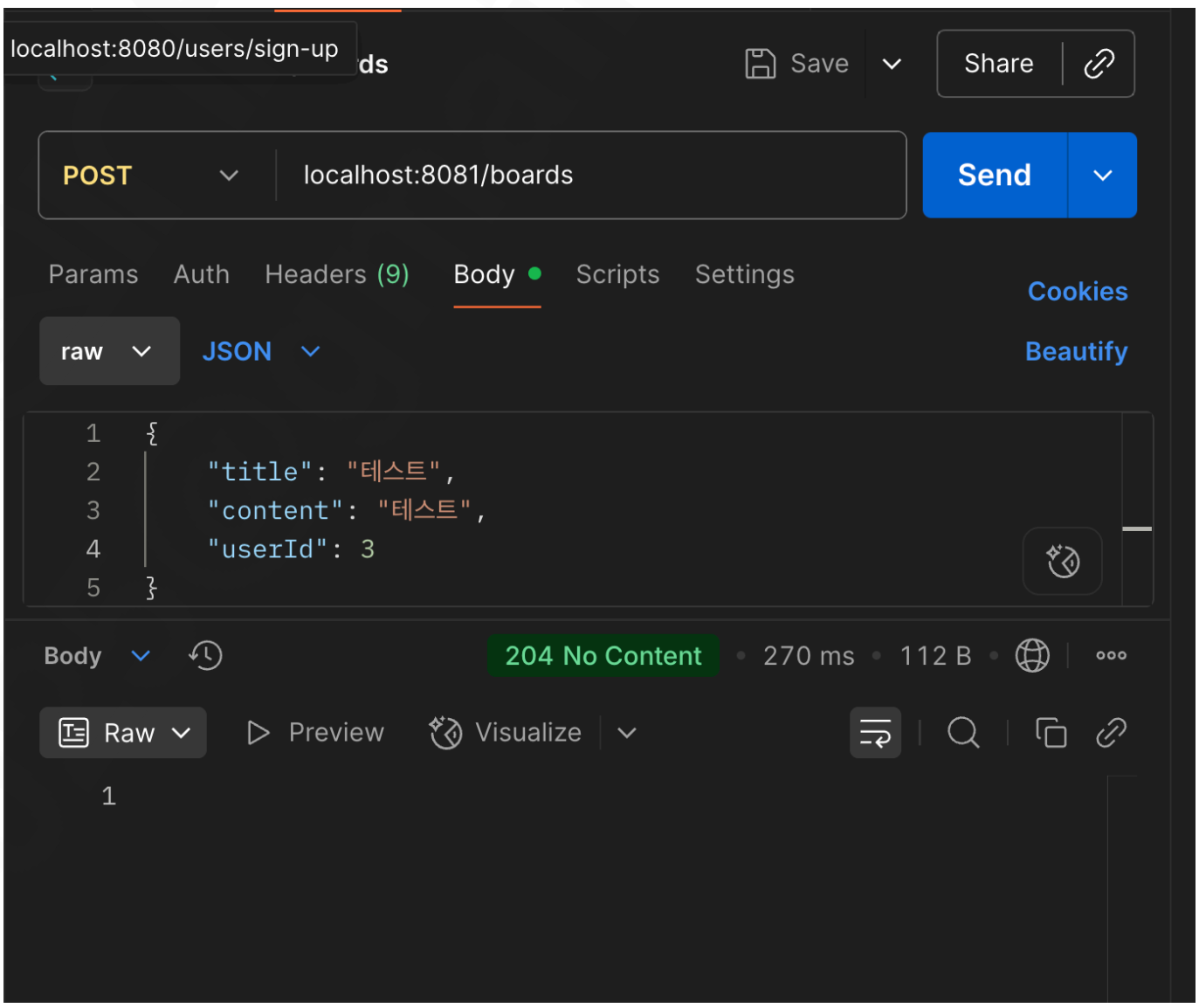
   b. **포인트 데이터 1000점으로 맞추기**



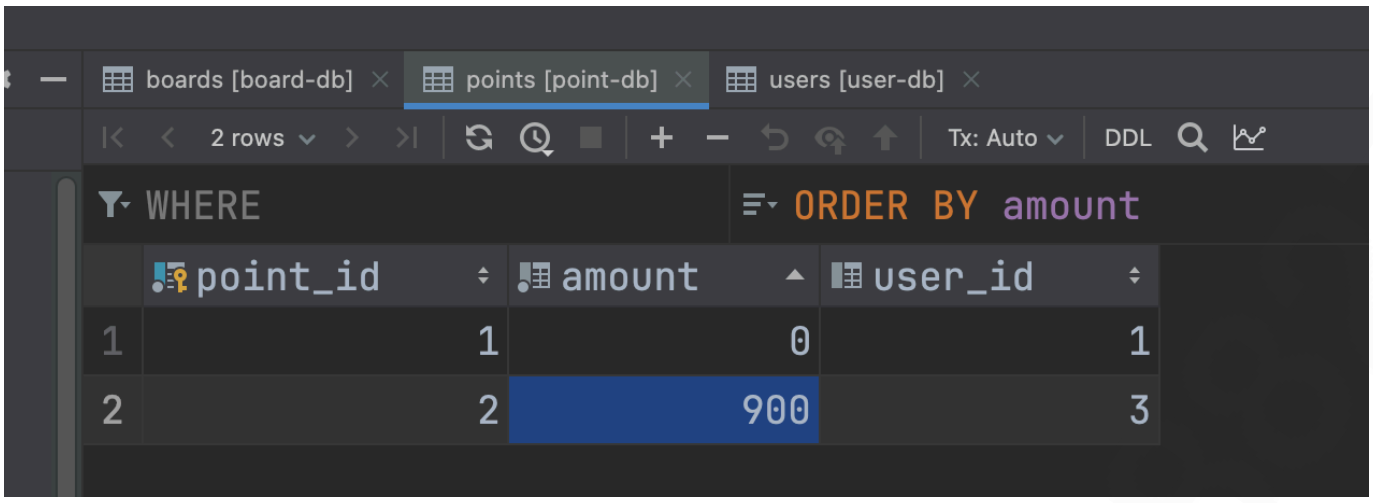
c. **사용자 활동 점수 0점으로 바꾸기**

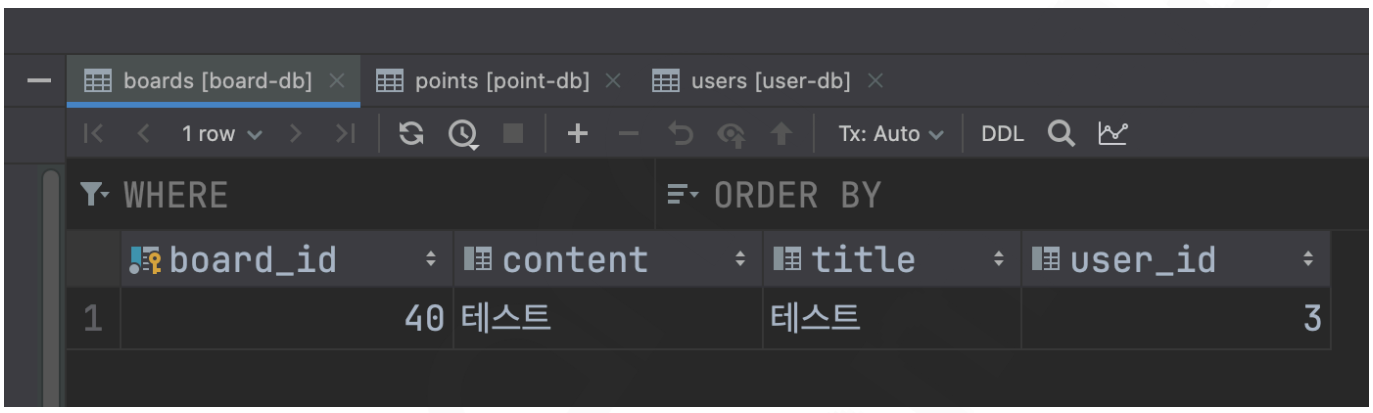


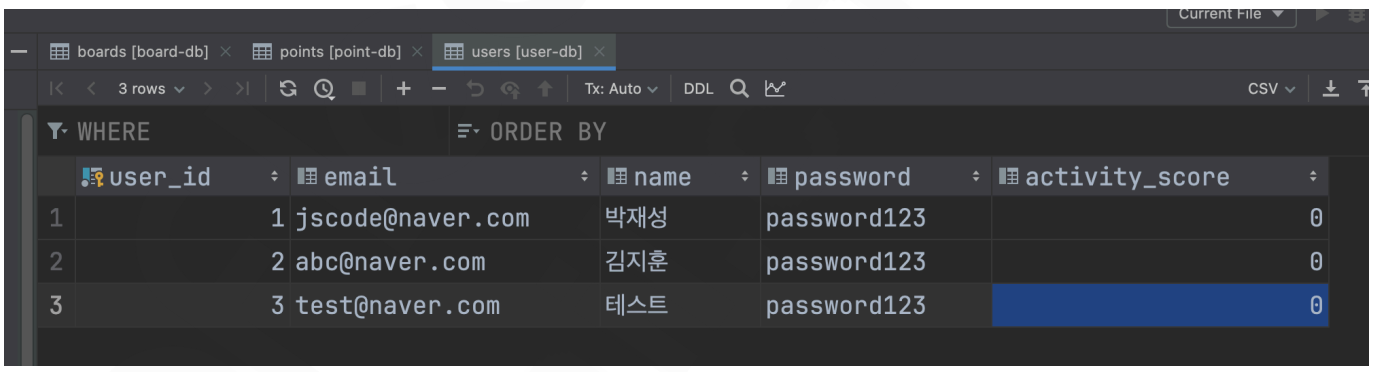
4. **API 요청 보내자마자 DB 다시 확인하기**



[API 요청 보낸 직후에 조회한 DB 데이터]



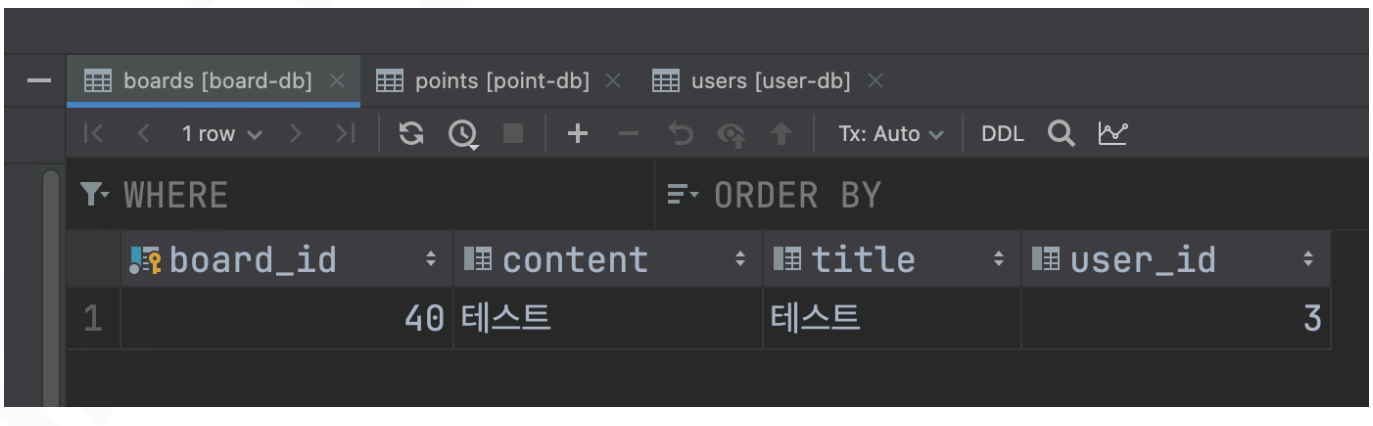


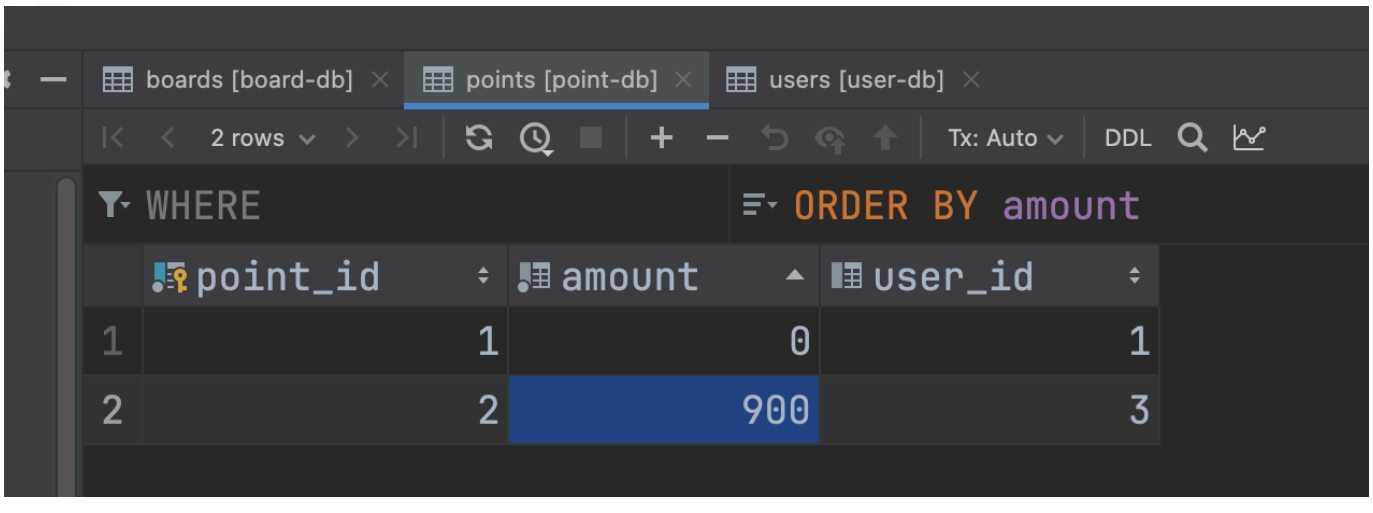


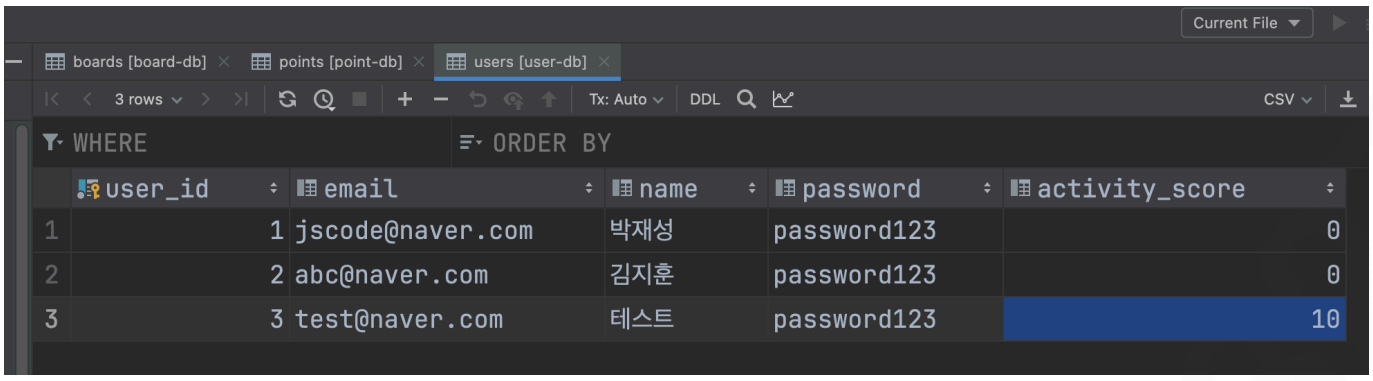
API 요청을 보낸 직후에 DB 데이터를 조회해보면 데이터 일관성이 맞지 않는 순간이 발생한다. 즉, 데이터만 봤을 때 서비스 기획상 모순되는 상황(게시글은 작성됐는 데 활동 점수만 적립되지 않은 상황)이 발생했다.

> 포인트 차감 O, 게시글 데이터 O, **활동 점수 적립 X**

[API 요청 보내고 10초 이후에 조회한 DB 데이터]







하지만 일정 시간(10초)이 지난 이후에 DB 데이터를 다시 조회해보니 데이터 일관성이 맞아졌다.

> 포인트 차감 O, 게시글 데이터 O, **활동 점수 적립 O**

이런 것처럼 **처음에는 잠깐 불일치하는 데이터가 발생하지만, 일정 시간 이후에는 모든 데이터의 일관성이 맞아지는 현상**을 보고 **Eventual Consistency(최종적 일관성)**이라고 한다.

✅ Eventual Consistency에서 데이터 일관성이 잠깐 깨져도 괜찮은 이유

Eventual Consistency(최종적 일관성)는 처음에는 데이터 일관성이 맞지 않는 순간이 잠깐 생겼다가, 일정 시간 이후에는 모든 데이터 일관성이 맞아지는 현상을 의미한다. 이 개념을 처음 들으면 이런 생각이 들 수도 있다.

> **"잠깐이라도 데이터가 불일치한 상태가 존재해도 되는거야?**
> **그 사이에 누가 불일치한 상태의 데이터에 접근하면 문제되는 거 아냐?"**

그런데 실무에서 이런 Eventual Consistency를 허용해도 서비스상 크게 문제가 되지 않는 경우가 많다. 실제 규모 있는 기업의 사례를 살펴보자.

- Instagram에서 게시글을 작성하면, 다른 사용자에게 바로 보이는 게 아니라 일정 시간 이후에 보인다.
  - 작성한 게시글이 바로 즉시 안 보인다고 해서 큰 문제 될 게 없다.
- Instagram에서 누군가가 게시글에 '좋아요'를 눌렀는데, 친구에게는 그게 즉시 반영되지 않고 일정 시간 이후에 반영된다.
  - 좋아요 누른 게 바로 즉시 안 보인다고 해서 큰 문제 될 게 없다.
- Amazon에서 누군가가 재고가 1개 남은 상품을 구매했는데, 일정 시간 동안 "재고 있음"으로 보일 수 있다.
  - 재고가 있는 줄 알고 결제를 시도하지만, 서버가 결제 전에 재고를 체크했는데 재고가 없다고 판단했다고 치자. 그럼 사용자에게 "죄송합니다. 품절된 상품입니다"의 메시지와 함께 결제를 거절하면 된다.
    (사용자 입장에서 살짝 불편할 수는 있으나 큰 문제가 되진 않는다.)

실제 서비스에서도 Eventual Consistency의 방식으로 구현이 되어 있는데도 불구하고 데이터의 일관성이 깨진 순간을 체감하지 못하는 이유는, 데이터 일관성이 깨지는 순간이 너무 찰나의 순간이기 때문이다. 서버 자체에 장애가 발생한 게 아니라면 데이터 일관성이 깨지는 순간이 1초를 넘지 않는 경우가 대부분이다.

✅ 잠깐이라도 데이터 일관성이 깨지면 안 될 때는 어떻게 하나요?

Saga 패턴을 적용하면서 Eventual Consistency로 인해 데이터의 일관성이 깨지는 찰나의 순간 조차도 막는 방법이 존재한다. 하지만 이걸 실제로 코드로 구현하려면 생각보다 많이 복잡해지기 때문에 MSA 입문 강의에서는 생략할 예정이다. 너무 궁금한 분들은 아래 링크를 참고하도록 하자.

스프링캠프 2025 [Track 1] 4. 실전! MSA 트랜잭션 개발 가이드 (김용욱)

🎙️ 김용욱 (삼성 SDS)
🗓️ 스프링캠프 2025 Track 1 - 4 (2025년 06월 28일)
🧭 스페이스쉐어 삼성
https://youtu.be/0UiE9YJi_f4

@naver.com
@gmail.com
khb302

📖

[참고] Saga 패턴을 구현할 때 고려해야 할 요소 (멱등성, 동시성 이슈 등)

✅ Saga 패턴을 구현할 때 고려해야 할 요소 (멱등성, 동시성 이슈, 시맨틱 락)

지금까지 Saga 패턴의 개념을 이해하고 Saga 패턴을 직접 구현해봤다. 하지만 Saga 패턴을 더 꼼꼼하고 완벽하게 구현하려면 고려해야 하는 요소들이 훨씬 많다.

실제 MSA 환경에서는 네트워크 장애, 특정 서비스 장애, 동시에 여러 번의 요청 등 다양한 예외 상황이 발생할 수 있다. 일어날 가능성이 작긴 하지만 혹시 모를 상황을 대비해 방어 로직을 짜는 편이다.

대표적으로 아래와 같은 요소를 고려해서 구현해야 한다.

- 멱등성
- 동시성 이슈
- 격리성(Isolation) 문제

위 요소들을 고려해서 Saga 패턴을 다루게 되면 난이도가 확 올라간다. 이런 이유 때문에 입문자들이 MSA에서 Saga 패턴을 공부하다가 많이 포기한다. 그래서 위 요소들을 고려해서 꼼꼼하게 Saga 패턴을 구현하는 건 MSA 후속 강의에서 따로 다루도록 하겠다. 이번 강의에서는 MSA의 전체 흐름에 익숙해지는 것을 목표로 하자.

khb3025@gmail.com

📖

사전 필요 지식

✅ 참고

이전 섹션에서 데이터 생성 및 변경을 처리할 때, 여러 마이크로 서비스를 거쳐서 통신을 해야 하는 Case를 경험했다. 그리고 여러 마이크로 서비스를 거쳐서 처리하는 여러 작업을 하나의 트랜잭션처럼 묶기 위해서 Saga 패턴을 활용해서 코드를 작성했다. 이 때, 서로 마이크로 서비스끼리 통신할 때 REST 기반의 동기 방식으로 통신을 했었다.

하지만 이번 섹션에서는 REST 기반의 동기 방식이 아닌, **메시징 기반의 비동기 방식으로 통신하는 방식**을 배워볼 것이다. 메시징 툴로는 **Kafka**를 활용할 예정이다.

따라서 강의 소개에서도 언급했다시피 **Kafka**를 이미 알고있다는 전제하에 강의를 진행할 예정이다. 혹시나 Kafka와 같은 메시징 툴을 아예 경험해보지 못했다면 아래 강의를 듣고 이어서 수강하는 걸 추천한다.

Kafka Intro: Hands-on| jscode - Course on Inflearn

This course has a rating of 5.0 and 306 students. I've created a 'Kafka Introduction' course that is easy for non-technical users to understand and imme

https://inf.run/AVnvB

khb3025@gmail.com

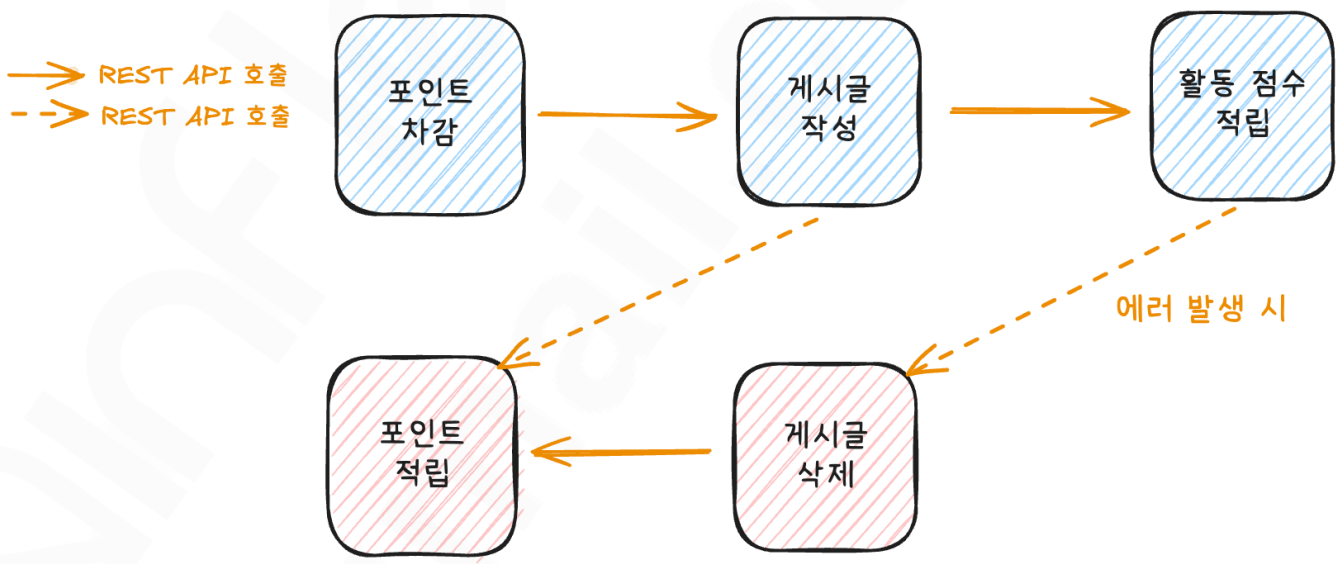
📖

MSA 환경에서 언제 메시징 기반(비동기 방식)으로 통신하면 좋을까?

👨🏻‍🏫 MSA 환경에서는 마이크로서비스끼리 통신을 해야 한다. 이 때, 통신 방식은 크게 **동기**와 **비동기** 방식으로 나뉜다. 이전까지 프로젝트에서 사용했던 **REST API 기반의 통신 방식**이 대표적인 **동기 통신 방식**이 있다. 그리고 Kafka와 같은 **메시지 큐를 활용해 메시징 기반으로 통신하는 방식**이 **비동기 통신 방식**이다.

MSA 환경에서는 이 2가지 통신 방식 중 하나의 통신 방식만 사용해야 하는 건 아니다. 두 가지 방식을 혼용해서 사용하는 경우가 많다. 그렇기 때문에 언제 동기 방식으로 통신하는 게 좋고, 언제 비동기 방식으로 통신하는 지를 판단할 수 있어야 한다. 그 차이점에 대해 알아보자.

✅ MSA 환경에서 언제 메시징 기반(비동기 방식)으로 통신하면 좋을까?



이전 프로젝트에서 게시글 작성 API를 구현할 때, 각 마이크로서비스는 **REST API 기반의 동기적으로 통신하는 방식**을 활용했다. 이 통신 방식의 가장 큰 특징은 '**요청'을 보내면 '응답'이 날라온다**는 특징이 있다.

이 특징 때문에 동기 방식의 통신은 '**요청을 보낸 뒤에 응답을 받아야만 하는 경우**'에 많이 활용한다. 기존에 구현한 게시글 작성 API 로직을 기반으로 생각해보자.

1. 사용자가 게시글 작성 API에 요청을 보내고 나서 게시글 작성이 완료됐는 지를 사용자에게 응답해주어야 하기 때문에 동기 방식으로 통신해야 한다.
2. '포인트 차감'이 확실하게 완료된 걸 확인한 뒤에 '게시글 작성' 작업을 해야 하기 때문에 동기 방식으로 통신해야 한다.

그럼 메시징 기반의 비동기 통신 방식은 언제 사용하면 좋을까? 이 방식은 요청 처리 결과에 대한 응답을 굳이 받지 않아도 되는 경우에 많이 사용한다. 기존에 구현한 게시글 작성 API 로직을 기반으로 생각해보자.

1. 사용자가 게시글 작성 API('포인트 차감 → 게시글 작성 → 활동 점수 적립')에 요청을 보냈을 때, '포인트 차감 → 게시글 작성' 로직까지만 확실하게 작동한 걸 확인하고 성공 응답을 바로 내려줘도 된다. 한 마디로 '활동 점수 적립' 처리가 끝날 때까지 기다렸다가 응답해줄 필요는 없다. '활동 점수 적립'은 메시징 기반의 비동기 방식으로 처리해도 된다.
2. 사용자가 게시글 작성 API에 요청을 보냈을 때, 특정 작업이 실패하면 보상 트랜잭션을 실행시켜야 한다. 그런데 보상 트랜잭션 처리가 전부 끝날 때까지 기다렸다가 실패 응답을 해줄 필요는 없다. 즉, 사용자에게 먼저 실패 응답을 해준 뒤에 보상 트랜잭션은 따로 메시징 기반의 비동기 방식으로 처리해도 된다.

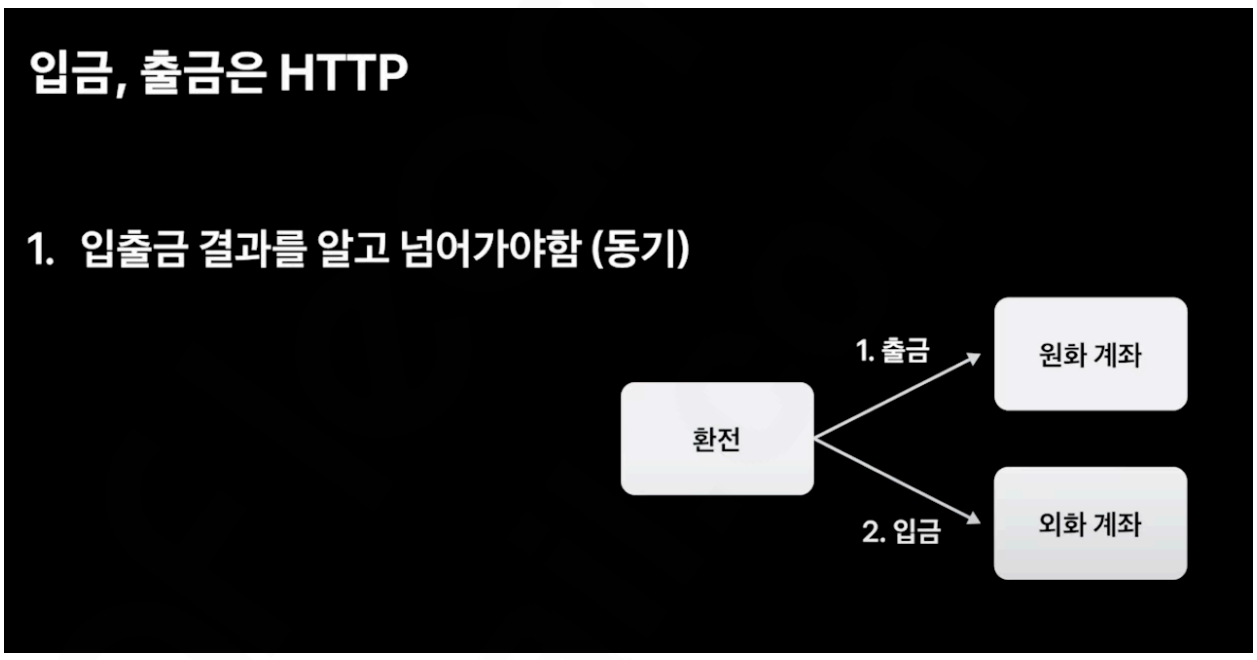
✅ MSA 환경에서 '동기 방식'으로 통신하는 게 잘못된 게 아니다!

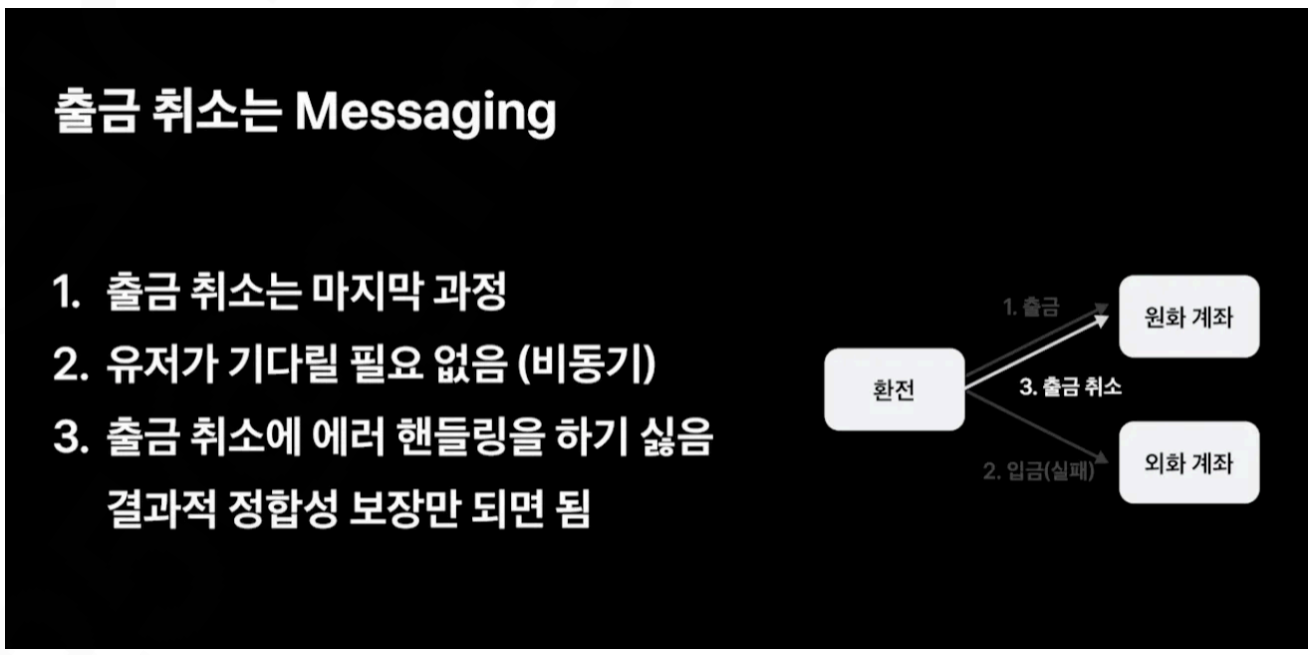
MSA에 관련된 강의와 책들을 찾아보면 대부분 이벤트를 기반으로 하는 비동기 방식을 위주로 서술해놨다. 그래서 사람들이 MSA 환경에서는 동기 통신 방식보다는 비동기 통신 방식 위주로 구현해야 한다고 착각한다. 하지만 위에서 설명했다시피 동기 통신 방식을 사용할 지, 비동기 통신 방식을 사용할 지 Case에 따라 판단하는 게 맞다.

실제 규모 있는 기업 중 토스의 사례를 살펴보자. 아래 영상을 보면 토스에서 환전 기능을 구현할 때도 동기 통신 방식과 비동기 통신 방식을 Case에 따라 혼용해서 사용한 걸 알 수 있다.

토스 | SLASH 24 - 보상 트랜잭션으로 분산 환경에서도 안전하게 환전하기
거대한 은행 계정계 시스템과 새로운 MSA 서비스는 어떻게 협력할 수 있을까요?
계정계 시스템의 원화 계좌와 분리된 외화 계좌를 오가는 환전 트랜잭션 구현법을 소개합니다.

https://youtu.be/xpwRTu47fqY?t=590



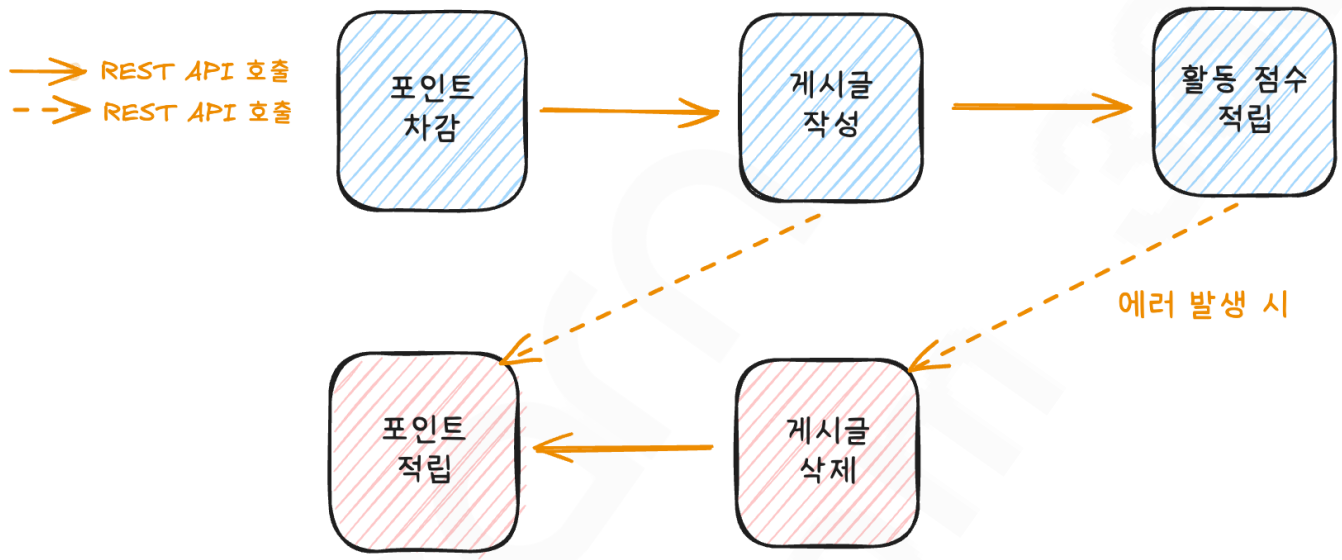


이처럼 MSA 환경에서 **메시징 기반의 비동기 방식**과 **REST API 기반의 동기 방식**을 **혼용해서 사용**하는 경우가 많다는 걸 기억하고 넘어가야 한다.

👨‍🏫 다음 강의에서는 **메시징 기반의 비동기 방식으로 통신하는 방법**에 대해 알아보자.

어떻게 구현할 지 설계하기

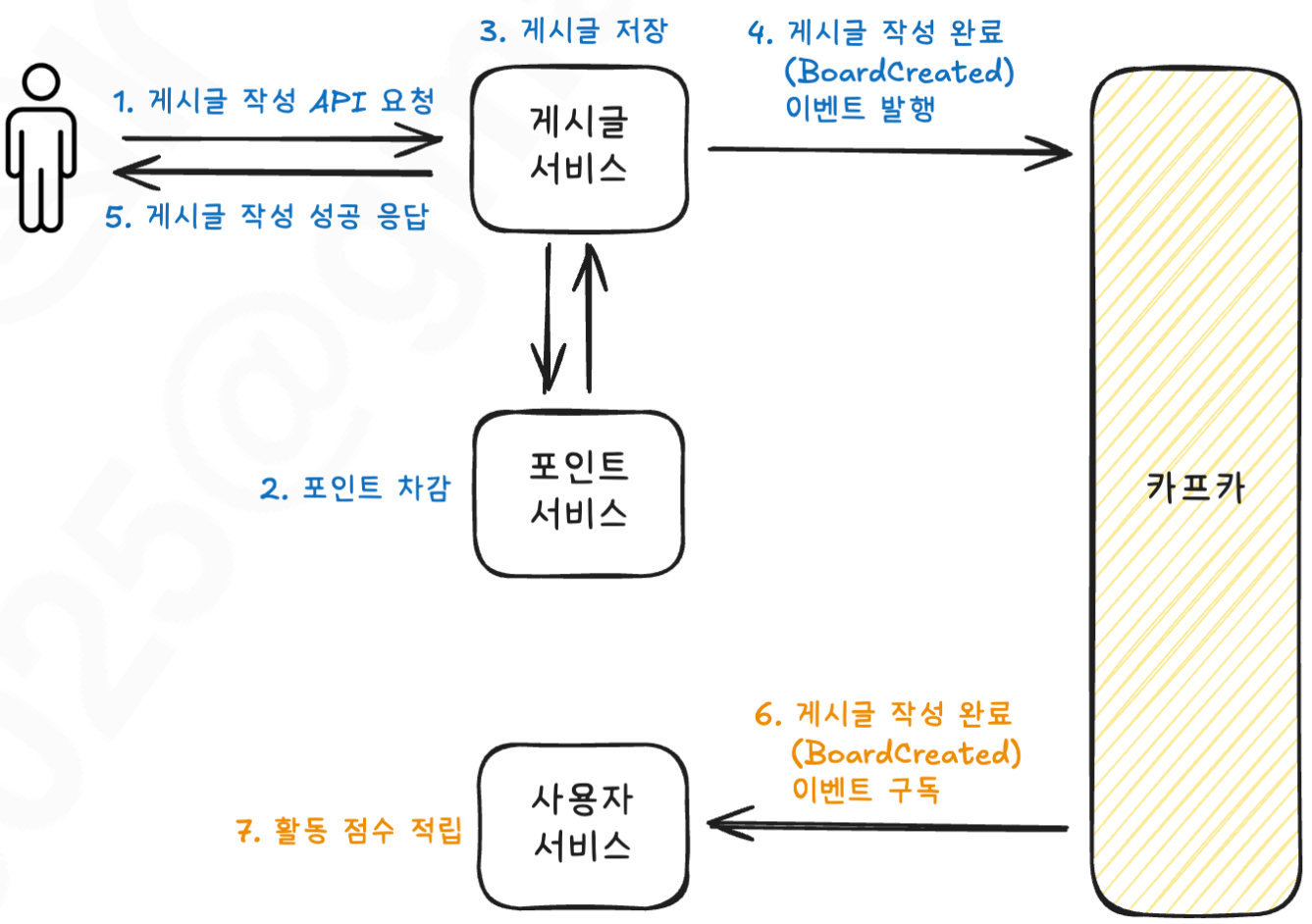
✅ 어떻게 구현할 지 설계하기



이전 강의에서 아래와 같이 메시징 기반의 비동기 방식으로 통신을 해도 괜찮다고 설명을 했었다.

> 사용자가 게시글 작성 API에 요청을 보냈을 때, '포인트 차감 → 게시글 작성' 로직까지만 확실하게 작동한 걸 확인하고 성공 응답을 내려줘도 된다. '활동 점수 적립' 처리가 끝날 때까지 기다렸다가 응답해줄 필요는 없다. 즉, '활동 점수 적립'은 메시징 기반의 비동기 방식으로 처리해도 된다.

위의 내용을 그림으로 표현하면 아래와 같다. 아래의 방식으로 코드를 수정해서 구현해보자.



khb3021

[실습] 게시글 서비스에서 Kafka로 '게시글 작성 완료 이벤트' 발행하기

✅ 게시글 서비스에 Kafka 연결하기

1. **Docker로 Kafka 띄우기**

```
$ docker run -d -p 9092:9092 apache/kafka:4.1.0

# 잘 실행됐는 지 확인하기
$ docker ps
```

- **참고)** Kafka를 편하게 셋팅하기 위해 Docker를 활용한 것이다.

2. **게시글 서비스에 Kafka 의존성 추가하기**

build.gradle

```
...

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.kafka:spring-kafka'
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    runtimeOnly 'com.mysql:mysql-connector-j'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

...
```

3. **application.yml에 Kafka 연결을 위한 정보 작성하기**

   application.yml

```
server:
  port: 8081

spring:
  datasource:
    url: jdbc:mysql://localhost:3307/board-db
    username: root
    password: password
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  kafka:
    # Kafka 서버 주소
    bootstrap-servers: localhost:9092
    # 게시글 서비스에서는 메시지를 produce 하기만 함
    producer:
      # 메시지의 key 직렬화 방식 : 자바 객체를 문자열(String)로 변환해서 Kafka에 전송
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      # 메시지의 value 직렬화 방식 : 자바 객체를 문자열(String)로 변환해서 Kafka에 전송
      value-serializer: org.apache.kafka.common.serialization.StringSerializer

client:
  user-service:
    url: http://localhost:8080
  point-service:
    url: http://localhost:8082
```

4. **Kafka로 전달할 메시지 객체 만들기**

   Kafka 메시지를 받아서 처리하는 곳(사용자 서비스의 활동 점수 적립 기능)에서 필요한 정보만 넘겨주면 된다.

   event/BoardCreatedEvent

```
public class BoardCreatedEvent {
  private Long userId;

  public BoardCreatedEvent(Long userId) {
    this.userId = userId;
  }

  public Long getUserId() {
    return userId;
  }
}
```

5. **게시글 작성을 완료하는 대로 Kafka 메시지 발행하도록 처리하기**

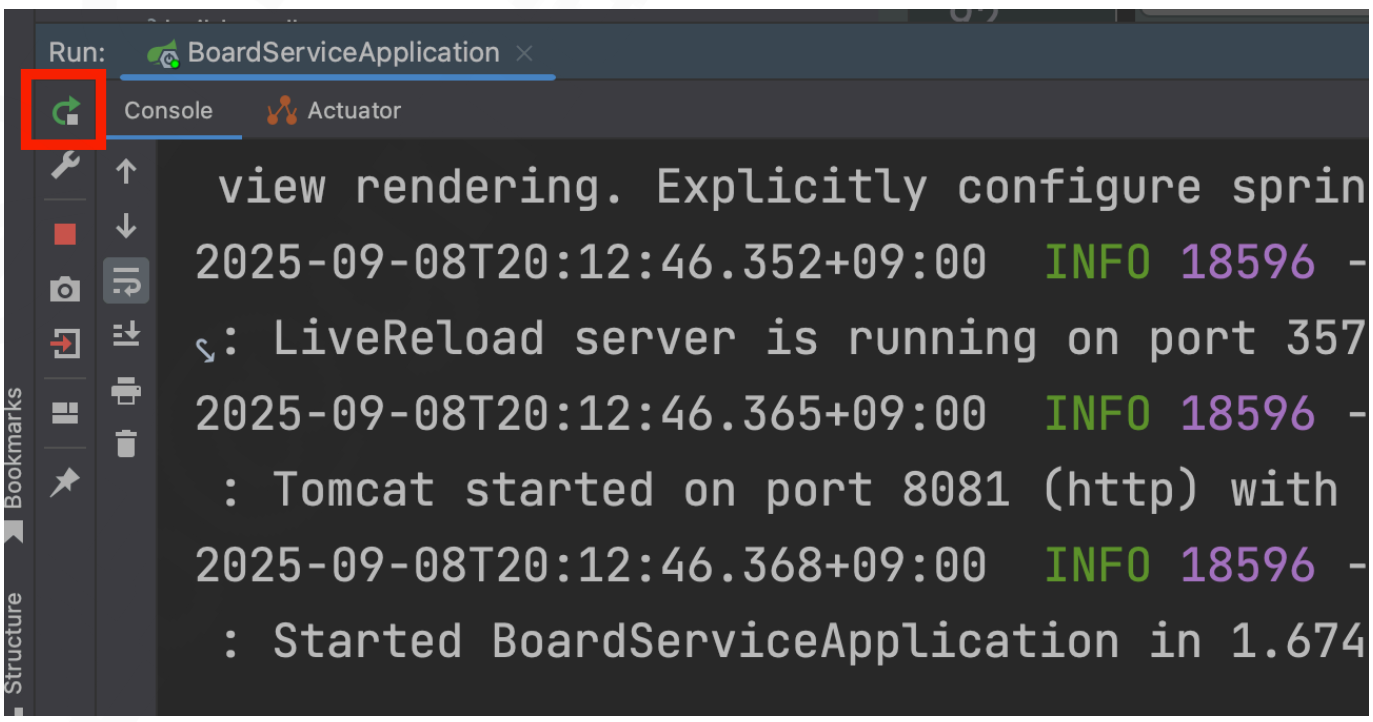
   service/BoardService

```
@Service
public class BoardService {
  private final BoardRepository boardRepository;
```

```
private final UserClient userClient;
private final PointClient pointClient;
private final KafkaTemplate<String, String> kafkaTemplate;

public BoardService(
    BoardRepository boardRepository,
    UserClient userClient,
    PointClient pointClient,
    KafkaTemplate<String, String> kafkaTemplate
) {
  this.boardRepository = boardRepository;
  this.userClient = userClient;
  this.pointClient = pointClient;
  this.kafkaTemplate = kafkaTemplate;
}

...

public void create(CreateBoardRequestDto createBoardRequestDto) {
  // 게시글 저장을 성공했는 지 판단하는 플래그
  boolean isBoardCreated = false;
  Long savedBoardId = null;

  // 포인트 차감을 성공했는 지 판단하는 플래그
  boolean isPointDeducted = false;

  try {
    // 게시글 작성 전 100 포인트 차감
    pointClient.deductPoints(createBoardRequestDto.getUserId(), 100);
    isPointDeducted = true; // 포인트 차감 성공 플래그
    System.out.println("포인트 차감 성공");

    // 게시글 작성
    Board board = new Board(
        createBoardRequestDto.getTitle(),
        createBoardRequestDto.getContent(),
        createBoardRequestDto.getUserId()
    );

    Board savedBoard = this.boardRepository.save(board);
    savedBoardId = savedBoard.getBoardId();
    isBoardCreated = true; // 게시글 저장 성공 플래그
    System.out.println("게시글 저장 성공");

    // 게시글 작성 시 작성자에게 활동 점수 10점 부여
    userClient.addActivityScore(createBoardRequestDto.getUserId(), 10);
    System.out.println("활동 점수 적립 성공");

    // '게시글 작성 완료' 이벤트 발행
    BoardCreatedEvent boardCreatedEvent
        = new BoardCreatedEvent(createBoardRequestDto.getUserId());
    this.kafkaTemplate.send("board.created", toJsonString(boardCreatedEvent));
    System.out.println("게시글 작성 완료 이벤트 발행");

  } catch (Exception e) {
    if (isBoardCreated) {
      // 게시글 작성 보상 트랜잭션 => 게시글 삭제
      this.boardRepository.deleteById(savedBoardId);
```

```
      System.out.println("[보상 트랜잭션] 게시글 삭제");
    }
    if (isPointDeducted) {
      // 포인트 차감 보상 트랜잭션 => 포인트 적립
      pointClient.addPoints(createBoardRequestDto.getUserId(), 100);
      System.out.println("[보상 트랜잭션] 포인트 적립");
    }

    // 실패 응답으로 처리하기 위해 예외 던지기
    throw e;
  }
}

// 객체를 Json 형태의 String으로 만들어주는 메서드
// (클래스로 분리하면 더 좋지만 편의를 위해 메서드로만 분리)
private String toJsonString(Object object) {
  ObjectMapper objectMapper = new ObjectMapper();
  try {
    String message = objectMapper.writeValueAsString(object);
    return message;
  } catch (JsonProcessingException e) {
    throw new RuntimeException("Json 직렬화 실패");
  }
 }
}
```

6. **서버 다시 실행시키기**



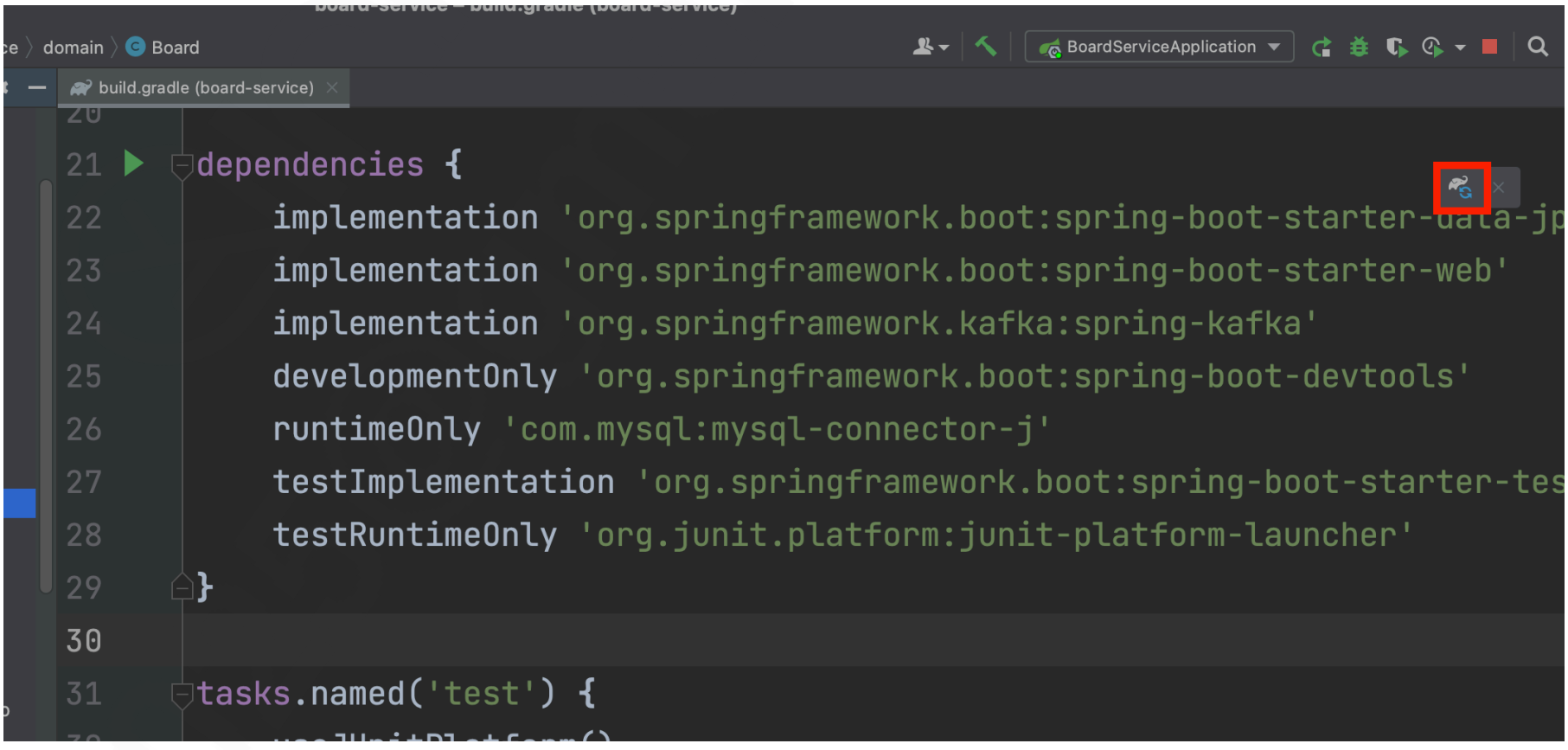
[실습] 사용자 서비스에서 Kafka로부터 '게시글 작성 완료 이벤트' 구독하기

✅ 사용자 서비스에서 Kafka로부터 '게시글 작성 완료 이벤트' 구독하기

1. **사용자 서비스에 Kafka 의존성 추가하기**

   build.gradle

```
...

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.kafka:spring-kafka'
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    runtimeOnly 'com.mysql:mysql-connector-j'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

...
```



2. **application.yml에 Kafka 연결을 위한 정보 작성하기**

   application.yml

```
server:
  port: 8080

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/user-db
```

```
    username: root
    password: password
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  kafka:
    # Kafka 서버 주소
    bootstrap-servers: localhost:9092
    # 사용자 서비스에서는 메시지를 consume 하기만 함
    consumer:
      # 메시지의 key 역직렬화 방식 : Kafka에서 받아온 메시지를 String으로 변환
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      # 메시지의 value 역직렬화 방식 : Kafka에서 받아온 메시지를 String으로 변환
      value-deserializer: org.apache.kafka.common.serialization.StringDeserializer

client:
  point-service:
    url: http://localhost:8082
```

3. **Kafka로부터 전달받을 메시지 객체 만들기**

   event/BoardCreatedEvent

```
public class BoardCreatedEvent {
  private Long userId;

  // 역직렬화(String 형태의 카프카 메시지 -> Java 객체)시 빈생성자 필요함
  public BoardCreatedEvent() {
  }

  // Json 값을 BoardCreatedEvent로 역직렬화하는 메서드
  public static BoardCreatedEvent fromJson(String json) {
    try {
      ObjectMapper objectMapper = new ObjectMapper();
      return objectMapper.readValue(json, BoardCreatedEvent.class);
    } catch (JsonProcessingException e) {
      throw new RuntimeException("JSON 파싱 실패");
    }
  }

  public Long getUserId() {
    return userId;
  }
}
```

4. **Kafka에 있는 메시지 Consume 하기**

   consumer/BoardCreatedEventConsumer

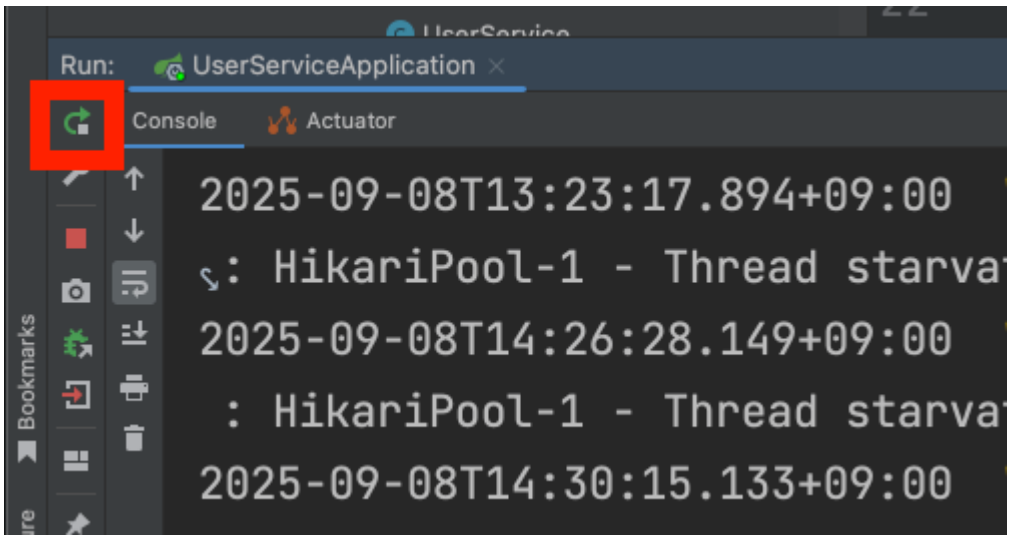
```
@Component
public class BoardCreatedEventConsumer {
  private final UserService userService;

  public BoardCreatedEventConsumer(UserService userService) {
```

```
  this.userService = userService;
}

@KafkaListener(
    topics = "board.created",
    groupId = "user-service"
)
public void consume(String message) {
  BoardCreatedEvent boardCreatedEvent
      = BoardCreatedEvent.fromJson(message);

  // 게시글 작성 시 활동 점수 10점 추가
  AddActivityScoreRequestDto addActivityScoreRequestDto
      = new AddActivityScoreRequestDto(
          boardCreatedEvent.getUserId(),
          10
      );
  userService.addActivityScore(addActivityScoreRequestDto);
    System.out.println("활동 점수 적립 완료");
  }
}
```

dto/AddActivityScoreRequestDto

```
public class AddActivityScoreRequestDto {
  private Long userId;
  private int score;

  public AddActivityScoreRequestDto(Long userId, int score) {
    this.userId = userId;
    this.score = score;
  }

  public Long getUserId() {
    return userId;
  }

  public int getScore() {
    return score;
  }
}
```

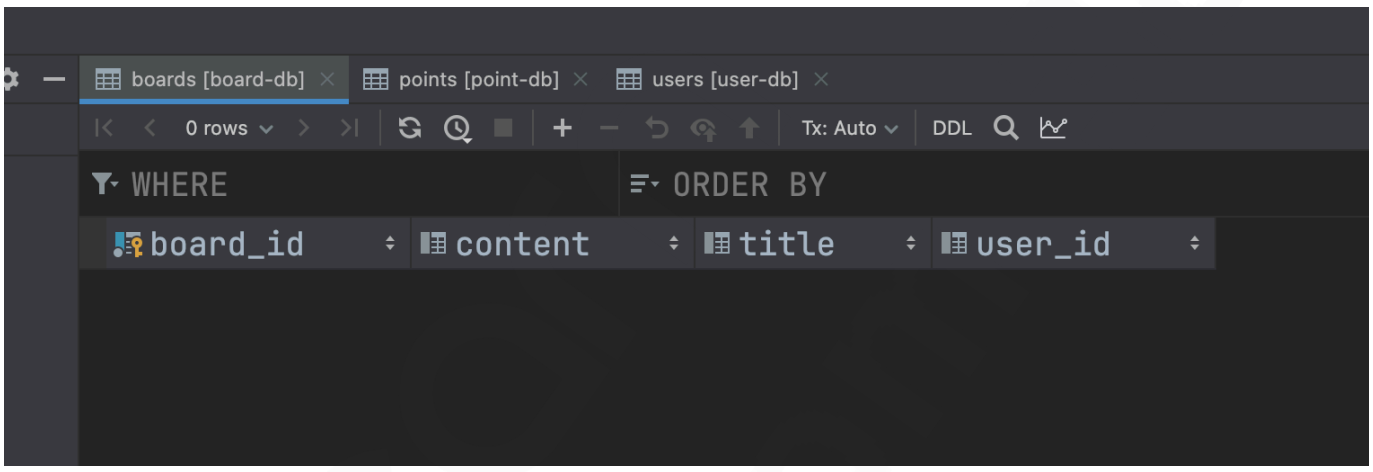
6. 서버 다시 실행시키기



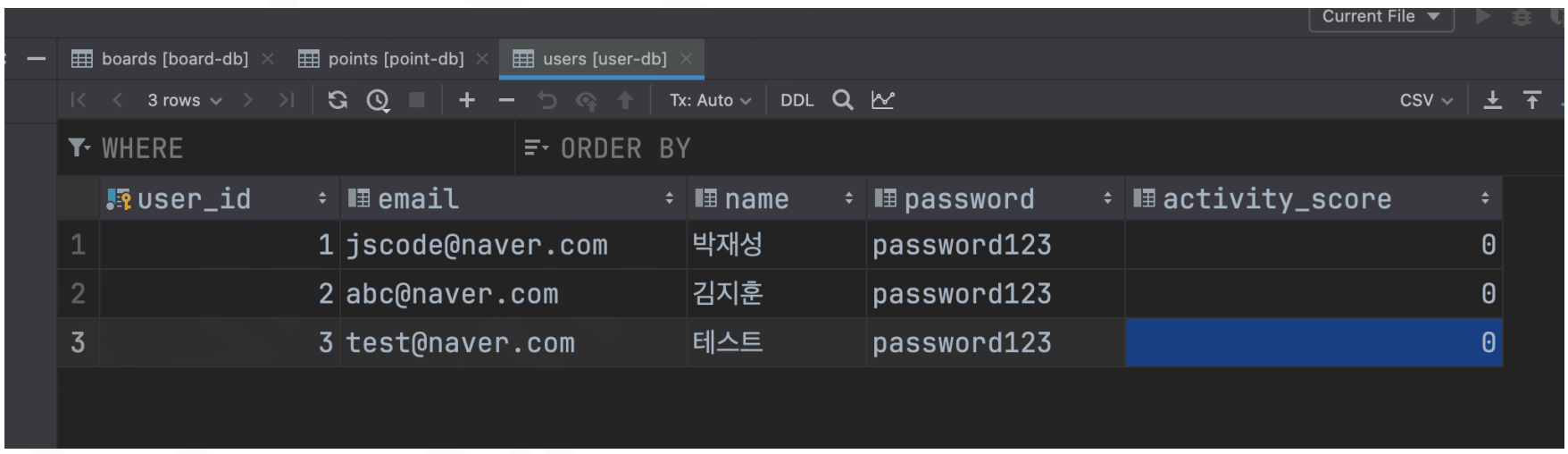
[실습] 잘 작동하는 지 테스트하기

✅ 잘 작동하는 지 테스트하기

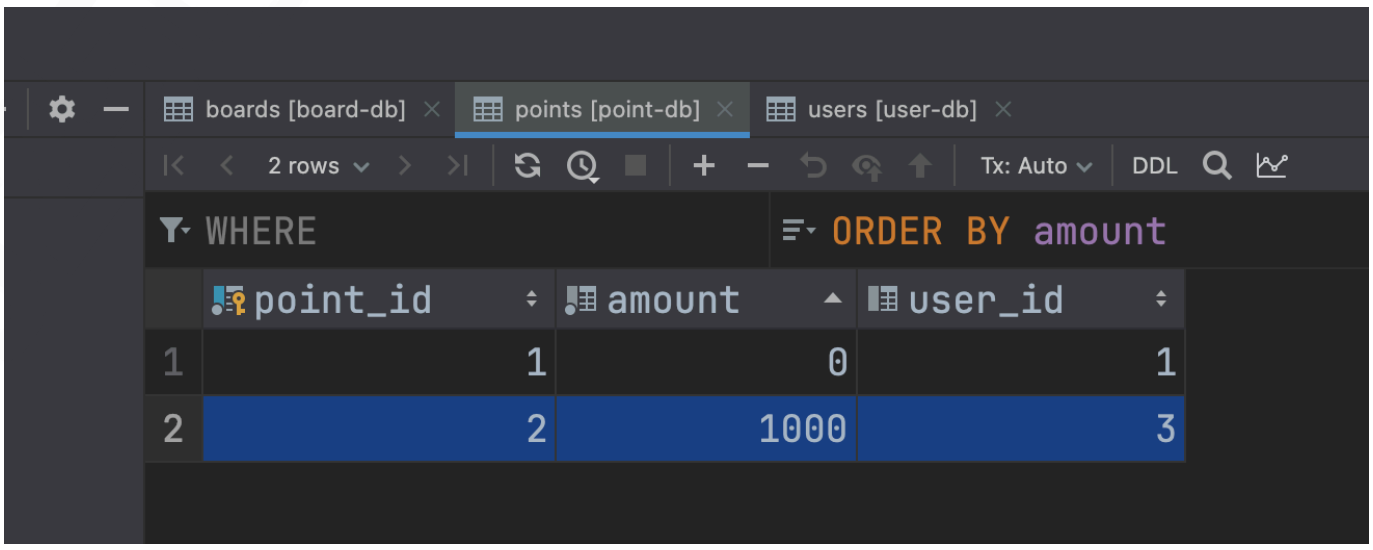
1. **테스트 전에 DB 데이터 정리하기**
    a. **모든 게시글 데이터 삭제하기**



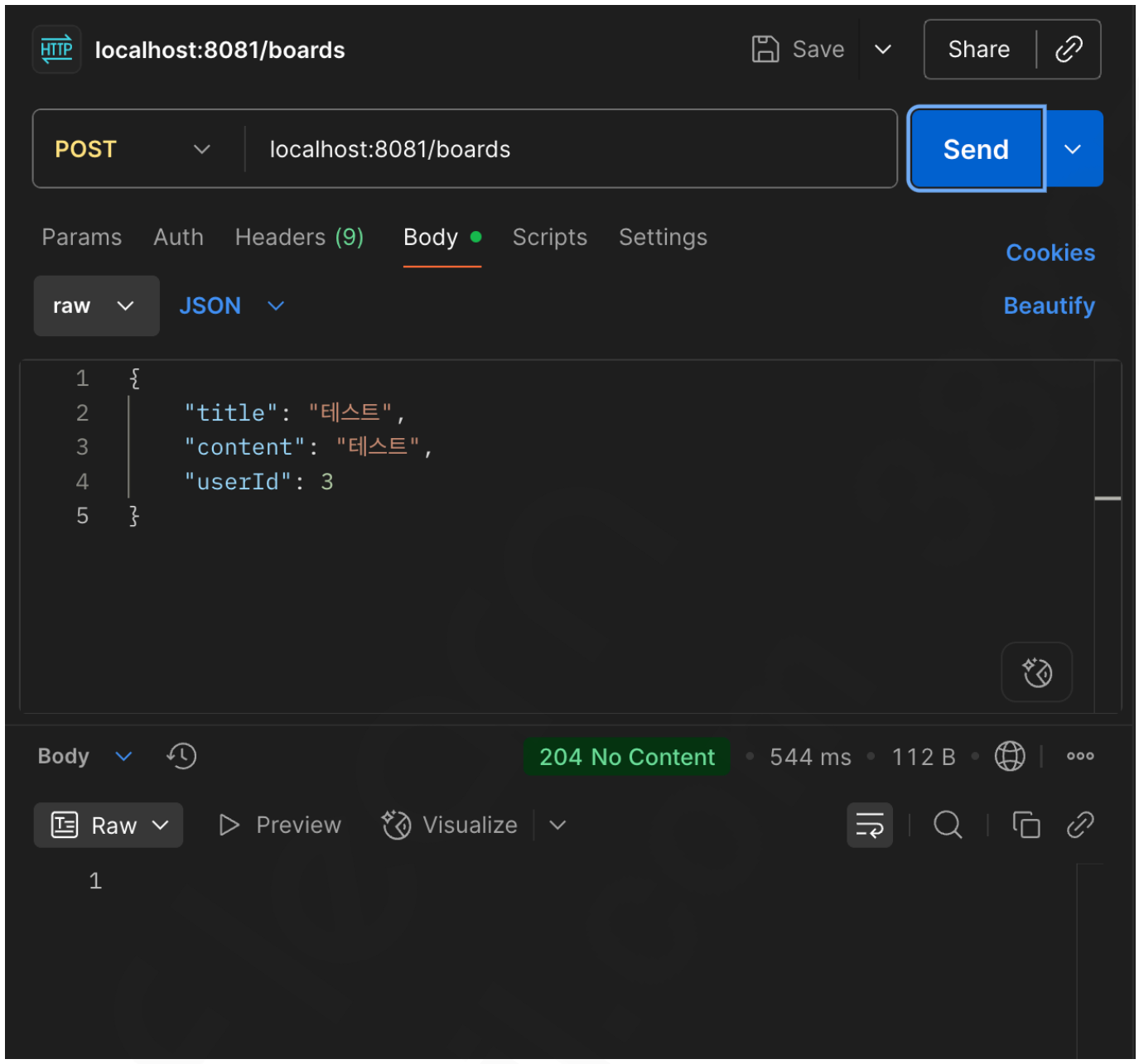
b. **활동 점수 0점으로 맞추기**



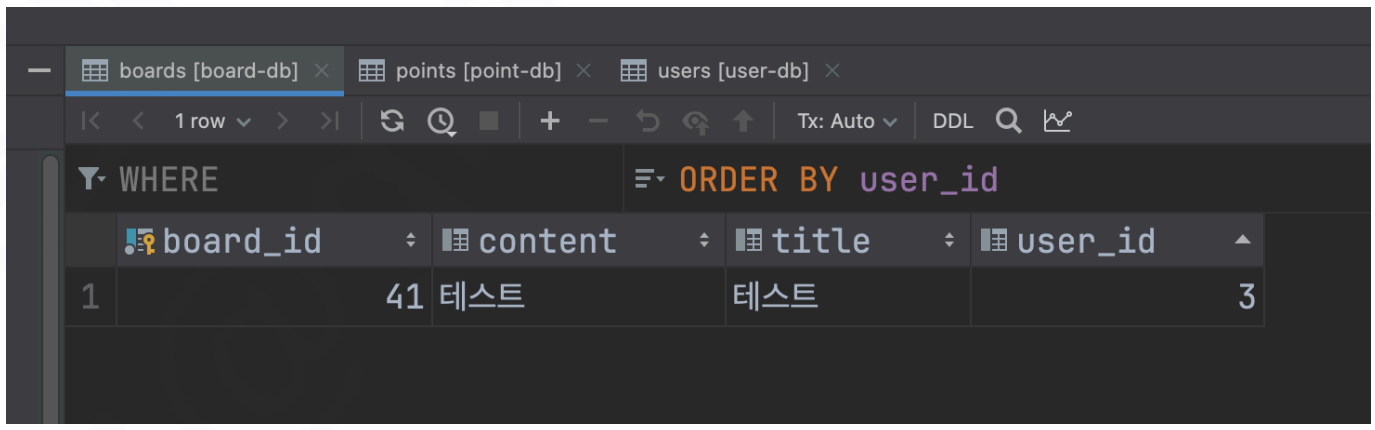
c. **포인트 1000점으로 맞추기**

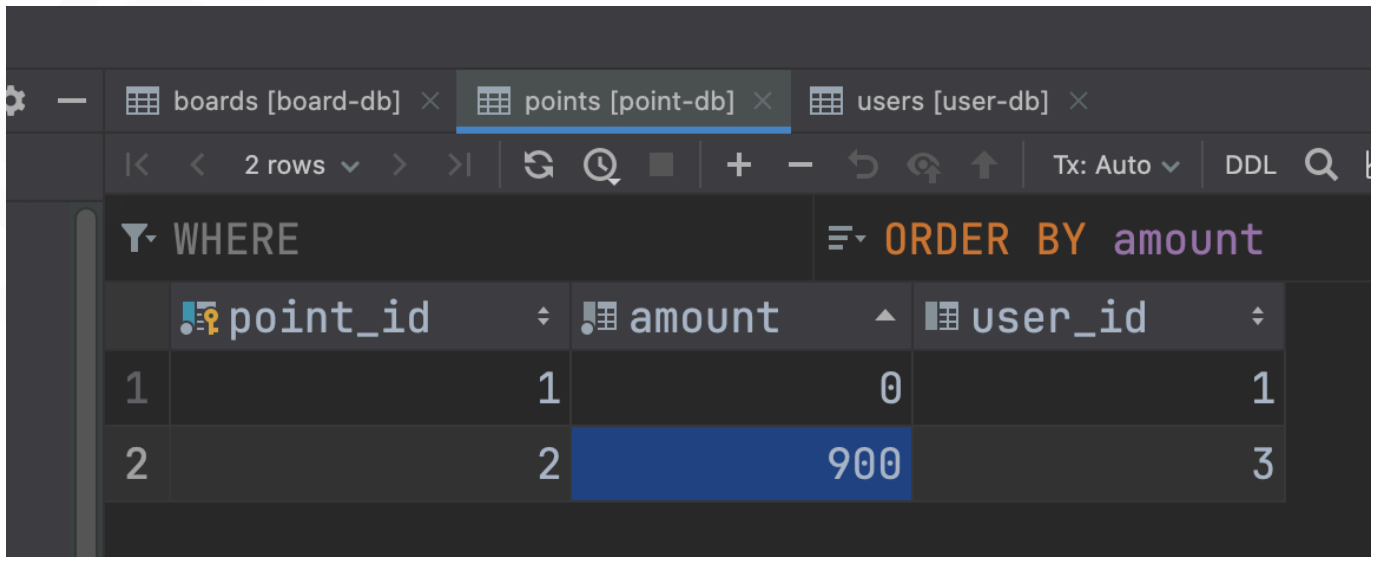


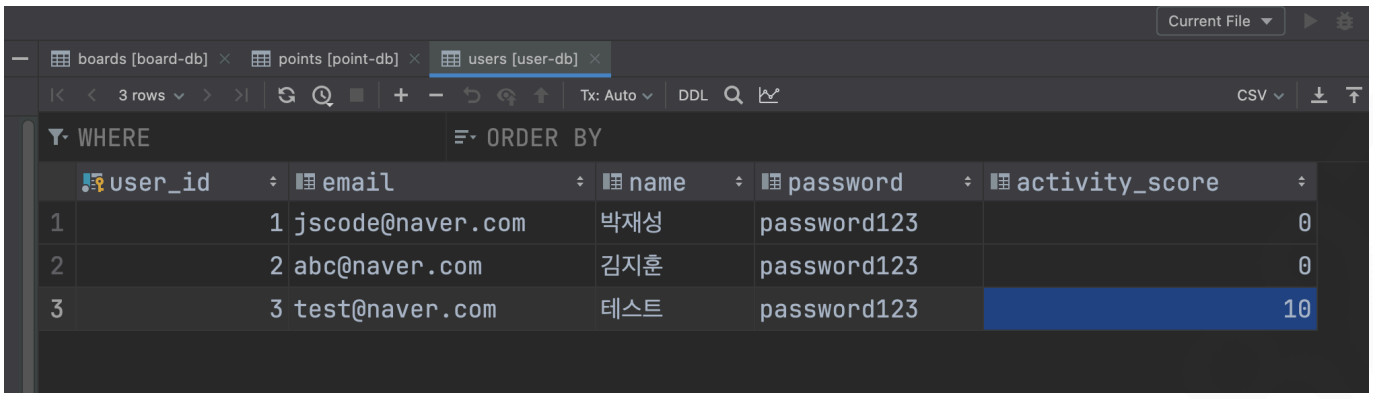
2. **API 요청 보내기**



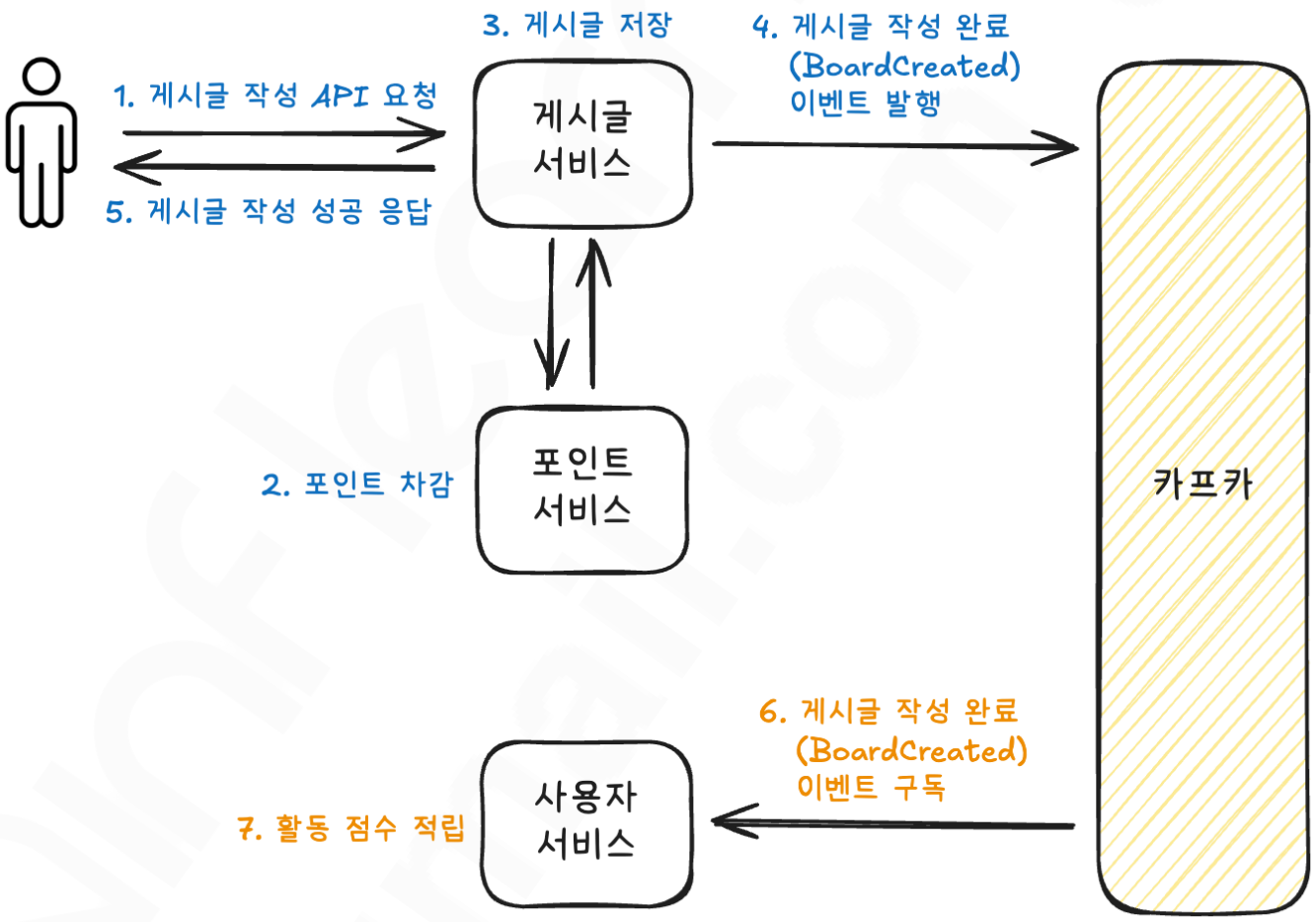
3. **성공적으로 잘 처리했는 지 DB 확인하기**







✅ 정리하기



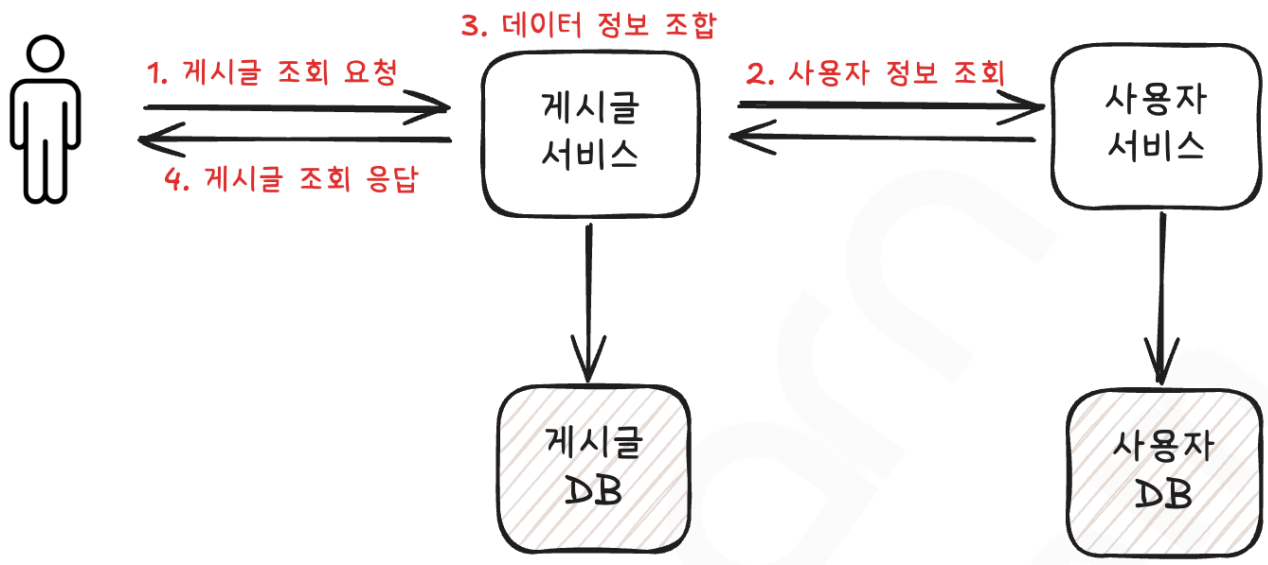
설계했던 대로 사용자가 게시글 작성 API에 요청을 보냈을 때 '포인트 차감 → 게시글 작성' 로직까지만 처리하고 성공 응답을 내려주는 방식으로 코드를 변경했다. 그리고 '활동 점수 적립'은 메시징 기반의 비동기 방식으로 처리하도록 코드를 수정했다. 그리고 성공적으로 잘 작동하는지까지 확인했다.

이와 같이 실무에서는 Saga 패턴을 구현할 때 **REST API 기반의 동기 방식**과 **메시징 기반의 비동기 방식**을 각 작업의 특징에 맞게 혼용해서 사용하는 경우가 많다.

khb3025@gmail.com

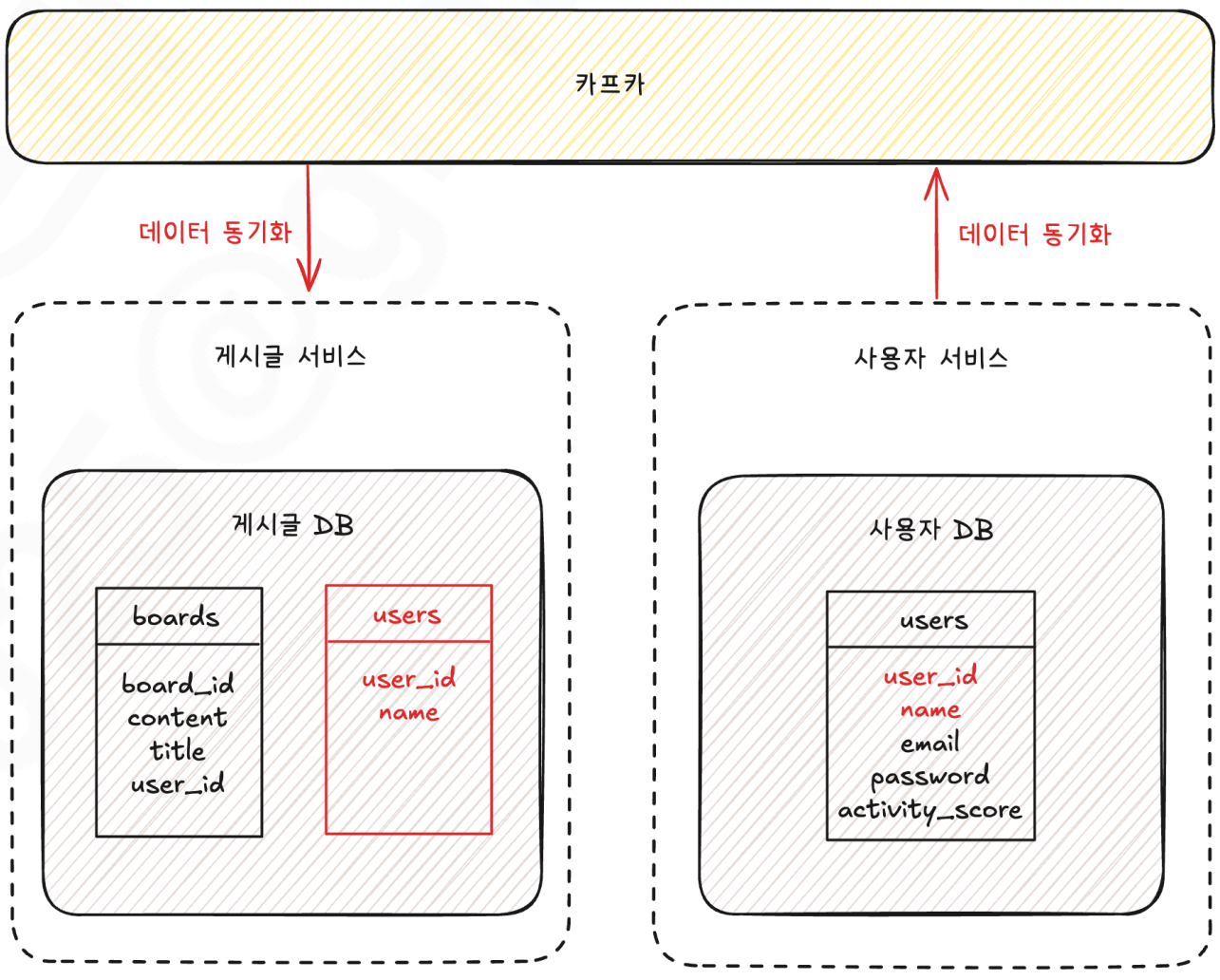
‘데이터 동기화’를 활용한 게시글 조회 API 최적화하기

✅ ‘데이터 동기화’를 활용한 게시글 조회 API 최적화하기



‘게시글 조회 로직’을 구현할 때 위의 방식처럼 구현했었다. 위의 방식도 충분히 괜찮은 방식이지만, 게시글 조회 로직의 트래픽이 점차 증가하면서 사용자 서비스의 ‘사용자 DB’에 부담을 줄 수 있다. 또한 ‘게시글 조회 로직’이 더 복잡해져서 사용자 서비스 뿐만 아니라 여러 서비스에 데이터를 조회해서 사용해야 한다면 로직이 복잡해진다.

이런 문제가 발생한다면 게시글 조회 로직을 최적화 할 필요가 있다. 조회 로직의 가장 간단한 최적화 방법은 ‘**데이터 동기화**’이다. 그림으로 살펴보자.



게시글 서비스에서 게시글 조회를 할 때 사용자 서비스의 사용자 정보(`user_id`, `name`)를 같이 조회한다. 즉, 게시글 서비스는 사용자 정보(`user_id`, `name`)가 필요하다. 이걸 해결하기 위해 게시글 서비스에서 필요한 사용자 정보를 관리할 수 있는 테이블을 별도로 두는 것이다.

그러고 사용자 DB에서 데이터 변경이 일어날 때마다 카프카로 ‘사용자 정보 변경 메시지’를 발행한다. 그럼 게시글 서비스가 해당 메시지를 구독하고 있다가, 사용자 정보가 변경될 때마다 게시글 DB에 사용자 정보를 같이 동기화시켜주는 방식이다.

이런 식으로 구성하게 되면 게시글 서비스가 자체적으로 사용자 정보(`user_id`, `name`)을 가지고 있기 때문에, 게시글 정보를 조회할 때 사용자 서비스에 데이터를 따로 요청하지 않아도 된다. 그래서 사용자 DB에 부담을 주지 않게 되고, 게시글 조회 로직도 훨씬 간단하게 바뀌게 된다.

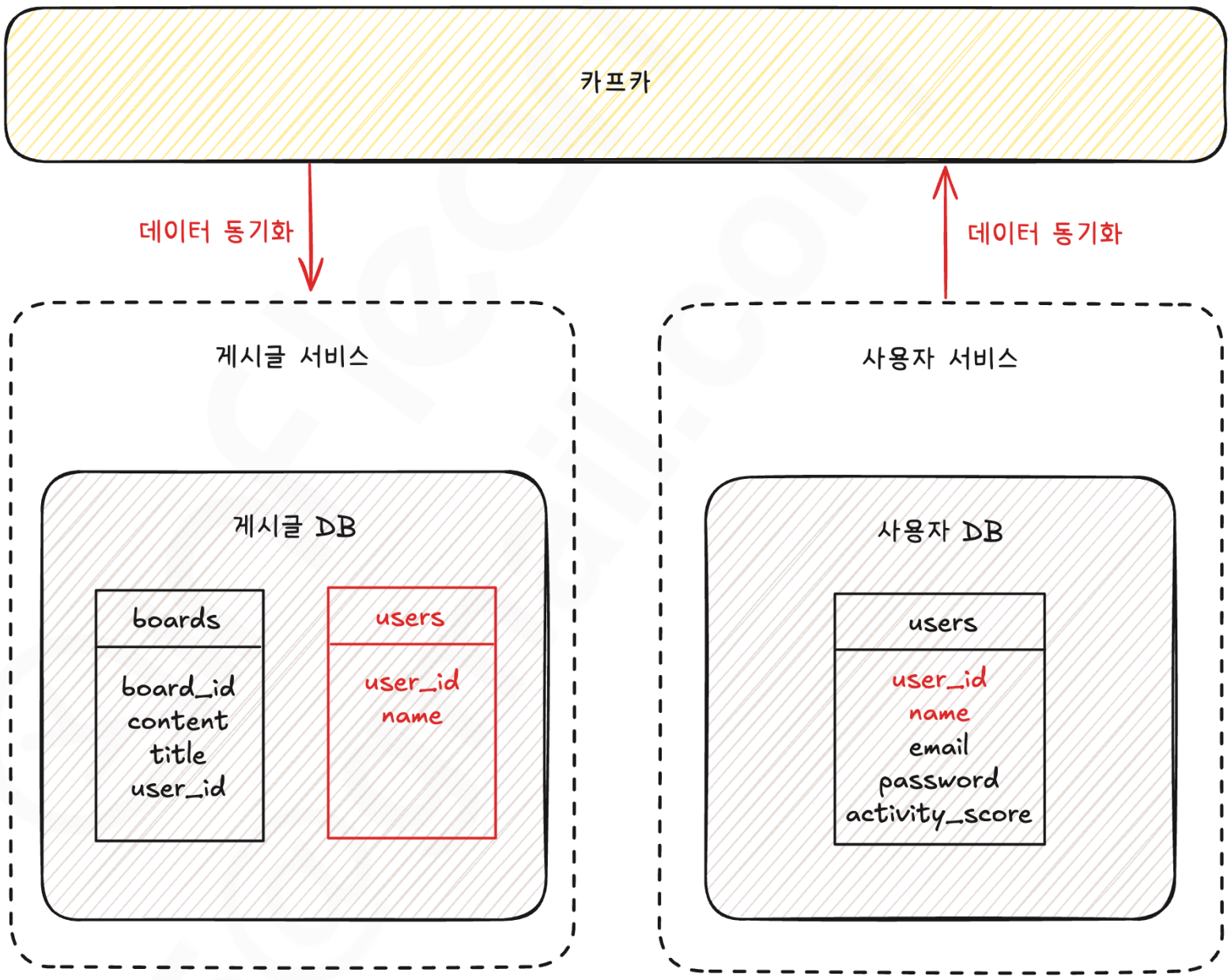
다음 강의에서 위의 구조대로 코드를 작성해보자.

@kfc3882.com
kfc3025

🎯 [실습] 사용자 서비스에서 Kafka로 '회원가입 완료 이벤트' 발행하기

✅ 사용자 서비스에서 Kafka로 '회원가입 완료 이벤트' 발행하기

게시글 서비스에서 사용해야 하는 사용자 데이터(`user_id`, `name`)를 동기화하기 위해서, 사용자 서비스에서 `user_id`, `name`의 데이터가 **생성/변경/삭제가 될 때마다 카프카로 이벤트를 발행**해주어야 한다. 현재 프로젝트에서는 사용자 데이터를 **생성**하는 회원가입 API가 구현되어 있다. 따라서 데이터 동기화를 위해 회원가입 API에 이벤트 발행 로직을 추가해보자.



✅ 코드 작성하기

> user-service에서 아래 코드 작성하기

1. **application.yml에 Kafka 설정 추가해주기**

```
server:
  port: 8080

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/user-db
    username: root
```

```
    password: password
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  kafka:
    # Kafka 서버 주소
    bootstrap-servers: localhost:9092
    consumer:
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.apache.kafka.common.serialization.StringDeserializer
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.apache.kafka.common.serialization.StringSerializer

client:
  point-service:
    url: http://localhost:8082
```

2. **Kafka로 전달할 메시지 객체 만들기**

   event/UserSignedUpEvent

```
public class UserSignedUpEvent {
  private Long userId;
  private String name;

  public UserSignedUpEvent(Long userId, String name) {
    this.userId = userId;
    this.name = name;
  }

  public Long getUserId() {
    return userId;
  }

  public String getName() {
    return name;
  }
}
```

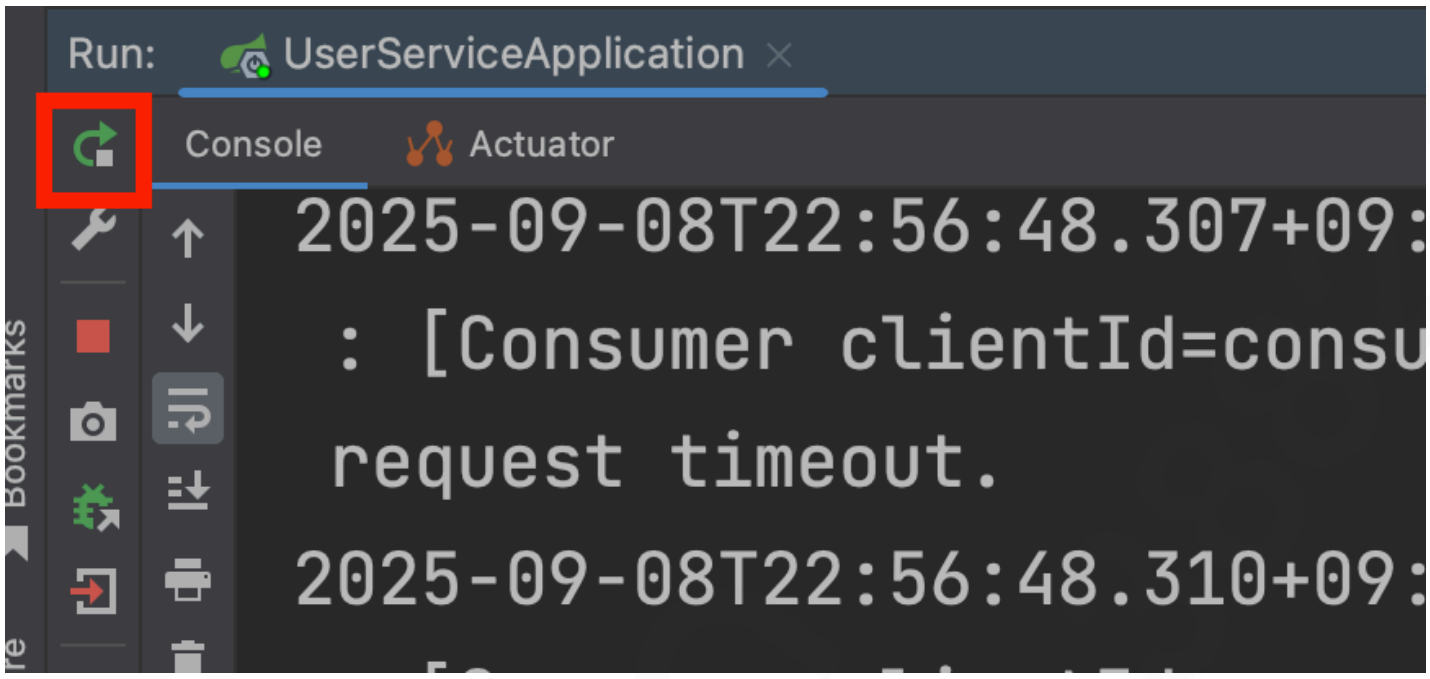
3. **회원 가입을 완료하는 대로 Kafka 메시지 발행하도록 처리하기**

   service/UserService

```
@Service
public class UserService {
  private final UserRepository userRepository;
  private final PointClient pointClient;
  private final KafkaTemplate<String, String> kafkaTemplate;

  public UserService(UserRepository userRepository,
      PointClient pointClient,
      KafkaTemplate<String, String> kafkaTemplate) {
    this.userRepository = userRepository;
```

```
  this.pointClient = pointClient;
  this.kafkaTemplate = kafkaTemplate;
}

@Transactional
public void signUp(SignUpRequestDto signUpRequestDto) {
  User user = new User(
    signUpRequestDto.getEmail(),
    signUpRequestDto.getName(),
    signUpRequestDto.getPassword()
  );

  User savedUser = this.userRepository.save(user);

  // 회원가입하면 포인트 1000점 적립
  pointClient.addPoints(savedUser.getUserId(), 1000);

  // '회원가입 완료' 이벤트 발행
  UserSignedUpEvent userSignedUpEvent = new UserSignedUpEvent(
      savedUser.getUserId(),
      savedUser.getName()
  );
  this.kafkaTemplate.send(
      "user.signed-up",
      toJsonString(userSignedUpEvent)
  );
}

// 객체를 Json 형태의 String으로 만들어주는 메서드
// (클래스로 분리하면 더 좋지만 편의를 위해 메서드로만 분리)
private String toJsonString(Object object) {
  ObjectMapper objectMapper = new ObjectMapper();
  try {
    String message = objectMapper.writeValueAsString(object);
    return message;
  } catch (JsonProcessingException e) {
    throw new RuntimeException("Json 직렬화 실패");
  }
}

  ...
}
```

4. **서버 다시 실행시키기**



@naver.com
@gmail.com
khb3025

🎯 [실습] 게시글 서비스에서 Kafka로부터 '회원가입 완료 이벤트' 구독하기

✅ 게시글 서비스에서 Kafka로부터 '회원가입 완료 이벤트' 구독하기

> board-service에서 아래 코드 작성하기

1. **User 엔티티 만들기**

   domain/User

```
@Entity
@Table(name = "users")
public class User {

// Auto Increment 옵션을 사용하지 않을 거기 때문에
// @GeneratedValue() 어노테이션 작성하지 않음
@Id
private Long userId;

private String name;

public User() {
}

public User(Long userId, String name) {
  this.userId = userId;
  this.name = name;
}

public Long getUserId() {
  return userId;
}

public String getName() {
  return name;
  }
}
```

2. **UserRepository 만들기**

   domain/UserRepository

```
public interface UserRepository extends JpaRepository<User, Long> {
}
```

3. **사용자 정보 저장 Service 로직 만들기**

   dto/SaveUserRequestDto

```
public class SaveUserRequestDto {
  private Long userId;
```

```
  private String name;

  public SaveUserRequestDto(Long userId, String name) {
    this.userId = userId;
    this.name = name;
  }

  public Long getUserId() {
    return userId;
  }

  public String getName() {
    return name;
  }
}
```

service/UserService

```
@Service
public class UserService {

  private final UserRepository userRepository;

  public UserService(UserRepository userRepository) {
    this.userRepository = userRepository;
  }

    @Transactional
  public void save(SaveUserRequestDto saveUserRequestDto) {
    User user = new User(
        saveUserRequestDto.getUserId(),
        saveUserRequestDto.getName()
    );

    this.userRepository.save(user);
  }
}
```

4. **application.yml에 Kafka 연결을 위한 정보 작성하기**

application.yml

```
server:
  port: 8081

spring:
  datasource:
    url: jdbc:mysql://localhost:3307/board-db # DB ??
    username: root # DB ??
    password: password # DB ????
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  kafka:
    # Kafka 서버 주소
```

```
    bootstrap-servers: localhost:9092
    producer:
      # 메시지의 key 직렬화 방식 : 자바 객체를 문자열(String)로 변환해서 Kafka에 전송
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      # 메시지의 value 직렬화 방식 : 자바 객체를 문자열(String)로 변환해서 Kafka에 전송
      value-serializer: org.apache.kafka.common.serialization.StringSerializer
    consumer:
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.apache.kafka.common.serialization.StringDeserializer
client:
  user-service:
    url: http://localhost:8080
  point-service:
    url: http://localhost:8082
```

2. **Kafka로부터 전달받을 메시지 객체 만들기**

event/UserSignedUpEvent

```
public class UserSignedUpEvent {
  private Long userId;
  private String name;

  // 역직렬화(String 형태의 카프카 메시지 -> Java 객체)시 빈생성자 필요함
  public UserSignedUpEvent() {
  }

  // Json 값을 UserSignedUpEvent로 역직렬화하는 메서드
  public static UserSignedUpEvent fromJson(String json) {
    try {
      ObjectMapper objectMapper = new ObjectMapper();
      return objectMapper.readValue(json, UserSignedUpEvent.class);
    } catch (JsonProcessingException e) {
      throw new RuntimeException("JSON 파싱 실패");
    }
  }

  public Long getUserId() {
    return userId;
  }

  public String getName() {
    return name;
  }
}
```

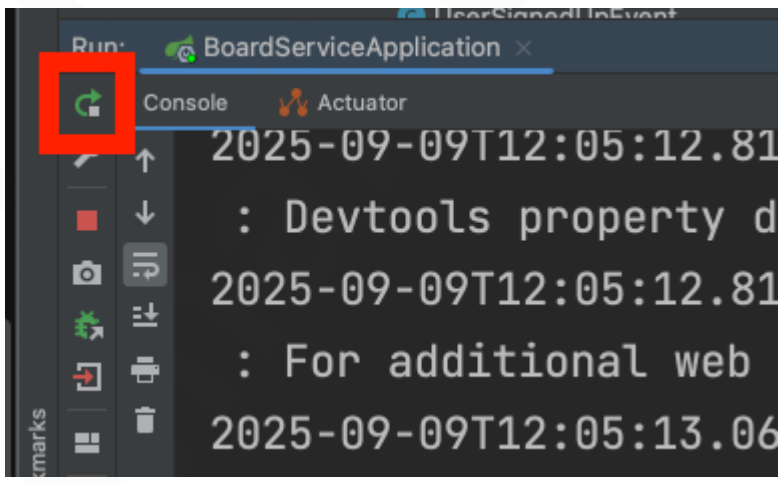
3. **Kafka에 있는 메시지 Consume 하기**

consumer/UserSignedUpEventConsumer

```
@Component
public class UserSignedUpEventConsumer {

  private final UserService userService;

  public UserSignedUpEventConsumer(UserService userService) {
```

```
  this.userService = userService;
}

@KafkaListener(
    topics = "user.signed-up",
    groupId = "board-service"
)
public void consume(String message) {
  UserSignedUpEvent userSignedUpEvent
      = UserSignedUpEvent.fromJson(message);

  // 사용자 정보 저장
  SaveUserRequestDto saveUserRequestDto
      = new SaveUserRequestDto(
          userSignedUpEvent.getUserId(),
          userSignedUpEvent.getName()
      );
  userService.save(saveUserRequestDto);
  }
}
```

7. **서버 다시 실행시키기**



khb3025@gmail.com

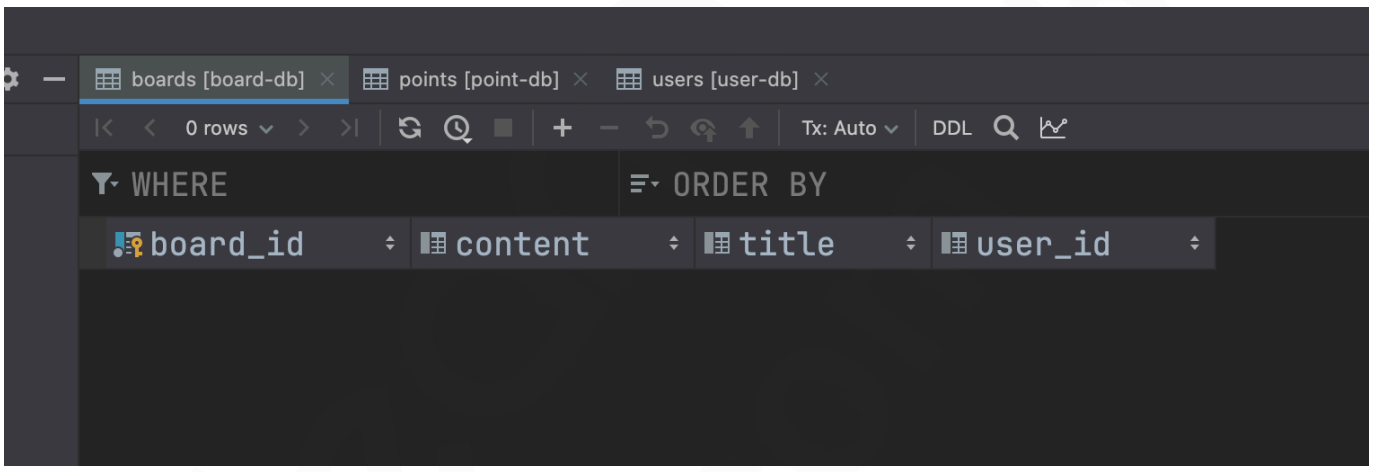
[실습] 잘 작동하는 지 테스트하기

✅ 잘 작동하는 지 테스트하기

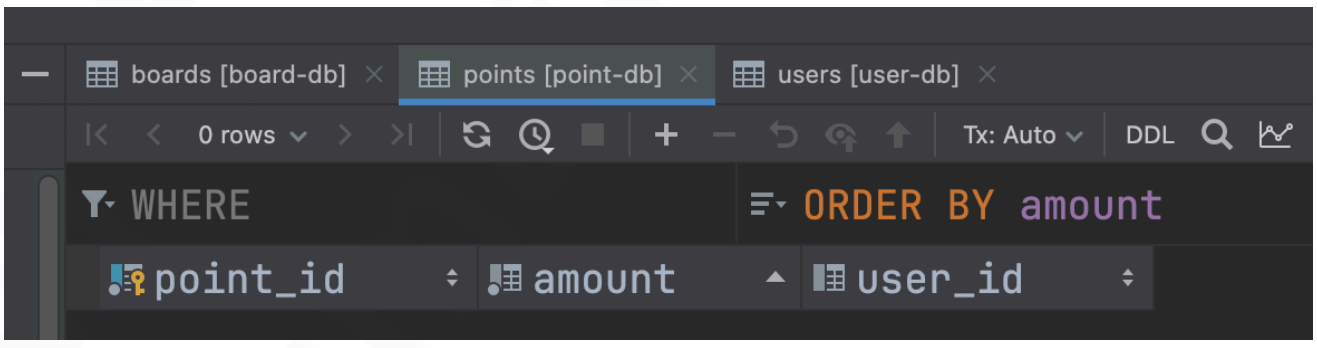
1. **테스트 전에 DB 데이터 정리하기**

   깔끔하게 테스트하기 위해 기존에 만들었던 모든 데이터를 삭제해주자.

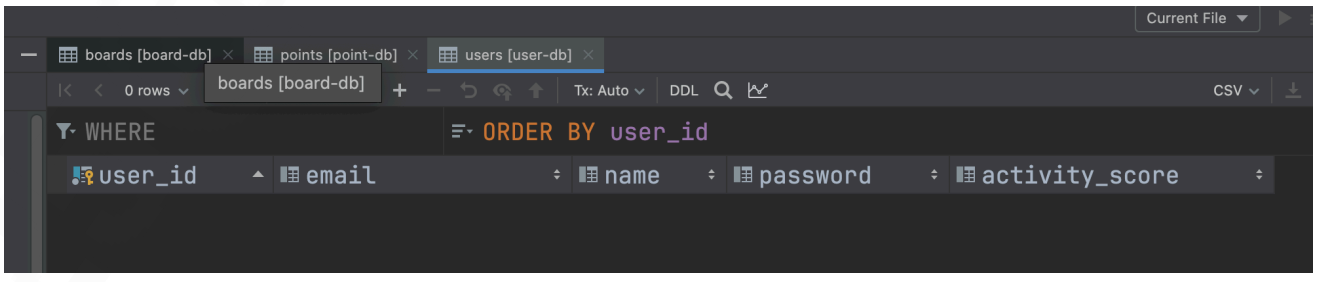
   a. **모든 게시글 데이터 삭제하기**



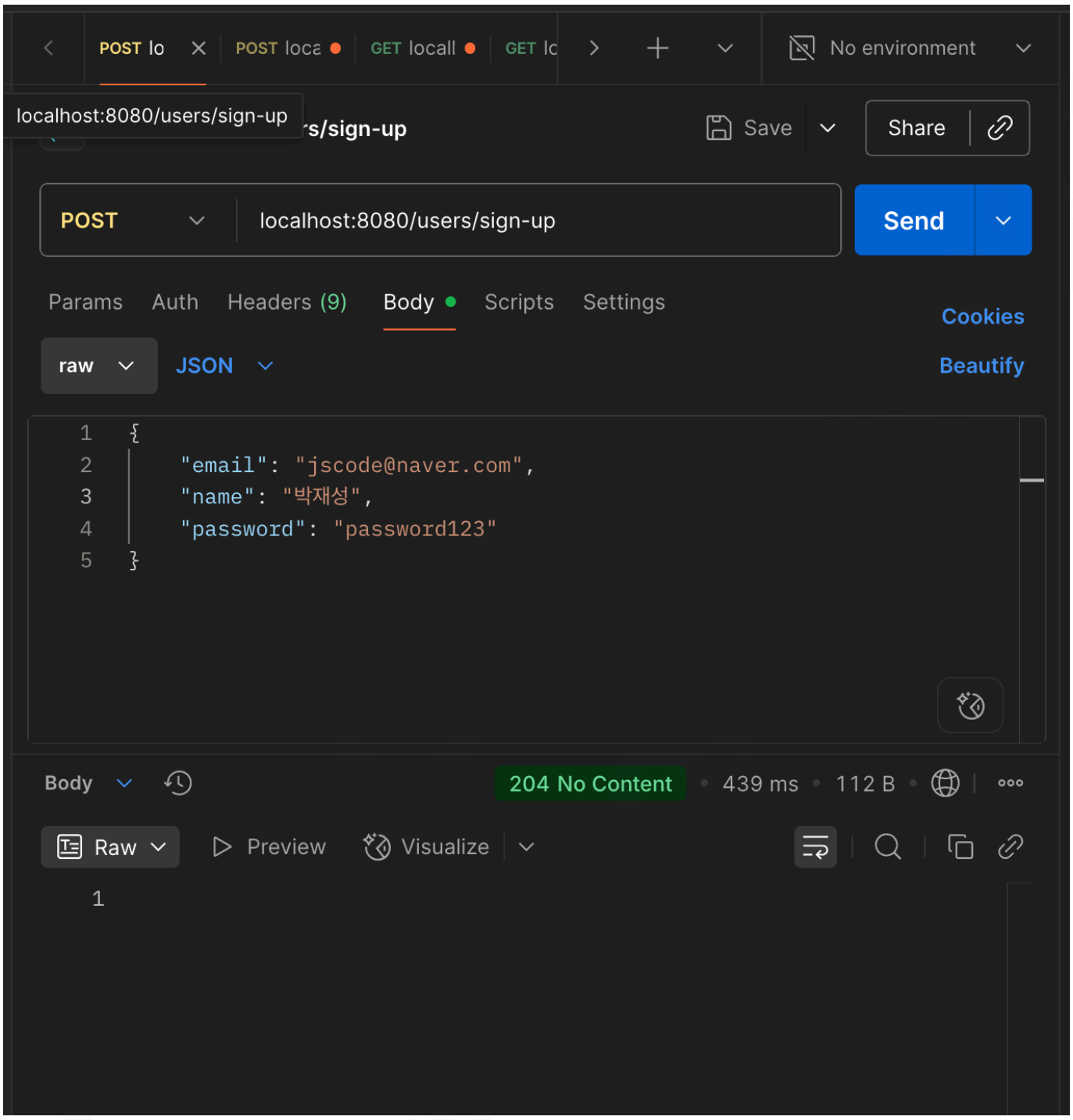
   b. **모든 포인트 데이터 삭제하기**



   c. **모든 사용자 데이터 삭제하기**

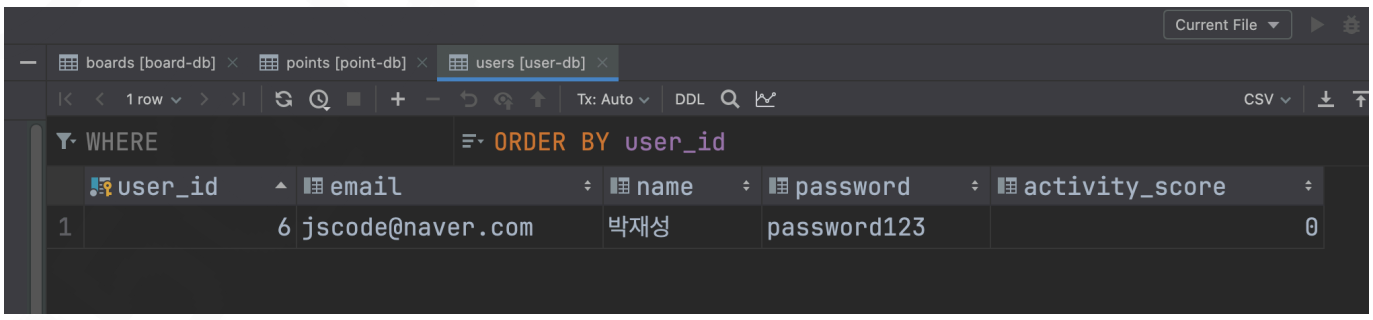


2. **API 요청 보내기**

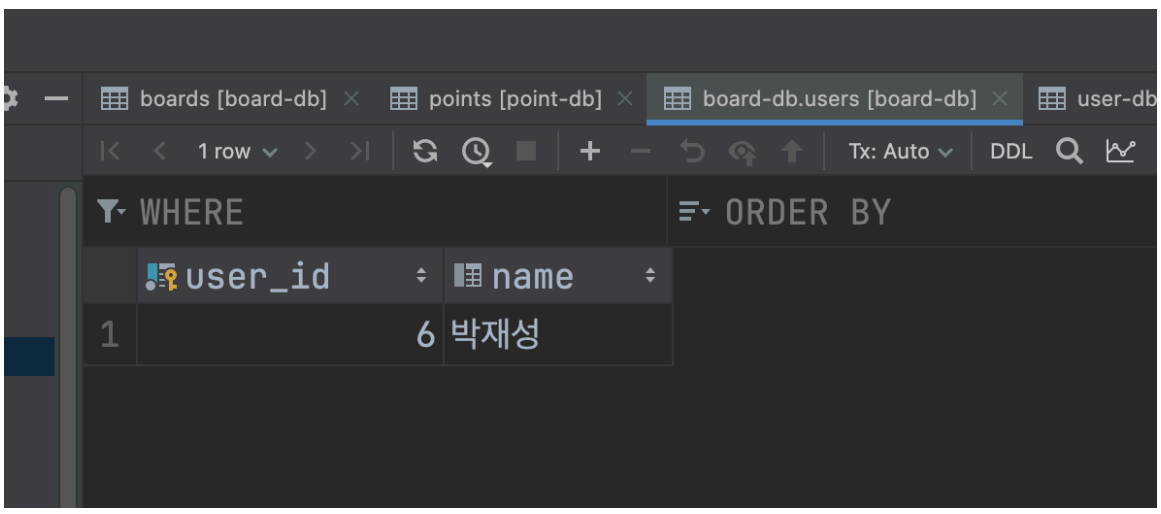


3. **성공적으로 잘 처리했는 지 DB 확인하기**

   [사용자 DB의 users 테이블]

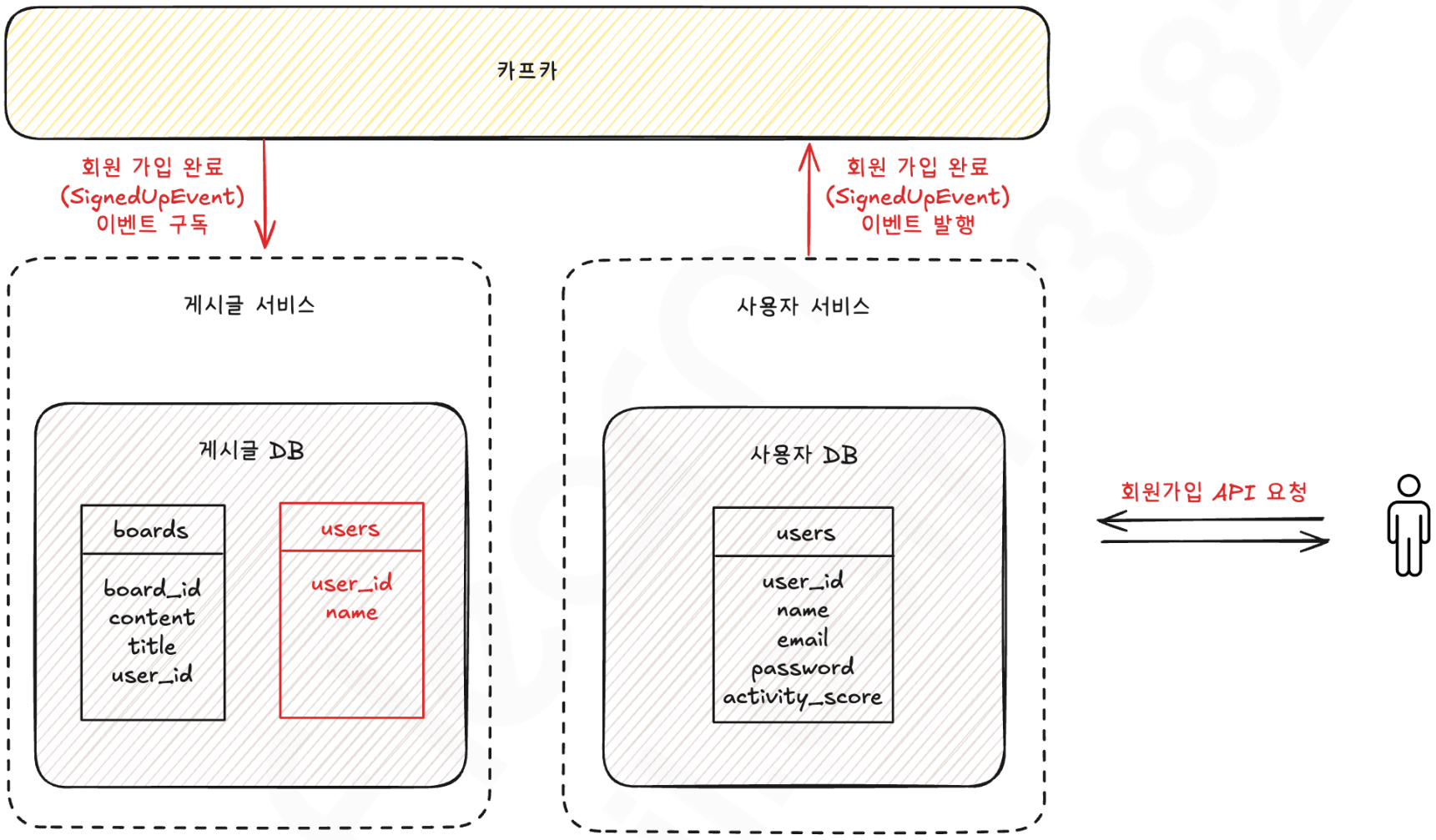


[게시글 DB의 users 테이블]



DB에 저장된 데이터를 보니 회원가입을 통해 생성된 사용자 데이터가 게시글 서비스의 게시글 DB에도 잘 저장된 걸 확인할 수 있다.

✅ 정리하기



1. 사용자(클라이언트)가 회원가입 API 요청
2. 사용자 서비스의 users DB에 사용자 정보 저장
3. Kafka에 회원가입 완료 이벤트 발행
4. 회원가입 API 완료 응답
5. 게시글 서비스에서 회원가입 완료 이벤트 구독
6. 게시글 서비스의 users DB에 사용자 정보 저장

위의 과정을 거쳐 사용자 서비스의 사용자 정보를 게시글 서비스에도 동기화시켰다. 그럼 다음 강의에서 게시글 서비스에서 게시글을 조회할 때 사용자 정보를 사용자 서비스로부터 가져오는 게 아닌, 게시글 DB로부터 가져오도록 수정해보자.

khb3025

🎯 [실습] 게시글 조회 로직 최적화하기

✅ 게시글 조회 로직 최적화하기

> board-service에서 아래 코드 작성하기

1. **Board 엔티티에 연관 관계 매핑하기**

   domain/Board

```
@Entity
@Table(name = "boards")
public class Board {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long boardId;

  private String title;

  private String content;

  @ManyToOne
  @JoinColumn(name = "user_id", insertable = false, updatable = false) // 조회용
  private User user;

  @Column(name = "user_id")
  private Long userId;

  public Board() {
  }

  public Board(String title, String content, Long userId) {
    this.title = title;
    this.content = content;
    this.userId = userId;
  }

  ...

  public User getUser() {
    return user;
  }
}
```

2. **게시글 조회 로직 수정하기**

   service/BoardService

```
@Service
public class BoardService {
  ...

  // 연관관계를 활용한 게시글 조회
  public BoardResponseDto getBoard2(Long boardId) {
```

```
  Board board = boardRepository.findById(boardId)
      .orElseThrow(() -> new IllegalArgumentException("게시글을 찾을 수 없습니다."));

  // BoardResponseDto 생성
  BoardResponseDto boardResponseDto = new BoardResponseDto(
      board.getBoardId(),
      board.getTitle(),
      board.getContent(),
      new UserDto(
          board.getUser().getUserId(),
          board.getUser().getName()
      )
  );

  return boardResponseDto;
}

// 연관관계를 활용한 게시글 전체 조회
  public List<BoardResponseDto> getBoards2() {
    List<Board> boards = boardRepository.findAll();

    return boards.stream()
        .map(board -> new BoardResponseDto(
            board.getBoardId(),
            board.getTitle(),
            board.getContent(),
            new UserDto(
                board.getUser().getUserId(),
                board.getUser().getName()
            )
        ))
        .toList();
  }
}
```

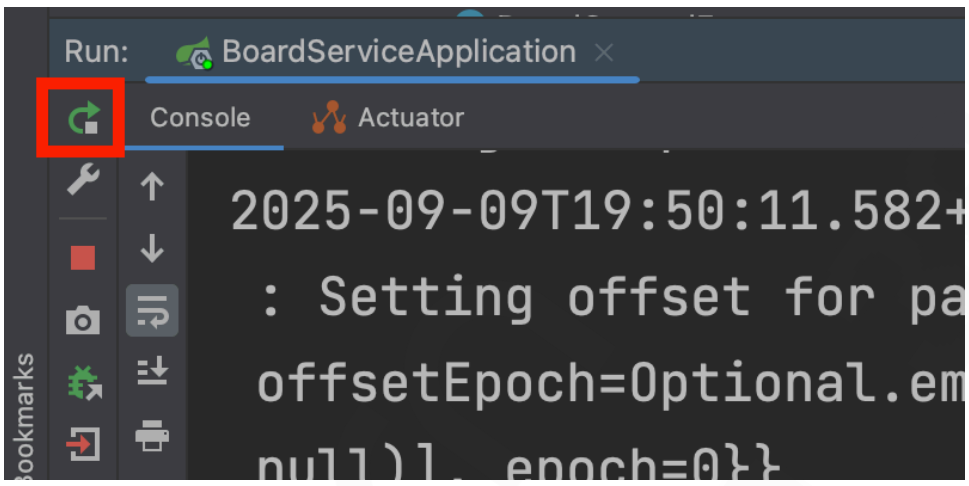
3. **컨트롤러 로직 바꾸기**

   controller/BoardController

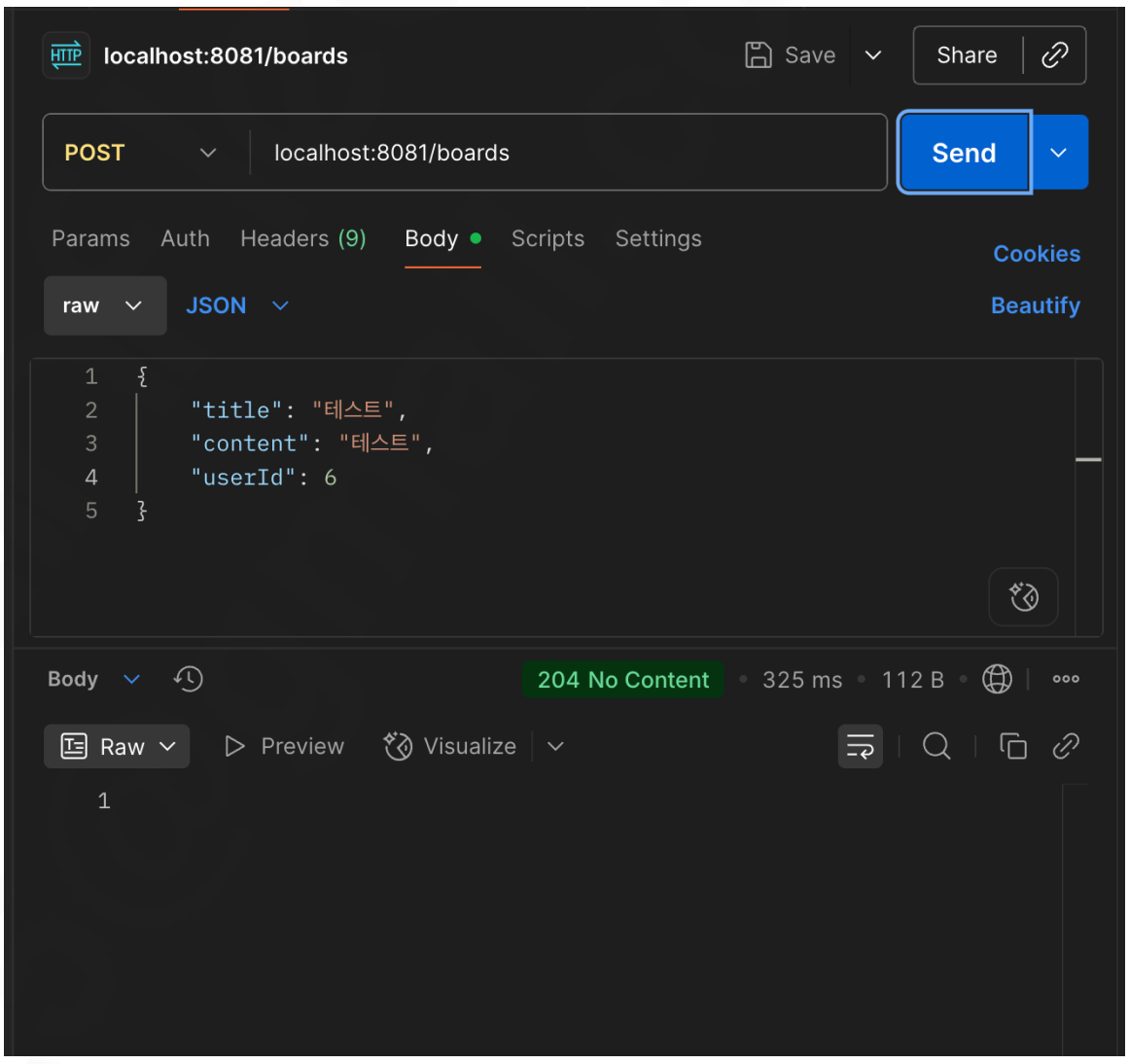
```
@RestController
@RequestMapping("/boards")
public class BoardController {
    ...

  @GetMapping("/{boardId}")
  public ResponseEntity<BoardResponseDto> getBoard(@PathVariable Long boardId) {
      // BoardResponseDto boardResponseDto = boardService.getBoard(boardId);
    BoardResponseDto boardResponseDto = boardService.getBoard2(boardId);
    return ResponseEntity.ok(boardResponseDto);
  }

  @GetMapping()
  public ResponseEntity<List<BoardResponseDto>> getBoards() {
    // List<BoardResponseDto> boardResponseDtos = boardService.getBoards();
    List<BoardResponseDto> boardResponseDtos = boardService.getBoards2();
    return ResponseEntity.ok(boardResponseDtos);
```

```
  }
}
```

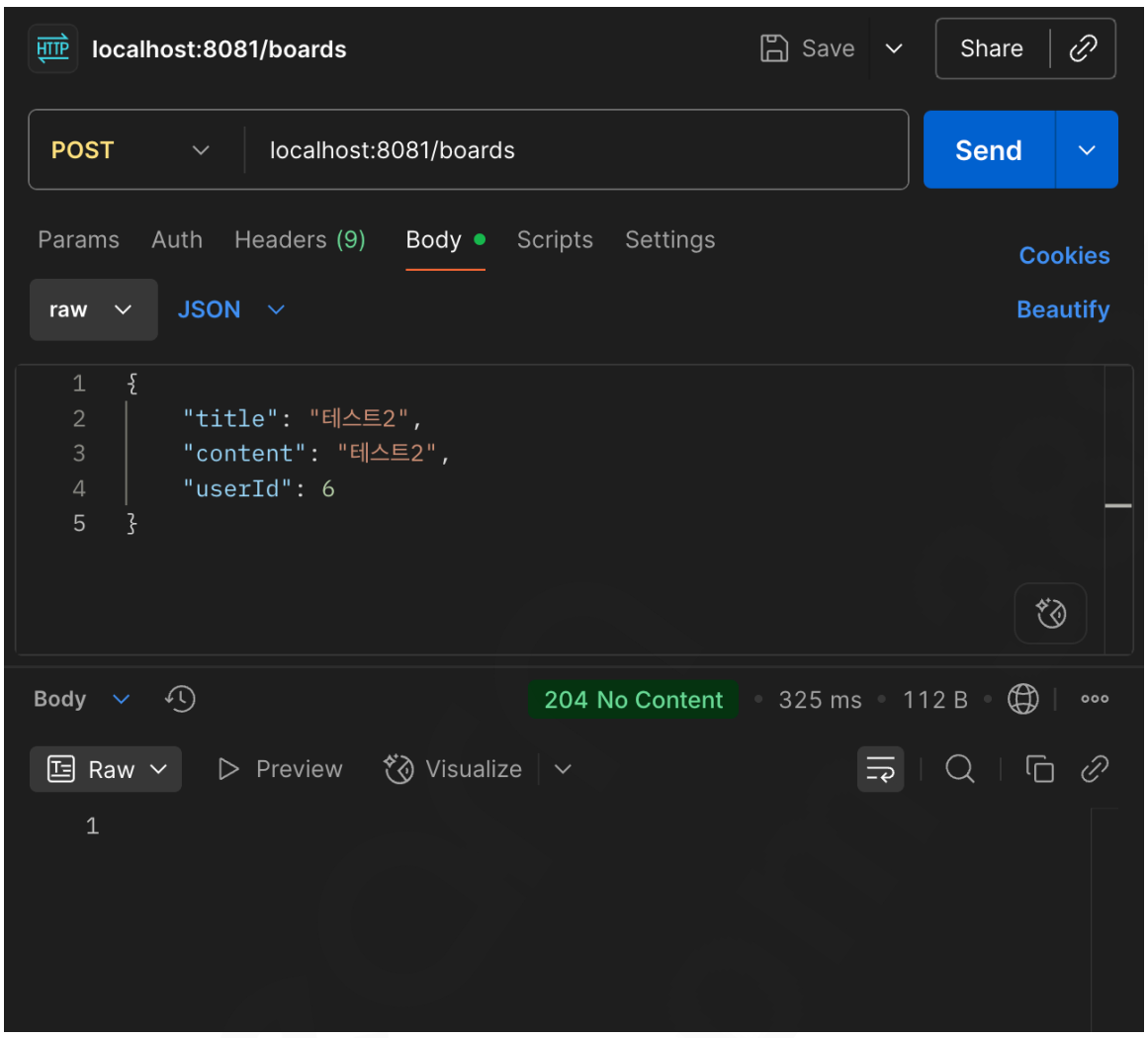
4. **서버 다시 실행시키기**



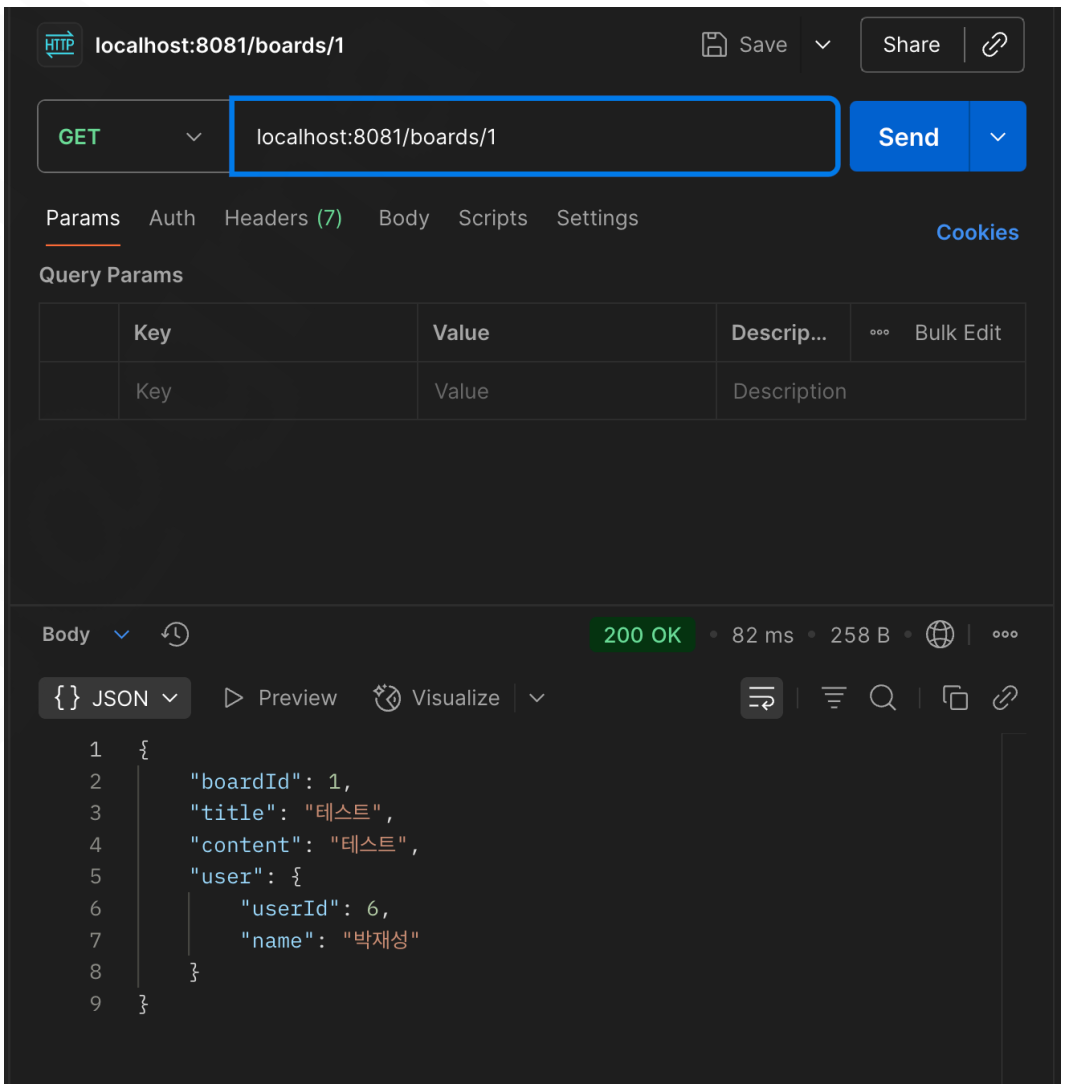
5. **테스트를 위해 게시글 데이터 2개 넣기**



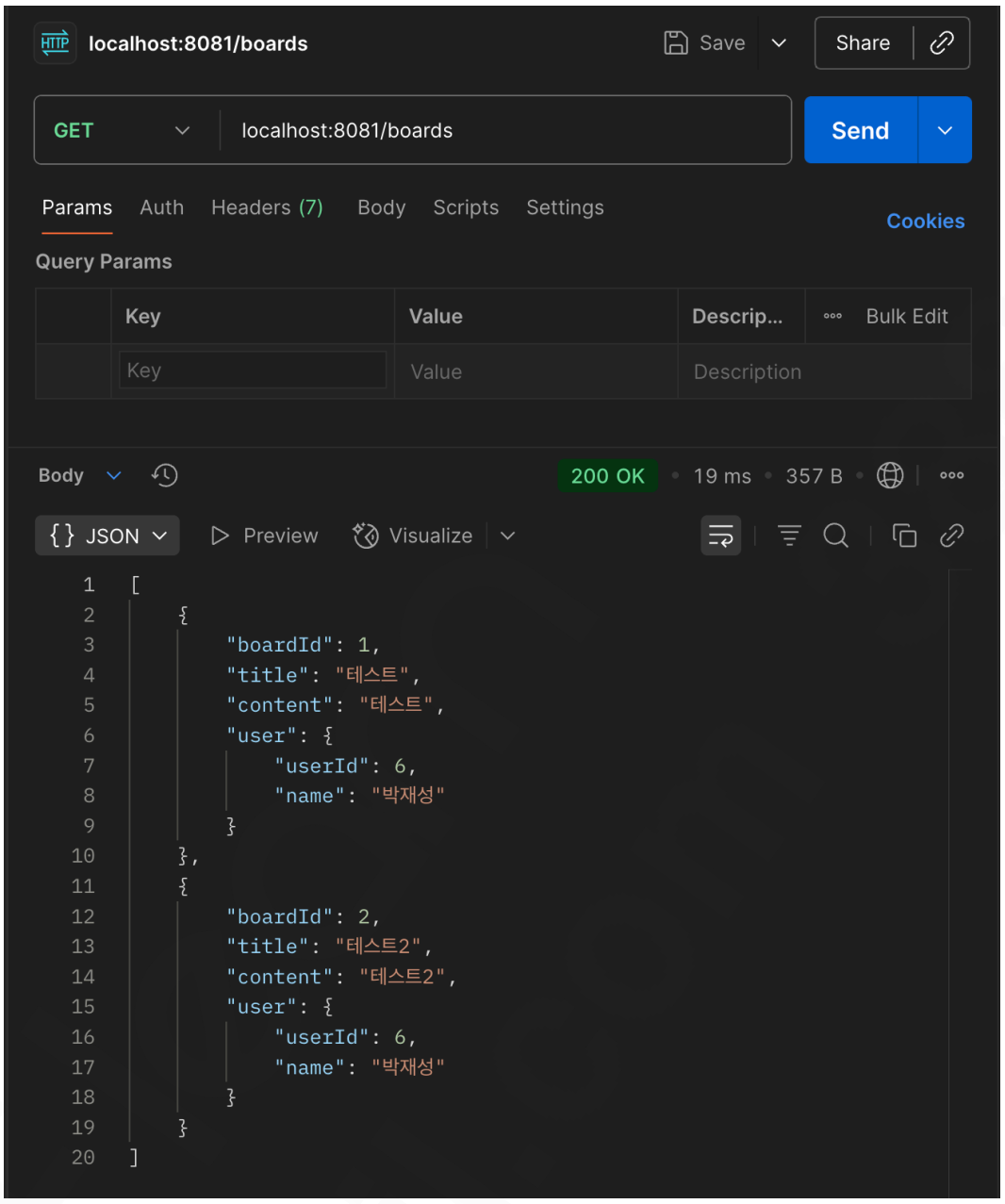
khb3021



6. **API 테스트 해보기**
    a. **특정 게시글 조회하기**



    b. **전체 게시글 조회하기**



마이크로서비스 간 통신 방식에 비해 조회 로직이 훨씬 단순해졌다. 또한 게시글 서비스가 사용자 정보를 가져오기 위해 사용자 서비스에 API 요청을 보내지 않아, 사용자 서비스나 사용자 DB에 불필요한 부하가 발생하지 않는다.

khb3025@gmail.com

📖

API Gateway란?

✅ API Gateway란?

우선 Gateway라는 의미를 먼저 알아보자.

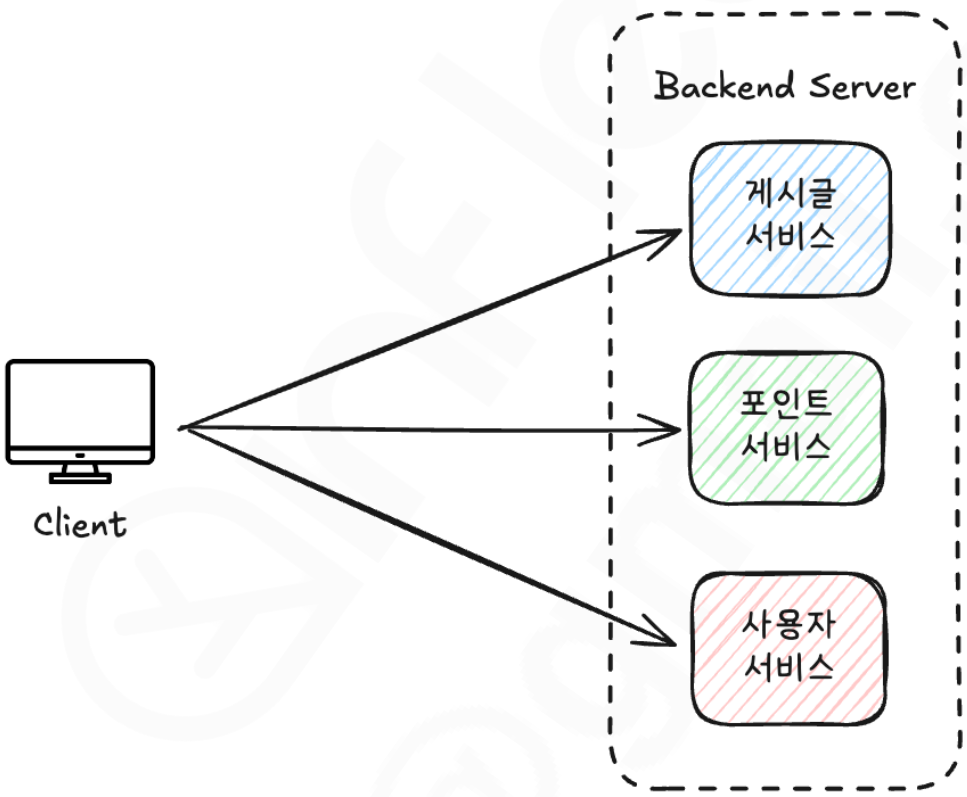
> **Gateway**는 **네트워크 상에서 다른 네트워크로 들어가는 입구 역할을 하는 지점**을 의미한다.

그럼 API Gateway의 뜻은 뭘까?

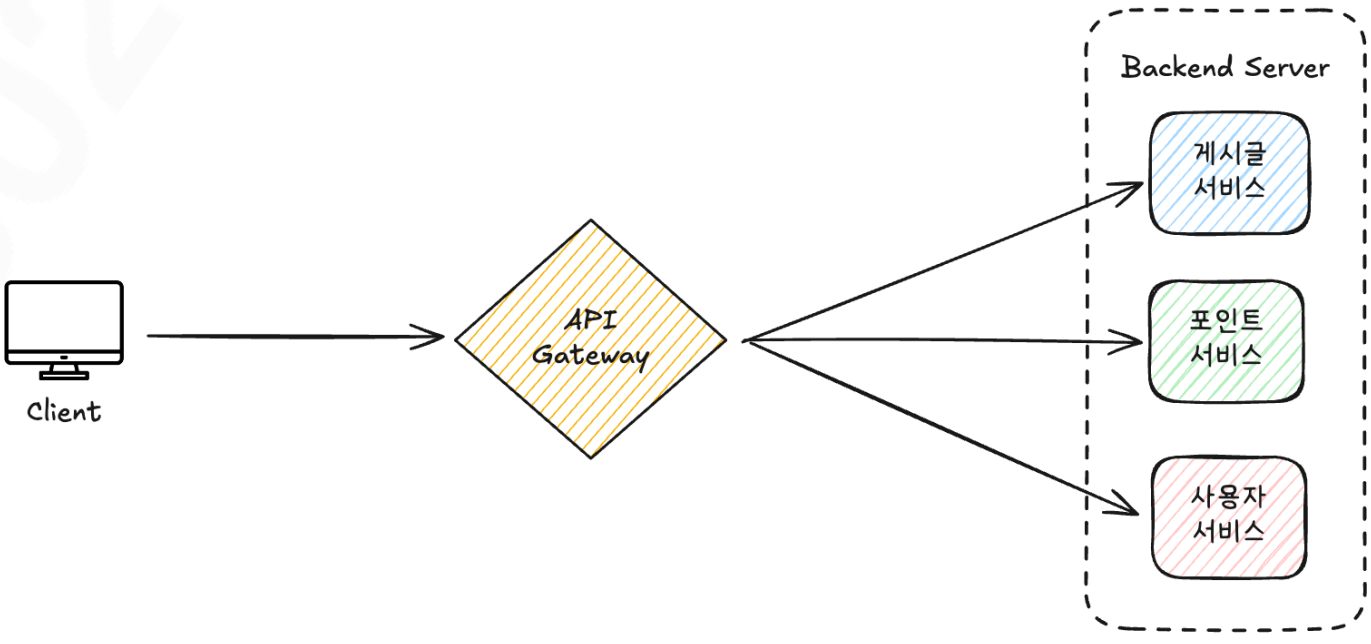
> **API Gateway**는 **클라이언트(웹, 모바일 등)가 백엔드 API 서버에 접근하기 전에 거쳐가는 입구 역할을 하는 서버**를 의미한다.

이 API Gateway의 의미는 그림으로 보면 조금 더 이해가 쉽다.

[API Gateway가 없을 때 통신하는 방식]



[API Gateway 있을 때 통신하는 방식]



✅ API Gateway를 사용하는 이유

> API Gateway를 사용하는 이유를 찾아보면 여러가지가 있지만, 그 중에서 대표적인 이유는 크게 2가지이다.

1. **라우팅 (Routing) :** 클라이언트 요청을 적절한 서비스로 전달하기 위해 사용

   지금까지 구성한 프로젝트를 보면 클라이언트가 각 서비스에 통신을 하려면, 각 서비스의 서로 다른 주소를 전부 알고 있어야 한다.

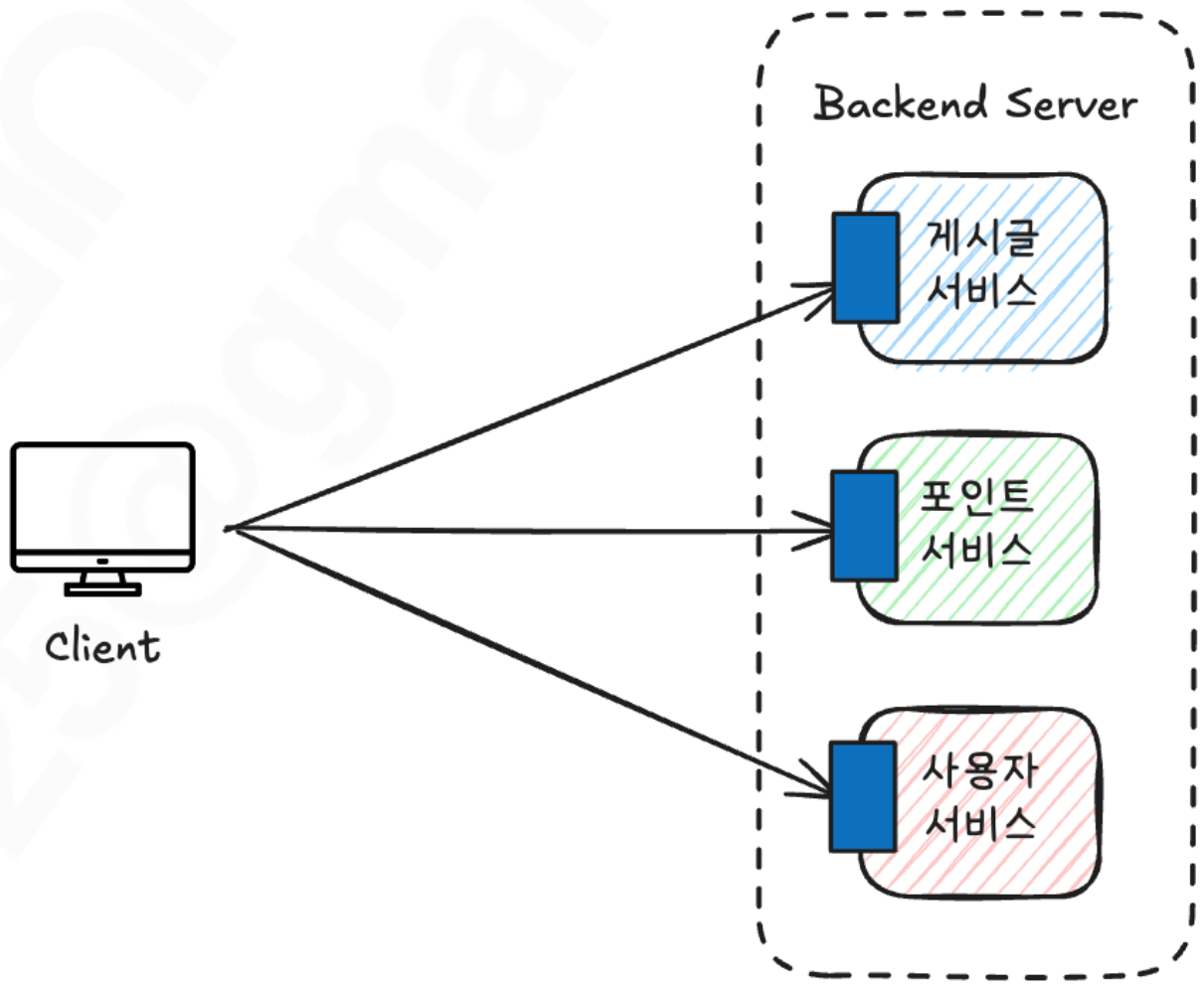
   - localhost:8080 (사용자 서비스)
   - locahlost:8081 (게시글 서비스)
   - localhost:8082 (포인트 서비스)

   하지만 API Gateway를 사용하면 클라이언트가 각 서비스에 통신을 해야 할 때, API Gateway 주소 하나로만 통신하면 된다.

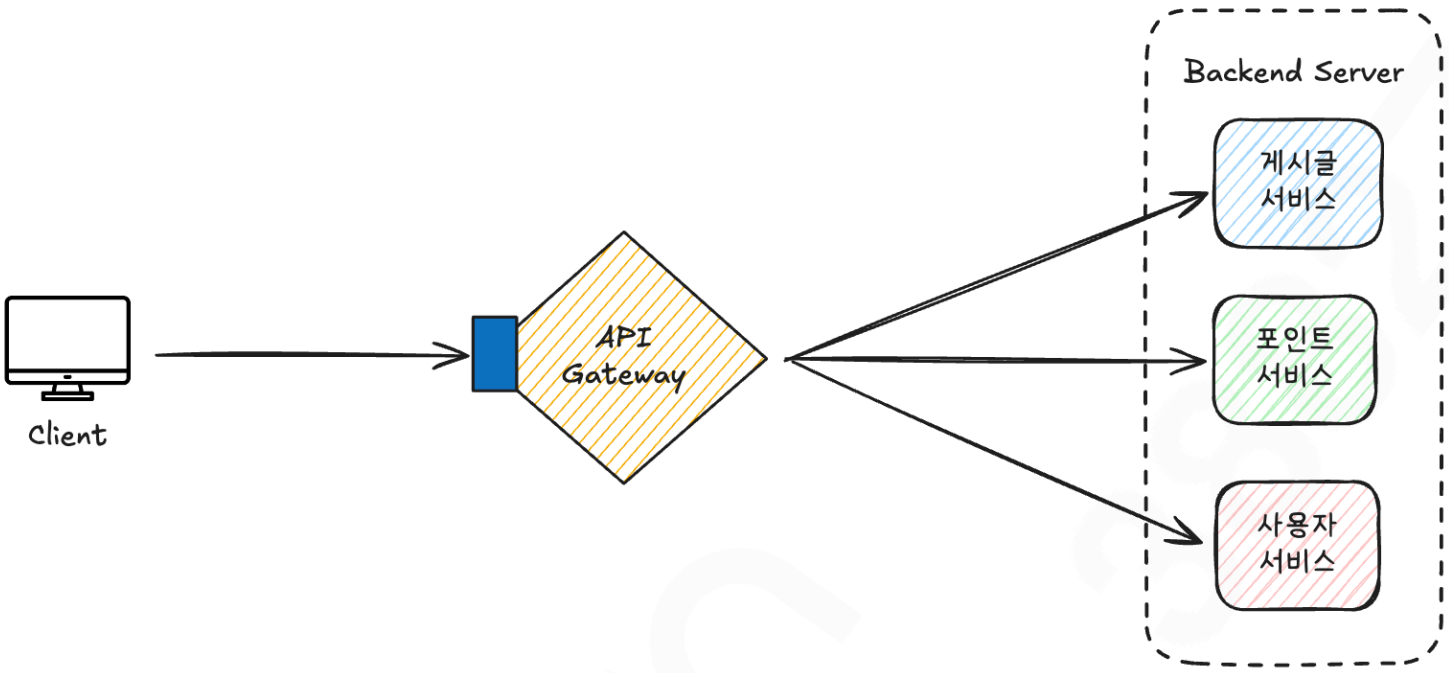
   - localhost:8000 (사용자 서비스)
   - localhost:8000 (포인트 서비스)

2. **공통 로직 처리** : 인증, 로깅과 같이 여러 서비스에서 공통으로 사용하는 횡단 관심사 로직을 한 곳에서 처리

   예를 들어, 각 서비스에 클라이언트 요청을 검증하기 위한 **JWT 인증 로직**이 각각 중복되어 있다고 해보자. 이 경우, 인증 로직을 일부 수정해야 한다면 모든 서비스 코드를 일일이 변경해야 하는 불편이 발생한다.



그래서 API Gateway를 활용하면, 아래 그림과 같이 각 서비스에 중복된 코드를 넣을 필요 없이 공통 로직을 API Gateway에서 일괄적으로 처리하도록 구성할 수 있다.



✅ API Gateway를 구현할 때 많이 사용하는 툴

API Gateway를 구현할 때 활용할 수 있는 툴은 굉장히 많다. 현업에서는 대표적으로 크게 2가지를 많이 사용한다.

- Spring Cloud Gateway : 프레임워크를 기반으로 직접 구현해야 하는 API Gateway 서버
- AWS API Gateway : AWS에서 자체적으로 제공하는 API Gateway 서버

이 강의에서는 Spring Cloud Gateway를 활용해서 API Gateway를 구현해볼 예정이다.

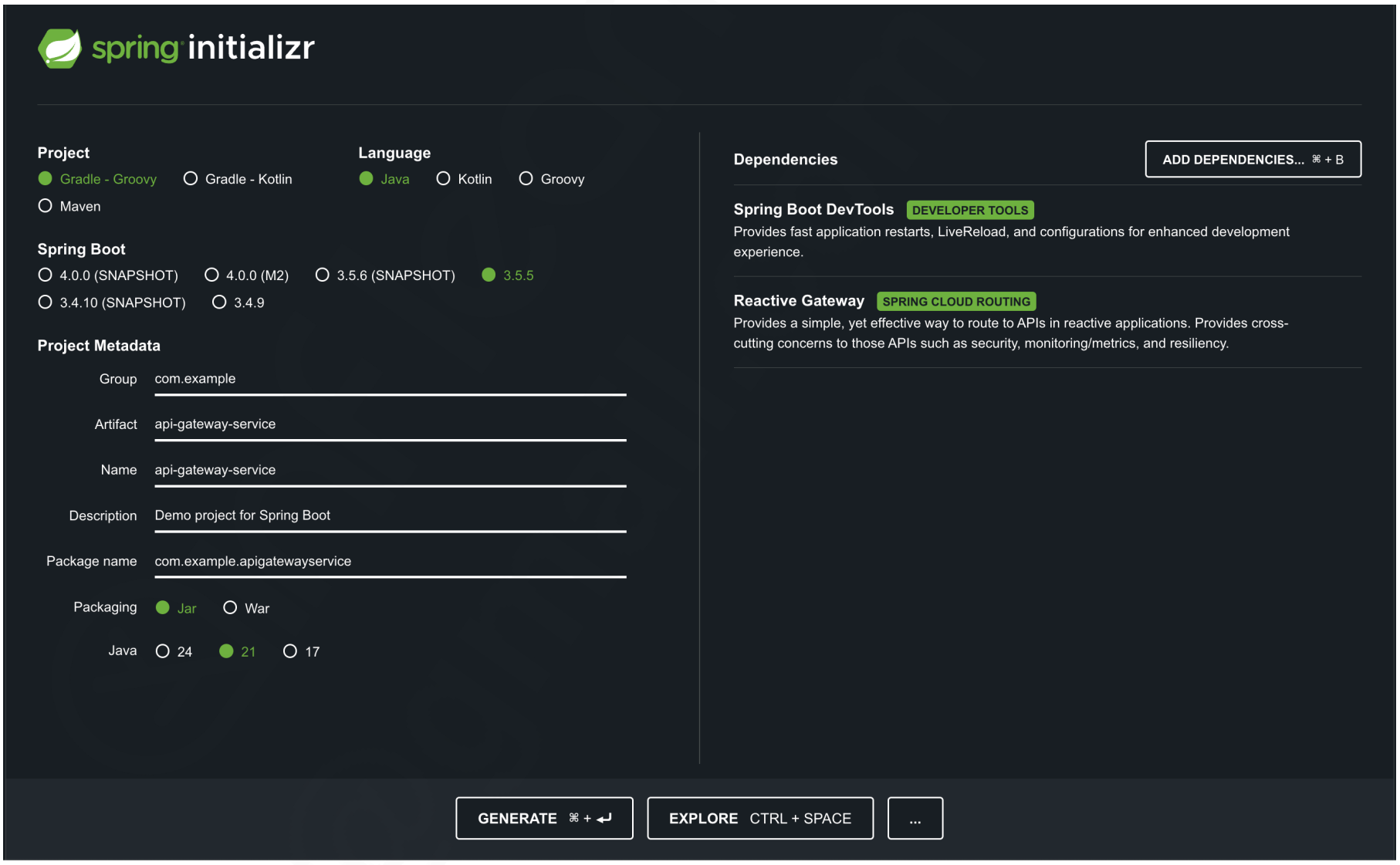
khb3025@gmail.com

🎯 [실습] Spring Cloud Gateway 셋팅하기 / 라우팅 설정하기

✅ Spring Cloud Gateway 서버 기본 셋팅하기 / 라우팅 설정하기

1. **Spring Boot 프로젝트 셋팅**

> start.spring.io
>
> https://start.spring.io/



- `Artifact`와 `Name`을 `api-gateway-service`라고 지어주자.
- `Package name`을 Java 컨벤션에 맞게 `apigatewayservice`라고 지어주자.
- Java 21 버전을 선택하자. 아래 과정을 Java 21 버전을 기준으로 진행할 예정이다.
- Dependencies는 `Spring Boot DevTools`, `Reactive Gateway`를 선택하자.

2. **application.yml 작성하기**

> 이 프로젝트에서는 `application.properties`를 지우고 `application.yml`을 생성했다.

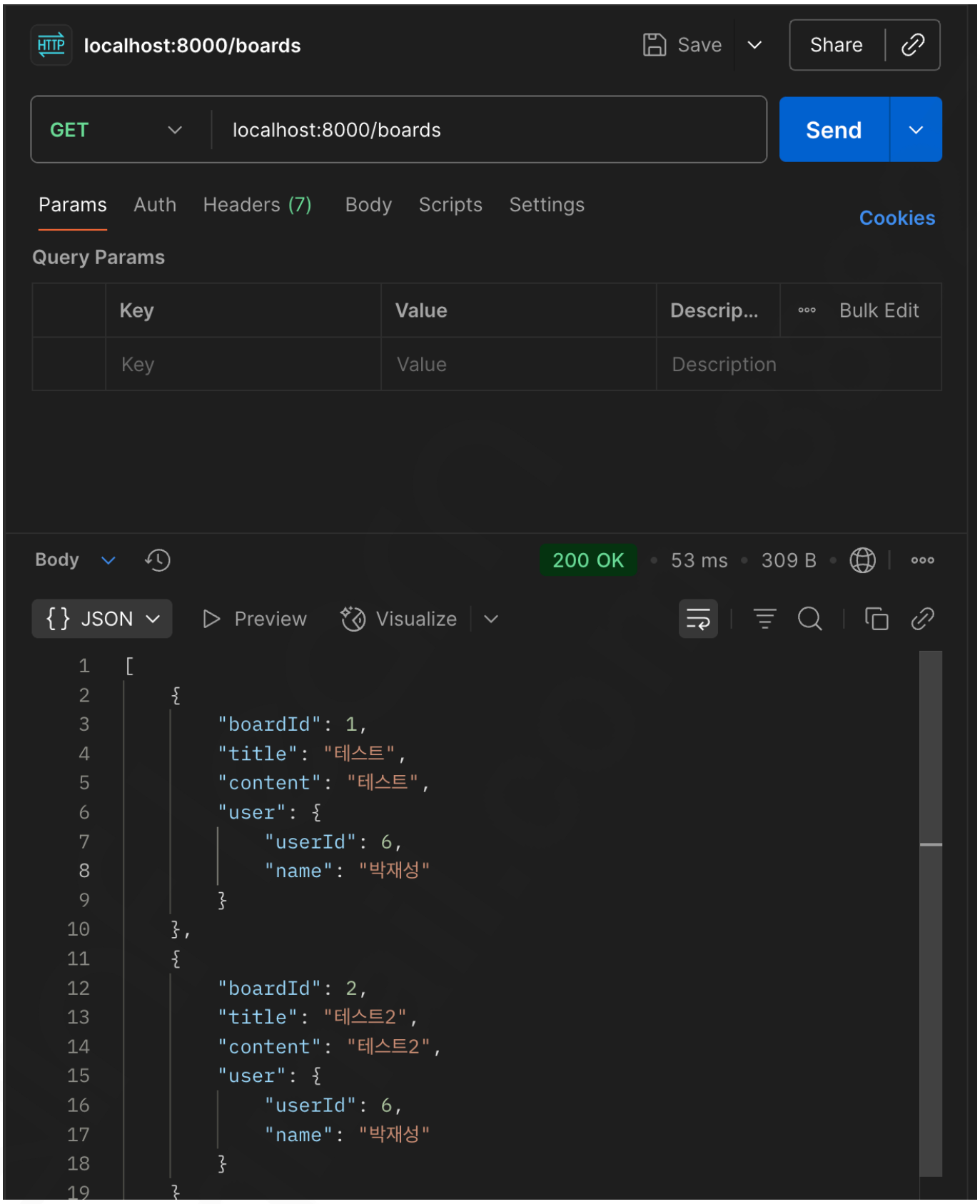
```
server:
  port: 8000

spring:
  cloud:
    gateway:
        server:
```

```
webflux:
routes:
    # /users로 시작하는 모든 경로의 요청은 user-service(localhost:8080)으로 전달
  - id: user-service
    uri: http://localhost:8080
    predicates:
      - Path=/users/**
  # /boards로 시작하는 모든 경로의 요청은 board-service(localhost:8081)으로 전달
  - id: board-service
    uri: http://localhost:8081
    predicates:
      - Path=/boards/**
  # /points로 시작하는 모든 경로의 요청은 point-service(localhost:8082)으로 전달
  - id: point-service
    uri: http://localhost:8082
    predicates:
      - Path=/points/**
```

- `id` : 각 라우팅 식별자
- `uri` : 클라이언트로부터 받은 요청을 전달할 주소
- `predicates` : 어떤 요청에 대해 특정 서비스로 요청을 전달할 지 패턴을 지정

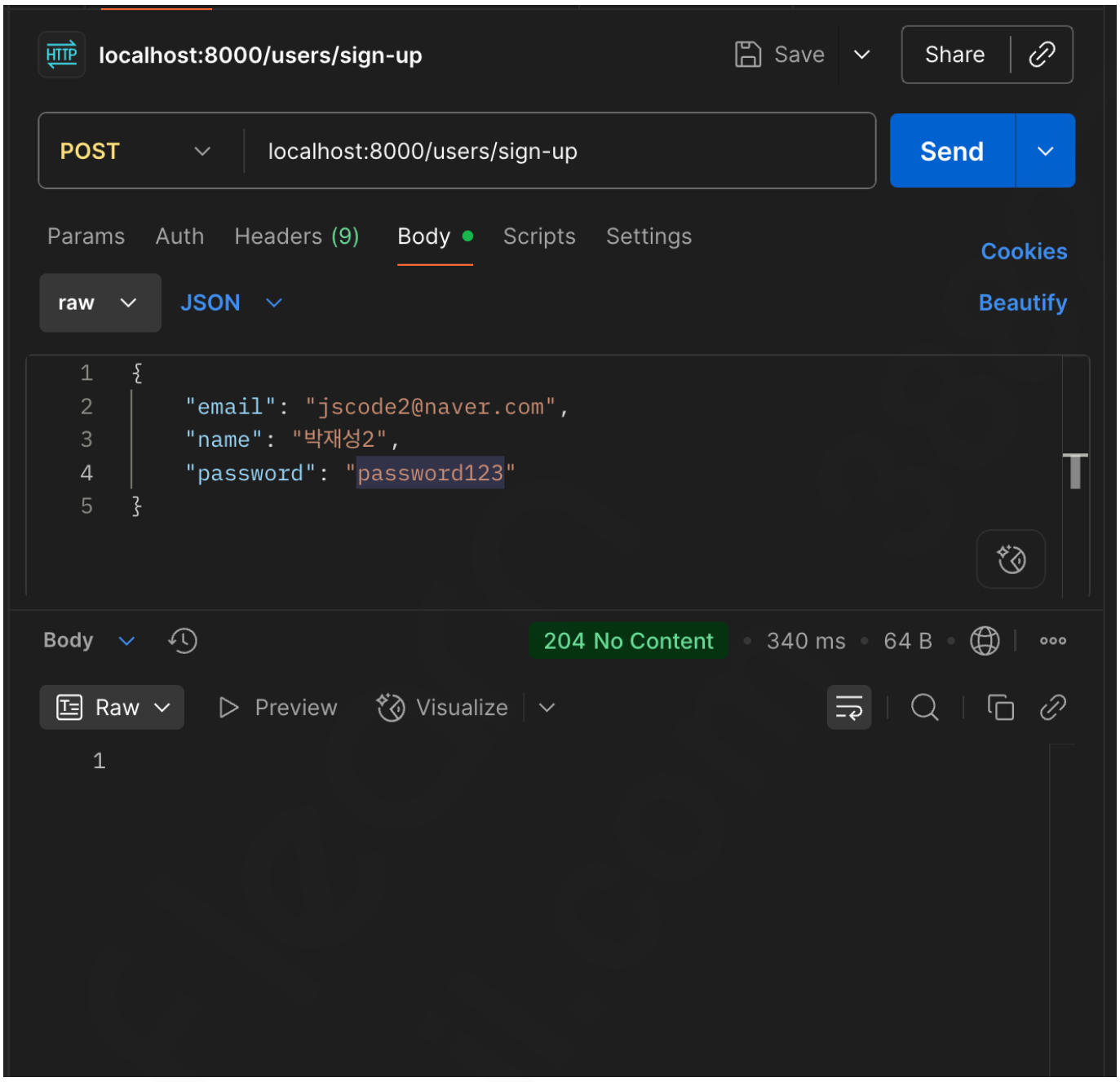
3. **요청이 잘 라우팅되는 지 테스트해보기**
    a. **게시글 서비스의 게시글 조회 API 테스트**

khb3025@gmail.com

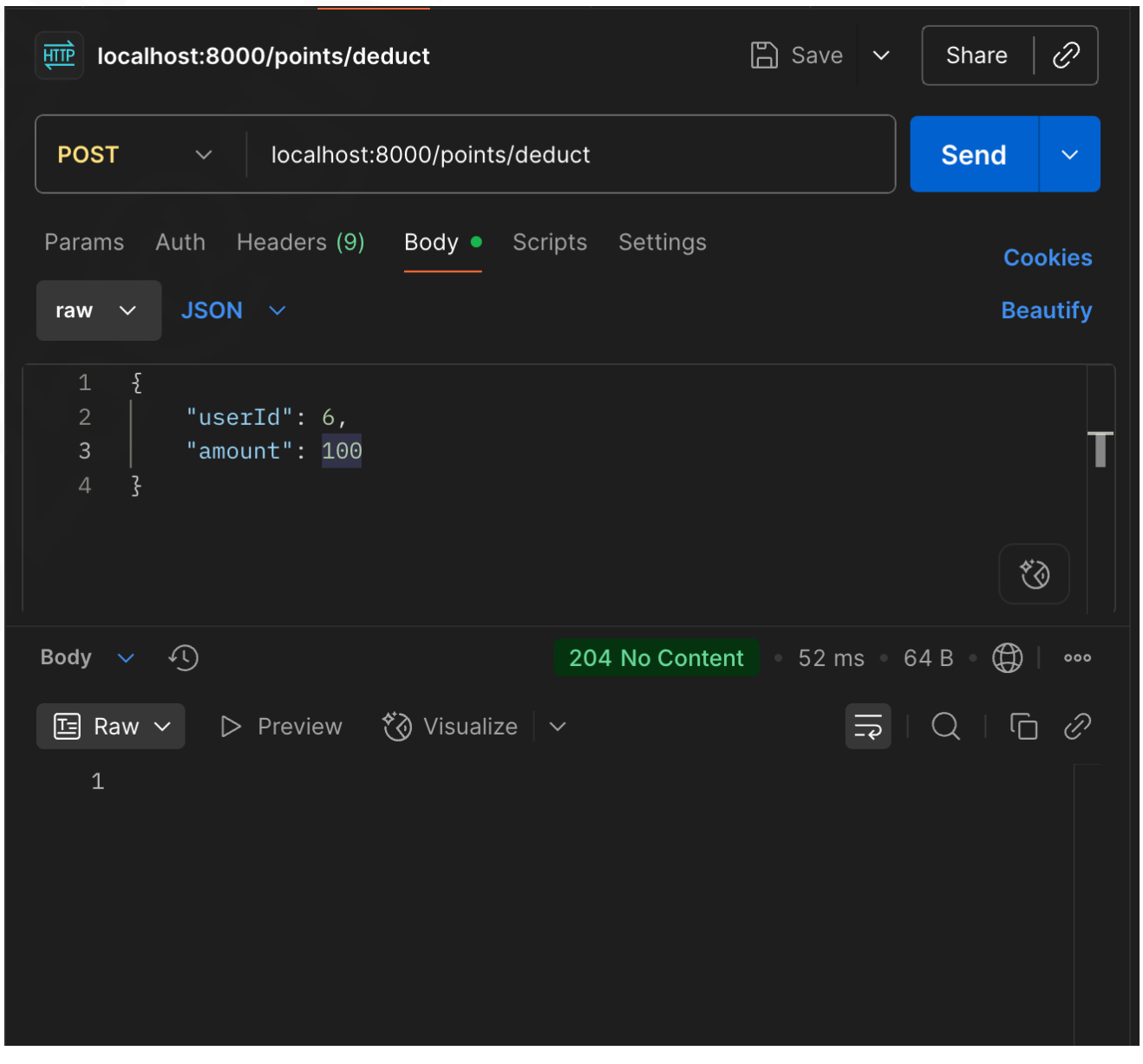


b. **사용자 서비스의 회원가입 API 테스트**

khb3025



c. **포인트 서비스의 포인트 차감 API 테스트**



위 API 들을 실행시켰을 때 성공적으로 응답하는 걸로 봐서, API Gateway가 클라이언트의 요청을 각 마이크로서비스로 잘 전달해준다는 걸 알 수 있다.

@kfc38827@gmail.com
khb302500

🎯 [실습] '외부용 API'와 '마이크로서비스간 통신용 API' 구분하기

✅ '외부용 API'와 '마이크로서비스간 통신용 API' 구분하기

현재까지 프로젝트에서 아래의 API를 구현했다.

[사용자 서비스]

- 회원가입 (`POST /users/sign-up`) - **외부용 API**
- 특정 사용자 정보 조회 (`GET /users/{userId}`) - 내부용 API
- 여러 사용자 정보 조회 (`GET /users`) - 내부용 API
- 활동 점수 적립 (`POST /users/activity-score/add`) - 내부용 API
- 활동 점수 차감 (`POST /users/activity-score/deduct`) - 내부용 API

[게시글 서비스]

- 게시글 생성 (`POST /boards`) - **외부용 API**
- 특정 게시글 조회 (`GET /boards/{boardId}`) - **외부용 API**
- 전체 게시글 조회 (`GET /boards`) - **외부용 API**

[포인트 서비스]

- 포인트 적립 (`POST /points/add`) - 내부용 API
- 포인트 차감 (`POST /points/deduct`) - 내부용 API

MSA에서는 위와 같이 반드시 **외부 클라이언트가 사용할 API**와 **내부 마이크로서비스끼리 사용하는 API**를 구분해야 한다. 그래야 나중에 구분해서 보안 처리를 하기가 쉽다. 한 마디로, 외부 클라이언트가 '내부 마이크로서비스끼리 사용하는 API'에 접근하지 못하도록 설정하기가 쉽다.

외부 클라이언트용 API 주소와 **내부 마이크로서비스용 API**를 구분하기 위해 아래와 같이 URL을 구분하자.

- 외부 클라이언트용 API 주소 : `/api/___`
- 내부 클라이언트용 API 주소 : `/internal/___`

✅ 사용자 서비스 코드 수정하기

1. **UserController 복사해서 UserInternalController 만들기**

   controller/UserInternalController

```
@RestController
@RequestMapping("/internal/users")
public class UserInternalController {
    ...
    @PostMapping("sign-up")
  public ResponseEntity<Void> signUp() {...}

  @GetMapping("{userId}")
  public ResponseEntity<UserResponseDto> getUser(...) {...}
```

```
  @GetMapping()
  public ResponseEntity<List<UserResponseDto>> getUsersByIds() {...}

    @PostMapping("activity-score/add")
  public ResponseEntity<Void> addActivityScore() {...}

}
```

2. **UserController 코드 수정하기**

   controller/UserController

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    ...
  @PostMapping("sign-up")
  public ResponseEntity<Void> signUp() {...}

  @GetMapping("{userId}")
  public ResponseEntity<UserResponseDto> getUser(...) {...}

  @GetMapping()
  public ResponseEntity<List<UserResponseDto>> getUsersByIds() {...}

    @PostMapping("activity-score/add")
  public ResponseEntity<Void> addActivityScore() {...}
}
```

3. **Client 코드 수정하기**

   client/PointClient

```
@Component
public class PointClient {

  ...
  public void addPoints(Long userId, int amount) {
    AddPointsRequestDto addPointsRequestDto
        = new AddPointsRequestDto(userId, amount);
    this.restClient.post()
        .uri("/internal/points/add")
        .contentType(MediaType.APPLICATION_JSON)
        .body(addPointsRequestDto)
        .retrieve()
        .toBodilessEntity();
  }
}
```

4. **서버 다시 실행시키기**

✅ 게시글 서비스 코드 수정하기

> 게시글 서비스는 외부용 API만 존재한다.

1. **BoardController 코드 수정하기**

   controller/BoardController

```
@RestController
@RequestMapping("/api/boards")
public class BoardInternalController {
    ...
    @PostMapping
  public ResponseEntity<Void> create(...) {...}

  @GetMapping("/{boardId}")
  public ResponseEntity<BoardResponseDto> getBoard(...) {...}

  @GetMapping()
  public ResponseEntity<List<BoardResponseDto>> getBoards() {...}
}
```

2. **Client 코드 수정하기**

   client/PointClient

```
@Component
public class PointClient {

    ...

  public void deductPoints(Long userId, int amount) {
    DeductPointsRequestDto deductPointsRequestDto
        = new DeductPointsRequestDto(userId, amount);
    this.restClient.post()
        .uri("/internal/points/deduct")
        .contentType(MediaType.APPLICATION_JSON)
        .body(deductPointsRequestDto)
        .retrieve()
        .toBodilessEntity();
  }

  public void addPoints(Long userId, int amount) {
    AddPointsRequestDto addPointsRequestDto
        = new AddPointsRequestDto(userId, amount);
    this.restClient.post()
        .uri("/internal/points/add")
        .contentType(MediaType.APPLICATION_JSON)
        .body(addPointsRequestDto)
        .retrieve()
        .toBodilessEntity();
  }
}
```

   client/UserClient

```
@Component
public class UserClient {

  ...

  public Optional<UserResponseDto> fetchUser(Long userId) {
    try {
      UserResponseDto userResponseDto = this.restClient.get()
          .uri("/internal/users/{userId}", userId)
          .retrieve()
          .body(UserResponseDto.class);
      return Optional.ofNullable(userResponseDto);
    } catch (RestClientException e) {
      // 로깅 : 예외 발생 시 로그를 남겨 문제를 파악할 수 있게 해야 함
      // log.error("사용자 정보 조회 실패: {}", e.getMessage(), e);
      return Optional.empty();
    }
  }

  public List<UserResponseDto> fetchUsersByIds(List<Long> ids) {
    try {
      return this.restClient.get()
          .uri(uriBuilder -> uriBuilder
              .path("/internal/users")
              .queryParam("ids", ids)
              .build()
          )
          .retrieve()
          .body(new ParameterizedTypeReference<>() {});
    } catch (RestClientException e) {
      // 로깅 : 예외 발생 시 로그를 남겨 문제를 파악할 수 있게 해야 함
      // log.error(...)
      return Collections.emptyList();
    }
  }

  public void addActivityScore(Long userId, int score) {
    AddActivityScoreRequestDto addActivityScoreRequestDto
        = new AddActivityScoreRequestDto(userId, score);
    this.restClient.post()
        .uri("/internal/users/activity-score/add")
        .contentType(MediaType.APPLICATION_JSON)
        .body(addActivityScoreRequestDto)
        .retrieve()
        .toBodilessEntity();
  }
}
```

3. **서버 다시 실행시키기**

✅ 포인트 서비스 코드 수정하기

> 포인트 서비스는 내부용 API만 존재한다.

1. **PointController를 PointInternalController로 이름 바꾸기**

```
@RestController
@RequestMapping("/points")
public class PointInternalController {
    ...
}
```

2. **PointInternalController로 코드 수정하기**

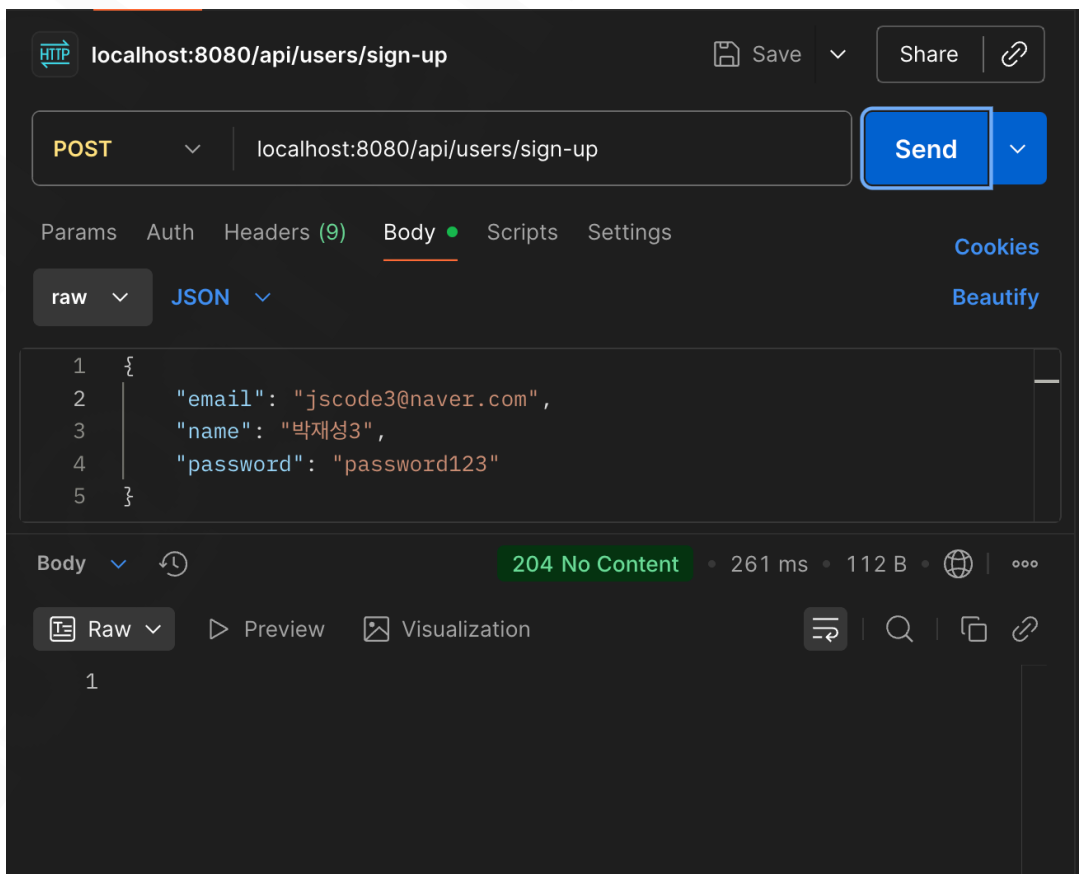
```
@RestController
@RequestMapping("/internal/points")
public class PointInternalController {
    ...
}
```

3. **서버 다시 실행시키기**

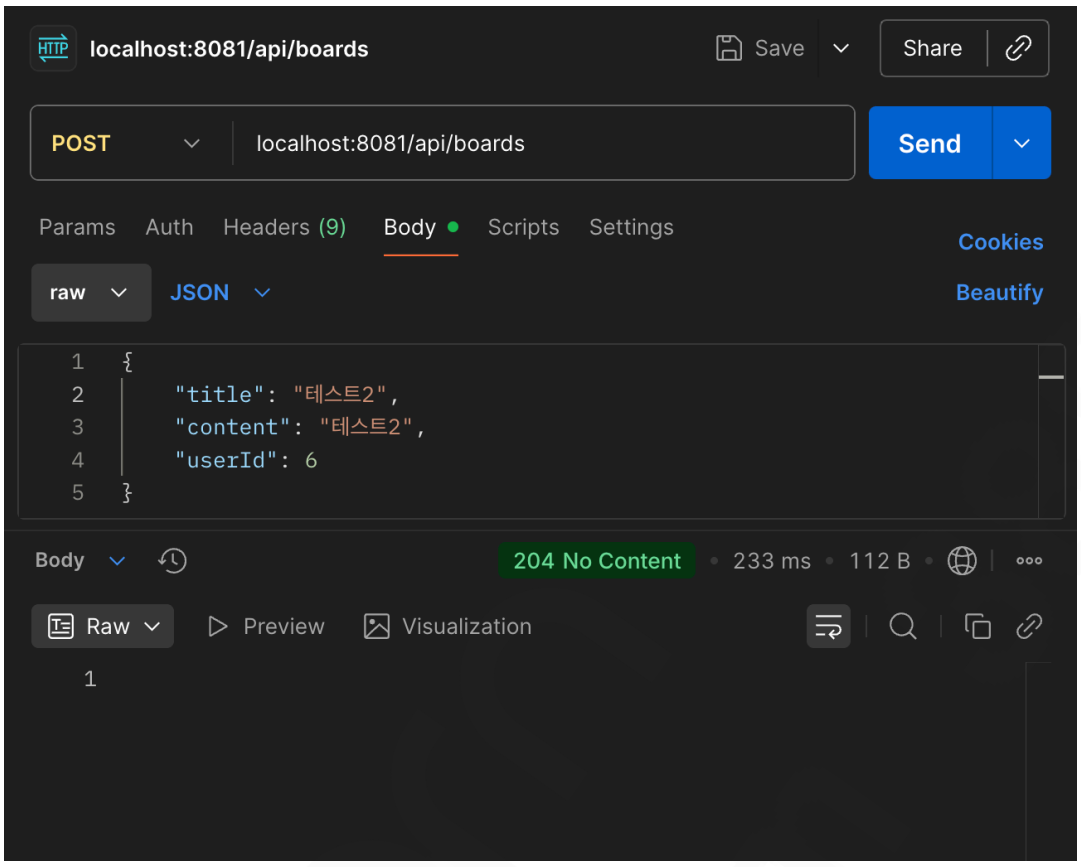
✅ 잘 작동하는 지 테스트하기

> 클라이언트가 사용하는 API에 통신을 보내서 성공 응답이 날라오는 지 확인해보자.

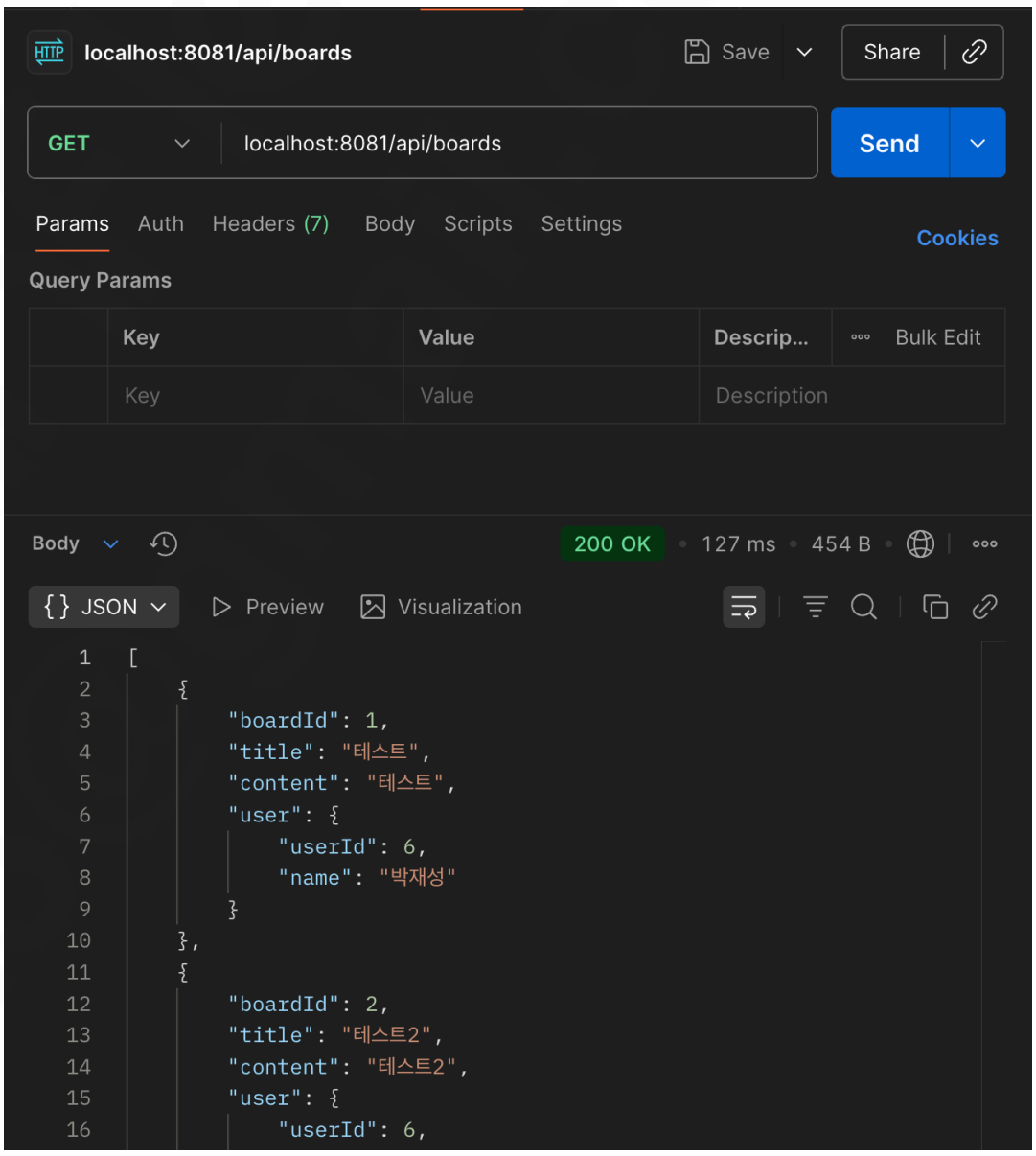
1. **회원가입 API**



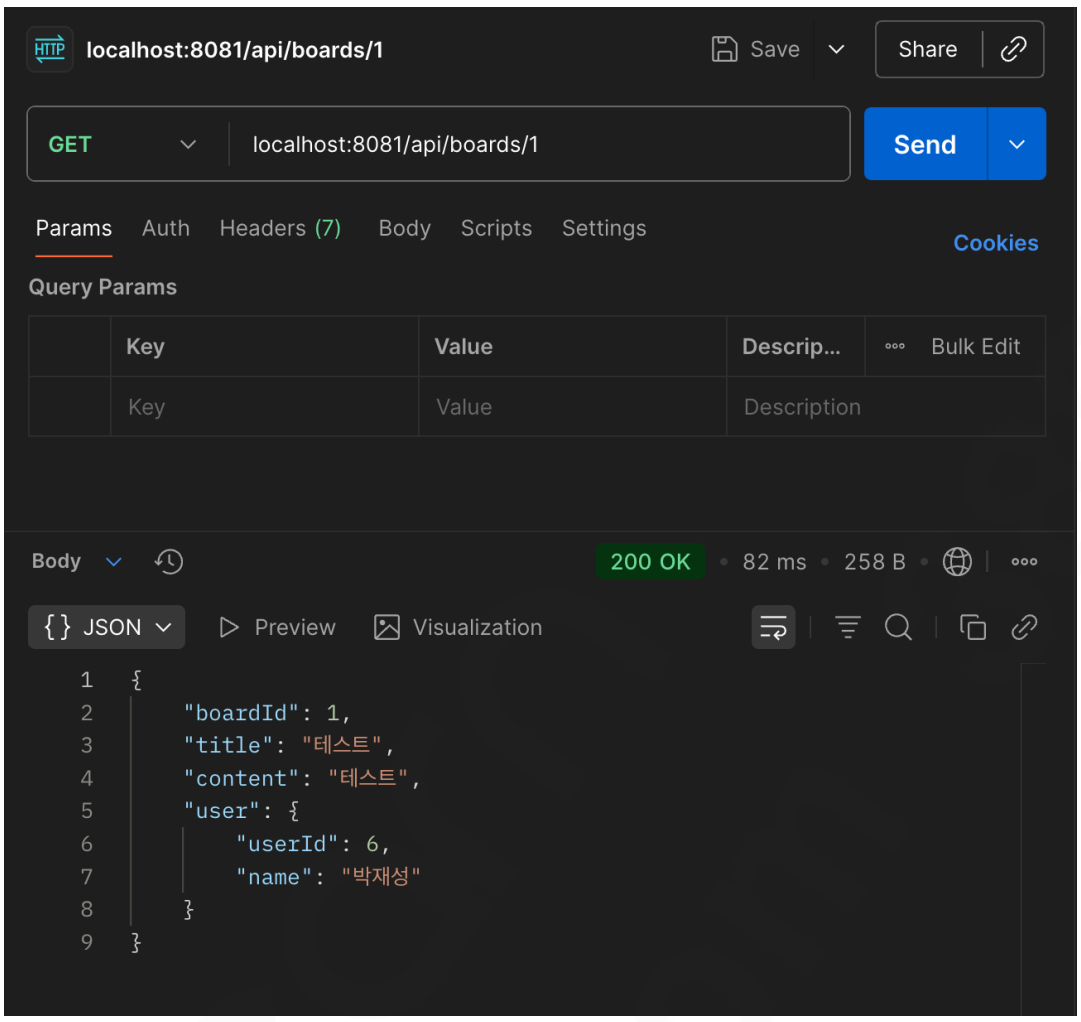
2. **게시글 작성 API**



3. **게시글 전체 조회 API**



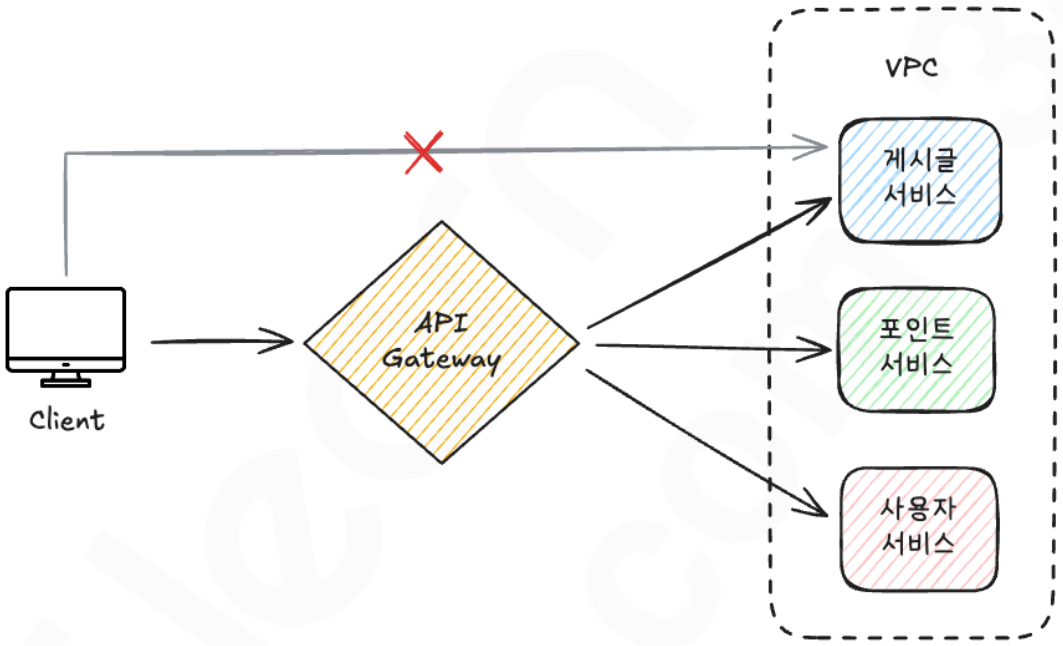
4. **특정 게시글 조회 API**



khb3025@gmail.com

🎯 [실습] API Gateway를 활용해 클라이언트가 접근할 API 제한하기

✅ API Gateway를 활용해 클라이언트가 접근할 API 제한하기



이전 강의에서 **클라이언트가 사용하는 외부용 API**와 **마이크로서비스 내부에서만 사용하는 내부용 API**를 분리했다. 이 구성에서 클라이언트는 내부용 API에 접근할 수 없게 막아야 한다. 그러려면 2가지 처리를 해주면 된다.

1. **클라이언트는 API Gateway로만 요청을 보낼 수 있게 만들면 된다.**
   (마이크로서비스를 VPC 내부에 구성함으로써 외부에서 접근할 수 없게 만들면 된다.)
2. 클라이언트가 API Gateway를 통해 요청을 보낼 수 있는 API를 외부용 API에 한해서만 오픈하면 된다.

여기서 첫 번째 언급한 VPC 구성은 클라우드 환경 내에서 구성해야 하기 때문에 이번 강의에서는 다루지 않을 것이다. 대신에 두 번째 언급한 내용은 API Gateway에 작성한 코드 일부만 수정해주면 된다. 따라서 API Gateway의 코드를 수정해 클라이언트가 외부용 API에만 접근할 수 있도록 바꿔보자.

✅ 코드 수정하기

> api-gateway-service에서 코드를 수정하면 된다.

1. **API Gateway의 라우팅 설정 변경하기**

   외부 API가 존재하는 user-service와 board-service만 라우팅을 설정하면 된다.

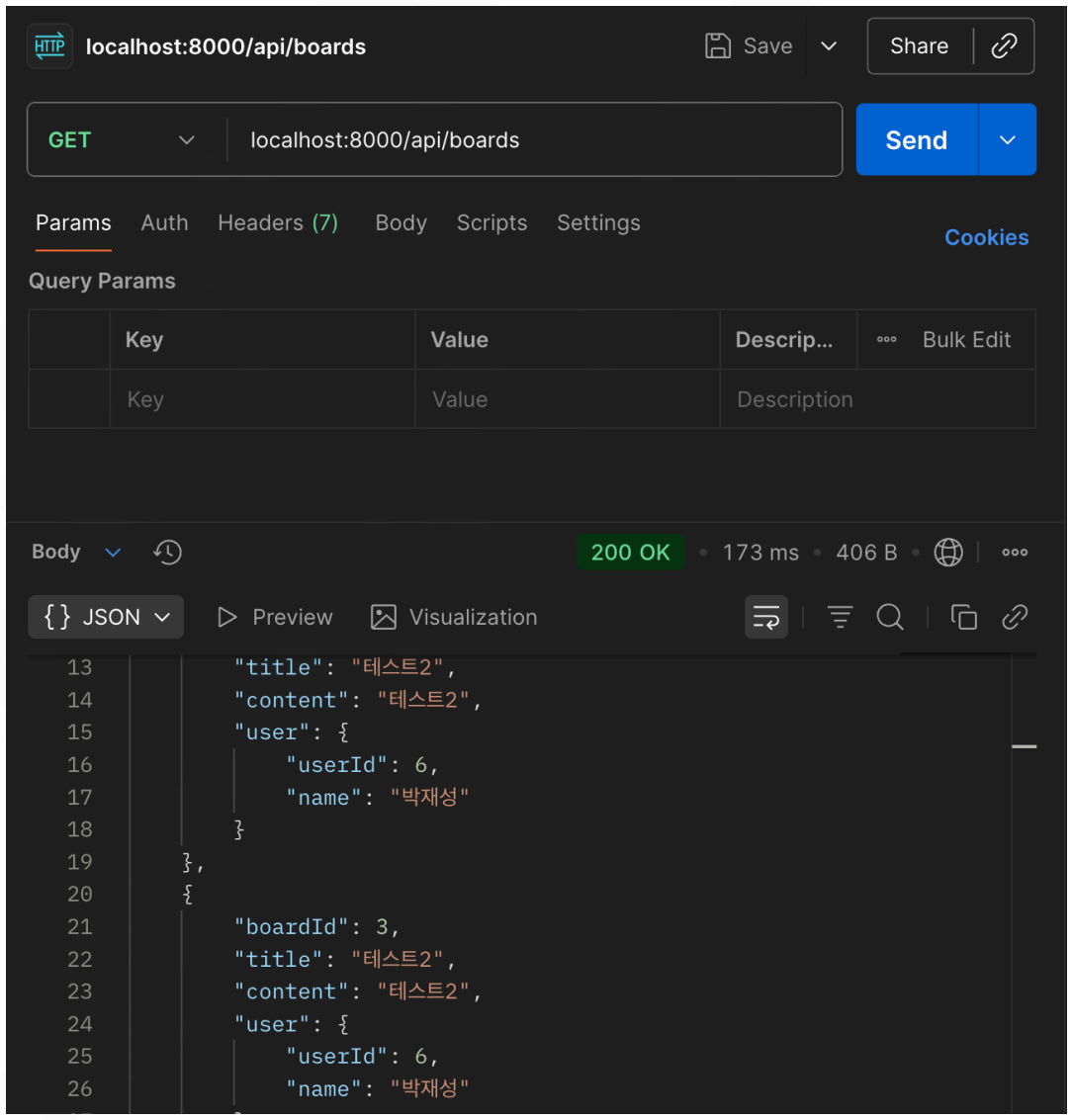
   application.yml

```
server:
  port: 8000

spring:
  cloud:
    gateway:
      server:
        webflux:
          routes:
```

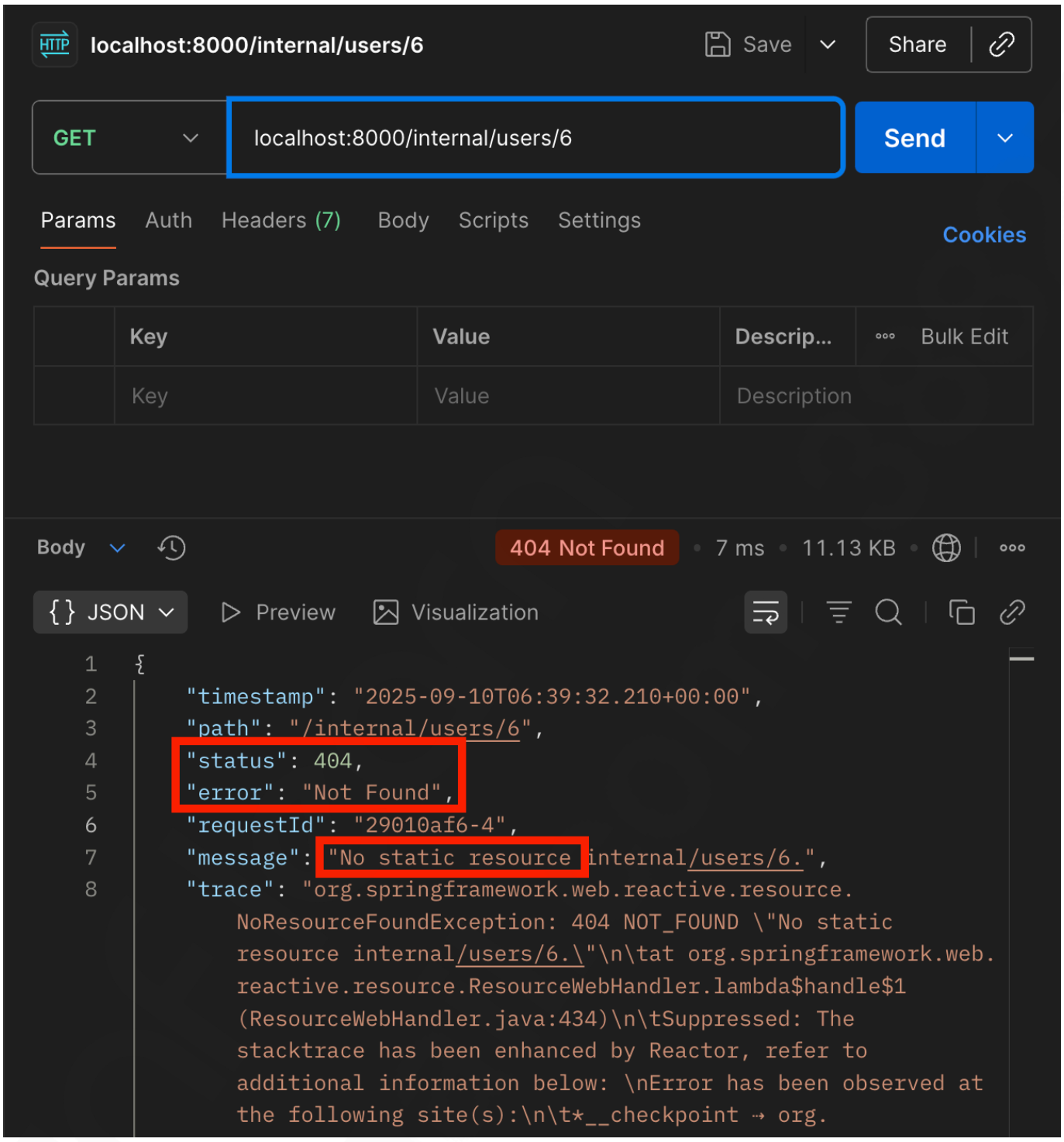
```
      - id: user-service
        uri: http://localhost:8080
        predicates:
          - Path=/api/users/**
      - id: board-service
        uri: http://localhost:8081
        predicates:
          - Path=/api/boards/**
      - id: point-service
        uri: http://localhost:8082
        predicates:
          - Path=/points/**
```

2. **잘 작동하는 지 테스트하기**

   [게시글 전체 조회 API - 외부용 API]



   [사용자 정보 조회 API - 내부용 API]



API Gateway 설정에 해당 주소를 작성해주지 않았더니, 해당 마이크로서비스에 접근이 차단되는 걸 알 수 있다. 즉, 클라이언트가 API Gateway에서 설정해놓은 외부용 API에만 접근이 가능하고, 내부용 API에는 접근이 차단된 걸 확인할 수 있다.

kb302500@gmail.com

📖

MSA에서 '인증 로직' 구현하는 방법

✅ MSA에서의 API 인증(Authentication)

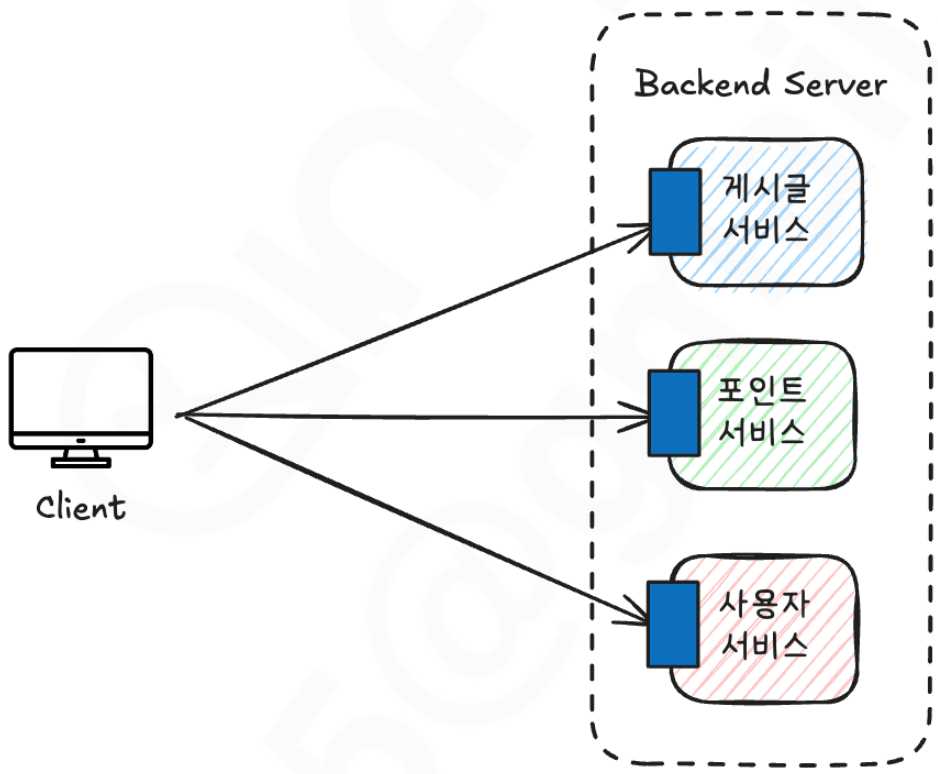
지금까지 구현한 프로젝트는 '인증(Authentication)'을 신경쓰지 않고 구현했다. 하지만 해당 프로젝트에서 악의적으로 다른 사용자인척 게시글을 작성하는 일이 발생하면 안 된다. 따라서 인증 로직을 구현해야 한다.

이전 강의를 통해 MSA에서는 2가지 형태의 API가 있다는 걸 알았다.

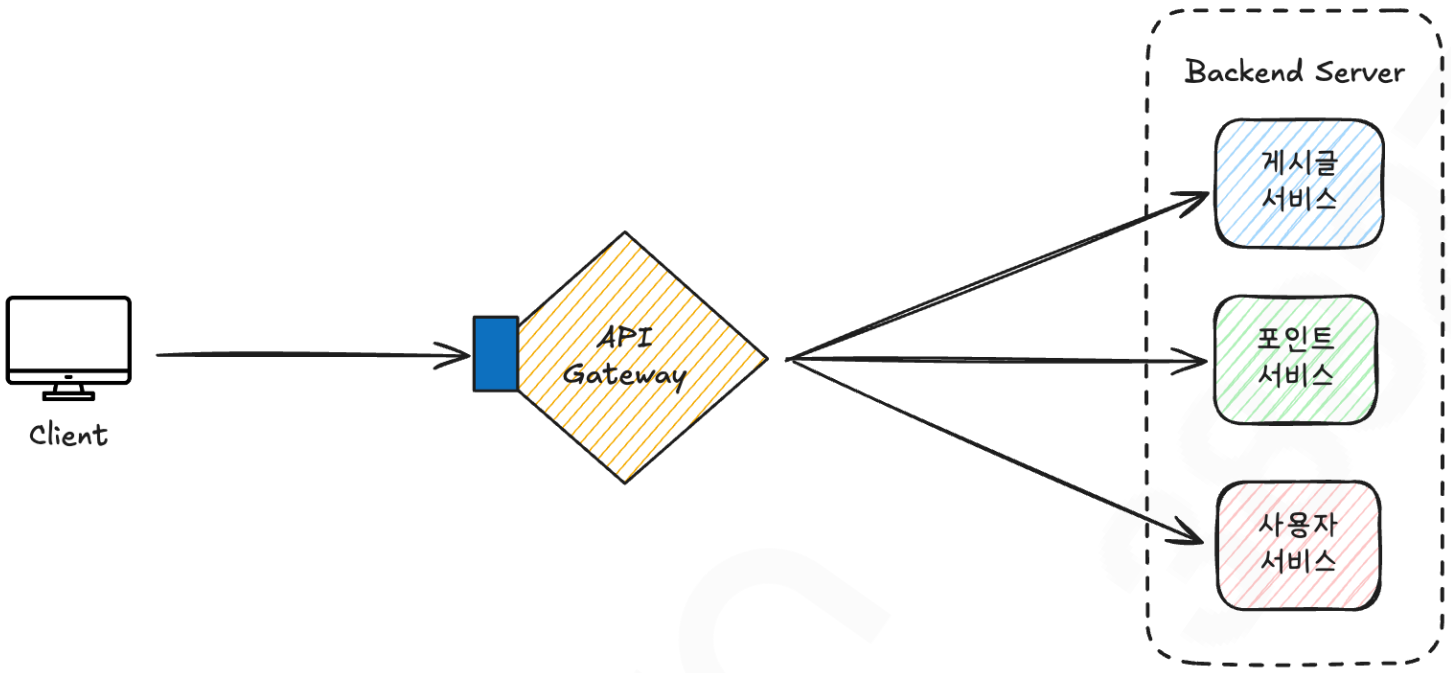
1. 클라이언트(웹, 모바일 등)가 사용하는 외부용 API
2. 마이크로서비스끼리 통신할 때 사용하는 내부용 API

여기서 내부용 API는 마이크로서비스끼리만 통신하기 때문에 악의적인 사용자를 고려해야 할 필요가 없다. 그래서 내부용 API에서는 인증 로직이 별도로 필요하지 않다. 하지만 외부용 API는 악의적인 사용자가 요청을 조작해 보낼 수도 있기 때문에 인증 로직이 필요한 경우가 많다.

✅ MSA에서 '인증 로직' 구현하는 방법



각 서비스마다 인증이 필요한 API를 가지고 있다면, 각 서비스에 인증 로직을 전부 구현해야 한다. 구체적으로 예를 들자면, 인증을 JWT로 한다고 했을 때 모든 서비스에 JWT 토큰을 검증하는 로직을 추가해야 한다. 하지만 중복된 코드가 발생하기 때문에 유지보수가 힘들어진다. 이 때문에 아래와 같이 API Gateway에만 인증 로직을 구현한다.



@naver.com
@gmail.com
khb3025

🎯 [실습] 사용자 서비스에 로그인 API 구현하기

✅ 사용자 서비스에 로그인 API 구현하기

1. **DTO 구현하기**

   dto/LoginRequestDto

```
public class LoginRequestDto {
  private String email;
  private String password;

  public String getEmail() {
    return email;
  }

  public String getPassword() {
    return password;
  }
}
```

dto/LoginResponseDto

```
public class LoginResponseDto {
  private String token;

  public LoginResponseDto(String token) {
    this.token = token;
  }

  public String getToken() {
    return token;
  }
}
```

2. **Controller 로직 구현하기**

   service/UserController

```
@RestController
@RequestMapping("/api/users")
public class UserController {
  ...

  @PostMapping("login")
  public ResponseEntity<LoginResponseDto> login(
      @RequestBody LoginRequestDto loginRequestDto
  ) {
    LoginResponseDto loginResponseDto = userService.login(loginRequestDto);
    return ResponseEntity.ok(loginResponseDto);
  }
}
```

3. **Service 로직 구현하기**

service/UserService

```
@Service
public class UserService {
  ...

  public LoginResponseDto login(LoginRequestDto loginRequestDto) {
    User user = userRepository.findByEmail(loginRequestDto.getEmail())
        .orElseThrow(() -> new IllegalArgumentException("사용자를 찾을 수 없습니다."));

    if (!user.getPassword().equals(loginRequestDto.getPassword())) {
      throw new IllegalArgumentException("비밀번호가 일치하지 않습니다.");
    }

    // (JWT 로직)
    String token = null;

    return new LoginResponseDto(
      token
    );
  }
}
```

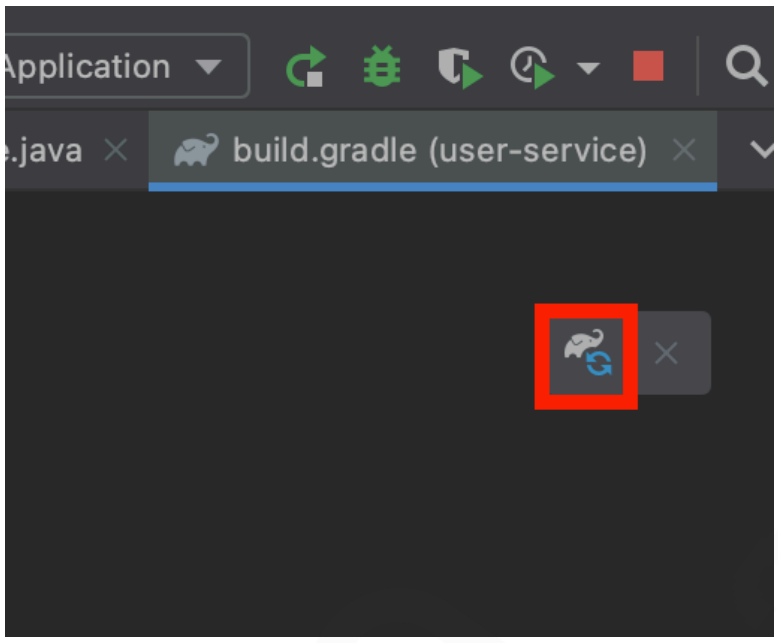
domain/UserRepository

```
public interface UserRepository extends JpaRepository<User, Long> {

  Optional<User> findByEmail(String email);
}
```

5. **JWT 의존성 추가하기**

build.gradle

```
...

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.kafka:spring-kafka'
    implementation 'io.jsonwebtoken:jjwt-api:0.13.0'
    runtimeOnly 'io.jsonwebtoken:jjwt-impl:0.13.0'
    runtimeOnly 'io.jsonwebtoken:jjwt-jackson:0.13.0'
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    runtimeOnly 'com.mysql:mysql-connector-j'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

...
```



6. **JWT 로직 추가하기**

application.yaml

```
...

jwt:
  secret: jscode-secret-1234-1234-1234-1234
```

service/UserService

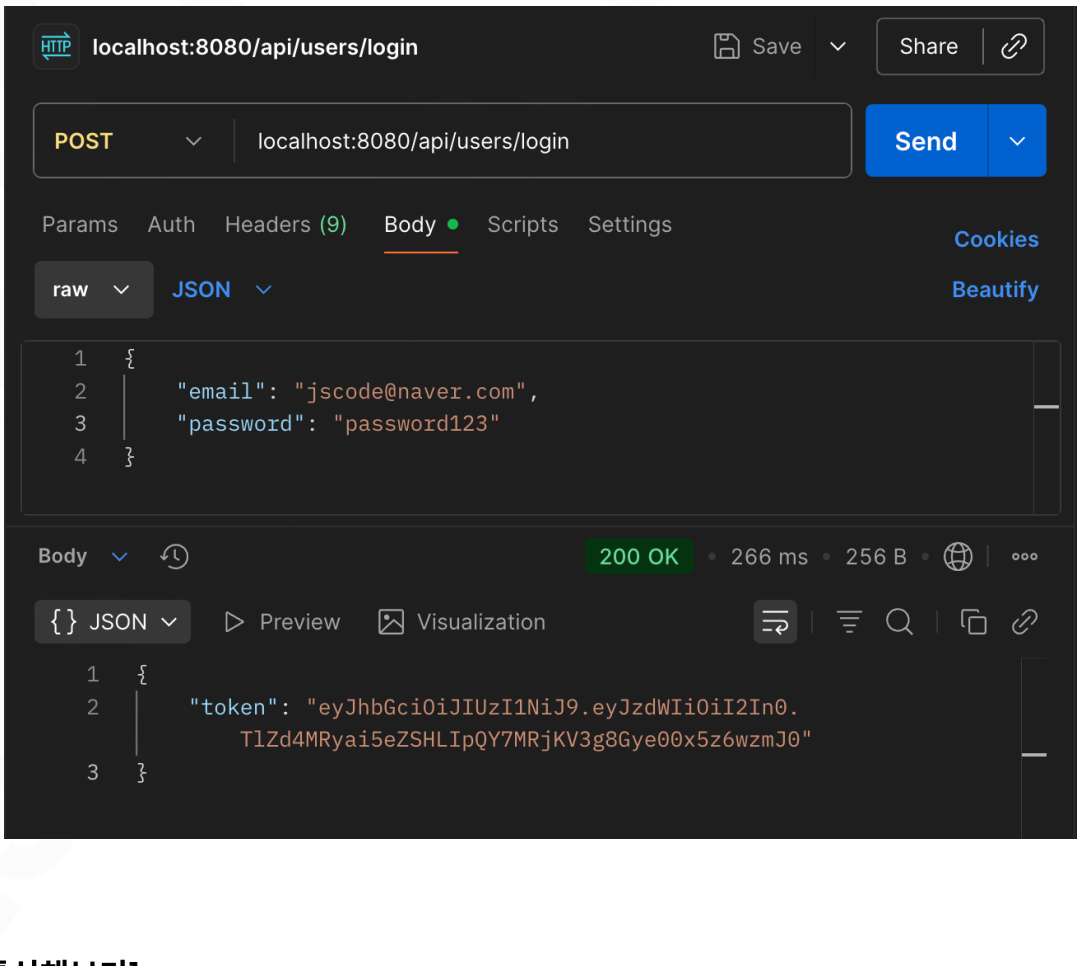
```
@Service
public class UserService {
  private final UserRepository userRepository;
  private final PointClient pointClient;
  private final KafkaTemplate<String, String> kafkaTemplate;
  private final String jwtSecret;

  public UserService(
      UserRepository userRepository,
      PointClient pointClient,
      KafkaTemplate<String, String> kafkaTemplate,
      @Value("${jwt.secret}") String jwtSecret
  ) {
    this.userRepository = userRepository;
    this.pointClient = pointClient;
    this.kafkaTemplate = kafkaTemplate;
    this.jwtSecret = jwtSecret;
  }

  ...

  public LoginResponseDto login(LoginRequestDto loginRequestDto) {
    User user = userRepository.findByEmail(loginRequestDto.getEmail())
        .orElseThrow(() -> new IllegalArgumentException("사용자를 찾을 수 없습니다."));

    if (!user.getPassword().equals(loginRequestDto.getPassword())) {
      throw new IllegalArgumentException("비밀번호가 일치하지 않습니다.");
    }

    // JWT를 만들 때 사용하는 Key 생성 (공식 문서 방식)
```

```
    SecretKey secretKey = Keys.hmacShaKeyFor(
            jwtSecret.getBytes(StandardCharsets.UTF_8)
        );

    // JWT 토큰 만들기
    String token = Jwts.builder()
        .subject(user.getUserId().toString())
        .signWith(secretKey)
        .compact();

    return new LoginResponseDto(
      token
    );
  }
}
```

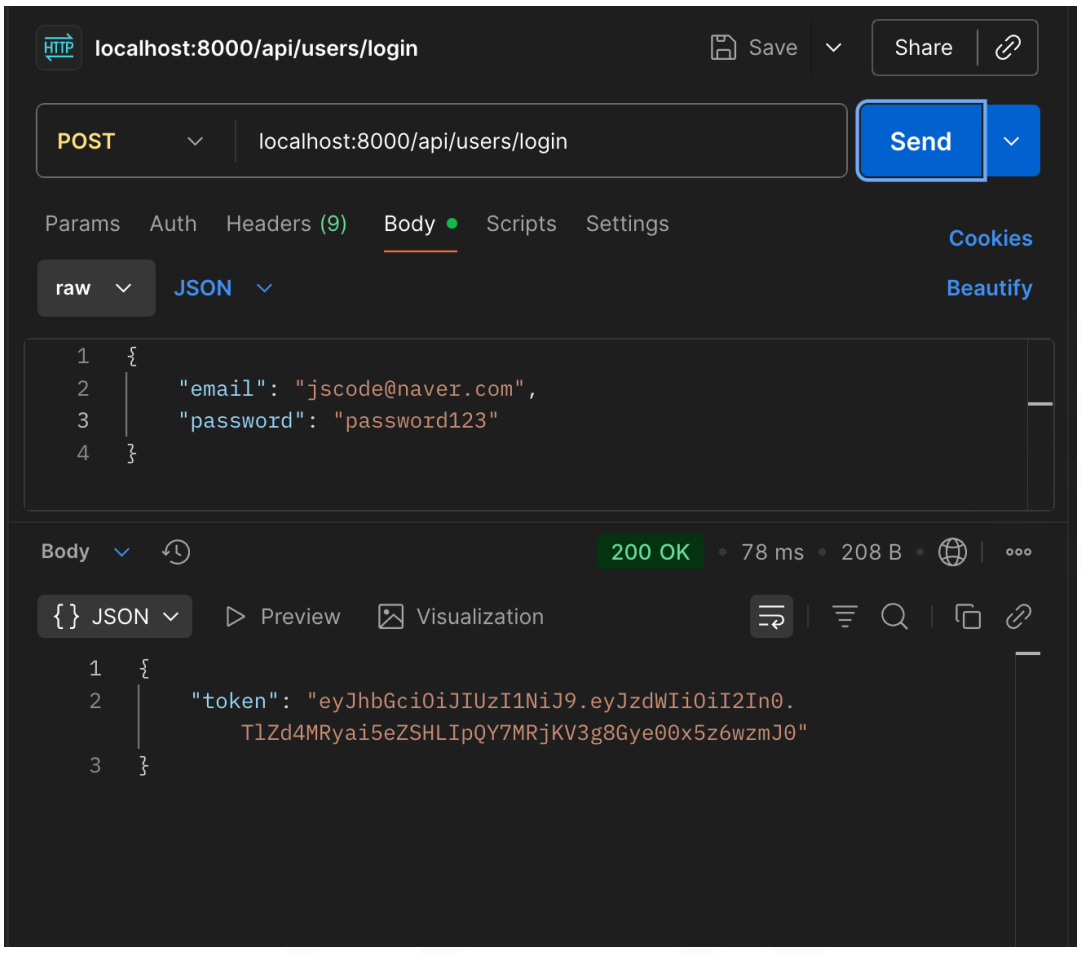
- **참고)** 간단한 로직 구조를 위해 Spring Security를 사용하지 않았고, 디테일한 JWT 설정도 하지 않았고, 별도의 클래스로 분리하지도 않았다.

6. **서버 다시 실행시키기**

7. **API 테스트 해보기**



[API Gateway 주소로도 통신해보기]

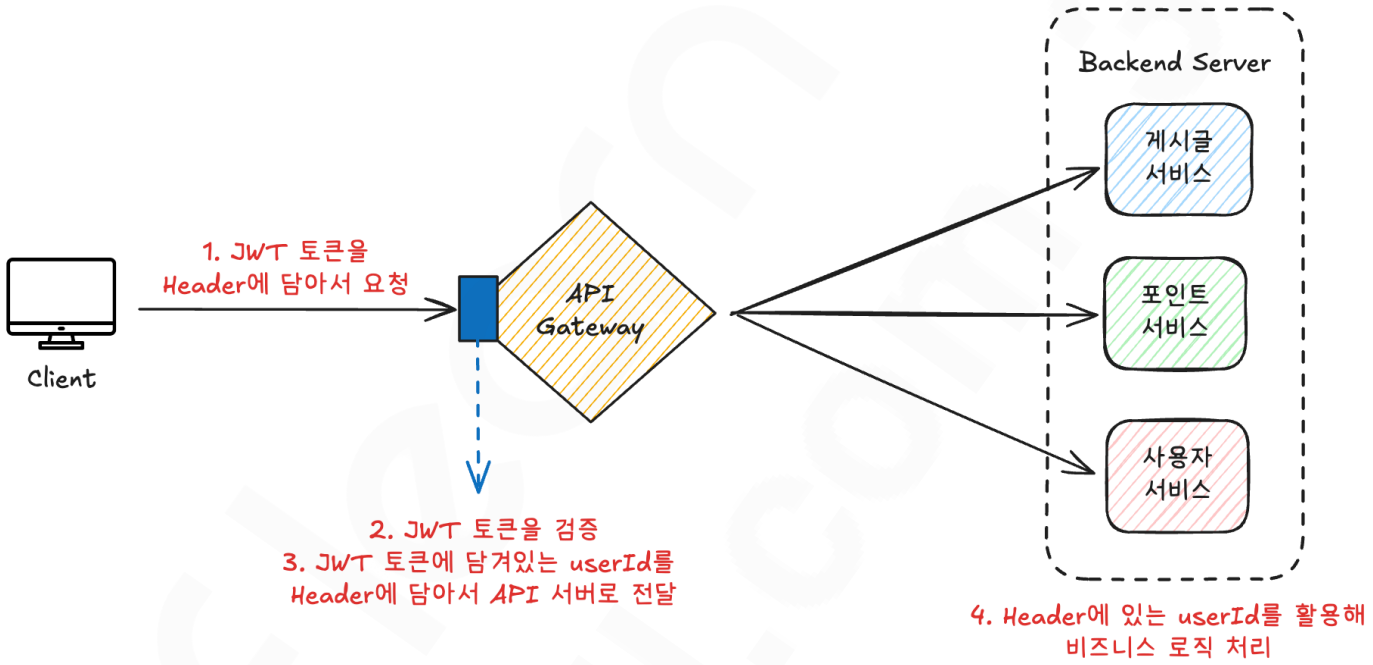


khb3025@gmail.com

🎯 [실습] API Gateway에 JWT 인증 로직 구현하기

✅ 설계

API Gateway로 들어오는 요청에 대해 비즈니스 로직을 처리할 수 있는 Filter 기능을 가지고 있다. 그래서 이 Filter 기능을 활용해 아래와 같이 JWT 인증 로직을 구현하고자 한다.

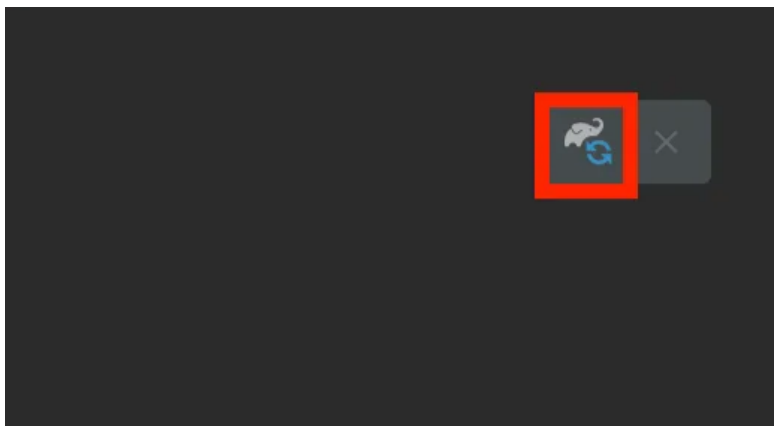


✅ API Gateway에 JWT 인증 로직 구현하기

1. **JWT 의존성 추가하기**

   build.gradle

```
...

dependencies {
    implementation 'org.springframework.cloud:spring-cloud-starter-gateway-server-webflux'
    implementation 'io.jsonwebtoken:jjwt-api:0.13.0'
    runtimeOnly 'io.jsonwebtoken:jjwt-impl:0.13.0'
    runtimeOnly 'io.jsonwebtoken:jjwt-jackson:0.13.0'
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testImplementation 'io.projectreactor:reactor-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

...
```



2. **application.yml 수정하기**

   application.yml

```
...

jwt:
  secret: jscode-secret-1234-1234-1234-1234
```

3. **JwtAuthenticationFilter**

```
@Component
public class JwtAuthenticationFilter extends AbstractGatewayFilterFactory {

  @Value("${jwt.secret}")
  private String jwtSecret;

  @Override
  public GatewayFilter apply(Object config) {
    return (exchange, chain) -> {
      // Reqeuest Header에서 토큰 가져오기
      String token = exchange.getRequest()
          .getHeaders()
          .getFirst("Authorization");
      System.out.println("토큰 : " + token);

      // 토큰이 없을 경우 401 Unauthorized로 응답
      if (token == null) {
        exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
        return exchange.getResponse().setComplete();
      }

      // SecretKey로 토큰 검증 및 Payload(userId 담겨있음) 가져오기
      SecretKey secretKey = Keys.hmacShaKeyFor(jwtSecret.getBytes(StandardCharsets.UTF_8));
      String subject = Jwts.parser()
          .verifyWith(secretKey)
          .build()
          .parseSignedClaims(token)
          .getPayload()
          .getSubject();
      System.out.println("userId : " + subject);

      // Payload를 X-User-Id 헤더에 담아서 Request 전달
      // (= 다른 마이크로서비스에 요청 전달할 때 userId 정보를 담아서 보냄)
      return chain.filter(
          exchange.mutate()
```

```
            .request(
                exchange.getRequest()
                    .mutate()
                    .header("X-User-Id", subject)
                    .build()
            )
            .build()
      );
    };
  }
}
```

참고) JWT 토큰 검증 로직을 더 자세히 작성할 수도 있지만 프로젝트의 심플함을 위해 간단하게 작성했다.

✅ 라우팅 설정하기

외부 클라이언트가 사용하는 API는 아래의 API 밖에 없다. 이 중에서 JWT 인증을 거쳐야 하는 API는 **게시글 생성 API** 밖에 없다. 게시글 생성 API에만 `JwtAuthenticationFilter`를 적용시켜보자.

[사용자 서비스]

- 회원가입 (`POST /users/sign-up`) - **외부용 API**
- 로그인 (`POST /users/login`) - **외부용 API**

[게시글 서비스]

- 게시글 생성 (`POST /boards`) - **외부용 API**
- 특정 게시글 조회 (`GET /boards/{boardId}`) - **외부용 API**
- 전체 게시글 조회 (`GET /boards`) - **외부용 API**

1. **application.yml 코드 수정하기**

   application.yml

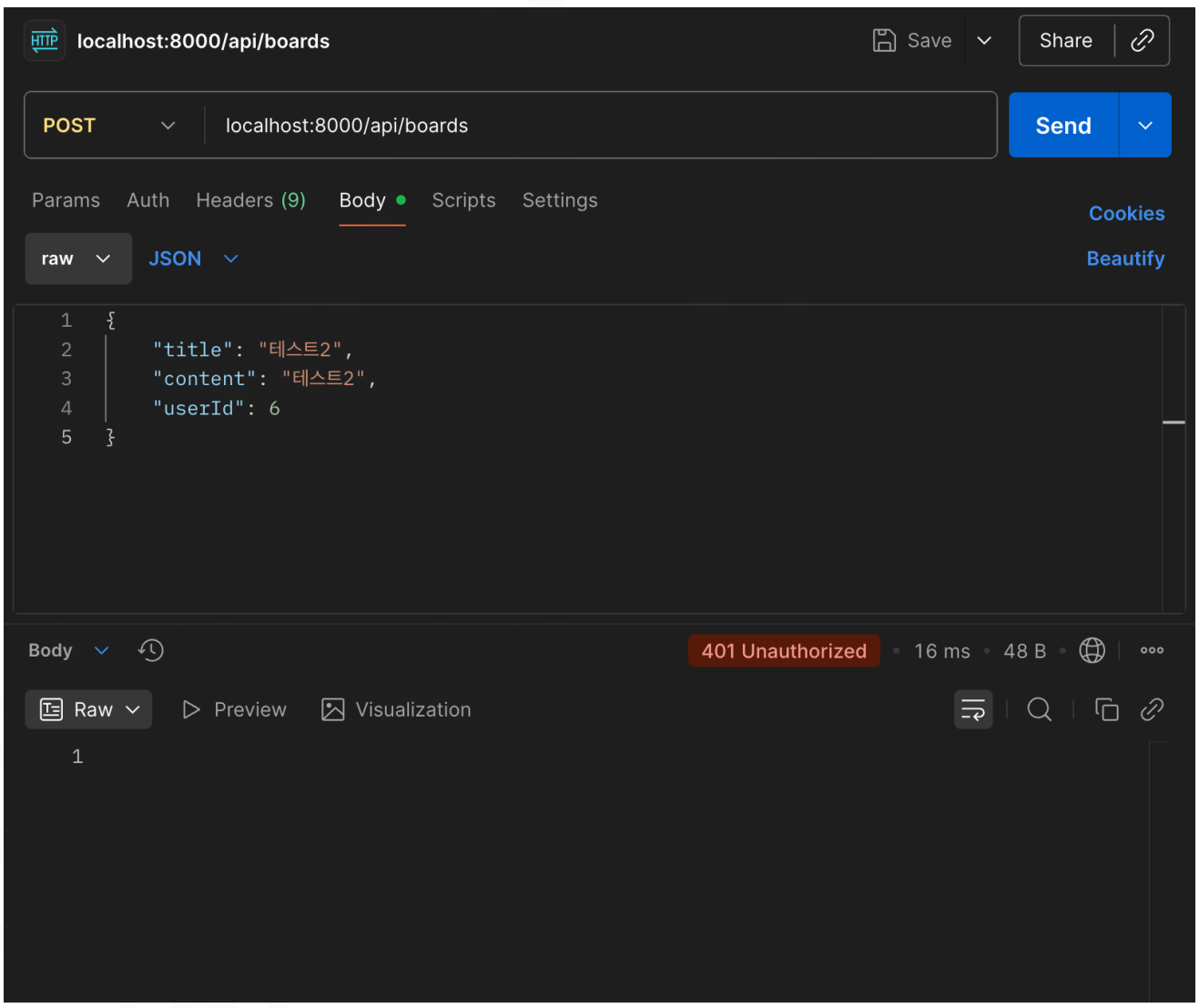
```
server:
  port: 8000

spring:
  cloud:
    gateway:
      server:
        webflux:
          routes:
            - id: user-service
              uri: http://localhost:8080
              predicates:
                - Path=/api/users/**
            - id: board-service-jwt
              uri: http://localhost:8081
              predicates:
                - Path=/api/boards
                - Method=POST
              filters:
                - JwtAuthenticationFilter
```

```
          - id: board-service
            uri: http://localhost:8081
            predicates:
              - Path=/api/boards/**
jwt:
  secret: jscode-secret-1234-1234-1234-1234
```

- **주의)** routes에서 설정한 순서대로 처리하기 때문에 `board-service` 위에 `board-service-jwt`에 대한 설정값을 작성해야 한다.

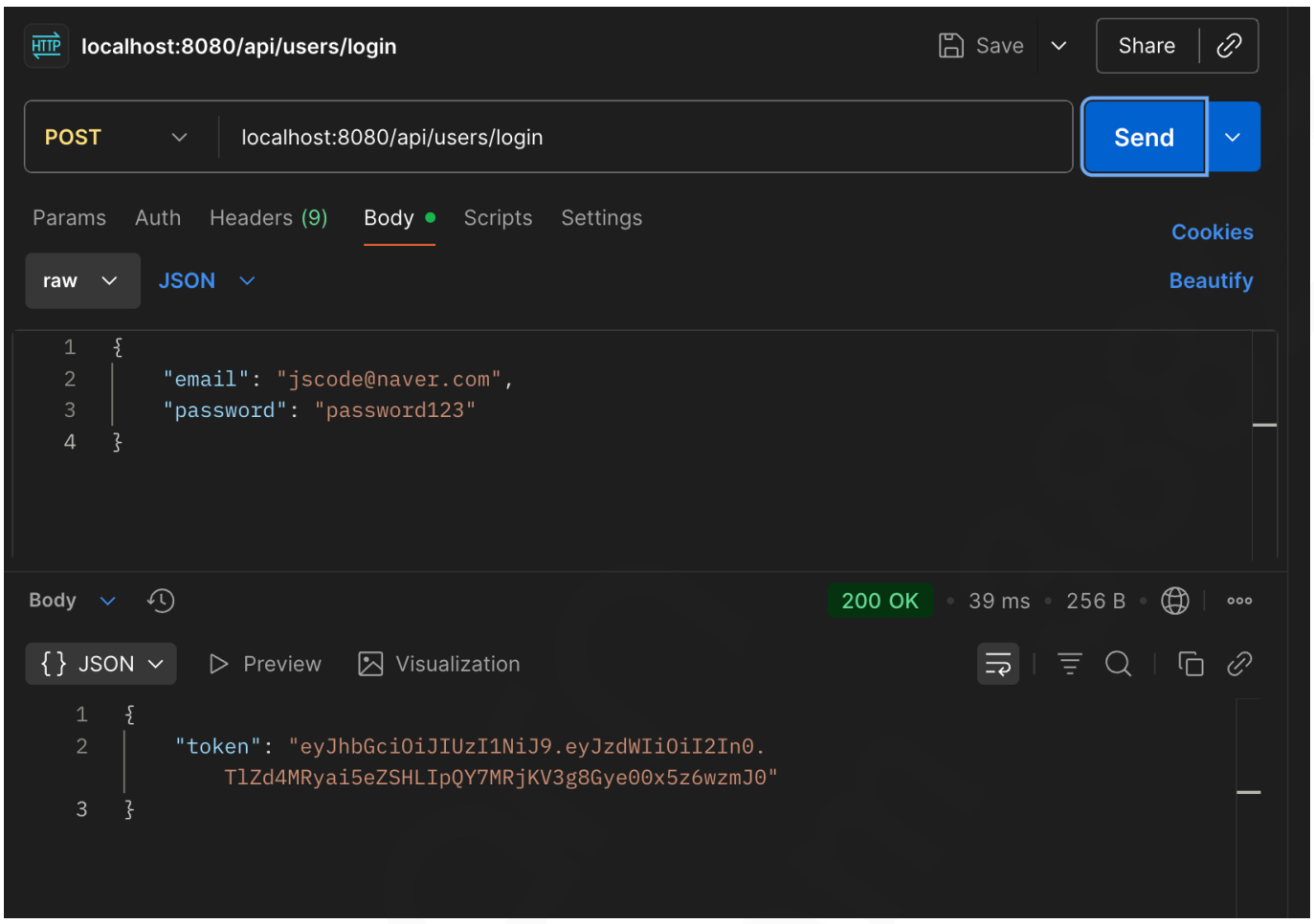
2. **서버 다시 실행시키기**

3. **Header에 토큰 안 넣고, 게시글 작성 API 요청 보내보기**

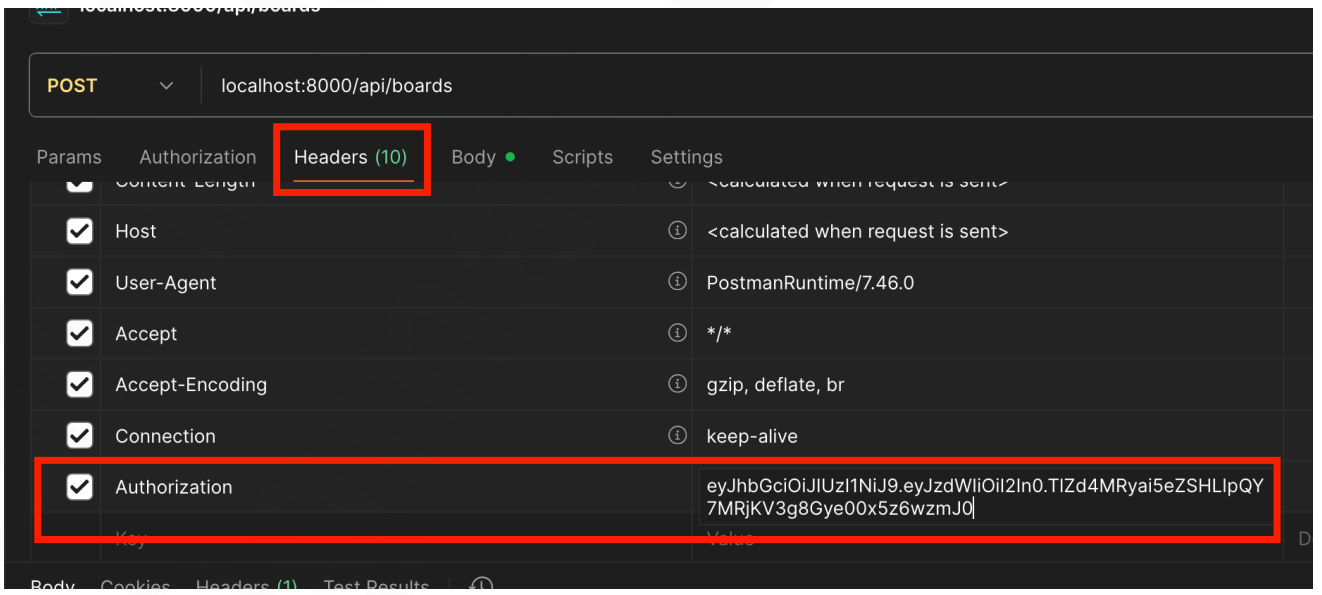


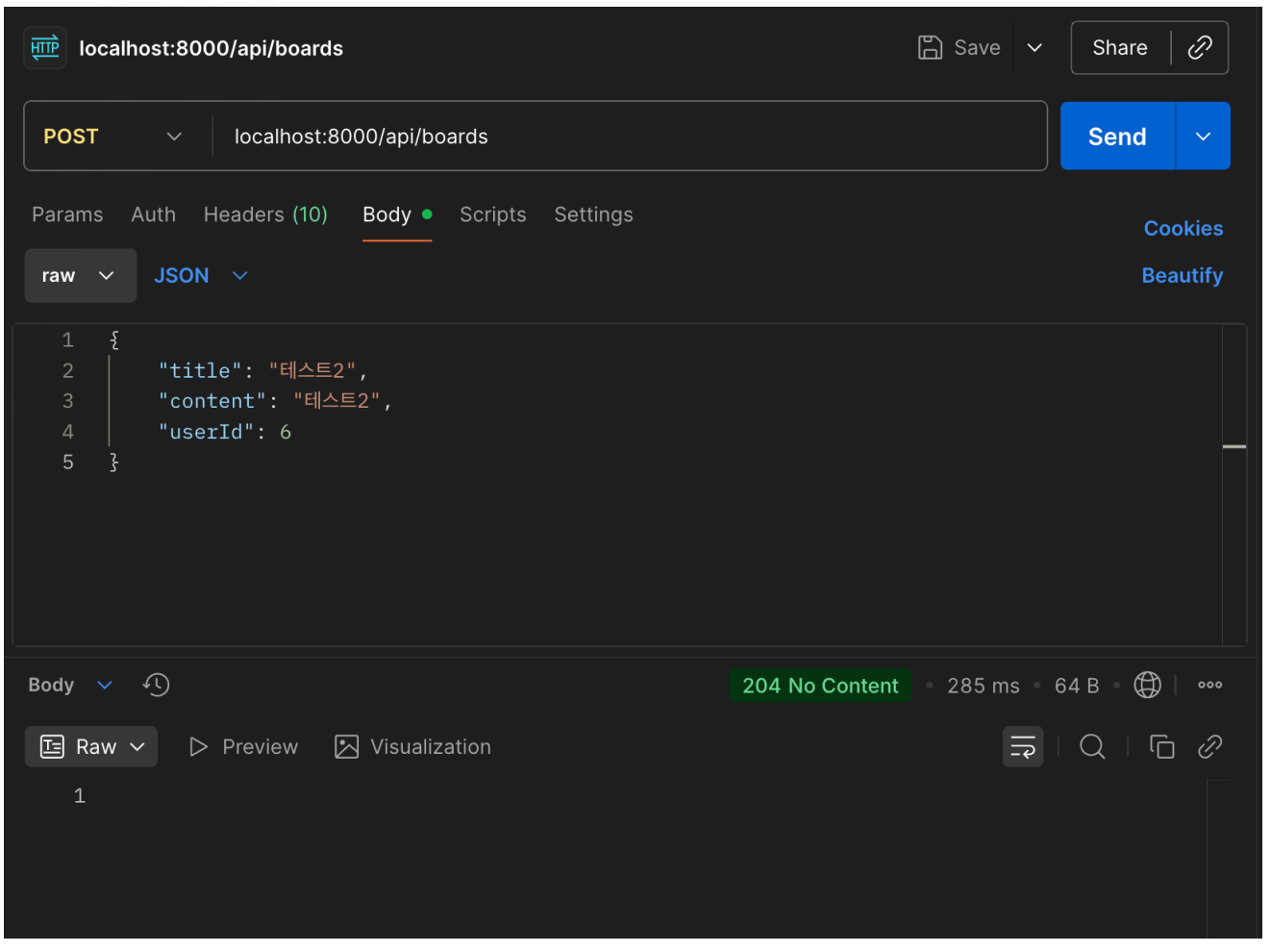
API Gateway의 인증 로직에 따라 요청이 잘 가로막혔다.

4. **Header에 토큰 넣고 요청 보내보기**
    a. **로그인 API로 토큰 발급받기**



b. **발급받은 토큰을 Header에 넣고, 게시글 작성 API 요청 보내보기**





정상적으로 게시글 작성이 된 걸 확인할 수 있다.

@kfc3882@gmail.com
kfc3025

🎯 [실습] 게시글 작성 로직 수정하기

✅ 게시글 작성 로직 수정하기

현재 게시글 작성 API는 `Request Body`에 포함된 `userId` 값을 기반으로 로직을 처리하고 있다. 하지만 이 방식은 클라이언트가 임의로 다른 사용자의 `userId`를 입력해 악의적인 요청을 보낼 수 있다는 보안 문제가 있다. 따라서 API Gateway에서 JWT 토큰을 파싱해 생성한 `X-User-Id` 헤더 값을 활용하도록 코드를 수정해야 한다.

✅ 코드 수정하기

> board-service에서 코드 수정하기

1. **DTO 수정하기**

   dto/CreateBoardRequestDto

```
public class CreateBoardRequestDto {
  private String title;

  private String content;

  private Long userId;

  public String getTitle() {
    return title;
  }

  public String getContent() {
    return content;
  }

  public Long getUserId() {
    return userId;
  }
}
```

2. **Controller 수정하기**

   controller/BoardController

```
@RestController
@RequestMapping("/api/boards")
public class BoardController {
    ...

   @PostMapping
  public ResponseEntity<Void> create(
     @RequestBody CreateBoardRequestDto createBoardRequestDto,
     @RequestHeader("X-User-Id") Long userId
  ) {
    boardService.create(createBoardRequestDto, userId);
    return ResponseEntity.noContent().build();
```

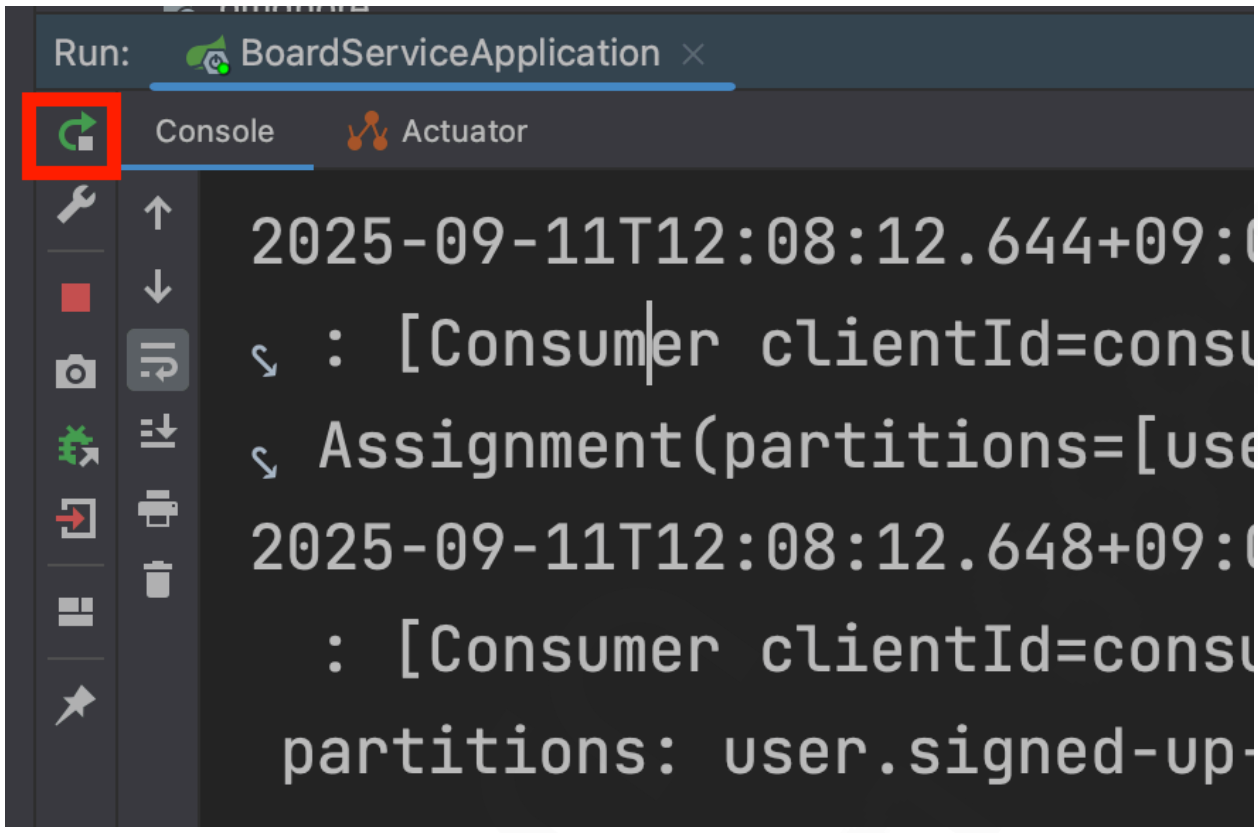
```
  }
}
```

3. **Service 수정하기**

   service/BoardService

```
public void create(CreateBoardRequestDto createBoardRequestDto, Long userId) {
  // 게시글 저장을 성공했는 지 판단하는 플래그
  boolean isBoardCreated = false;
  Long savedBoardId = null;

  // 포인트 차감을 성공했는 지 판단하는 플래그
  boolean isPointDeducted = false;

  try {
    // 게시글 작성 전 100 포인트 차감
    pointClient.deductPoints(userId, 100);
    isPointDeducted = true; // 포인트 차감 성공 플래그
    System.out.println("포인트 차감 성공");

    // 게시글 작성
    Board board = new Board(
        createBoardRequestDto.getTitle(),
        createBoardRequestDto.getContent(),
        userId
    );

    Board savedBoard = this.boardRepository.save(board);
    savedBoardId = savedBoard.getBoardId();
    isBoardCreated = true; // 게시글 저장 성공 플래그
    System.out.println("게시글 저장 성공");

    // '게시글 작성 완료' 이벤트 발행
    BoardCreatedEvent boardCreatedEvent
        = new BoardCreatedEvent(userId);
    this.kafkaTemplate.send("board.created", toJsonString(boardCreatedEvent));

  } catch (Exception e) {
    if (isBoardCreated) {
      // 게시글 작성 보상 트랜잭션 => 게시글 삭제
      this.boardRepository.deleteById(savedBoardId);
      System.out.println("[보상 트랜잭션] 게시글 삭제");
    }
    if (isPointDeducted) {
      // 포인트 차감 보상 트랜잭션 => 포인트 적립
      pointClient.addPoints(userId, 100);
      System.out.println("[보상 트랜잭션] 포인트 적립");
    }

    // 실패 응답으로 처리하기 위해 예외 던지기
    throw e;
  }
}
```

4. **서버 다시 실행시키기**



👨🏻‍🏫 다음 강의에서 '게시글 작성 API' 뿐만 아니라 구현한 모든 API가 정상적으로 잘 작동하는 지 테스트해보자.

khb3025@gmail.com

🎯 프로젝트의 모든 API 잘 작동하는 지 테스트하기

✅ 프로젝트의 모든 API 잘 작동하는 지 테스트하기

현재까지 아래의 기능들을 구현해왔다. 잘 작동하는 지 전부 테스트해보자.

- 회원가입 API → 포인트 적립
- 로그인 API → JWT 토큰 발급
- 게시글 작성 API (JWT 토큰 검증) → 포인트 차감 → 활동 점수 적립
- 전체 게시글 조회 API → 사용자 정보도 같이 조회
- 특정 게시글 조회 API → 사용자 정보도 같이 조회

✅ 테스트하기

> 실제 클라이언트가 요청을 보내듯이 API Gateway 주소(localhost:8000)로 테스트 하자.

1. **테스트를 위해 DB 비우기**

   [board-db]

   `boards`, `users` 테이블 데이터 전부 삭제하기

   [point-db]

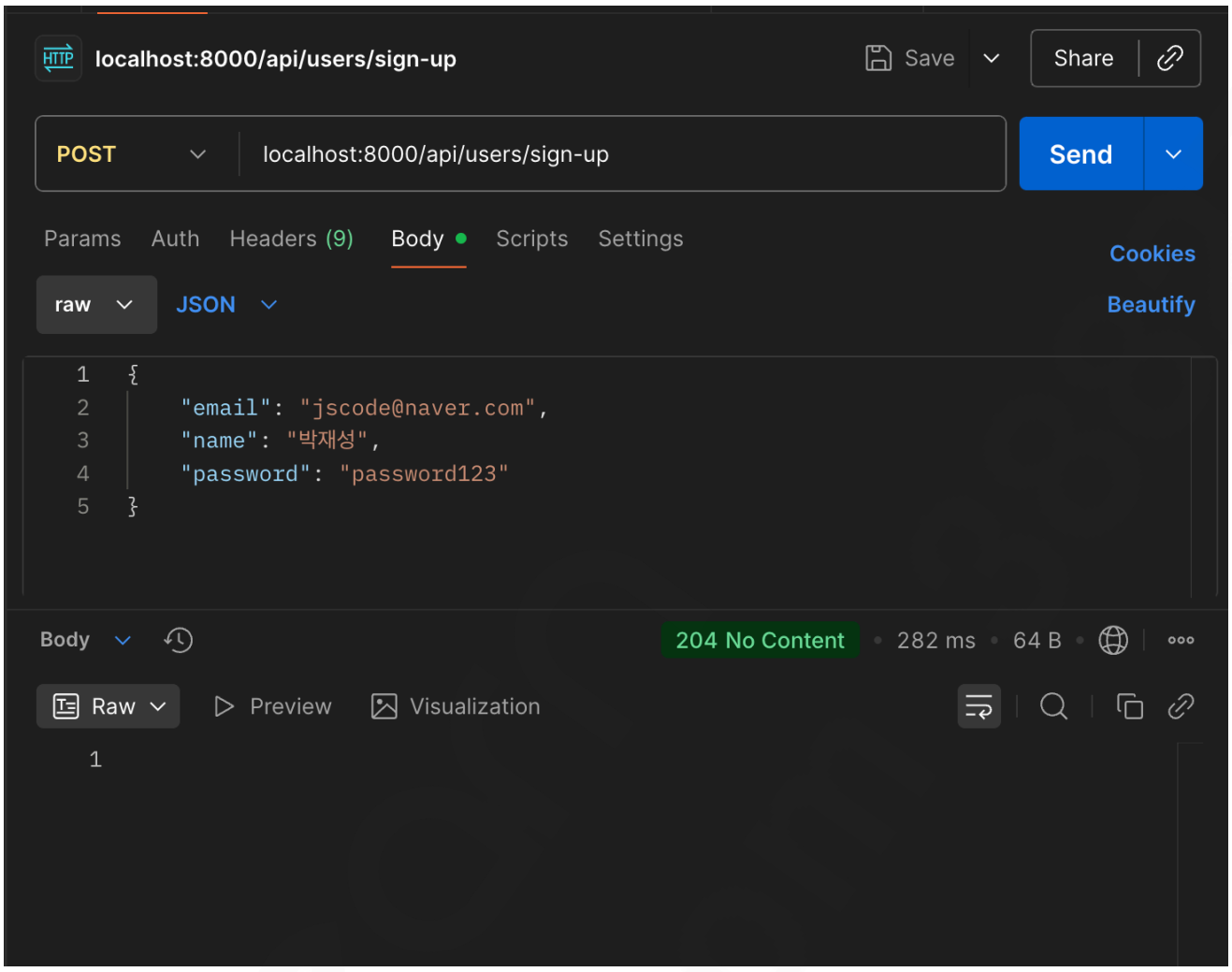
   `points` 테이블 데이터 전부 삭제하기

   [user-db]

   `users` 테이블 데이터 전부 삭제하기

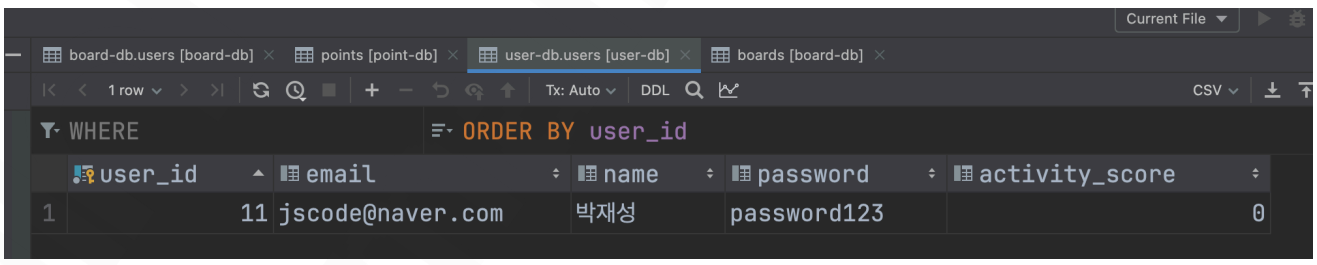
6. **회원가입 API 테스트하기**
   a. **API 요청 보내기**

khb3025@gmail.com 3882**

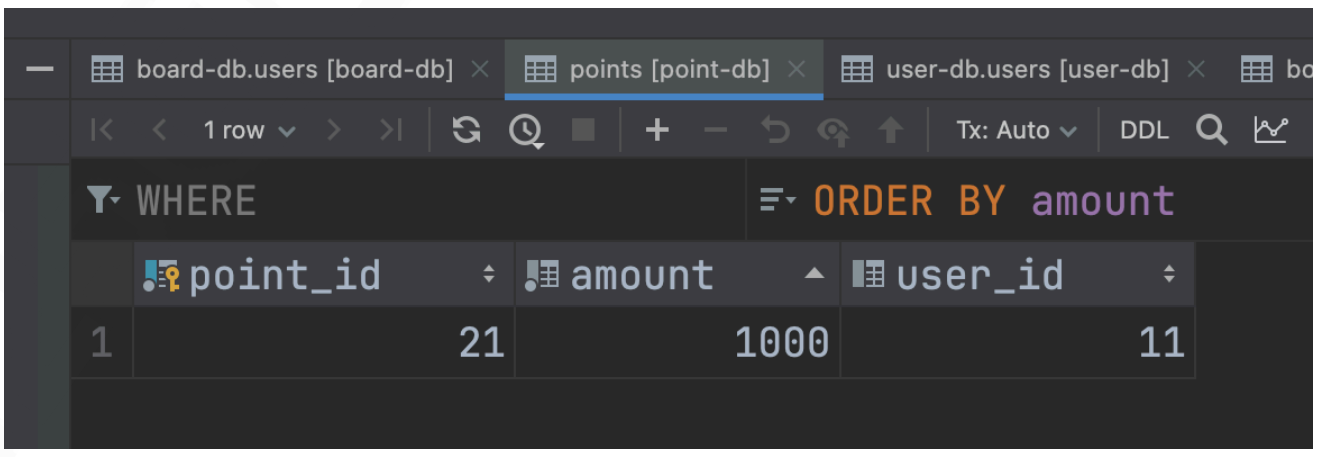


b. **DB 확인하기**

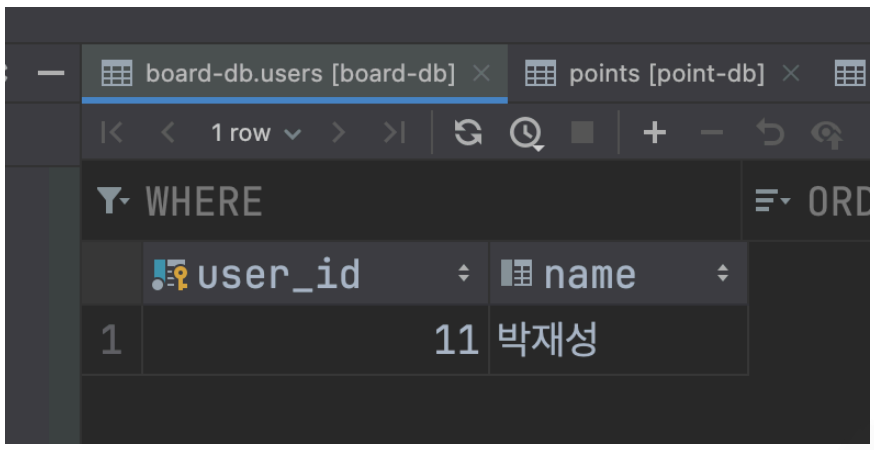
[user-db]



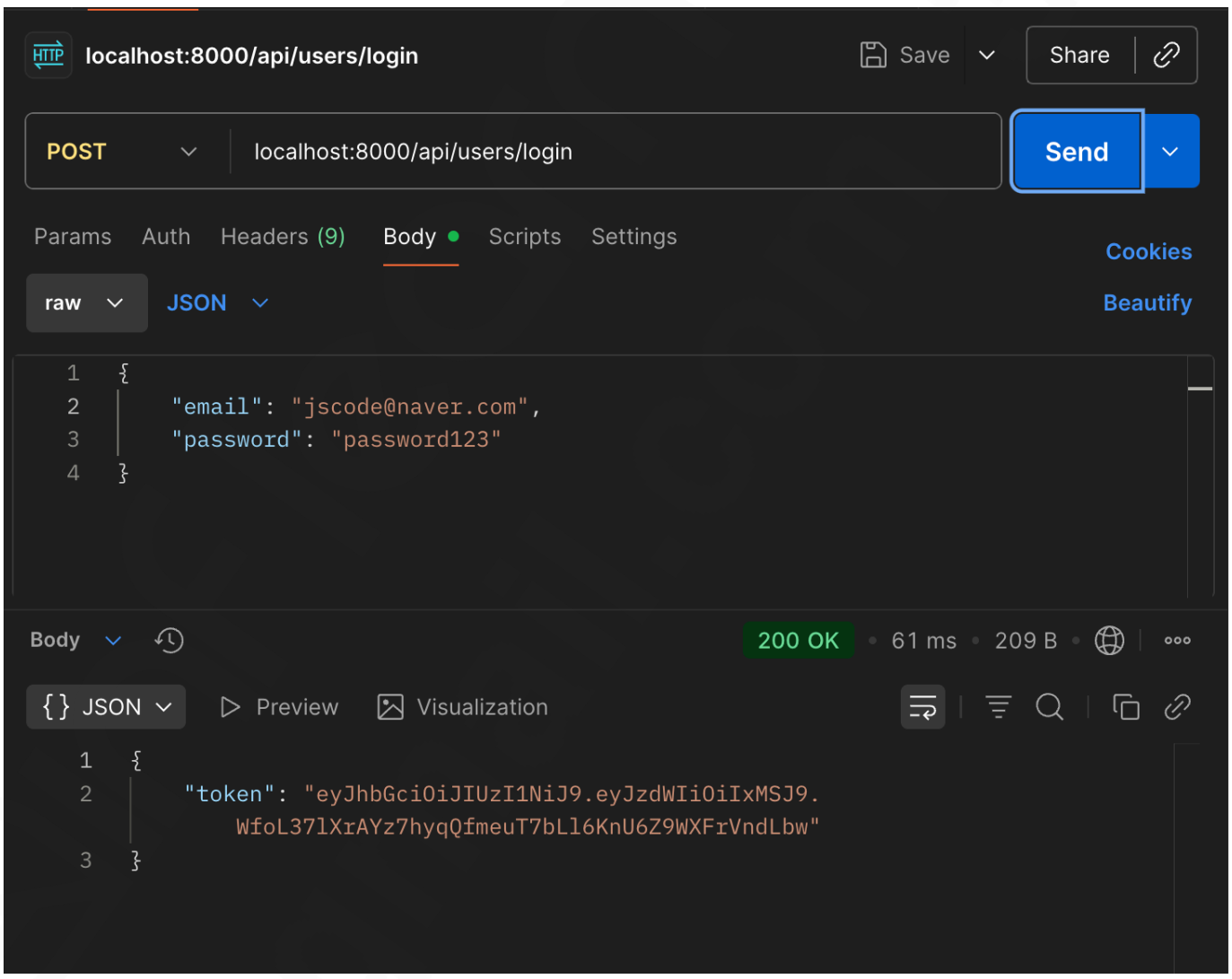
[point-db]



[board-db]



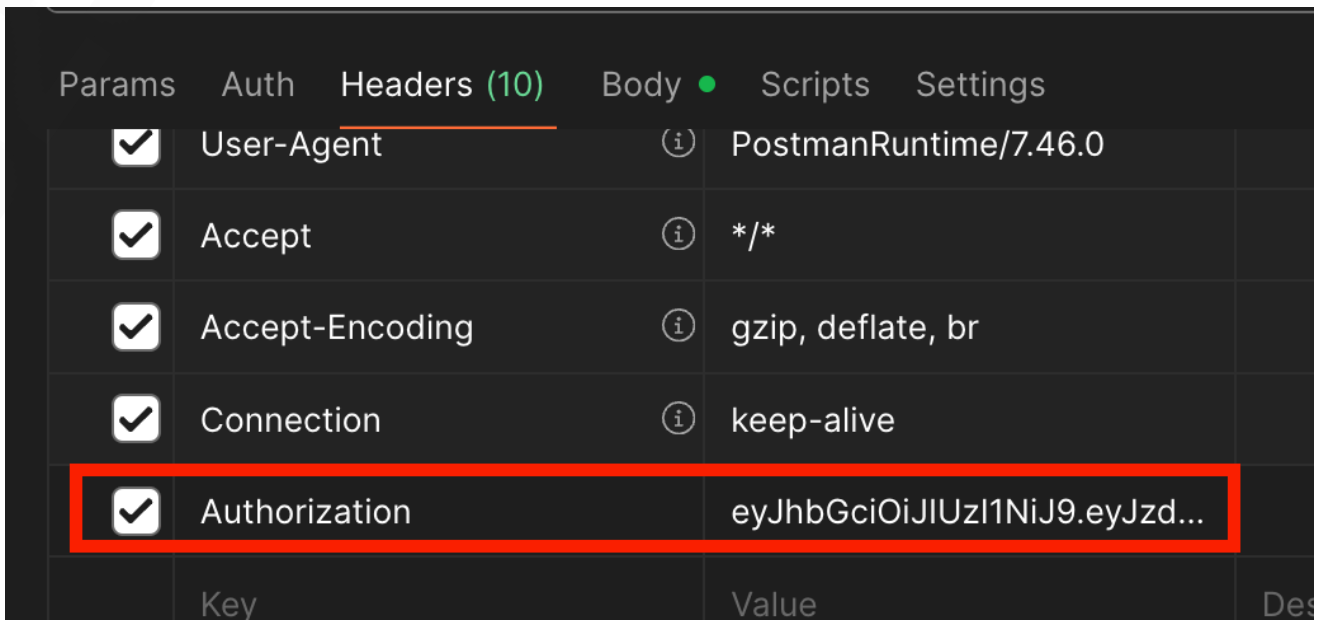
3. **로그인 API 테스트하기**

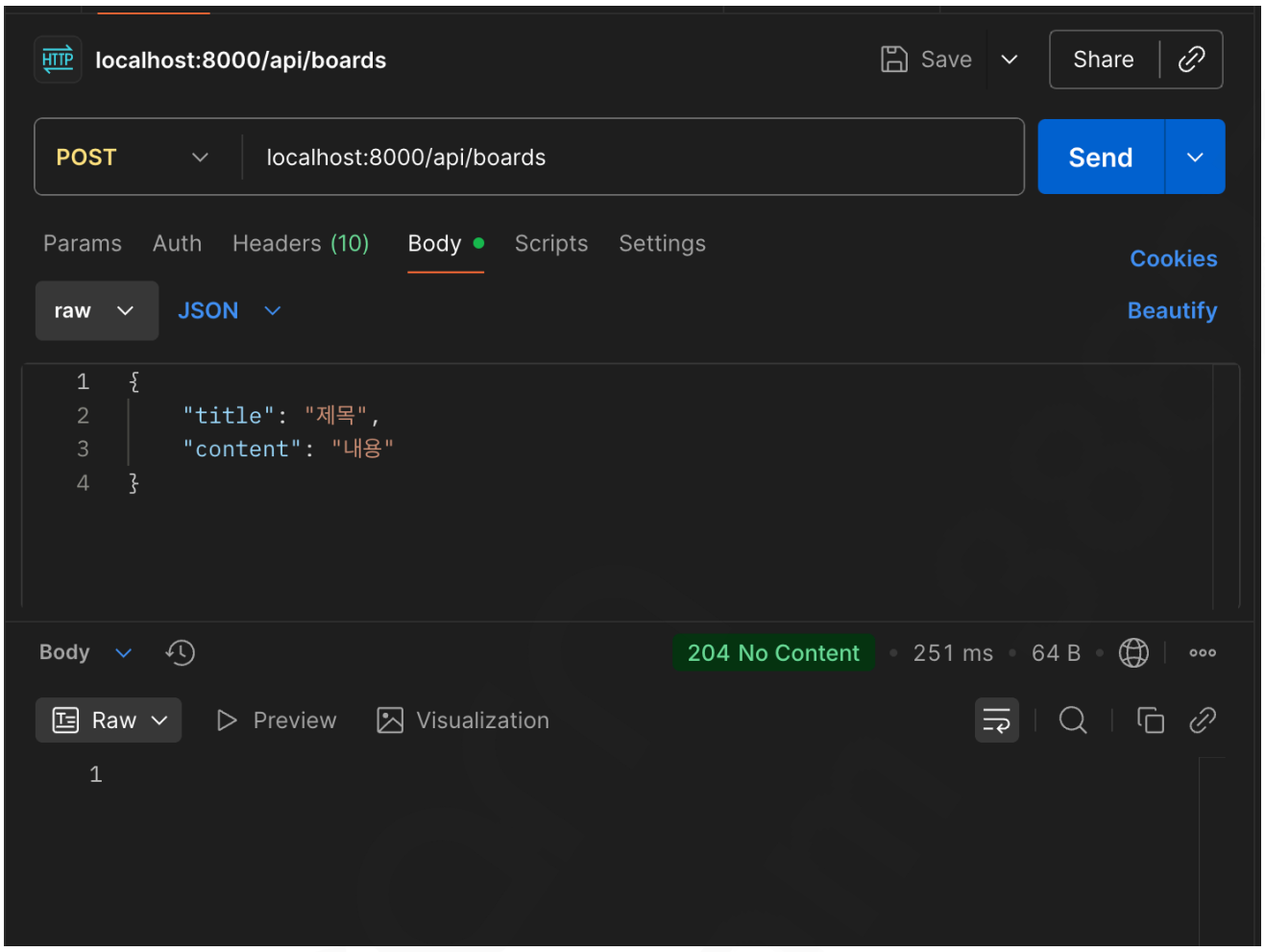


로그인해서 얻은 토큰을 잘 복사해두자.

4. **게시글 작성 API 테스트하기**

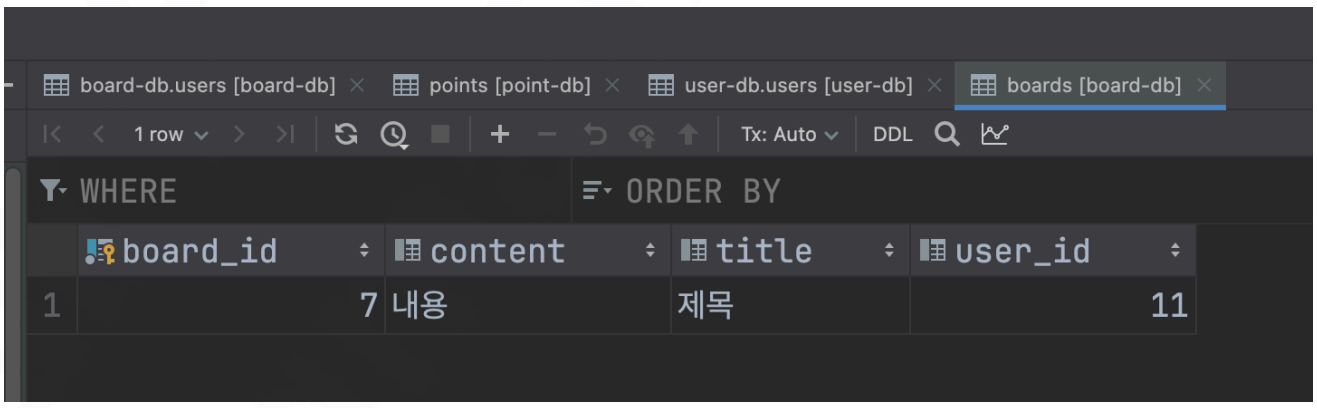
a. **API 요청 보내기**



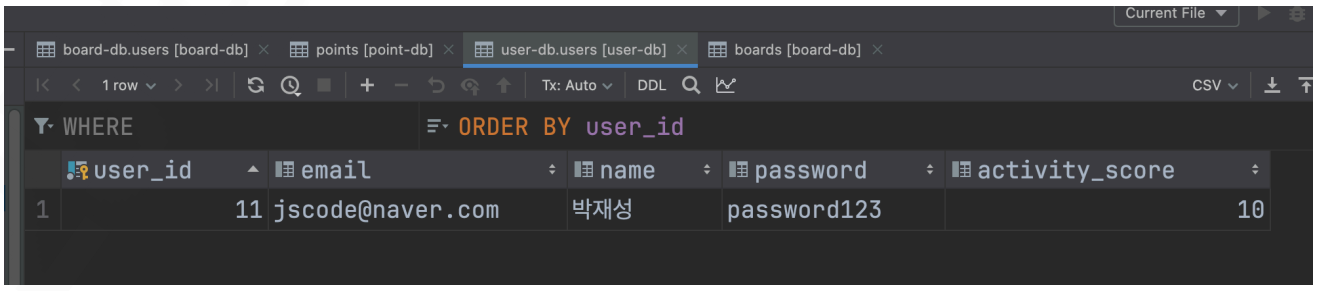


b. **DB 확인하기**

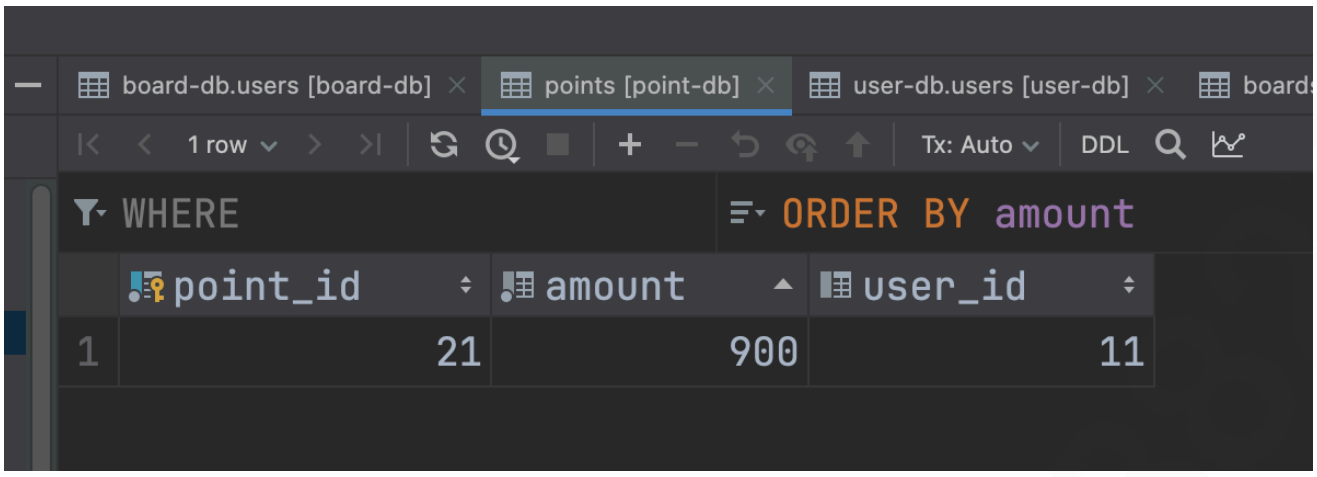
[board-db]



[user-db]

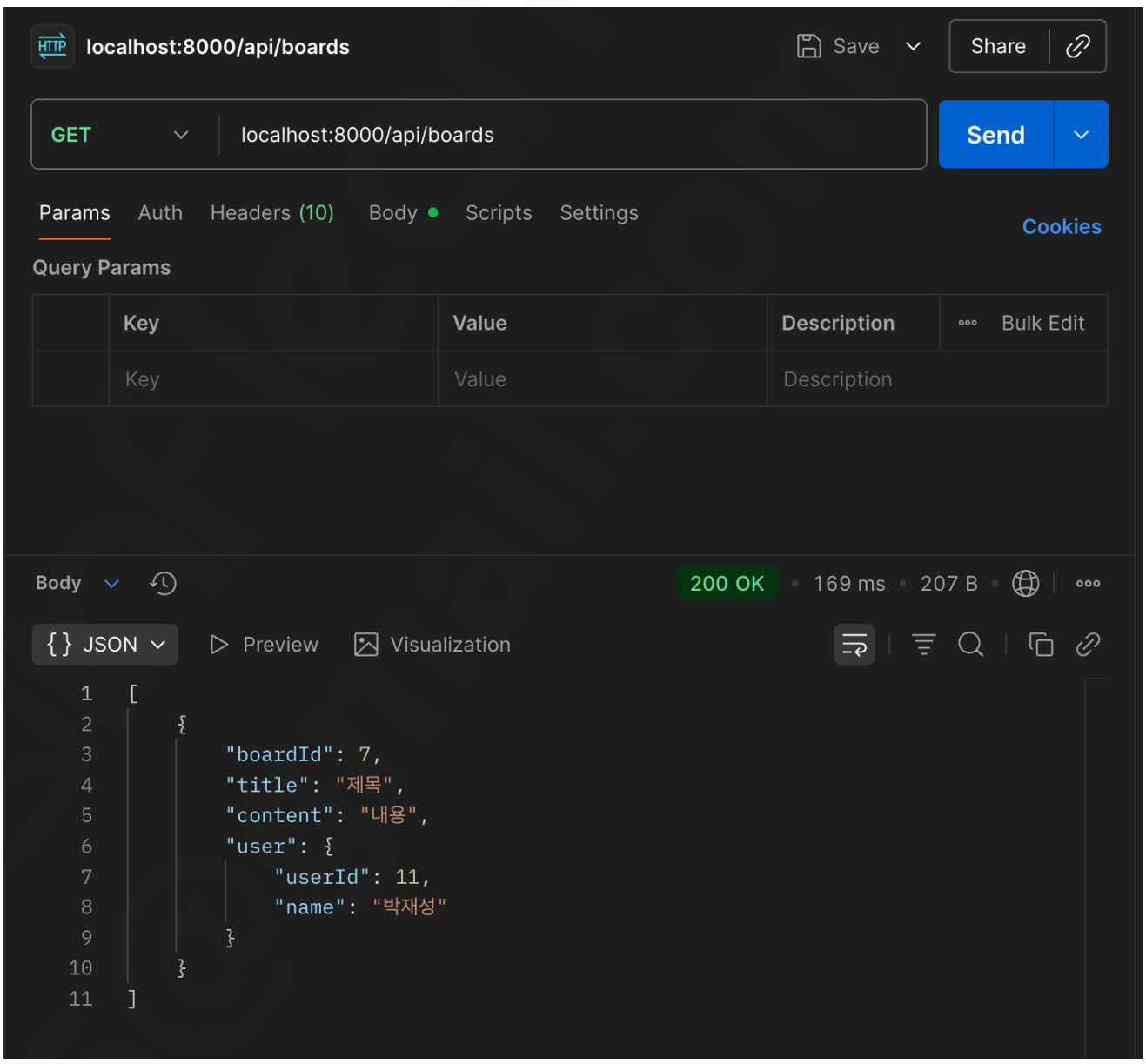


[point-db]



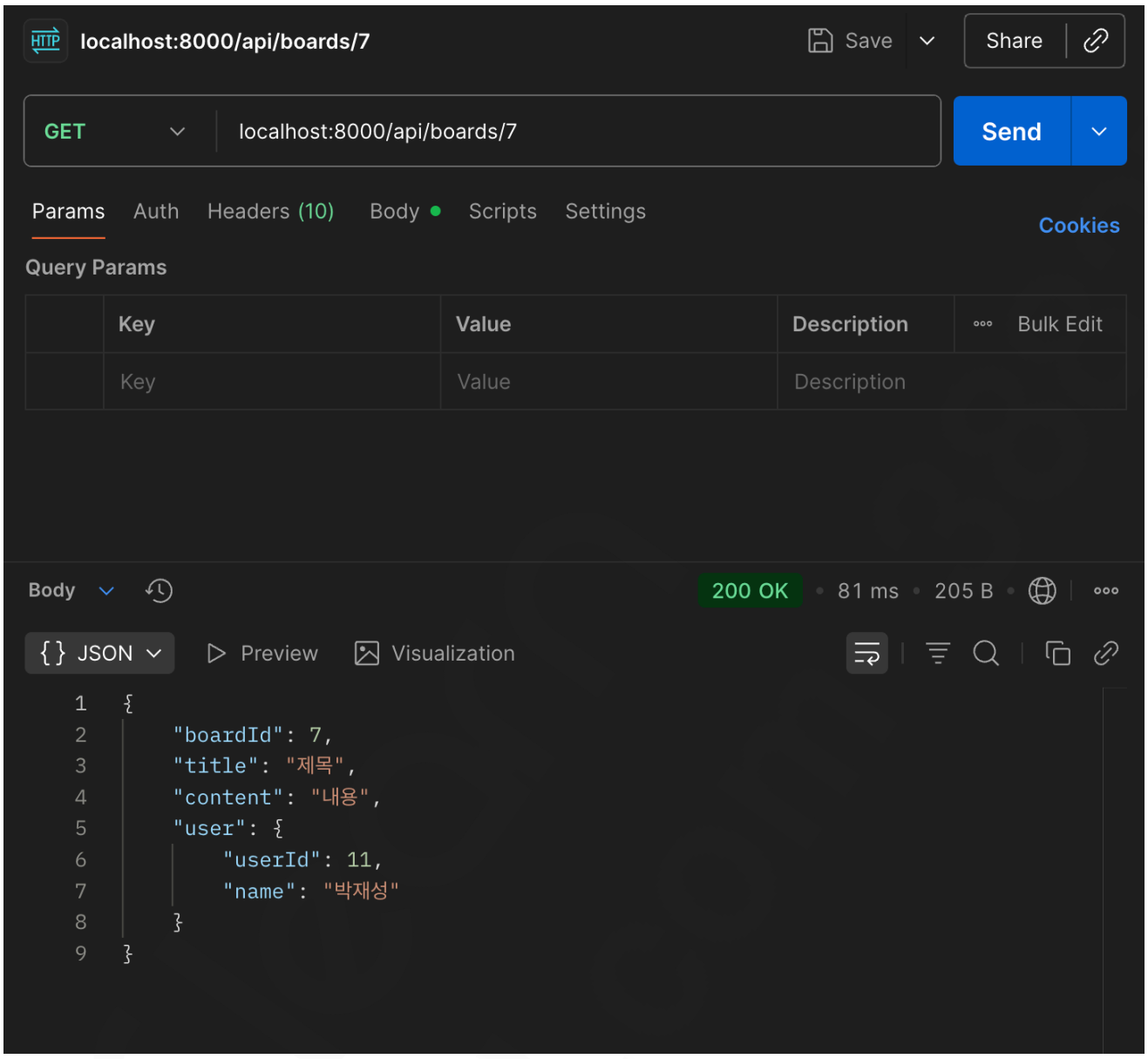
5. **게시글 조회 API 테스트**

   [전체 게시글 조회 API]



[특정 게시글 조회 API]

khb3025



kh03025@gmail.com

📖

MSA는 꼭 쿠버네티스로 배포해야 할까?

✅ MSA는 꼭 쿠버네티스로 배포해야 할까?

마이크로서비스의 개수가 3~5개 정도로 작을 때는 굳이 쿠버네티스를 안 써도 된다. 오히려 쿠버네티스를 쓰는 게 오버스펙인 경우가 많다. MSA 구조를 도입한 회사들 중에서 쿠버네티스(EKS)를 사용하지 않고 EC2, ECS로 운영하는 경우도 많다. 혹시나 'MSA = 쿠버네티스 운영'이라고 알고 있다면 잘못 알고 있는 것이다.

그리고 처음 MSA를 배우는 입장에서 쿠버네티스를 활용해 마이크로서비스를 배포하고 관리하려고 하다보면 난이도가 확 올라가서 어렵게 느껴질 수 밖에 없다. 따라서 처음에는 간단하게 AWS EC2를 활용해 마이크로서비스를 배포하고 운영하는 걸 연습하는 걸 추천한다. 지금까지 배운 MSA 개념이 익숙해져서 자유자재로 코드를 작성할 수 있게 되면, 그 다음에서야 쿠버네티스로 배포하는 연습을 해볼 것을 추천한다.

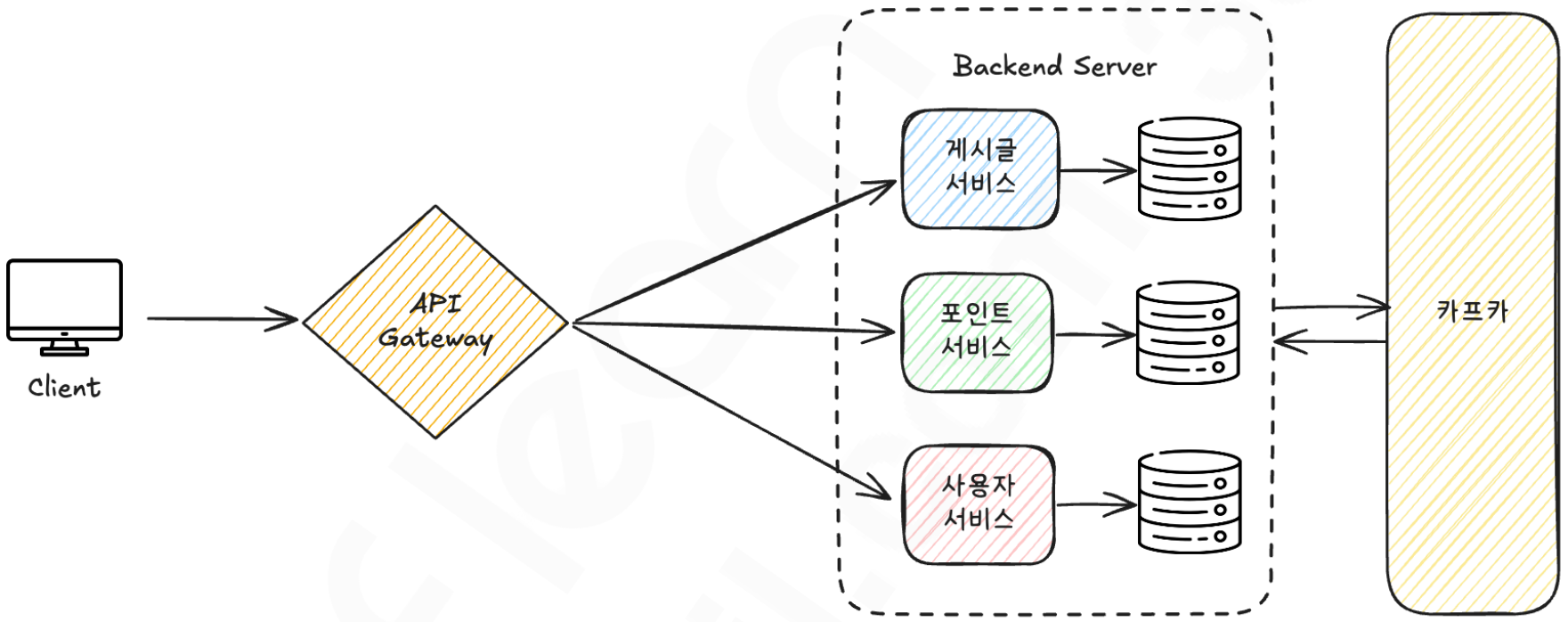
@kb302571@gmail.com
kb302571

💯

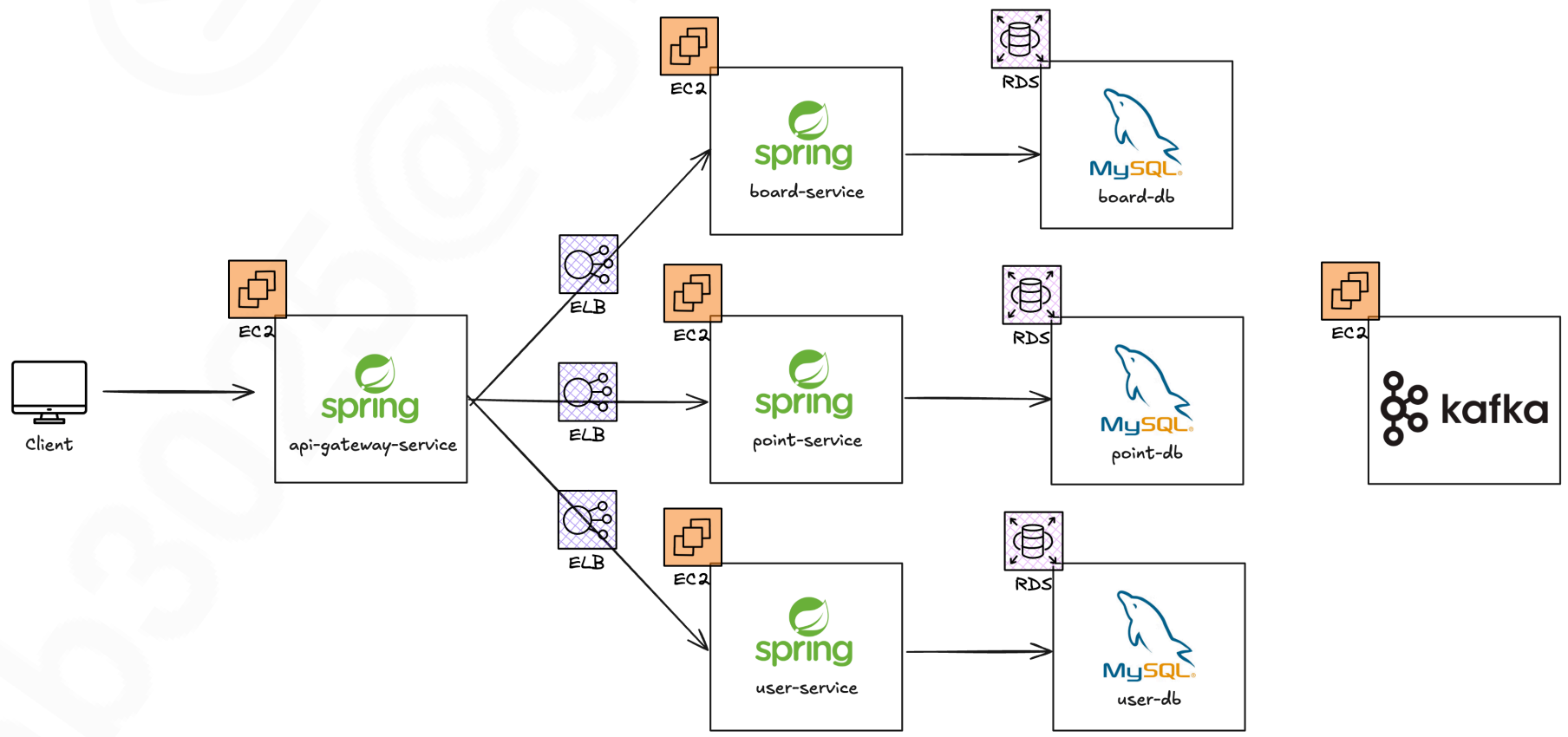
[미션] AWS에 MSA 프로젝트 배포하기

✅ AWS에 MSA 프로젝트 배포하기

현재까지 아래와 같은 구성으로 MSA 프로젝트를 구성했다. 이 MSA 구성을 클라우드 환경인 AWS에 배포하는 것까지 해보는 걸 추천한다. AWS로 서버를 배포해본 적이 있다면 충분히 구현할 수 있을 것이다.



✅ 구현 방식 1) 비용이 어느 정도 나와도 상관 없는 경우



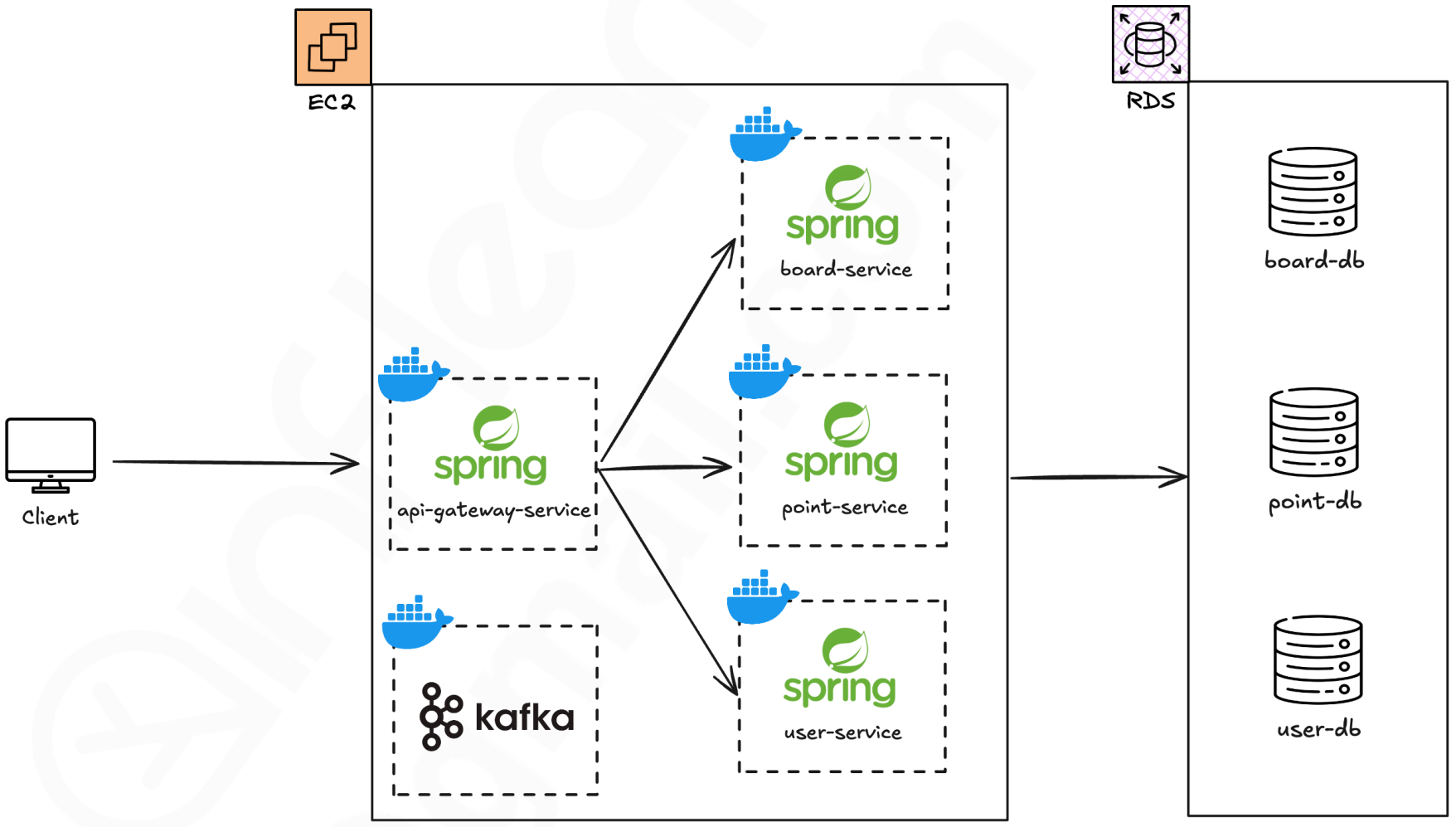
- 각 서버를 별개의 EC2에 따로 배포하기
- 각 마이크로서비스마다 ELB를 셋팅하기
- 각 마이크로서비스의 DB마다 RDS를 셋팅하기

- EC2에 Kafka를 설치해서 셋팅하거나, AWS MSK 셋팅하기

[심화]

- EC2에 오토 스케일링을 적용시켜 각 마이크로서비스의 확장성을 확보하기
- VPC의 Public Subnet과 Private Subnet을 활용해 각 리소스마다 접근 제한하기
  - api-gateway-service를 Public Subnet에 배치해 외부의 클라이언트가 접근할 수 있게 만들기
  - 마이크로 서비스와 DB 서버, Kafka 서버는 Private Subnet에 배치해 외부의 클라이언트가 직접적으로 접근하지 못하게 막기

✅ 구현 방식 2) 최저 비용으로 연습하고 싶은 경우



- 하나의 EC2 인스턴스에 Docker를 활용해 Spring 서버 4대와 Kafka를 띄우기
  - EC2 인스턴스의 사양이 부족해서 EC2가 멈추거나 버벅거릴 수 있으므로, 배포를 해보면서 EC2 사양을 선택하도록 하자. 또한 EC2에 메모리 Swap을 적용시켜서 최소한의 사양 안에서 돌릴 수 있게 시도하도록 하자.
- 하나의 RDS 인스턴스를 생성해 board-db, point-db, user-db 데이터베이스를 만들어서 사용하기

위의 방식으로 구현하다가 막히거나 궁금한 점이 생기면 언제든 질문 게시판에 질문해주세요😊